Java Guide

Core Java — Complete Reference

44 Topics	**72+ Java Files**	**OOP to Spring Boot**	**Java 8–17**

Generated from CoreJava_Basics/ — Java 8 through 17

Table of Contents

01 Java Introduction

- JavaIntroDemo.java

02 JRE JVM JDK

- JVMJREDemo.java

03 Java Applications

- ApplicationTypesDemo.java

04 Java Editions

- JavaEditionsDemo.java

05 Java Frameworks

- FrameworksDemo.java

06 Java Syntax

- SyntaxDemo.java

07 Output Statements

- OutputDemo.java

08 Input Programs

- InputDemo.java

09 Basic Java Program

- HelloWorld.java

10 Static vs Instance Methods

- AppConfigDemo.java
- StaticCounterDemo.java
- StaticVsInstanceDemo.java
- UserFactoryDemo.java

11 Variables and DataTypes

- ProductInventoryDemo.java
- UserModelDemo.java
- VariablesDemo.java

12 Scanner Class Examples

- FormInputDemo.java
- ScannerDemo.java

13 OOP Classes Objects

- AngularComponentDemo.java
- BankAccount.java
- Product.java
- Student.java

14 Inheritance Polymorphism

- AngularServiceDemo.java
- InheritanceDemo.java
- Person.java

15 Encapsulation Abstraction

- EncapsulationDemo.java

16 Control Flow

- ControlFlowDemo.java
- UserAccessControlDemo.java

17 Strings

- StringMethodsDemo.java
- UserInputValidationDemo.java

18 Arrays

- ArraysDemo.java
- ProductCatalogDemo.java

19 Exception Handling

- ExceptionHandlingDemo.java

19 Methods

- AngularServiceDemo.java

20 Collections Framework

- ArrayListDemo.java
- HashMapDemo.java
- HashSetDemo.java
- LinkedListDemo.java
- PriorityQueueDemo.java
- StackDemo.java

21 Generics

- AdvancedGenericsDemo.java

21 Inheritance

- AngularServiceHierarchyDemo.java

22 Generics Multithreading FileIO

- FileIODemo.java
- GenericsDemo.java
- MultithreadingDemo.java

22 Polymorphism

- PaymentProcessorDemo.java

23 Encapsulation

- UserModelDemo.java

23 JDBC Database Connectivity

- JDBCDemo.java

24 Interfaces

- DataProviderDemo.java

24 Serialization ObjectIO

- SerializationDemo.java

25 Lambda Stream API

- AngularIntegrationDemo.java
- LambdaExpressionsDemo.java
- StreamBasicDemo.java
- StreamIntermediateDemo.java
- StreamTerminalDemo.java

26 Advanced Stream Operations

- AdvancedStreamDemo.java

26 Lambda Streams

- AngularPipesDemo.java

27 Annotations Java8 DesignPatterns

- AnnotationsDemo.java

27 Exception Handling

- APIFaultHandlerDemo.java

28 Java 9 10 11 12 Features

- DateTimeAPIDemo.java

28 Wrapper Classes

- AngularDataTypeDemo.java

29 Backend Design Patterns

- DesignPatternsDemo.java
- ServiceFactoryDemo.java

30 REST API Backend Angular Integration

- User.java
- UserController.java
- UserCRUDDemo.java

31 File IO

- UserDataFileDemo.java

31 Spring Boot Basics

- SpringBootConceptsDemo.java

32 Database JPA Hibernate

- JPADemo.java

32 Generics

- AngularDataServiceDemo.java

33 JSON Jackson Integration

- JSONDemo.java

33 Java 17 Features

- ModernJavaFeaturesDemo.java

00

Complete Learning Path

Complete Java Learning Path for Angular Developers

Overview

This document provides a comprehensive list of Java core topics organized in a logical learning progression for developers who are learning Angular and want to build full-stack applications with Java backends.

Phase 1: Java Fundamentals (Topics 1-12)

Week 1: Getting Started

Topic	File	Description
1. Java Introduction	`01_Java_Introduction.md`	What is Java, history, features
2. JRE, JVM, JDK	`02_JRE_JVM_JDK.md`	Understanding Java architecture
3. Java Applications	`03_Java_Applications.md`	Types of Java applications
4. Java Editions	`04_Java_Editions.md`	SE, EE, ME, Jakarta
5. Java Frameworks	`05_Java_Frameworks.md`	Popular frameworks overview
6. Java Syntax	`06_Java_Syntax.md`	Basic syntax rules

Week 2: Basic Programming

Topic	File	Description
7. Output Statements	`07_Output_Statements.md`	System.out, printf, logging
8. Input Programs	`08_Input_Programs.md`	Scanner, BufferedReader
9. Basic Java Program	`09_Basic_Java_Program.java`	Hello World example

Topic	File	Description
10. Static vs Instance	`10_Static_vs_Instance_Methods.java`	Class vs object methods
11. Variables & DataTypes	`11_Variables_and_DataTypes.java`	Primitives, references
12. Scanner Examples	`12_Scanner_Class_Examples.java`	User input handling

Phase 2: Object-Oriented Programming (Topics 13-15)

Week 3: OOP Core Concepts

Topic	File	Examples
13. Classes & Objects	`13_OOP_Classes_and_Objects.md`	`BankAccount.java`, `Product.java`, `Student.java`
14. Inheritance & Polymorphism	`14_Inheritance_Polymorphism.md`	`InheritanceDemo.java`
15. Encapsulation & Abstraction	`15_Encapsulation_Abstraction.md`	Data hiding, access modifiers

Why OOP matters for Angular:

- Java backend uses classes to model data (entities, DTOs)
- REST APIs return objects that Angular consumes as JSON
- Understanding OOP helps design better data models
- Spring Boot relies heavily on OOP principles

Phase 3: Core Java Features (Topics 16-20)

Week 4: Control Flow & Data

Topic	File	Description
16. Control Flow	`16_Control_Flow_Statements.md`	if-else, switch, loops

Topic	File	Description
17. Strings	`17_Strings_and_String Handling.md`	String operations, immutability
18. Arrays	`18_Arrays.md`	Single and multidimensional arrays

Week 5: Error Handling & Collections

Topic	File	Examples
19. Exception Handling	`19_Exception_Handling .md`	`ExceptionHandlingDemo .java`
20. Collections Framework	`20_Collections_Framew ork.md`	`CollectionsDemo.java`

Angular Integration:

- ArrayList → JSON Array
- HashMap → JSON Object
- HashSet → Unique values array
- Exception handling → HTTP error responses

Phase 4: Advanced Java Features (Topics 21-29)

Week 6-7: Generics & Multithreading

Topic	File	Description
21. Generics	`21_Generics.md`	Type parameters, wildcards
22. Multithreading	`22_Multithreading.md`	Threads, synchronization
23. File I/O	`23_File_IO_Operations .md`	Reading/writing files

Week 8: Modern Java

Topic	File	Description
24. Lambda Expressions	`24_Lambda_Expressions.md`	Functional programming
25. Stream API	`25_Lambda_Stream_API.md`	`LambdaStreamDemo.java`
26. Optional	`26_Optional_Class.md`	Null handling
27. Annotations	`27_Annotations_Java8_DesignPatterns.md`	Metadata,■■
28. Date/Time API	`28_Java_Date_Time_API.md`	java.time package
29. Design Patterns	`29_Backend_Design_Patterns.md`	Singleton, Factory, Repository

Phase 5: Full-Stack Integration (Topics 30-33)

Week 9-10: REST API & Database

Topic	File	Examples
30. REST API	`30_REST_API_Backend_Angular_Integration.md`	`UserController.java`
31. Spring Boot Basics	`31_Spring_Boot_Basics.md`	Dependency injection
32. Database & JPA	`32_Database_JPA_Hibernate.md`	ORM, entities
33. JSON & Jackson	`33_JSON_Jackson_Integration.md`	JSON serialization

Quick Reference: Angular-Java Mapping

Angular/TypeScript	Java Backend	Use Case
`interface`	`class`	Data models

Angular/TypeScript	Java Backend	Use Case
`class`	`class` + annotations	Entity classes
`enum`	`enum`	Constant values
`Observable<T>`	`ResponseEntity<T>`	HTTP responses
`HttpClient`	`@RestController`	API calls
`[()] two-way binding`	`@RequestBody`	Data input
`map()` operator	`Stream.map()`	Data transformation
`filter()` operator	`Stream.filter()`	Data filtering
`async/await`	`CompletableFuture`	Async operations

File Structure

```
CoreJava_Basics/
■■■ 00_Complete_Learning_Path.md      # This file
■■■ 01_Java_Introduction.md
■■■ 02_JRE_JVM_JDK.md
■■■ 03_Java_Applications.md
■■■ 04_Java_Editions.md
■■■ 05_Java_Frameworks.md
■■■ 06_Java_Syntax.md
■■■ 07_Output_Statements.md
■■■ 08_Input_Programs.md
■■■ 09_Basic_Java_Program.java
■■■ 10_Static_vs_Instance_Methods.java
■■■ 11_Variables_and_DataTypes.java
■■■ 12_Scanner_Class_Examples.java
■■■ 13_OOP_Classes_Objects/
■   ■■■ 13_OOP_Classes_and_Objects.md
■   ■■■ examples/
■       ■■■ BankAccount.java
■       ■■■ Product.java
■       ■■■ Student.java
■■■ 14_Inheritance_Polymorphism/
■   ■■■ 14_Inheritance_and_Polymorphism.md
■   ■■■ examples/
■       ■■■ InheritanceDemo.java
■■■ 15_Encapsulation_Abstraction/
■   ■■■ ...
■■■ 16_Control_Flow/
■   ■■■ ...
■■■ 17_Strings/
```

```
■   ■■■ ...
■■■ 18_Arrays/
■   ■■■ ...
■■■ 19_Exception_Handling/
■   ■■■ 19_Exception_Handling.md
■   ■■■ examples/
■       ■■■ ExceptionHandlingDemo.java
■■■ 20_Collections_Framework/
■   ■■■ 20_Collections_Framework.md
■   ■■■ examples/
■       ■■■ CollectionsDemo.java
■■■ 21_Generics/
■   ■■■ ...
■■■ 22_Generics_Multithreading_FileIO/
■   ■■■ ...
■■■ 23_File_IO_Operations/
■   ■■■ ...
■■■ 24_Lambda_Expressions/
■   ■■■ ...
■■■ 25_Lambda_Stream_API/
■   ■■■ 25_Lambda_Stream_API.md
■   ■■■ examples/
■       ■■■ LambdaStreamDemo.java
■■■ 26_Advanced_Stream_Operations/
■   ■■■ ...
■■■ 27_Annotations_Java8_DesignPatterns/
■   ■■■ ...
■■■ 28_Java_9_10_11_12_Features/
■   ■■■ ...
■■■ 29_Backend_Design_Patterns/
■   ■■■ ...
■■■ 30_REST_API_Backend_Angular_Integration/
■   ■■■ 30_REST_API_Backend_Angular_Integration.md
■   ■■■ examples/
■       ■■■ UserController.java
■■■ 31_Spring_Boot_Basics/
■   ■■■ ...
■■■ 32_Database_JPA_Hibernate/
■   ■■■ ...
■■■ 33_JSON_Jackson_Integration/
    ■■■ ...
```

Recommended Learning Sequence

1. **Start with Phase 1** - Master basics (2 weeks)
2. **Phase 2** - Understand OOP thoroughly (1 week)
3. **Phase 3** - Learn collections and error handling (1 week)
4. **Phase 4** - Advanced features (2 weeks)
5. **Phase 5** - Build full-stack apps (2 weeks)

Key Takeaways

- Java is the backbone of enterprise backends
- Spring Boot makes Java development faster
- Understanding Java helps you design better Angular services
- Collections and Stream API mirror JavaScript array methods
- REST APIs are the bridge between Angular and Java

Last Updated: 2026

01

Java Introduction

01_Java_Introduction\examples\01_Java_Introduction.md

Java Introduction

Short Definition

Java is a high-level, object-oriented programming language developed by Sun Microsystems (now Oracle). It was designed to be platform-independent, meaning Java programs can run on any device that has a Java Virtual Machine (JVM). Java is known for its simplicity, robustness, and security features.

Key Bullet Points

- **Object-Oriented**: Java follows object-oriented programming (OOP) principles, organizing code into classes and objects.
- **Platform Independent**: Java achieves platform independence through the "Write Once, Run Anywhere" (WORA) principle.
- **Simple and Easy to Learn**: Java syntax is clean and easy to understand for beginners.
- **Robust**: Java has strong memory management and exception handling.
- **Secure**: Java provides security features to protect against viruses and unauthorized access.
- **Multithreaded**: Java supports multithreading, allowing concurrent execution of tasks.
- **Distributed**: Java is designed for distributed computing and network-based applications.

Why Java is Platform Independent

Java achieves platform independence through the following mechanism:

1. **Source Code to Bytecode**: When you compile a Java program, the compiler translates your source code (`.java` file) into bytecode (`.class` file). Bytecode is a platform-neutral intermediate code.
1. **Java Virtual Machine (JVM)**: The JVM is a virtual machine that runs on different operating systems (Windows, macOS, Linux). It interprets the bytecode and executes it.
1. **Key Point**: The same bytecode can run on any platform that has a JVM installed. You don't need to recompile the code for different operating systems.

```
Source Code (.java)
        ↓
    Compiler
        ↓
```

```
  Bytecode (.class)
        ↓
   JVM (Windows)
        ↓
   JVM (macOS)
        ↓
   JVM (Linux)
```

Simple Example Program

```
// HelloWorld.java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Expected Output

```
Hello, World!
```

Code Examples

JavaIntroDemo.java

JavaIntroDemo.java

```
// JavaIntroDemo - Introduction to Java
// Java is a high-level, object-oriented programming language

public class JavaIntroDemo {

    public static void main(String[] args) {
        System.out.println("=== JAVA INTRODUCTION ===\n");

        System.out.println("--- What is Java? ---");
        System.out.println("Java is a high-level, class-based, object-oriented programming langua...
        System.out.println("Designed to have as few implementation dependencies as possible");
        System.out.println("Write Once, Run Anywhere (WORA)");

        System.out.println("\n--- Key Features ---");
        System.out.println("1. Object-Oriented - Everything is an object");
        System.out.println("2. Platform Independent - Runs on any platform with JVM");
        System.out.println("3. Secured - Security manager for runtime checks");
```

```
        System.out.println("4. Multithreaded - Built-in thread support");
        System.out.println("5. Distributed - Designed for distributed computing");

        System.out.println("\n--- Java History ---");
        System.out.println("1991: James Gosling creates Oak at Sun Microsystems");
        System.out.println("1995: Java 1.0 released");
        System.out.println("2006: Java Open Source");
        System.out.println("2010: Oracle acquires Sun Microsystems");
        System.out.println("2017: Java 9 with modular system");
        System.out.println("2023: Java 21 LTS");

        System.out.println("\n--- Angular Integration ---");
        System.out.println("Java is commonly used as backend with Angular frontend");
        System.out.println("REST APIs connect Java services to Angular apps");
    }
}
```

02

JRE JVM JDK

02_JRE_JVM_JDK\examples\02_JRE_JVM_JDK.md

JRE, JVM, and JDK

Short Definition

Understanding the difference between JVM, JRE, and JDK is essential for Java developers. These are the three main components that enable Java programs to run on any platform.

Key Bullet Points

- **JVM (Java Virtual Machine)**: The engine that executes Java bytecode. It converts bytecode into machine-specific instructions.
- **JRE (Java Runtime Environment)**: Contains the JVM and libraries needed to run Java applications. It's what end-users need.
- **JDK (Java Development Kit)**: Contains JRE plus development tools (compiler, debugger). It's what developers need to create Java programs.
- All three work together to enable Java's platform-independent feature.

Relationship Diagram

```
■■■
```

Component	Full Name	Purpose	What It Contains
JVM	Java Virtual Machine	Executes bytecode	Interpreter, JIT Compiler, Garbage Collector
JRE	Java Runtime Environment	Runs Java applications	JVM + Core Libraries
JDK	Java Development Kit	Develop Java programs	JRE + Development Tools (javac, java, jar, etc.)

Simple Example

```
// Test.java
public class Test {
    public static void main(String[] args) {
        System.out.println("Java is platform independent!");
    }
}
```

How it works:

1. **Write**: You write the code in a `.java` file
2. **Compile**: JDK's `javac` compiler converts `.java` to `.class` (bytecode)
3. **Run**: JRE's JVM reads the `.class` file and executes it

Key Takeaway

- **For running Java apps**: Install JRE
- **For developing Java apps**: Install JDK
- **JVM**: Included in both JRE and JDK

Code Examples

JVMJREDemo.java

JVMJREDemo.java

```
// JVMJREDemo - Demonstrates JVM, JRE, JDK concepts
// Shows the Java platform architecture
```

```java
public class JVMJREDemo {

    public static void main(String[] args) {
        System.out.println("=== JVM, JRE, JDK DEMO ===\n");

        // Show Java version (runtime info)
        System.out.println("--- Java Runtime Information ---");
        System.out.println("Java Version: " + System.getProperty("java.version"));
        System.out.println("Java Vendor: " + System.getProperty("java.vendor"));
        System.out.println("Java Home: " + System.getProperty("java.home"));
        System.out.println("OS Name: " + System.getProperty("os.name"));
        System.out.println("OS Arch: " + System.getProperty("os.arch"));

        System.out.println("\n--- JDK vs JRE vs JVM ---");
        System.out.println("JDK (Java Development Kit):");
        System.out.println("  - Includes JRE + Development tools");
        System.out.println("  - Contains: javac, java, jar, jdb, etc.");
        System.out.println("  - Used for: Writing and compiling Java code");

        System.out.println("\nJRE (Java Runtime Environment):");
        System.out.println("  - Includes JVM + Libraries");
        System.out.println("  - Used for: Running Java applications");

        System.out.println("\nJVM (Java Virtual Machine):");
        System.out.println("  - Executes bytecode (.class files)");
        System.out.println("  - Provides platform independence");
        System.out.println("  - Components: Class Loader, Bytecode Verifier, JIT Compiler");

        System.out.println("\n--- How Java Works ---");
        System.out.println("1. Write: Developer writes .java source file");
        System.out.println("2. Compile: javac compiles to .class bytecode");
        System.out.println("3. Load: ClassLoader loads .class into JVM");
        System.out.println("4. Verify: BytecodeVerifier checks code");
        System.out.println("5. Execute: JIT compiles to native machine code");

        // Memory info
        System.out.println("\n--- JVM Memory ---");
        Runtime rt = Runtime.getRuntime();
        System.out.println("Max Memory: " + rt.maxMemory() / 1024 / 1024 + " MB");
        System.out.println("Total Memory: " + rt.totalMemory() / 1024 / 1024 + " MB");
        System.out.println("Free Memory: " + rt.freeMemory() / 1024 / 1024 + " MB");
    }
}
```

03

Java Applications

03_Java_Applications\examples\03_Java_Applications.md

Java Applications

Short Definition

Java is a versatile programming language used to build a wide variety of applications. From desktop software to enterprise, systems Java powers millions of applications around the world.

Key Bullet Points

- **Desktop Applications**: Standalone programs that run on personal computers
- **Web Applications**: Server-side applications delivered over the internet
- **Mobile Applications**: Apps for Android devices (Android uses Java)
- **Enterprise Applications**: Large-scale business applications
- **Embedded Systems**: Software embedded in electronic devices
- Java's "Write Once, Run Anywhere" makes it ideal for all these applications

---

Types of Java Applications

1. Desktop Applications

Standalone applications that run on desktop computers without needing a web browser.

Examples:

- Eclipse IDE (development tool)
- NetBeans IDE
- Adobe Acrobat Reader

2. Web Applications

Server-side applications accessed through web browsers.

Examples:

- Gmail (Google)
- Amazon.com
- Many banking websites

3. Mobile Applications

Applications designed for mobile devices, especially Android.

Examples:

- Android apps (WhatsApp, Instagram, Spotify)
- Mobile banking apps
- Games

4. Enterprise Applications

Large-scale business applications that handle complex business processes.

Examples:

- ERP (Enterprise Resource Planning) systems
- CRM (Customer Relationship Management) systems
- Financial transaction systems

5. Embedded Systems

Software embedded in devices like smart cards, ATMs, and IoT devices.

Examples:

- ATM machines
- Smart cards
- Television remote controls

Real-World Examples

Application Type	Examples
Desktop	Eclipse IDE, IntelliJ IDEA, Minecraft
Web	Gmail, Amazon, LinkedIn
Mobile	WhatsApp, Instagram, Pokémon GO
Enterprise	Oracle ERP, SAP systems
Embedded	Smart cards, Blu-ray players

Simple Example

```
// SimpleDesktopApp.java
public class SimpleDesktopApp {
    public static void main(String[] args) {
```

```
        System.out.println("Desktop Application Started!");
        System.out.println("This is a simple Java desktop application.");
    }
}
```

Expected Output:

```
Desktop Application Started!
This is a simple Java desktop application.
```

Code Examples

ApplicationTypesDemo.java

ApplicationTypesDemo.java

```
// ApplicationTypesDemo - Types of Java Applications
// Java can be used for various types of applications

public class ApplicationTypesDemo {

    public static void main(String[] args) {
        System.out.println("=== TYPES OF JAVA APPLICATIONS ===\n");

        System.out.println("--- 1. Standalone Applications ---");
        System.out.println("Desktop applications that run locally");
        System.out.println("Examples: Eclipse IDE, IntelliJ, Media Players");
        System.out.println("Technologies: AWT, Swing, JavaFX");

        System.out.println("\n--- 2. Web Applications ---");
        System.out.println("Server-side applications accessed via browser");
        System.out.println("Examples: E-commerce sites, Banking apps");
        System.out.println("Technologies: Servlets, JSP, Spring MVC");

        System.out.println("\n--- 3. Enterprise Applications ---");
        System.out.println("Large-scale business applications");
        System.out.println("Examples: ERP, CRM systems");
        System.out.println("Technologies: EJB, JPA, JTA, Spring Boot");

        System.out.println("\n--- 4. Mobile Applications ---");
        System.out.println("Android applications");
        System.out.println("Examples: WhatsApp, Instagram (Android)");
        System.out.println("Technologies: Android SDK, Kotlin, Jetpack Compose");

        System.out.println("\n--- 5. Distributed Applications ---");
        System.out.println("Applications running across multiple systems");
        System.out.println("Examples: Cloud services, Microservices");
        System.out.println("Technologies: RMI, EJB, Web Services, REST API");

        System.out.println("\n--- 6. Embedded Systems ---");
```

```
        System.out.println("Applications running on embedded devices");
        System.out.println("Examples: Smart cards, ATMs");
        System.out.println("Technologies: Java ME, IoT platforms");

        System.out.println("\n--- Angular Integration ---");
        System.out.println("Java (Spring Boot) + Angular = Full-stack web apps");
        System.out.println("Java handles: REST API, Business logic, Database");
        System.out.println("Angular handles: UI, User interaction, HTTP calls");
    }
}
```

04

Java Editions

04_Java_Editions\examples\04_Java_Editions.md

Java Editions

Short Definition

Java comes in different editions, each designed for a specific type of application development. Understanding these editions helps you choose the right one for your project.

Key Bullet Points

- **Java SE (Standard Edition)**: For desktop and basic server applications
- **Java EE (Enterprise Edition)**: For large-scale enterprise applications (now called Jakarta EE)
- **Java ME (Micro Edition)**: For mobile and embedded devices
- Each edition includes different libraries and tools

Java Editions Explained

1. Java SE (Standard Edition)

Purpose: Core Java platform for desktop and basic server applications.

What's included:

- Core libraries and APIs
- Basic language features
- GUI (Graphical User Interface) libraries (AWT, Swing, JavaFX)
- Database connectivity (JDBC)

Use for:

- Learning Java fundamentals
- Desktop applications
- Simple server applications
- Console-based programs

Example: `java.lang`, `java.util`, `java.io`, `java.awt`, `java.swing`

---

2. Java EE (Enterprise Edition) - Now Jakarta EE

Purpose: For building large-scale, distributed, enterprise-level applications.

What's included:

- All Java SE features plus
- Enterprise APIs (EJB, Servlet, JSP)
- Web services (JAX-WS, JAX-RS)
- Messaging (JMS)
- Transaction management
- Security

Use for:

- Web applications
- Enterprise systems
- Cloud-based applications

Note: Oracle transferred Java EE to the Eclipse Foundation, which renamed it to **Jakarta EE**.

---

3. Java ME (Micro Edition)

Purpose: For mobile phones, PDAs, and embedded devices.

What's included:

- Lightweight libraries
- Limited memory and processing requirements
- Mobile-specific APIs

Use for:

- Mobile applications (older feature phones)
- Embedded systems
- IoT devices

Comparison Table

Edition	Full Name	Target	Use Case
Java SE	Standard Edition	Desktop/Server	Learning, basic apps
Java EE	Enterprise Edition	Enterprise	Large web apps, distributed systems
Java ME	Micro Edition	Mobile/Embedded	Mobile phones, IoT

Simple Example (Java SE)

```
// HelloWorld.java (Java SE)
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello from Java SE!");
    }
}
```

Expected Output:

```
Hello from Java SE!
```

Which Edition to Learn?

For beginners: Start with Java SE

- Learn the fundamentals
- Understand object-oriented programming
- Practice with basic programs

After mastering Java SE:

- Move to Java EE/Jakarta EE for web development
- Use frameworks like Spring for modern enterprise apps

Code Examples

JavaEditionsDemo.java

JavaEditionsDemo.java

```
// JavaEditionsDemo - Java Editions and Platforms
// Different Java platforms for different purposes

public class JavaEditionsDemo {

    public static void main(String[] args) {
        System.out.println("=== JAVA EDITIONS ===\n");

        System.out.println("--- Java SE (Standard Edition) ---");
        System.out.println("Core Java platform");
        System.out.println("Includes: Core APIs, GUI, Networking, Database");
        System.out.println("Use for: Desktop applications, Applets");
        System.out.println("Package: java.*, javax.*");

        System.out.println("\n--- Java EE (Enterprise Edition) ---");
```

```
        System.out.println("Enterprise Java platform (now Jakarta EE)");
        System.out.println("Includes: Servlets, JSP, EJB, JPA, Web Services");
        System.out.println("Use for: Web apps, Enterprise systems");
        System.out.println("Package: javax.*, jakarta.*");

        System.out.println("\n--- Java ME (Micro Edition) ---");
        System.out.println("Lightweight platform for embedded devices");
        System.out.println("Includes: Limited APIs, CLDC, MIDP");
        System.out.println("Use for: Mobile phones, PDAs, Set-top boxes");

        System.out.println("\n--- Jakarta EE (Evolved from Java EE) ---");
        System.out.println("Open source enterprise platform");
        System.out.println("Backward compatible with Java EE");
        System.out.println("Now maintained by Eclipse Foundation");

        System.out.println("\n--- Java Card ---");
        System.out.println("Platform for secure smart cards");
        System.out.println("Use for: SIM cards, Payment cards");

        System.out.println("\n--- Quick Comparison ---");
        System.out.println("| Edition   | Target           | Use Case                |");
        System.out.println("|-----------|------------------|-------------------------|");
        System.out.println("| Java SE   | Desktop/Server  | Core programming        |");
        System.out.println("| Java EE   | Enterprise       | Web & Enterprise apps   |");
        System.out.println("| Java ME   | Embedded         | Mobile/IoT devices      |");
        System.out.println("| Jakarta EE| Enterprise       | Modern web services     |");

        System.out.println("\n--- Angular + Java EE/Jakarta EE ---");
        System.out.println("Spring Boot (based on Java EE/Jakarta EE) + Angular = Full-stack");
    }
}
```

05

Java Frameworks

05_Java_Frameworks\examples\05_Java_Frameworks.md

Java Frameworks

Short Definition

Java frameworks are pre-written code libraries that provide a foundation for developing Java applications. They simplify development by providing reusable components, tools, and structures.

Key Bullet Points

- **Frameworks save time**: You don't need to write common code from scratch
- **Structure**: Frameworks provide a standard architecture for your applications
- **Security**: Built-in security features protect your applications
- **Community support**: Popular frameworks have large communities and extensive documentation
- **Popular frameworks**: Spring, Hibernate, Struts, JSF

Major Java Frameworks

1. Spring Framework

Purpose: Enterprise application development framework.

Key Features:

- Dependency Injection (IoC - Inversion of Control)
- Aspect-Oriented Programming (AOP)
- Transaction management
- Web applications (Spring MVC)
- Security (Spring Security)

Use for:

- Building enterprise web applications
- Microservices architecture
- RESTful APIs

Example:

```
// Simple Spring Controller
@RestController
public class HelloController {
    @GetMapping("/hello")
    public String sayHello() {
        return "Hello from Spring!";
    }
}
```

---

2. Hibernate

Purpose: Object-Relational Mapping (ORM) framework.

Key Features:

- Maps Java objects to database tables
- Automatic SQL generation
- Database independence
- Caching support
- Transaction management

Use for:

- Database operations
- Object persistence
- Simplifying SQL queries

Example:

```
// Simple Hibernate Entity
@Entity
@Table(name = "employees")
public class Employee {
    @Id
    @GeneratedValue
    private int id;

    private String name;
    private String department;

    // Getters and setters
}
```

---

3. Struts

Purpose: Web application framework.

Key Features:

- MVC architecture
- Form validation
- Integration with JSP
- Tiles for page layout
- Plugin support

Use for:

- Building web applications
- Enterprise web portals

---

4. JavaServer Faces (JSF)

Purpose: Component-based UI framework for web applications.

Key Features:

- Component-based architecture
- Event-driven programming
- Built-in AJAX support
- Tag libraries
- Easy integration with managed beans

Use for:

- Building user interfaces
- Enterprise web applications

Comparison Table

Framework	Purpose	Best For
Spring	Enterprise development	Large web apps, microservices
Hibernate	Database/ORM	Data persistence, database operations
Struts	Web applications	Web portals
JSF	UI framework	Enterprise web UIs

Simple Example (without frameworks)

```
// Plain Java (no framework)
public class SimpleApp {
    public static void main(String[] args) {
        System.out.println("Simple Java Application");
        System.out.println("Using plain Java without frameworks");
    }
}
```

Expected Output:

```
Simple Java Application
Using plain Java without frameworks
```

Which Framework to Learn?

1. **Start with Core Java** - Master the basics first
2. **Learn Spring** - Most popular and widely used
3. **Learn Hibernate** - For database-related applications

These frameworks are essential skills for any Java developer looking to work in enterprise application development.

Code Examples

FrameworksDemo.java

FrameworksDemo.java

```
// FrameworksDemo - Popular Java Frameworks
// Frameworks provide pre-built structures for faster development

public class FrameworksDemo {

    public static void main(String[] args) {
        System.out.println("=== POPULAR JAVA FRAMEWORKS ===\n");

        System.out.println("--- WEB FRAMEWORKS ---");

        System.out.println("\n1. Spring Framework");
        System.out.println("   - Most popular enterprise framework");
        System.out.println("   - Provides: DI, AOP, MVC, Data access");
        System.out.println("   - Use for: Enterprise applications");

        System.out.println("\n2. Spring Boot");
```

```
        System.out.println("   - Simplified Spring development");
        System.out.println("   - Auto-configuration, Embedded server");
        System.out.println("   - Use for: Microservices, REST APIs");

        System.out.println("\n3. Jakarta EE (Java EE)");
        System.out.println("   - Enterprise-grade platform");
        System.out.println("   - Includes: CDI, JPA, JAX-RS");
        System.out.println("   - Use for: Large enterprise systems");

        System.out.println("\n4. Struts");
        System.out.println("   - Classic MVC framework");
        System.out.println("   - Use for: Legacy web applications");

        System.out.println("\n--- DATA ACCESS ---");

        System.out.println("\n5. Hibernate");
        System.out.println("   - ORM framework");
        System.out.println("   - Maps Java objects to database tables");
        System.out.println("   - Use for: Database operations");

        System.out.println("\n6. MyBatis");
        System.out.println("   - SQL mapping framework");
        System.out.println("   - Use for: Direct SQL control");

        System.out.println("\n--- TESTING ---");
        System.out.println("7. JUnit - Unit testing framework");
        System.out.println("8. TestNG - Advanced testing");
        System.out.println("9. Mockito - Mocking framework");

        System.out.println("\n--- ANGULAR + JAVA ---");
        System.out.println("Recommended Stack:");
        System.out.println("  Frontend: Angular + TypeScript");
        System.out.println("  Backend: Spring Boot (REST API)");
        System.out.println("  Database: MySQL/PostgreSQL");
        System.out.println("  Build: Maven/Gradle");
    }
}
```

06

Java Syntax

06_Java_Syntax\examples\06_Java_Syntax.md

Java Syntax

Short Definition

Java syntax refers to the set of rules that define how a Java program is written and interpreted. Understanding syntax is essential for writing correct Java code.

Key Bullet Points

- **Case-sensitive**: Java treats uppercase and lowercase as different
- **Class names**: Must match the filename (e.g., `MyClass.java` contains `MyClass`)
- **Statements**: End with a semicolon (`;`)
- **Blocks**: Code inside braces `{ }` forms a block
- **Comments**: For documentation (ignored by compiler)

Structure of a Java Program

```
// This is a comment - ignored by the compiler

// Class declaration
public class MyFirstProgram {

    // Main method - entry point of the program
    public static void main(String[] args) {
        // This is a statement - must end with semicolon
        System.out.println("Hello, World!");
        private display();
    }
}
```

//dsgdzsfg

}

Key Elements Explained

1. Class

A class is a blueprint for creating objects. Every Java program must have at least one class.

```
public class ClassName {
    // class body
}
```

Rules:

- Class name should start with a capital letter
- The filename must match the class name with `.java` extension

---

2. main() Method

The `main()` method is the entry point of any Java application. The JVM starts execution from here.

```
public static void main(String[] args) {
    // program statements

}
```

Breakdown:

- `public`: Accessible from anywhere
- `static`: Can be called without creating an object
- `void`: Returns nothing
- `String[] args`: Command-line arguments

---

3. Statements

A statement is a complete instruction that performs an action. It must end with a semicolon (`;`).

```
int number = 10;           // Variable declaration
System.out.println(num);  // Print statement
```

---

4. Comments

Comments explain the code and are ignored by the compiler.

Single-line comment:

```
// This is a single-line comment
```

Multi-line comment:

```
/* This is a
   multi-line comment */
```

Documentation comment:

```
/**
 * This is a documentation comment
 */
```

Naming Rules

Rule	Example	Valid?
Start with letter, _, or $	`myVar`, `_value`, `$money`	✓
Cannot start with digit	`2name`	✗
No spaces allowed	`my var`	✗
Case-sensitive	`myVar`, `myvar`, `MYVAR`	Different!
Cannot use reserved words	`class`, `int`	✗

Simple Example Program

```
// MyFirstProgram.java
// This is my first Java program

public class MyFirstProgram {
    // Main method - program starts here
    public static void main(String[] args) {
        // Print a welcome message
        System.out.println("Welcome to Java!");

        // Another statement
        System.out.println("Learning Java is fun!");
    }
}
```

Expected Output

```
Welcome to Java!
Learning Java is fun!
```

Code Examples

SyntaxDemo.java

SyntaxDemo.java

```java
// SyntaxDemo - Basic Java Syntax Rules
// Fundamental syntax elements of Java programming

public class SyntaxDemo {

    // Class declaration (PascalCase)
    static String classVariable = "I am a class";

    // Main method - entry point
    public static void main(String[] args) {

        // Variable declaration (camelCase)
        String message = "Hello, Java!";
        int number = 42;
        double decimal = 3.14;

        System.out.println("=== JAVA SYNTAX RULES ===\n");

        System.out.println("--- Case Sensitivity ---");
        System.out.println("message vs Message vs MESSAGE are different");

        System.out.println("\n--- Class Names ---");
        System.out.println("MyFirstClass (PascalCase)");

        System.out.println("\n--- Method Names ---");
        System.out.println("myFirstMethod() (camelCase)");

        System.out.println("\n--- Variable Names ---");
        System.out.println("myFirstVariable (camelCase)");

        System.out.println("\n--- Constants ---");
        System.out.println("MAX_VALUE (UPPER_SNAKE_CASE)");

        System.out.println("\n--- Statement Termination ---");
        int a = 5;  // Statement ends with semicolon
        int b = 10; // Another statement

        System.out.println("\n--- Code Blocks ---");
        if (a < b) {
            System.out.println("a is less than b");
        } else {
```

```
            System.out.println("a is greater or equal");
        }

        System.out.println("\n--- Identifiers Rules ---");
        System.out.println("Can start with: letter, $, _");
        System.out.println("Can contain: letter, digit, $, _");
        System.out.println("Cannot: be keyword, start with digit");

        System.out.println("\n--- Comments ---");
        // This is single-line comment
        /* This is multi-line comment */
        /** This is Javadoc comment */
        System.out.println("Comments are ignored by compiler");
    }
}
```

07

Output Statements

07_Output_Statements\examples\07_Output_Statements.md

Output Statements in Java

Short Definition

Output statements in Java display information to the user. The most common way to output data is through the `System.out` object, which provides methods for printing to the console.

Key Bullet Points

- **System.out.println()**: Prints text and adds a new line after output
- **System.out.print()**: Prints text without a new line
- **System.out.printf()**: Prints formatted output (like C's printf)
- **String concatenation**: Use `+` to combine strings and variables

int nun = 10;

System.out.println("The number is: " + num);

sy

System.out.println()

Prints text and moves to a new line after the output.

Example 1: Simple Text

```
public class PrintlnExample {
    public static void main(String[] args) {
        System.out.println("Hello");
        System.out.println("World");
    }
}
```

Expected Output

```
Hello
World
```

System.out.print()

Prints text without moving to a new line. The next output will appear on the same line.

Example 2: Same Line Output

```
public class PrintExample {
    public static void main(String[] args) {
        System.out.print("Hello ");
        System.out.print("World");
    }
}
```

Expected Output

```
Hello World
```

System.out.printf()

Prints formatted output using format specifiers.

Common Format Specifiers

Specifier	Meaning	Example
%d	Integer	42
%f	Float/Decimal	3.14
%s	String	"Hello"
%c	Character	'A'
%n	New line	(line break)

Example 3: Formatted Output

```
public class PrintfExample {
    public static void main(String[] args) {
        String name = "John";
        int age = 25;
        double height = 5.9;
```

```
        System.out.printf("Name: %s%n", name);
        System.out.printf("Age: %d%n", age);
        System.out.printf("Height: %.1f%n", height);
    }
}
```

Expected Output

```
Name: John
Age: 25
Height: 5.9
```

String Concatenation

You can combine strings and variables using the + operator.

Example 4: Combining Strings

```
public class ConcatenationExample {
    public static void main(String[] args) {
        String firstName = "John";
        String lastName = "Doe";
        int score = 95;

        System.out.println("Name: " + firstName + " " + lastName);
        System.out.println("Score: " + score);
        System.out.println("Result: " + firstName + " scored " + score + " points!");
    }
}
```

Expected Output

```
Name: John Doe
Score: 95
Result: John scored 95 points!
```

Quick Reference

Method	Adds New Line	Use Case
`println()`	Yes	General output
`print()`	No	Output on same line
`printf()`	No (use `%n`)	Formatted output

Tips

1. Use `println()` for most output needs
2. Use `printf()` when you need formatted output (decimals, alignment)
3. Use `print()` when building output gradually on one line
4. Remember to add `%n` for new lines in `printf()`

Code Examples

OutputDemo.java

OutputDemo.java

```
// OutputDemo - Java Output Statements
// Different ways to display output in Java

public class OutputDemo {

    public static void main(String[] args) {
        System.out.println("=== JAVA OUTPUT METHODS ===\n");

        // System.out.println() - prints with newline
        System.out.println("--- println() ---");
        System.out.println("Line 1");
        System.out.println("Line 2");
        System.out.println("This prints and moves to new line");

        // System.out.print() - prints without newline
        System.out.println("\n--- print() ---");
        System.out.print("Hello ");
        System.out.print("World!");
        System.out.println(" (no new line in first two)");

        // System.out.printf() - formatted output
        System.out.println("\n--- printf() ---");
        String name = "John";
        int age = 30;
        double salary = 50000.50;

        System.out.printf("Name: %s%n", name);
```

```
        System.out.printf("Age: %d%n", age);
        System.out.printf("Salary: %.2f%n", salary);
        System.out.printf("Multiple: %s is %d years old%n", name, age);

        // Format specifiers
        System.out.println("\n--- Format Specifiers ---");
        System.out.printf("%%d = integer: %d%n", 42);
        System.out.printf("%%f = float: %.3f%n", 3.14159);
        System.out.printf("%%s = string: %s%n", "Java");
        System.out.printf("%%c = char: %c%n", 'A');
        System.out.printf("%%b = boolean: %b%n", true);

        // String.format()
        System.out.println("\n--- String.format() ---");
        String formatted = String.format("Welcome %s, age %d", "Alice", 25);
        System.out.println(formatted);

        // Angular Note
        System.out.println("\n--- Angular Note ---");
        System.out.println("In web apps, use console.log() in TypeScript");
        System.out.println("Angular displays data via templates, not console");
    }
}
```

08

Input Programs

08_Input_Programs\examples\08_Input_Programs.md

Input in Java

Short Definition

Input in Java allows programs to receive data from users during runtime. The most common way to read user input is through the `Scanner` class, which provides methods to read different types of data from the keyboard.

Key Bullet Points

- **Scanner class**: Built-in class for reading user input
- **Import required**: Must import `java.util.Scanner`
- **Create Scanner object**: `Scanner scanner = new Scanner(System.in);`
- **Read different types**: nextInt(), nextDouble(), nextLine(), next()
- **Close Scanner**: Always close the scanner when done

Scanner Class Basics

Step 1: Import the Scanner class

```
import java.util.Scanner;
import java.util.*;
```

Step 2: Create Scanner object

```
Scanner scanner = new Scanner(System.in);
```

Step 3: Read input

```
// Read integer
system.out.println("Enter a number: ");
int number = scanner.nextInt();

system.out.println("Enter a salary: ");
// Read decimal number
double price = scanner.nextDouble();
```

```
// Read text line
String name = scanner.nextLine();

// Read single word
String word = scanner.next();
```

Simple Input Example

Example 1: Reading a Name

```
import java.util.Scanner;

public class InputExample {
    public static void main(String[] args) {
        // Create Scanner object
        Scanner scanner = new Scanner(System.in);

        // Prompt user
        System.out.print("Enter your name: ");

        // Read input
        String name = scanner.nextLine();

        // Display output
        System.out.println("Hello, " + name + "!");

        // Close scanner
        scanner.close();
    }
}
```

Expected Output (Sample)

```
Enter your name: Alice
Hello, Alice!
```

Example 2: Reading Numbers

```
import java.util.Scanner;

public class NumberInputExample {
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);

        // Read integer
        System.out.print("Enter your age: ");
        int age = scanner.nextInt();

        // Read decimal
        System.out.print("Enter your height (in meters): ");
        double height = scanner.nextDouble();

        // Display results
        System.out.println("\n--- User Information ---");
        System.out.println("Age: " + age);
        System.out.println("Height: " + height + " meters");

        scanner.close();
    }
}
```

Expected Output (Sample)

```
Enter your age: 25
Enter your height (in meters): 1.75

--- User Information ---
Age: 25
Height: 1.75 meters
```

Important Notes

1. **Import Scanner**: Always include `import java.util.Scanner;` at the top
2. **Next vs NextLine**:
   - `next()` - reads single word (stops at space)
   - `nextLine()` - reads entire line (includes spaces)
1. **Type matching**: Use the correct method for the data type
2. **Close scanner**: Always call `scanner.close()` when done

Common Scanner Methods

Method	Description	Example
`nextInt()`	Reads an integer	42

Method	Description	Example
`nextDouble()`	Reads a double	3.14
`nextFloat()`	Reads a float	3.14f
`nextLine()`	Reads entire line	"Hello World"
`next()`	Reads single word	"Hello"
`nextBoolean()`	Reads boolean	true/false
`nextByte()`	Reads byte	127
`nextShort()`	Reads short	32767
`nextLong()`	Reads long	123456789

Tips for Beginners

1. **Import first**: Always import Scanner at the beginning
2. **Prompt user**: Always print a message asking for input
3. **Handle different types**: Use the right method for each data type
4. **Close resources**: Close Scanner to prevent memory leaks

Code Examples

InputDemo.java

InputDemo.java

```
// InputDemo - Java Input Methods
// Different ways to get user input in Java

import java.io.*;

public class InputDemo {

    public static void main(String[] args) throws IOException {
        System.out.println("=== JAVA INPUT METHODS ===\n");

        // Method 1: Command Line Arguments
        System.out.println("--- 1. Command Line Args ---");
        System.out.println("Pass args when running: java Program arg1 arg2");
        System.out.println("Access via: args[0], args[1]");

        // Method 2: BufferedReader (old way)
        System.out.println("\n--- 2. BufferedReader ---");
```

```
        System.out.println("BufferedReader reader = new BufferedReader(new InputStreamReader(Syst...
        System.out.println("String input = reader.readLine();");

        // Method 3: Scanner (modern way)
        System.out.println("\n--- 3. Scanner Class ---");
        System.out.println("Scanner scanner = new Scanner(System.in);");
        System.out.println("String name = scanner.nextLine();");
        System.out.println("int number = scanner.nextInt();");
        System.out.println("double value = scanner.nextDouble();");

        // Method 4: Console class
        System.out.println("\n--- 4. Console Class ---");
        System.out.println("Console console = System.console();");
        System.out.println("String input = console.readLine(\"Enter name: \");");

        // Method 5: GUI Dialogs (for desktop)
        System.out.println("\n--- 5. JOptionPane (GUI) ---");
        System.out.println("String input = JOptionPane.showInputDialog(\"Enter name\");");

        System.out.println("\n--- Scanner Example ---");
        java.util.Scanner scan = new java.util.Scanner("John 25 50000.50");

        String name = scan.next();
        int age = scan.nextInt();
        double salary = scan.nextDouble();

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Salary: $" + salary);

        scan.close();

        // Angular Note
        System.out.println("\n--- Angular Note ---");
        System.out.println("In web apps, use HTML forms and Angular inputs");
        System.out.println("Angular: <input [(ngModel)]=\"variable\">");
        System.out.println("Java backend receives via @RequestBody");
    }
}
```

09

Basic Java Program

09_Basic_Java_Program\examples\09_Basic_Java_Program.md

Basic Java Program - Hello World

Short Definition

A Basic Java Program is the simplest Java application that demonstrates the fundamental structure of any Java program. The traditional first program displays "Hello, World!" to the console.

Key Bullet Points

- **Entry Point**: The `main()` method is the entry point where the JVM starts execution
- **Class Declaration**: Every Java program requires at least one class
- **File Naming**: The class name must match the filename (e.g., `HelloWorld.java` for class `HelloWorld`)
- **System.out.println()**: Used to print output to the console
- **Compilation**: Java source code (.java) is compiled to bytecode (.class)
- **Execution**: The JVM interprets the bytecode

Basic Program Structure

```
class ClassName {

    public static void main(String[] args) {
        // Your code here
        System.out.println("Hello, World!");
    }
}
```

Components Explained

Component	Description
`class`	Keyword to declare a class
`ClassName`	Must match filename (PascalCase)
`public`	Access modifier - visible everywhere
`static`	Method belongs to class, not object

Component	Description
`void`	Return type - returns nothing
`main()`	Entry point of the program
`String[] args`	Command line arguments

How to Compile and Run

```
# Compile
javac HelloWorld.java

# Run
java HelloWorld
```

What Happens Behind the Scenes

```
Source Code (HelloWorld.java)
        ↓
    javac Compiler
        ↓
  Bytecode (HelloWorld.class)
        ↓
    java command
        ↓
      JVM
        ↓
   Output: Hello, World!
```

Example 1: Simple Output

```
class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Output:

```
Hello, World!
```

Example 2: Multiple Print Statements

```
class MultiplePrints {
    public static void main(String[] args) {
        System.out.println("Line 1");
        System.out.println("Line 2");
        System.out.println("Line 3");
    }
}
```

Output:

```
Line 1
Line 2
Line 3
```

Why This Matters for Angular Developers?

When building full-stack applications:

- Java backend returns JSON responses
- Console output helps debug Java applications locally
- Understanding basic Java structure helps when reading Spring Boot code
- The `main()` method is where Spring Boot applications start

Exercises

Exercise 1: Your First Program

Create a program that prints your name.

Exercise 2: Multiple Lines

Print three lines: your name, your city, and your favorite programming language.

Exercise 3: ASCII Art

Print a simple ASCII art pattern using `System.out.println()`.

Summary

- Every Java program needs a `main()` method as the entry point

- `System.out.println()` prints text to the console
- Java is compiled to bytecode and executed by the JVM
- This is the foundation for all Java applications including Spring Boot

Next Steps

- Learn about Variables and Data Types
- Learn about Control Flow Statements
- Understand Methods and Static vs Instance

Code Examples

HelloWorld.java

HelloWorld.java

```
// HelloWorld - Your First Java Program
// This demonstrates the basic structure of a Java application

public class HelloWorld {

    // main() method - entry point of any Java application
    public static void main(String[] args) {
        // Print to console
        System.out.println("Hello, World!");
        System.out.println("Welcome to Java Programming!");

        // Print without newline
        System.out.print("This is ");
        System.out.print("print ");
        System.out.println("statement.");

        // Print with formatting (printf)
        System.out.printf("Number: %d, String: %s, Float: %.2f%n", 42, "Java", 3.14159);

        // Variables
        String name = "Developer";
        int age = 25;
        double salary = 50000.50;

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Salary: $" + salary);
    }
}
```

10

Static vs Instance Methods

10_Static_vs_Instance_Methods\examples\10_Static_vs_Instance_Methods.md

Static vs Instance Methods in Java

Short Definition

Static methods belong to the class itself rather than to any instance (object). Instance methods require an object to be called. Understanding the difference is crucial for writing effective Java code.

Key Bullet Points

- **Static Methods**: Called using `ClassName.method()`, belong to the class
- **Instance Methods**: Called using `object.method()`, belong to objects
- **Static Variables**: Shared across all instances of a class
- **Instance Variables**: Each object has its own copy
- **Static Context**: Cannot use `this` or `super` keywords
- **Memory**: Static members are loaded when class is loaded

Static vs Instance Comparison

Feature	Static	Instance
Called by	ClassName.method()	object.method()
Keyword	static	(none)
Access to instance	Cannot access	Can access
Access to static	Can access	Can access
Memory	One copy per class	One copy per object
Loaded	At class loading	At object creation

Static Method Example

```
class Calculator {

    // Static method - belongs to class
```

```
    public static int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        // No object needed to call static method
        int result = Calculator.add(5, 3);
        System.out.println("Sum: " + result);  // Output: 8
    }
}
```

Instance Method Example

```
class Person {
    private String name;

    // Constructor
    public Person(String name) {
        this.name = name;
    }

    // Instance method - requires object
    public void introduce() {
        System.out.println("Hello, I am " + name);
    }

    public static void main(String[] args) {
        // Need to create object to call instance method
        Person p = new Person("Alice");
        p.introduce();  // Output: Hello, I am Alice
    }
}
```

Static Variable Example

```
class Counter {
    // Static variable - shared across all objects
    private static int count = 0;

    public Counter() {
        count++;  // Increment for each new object
    }

    public static void main(String[] args) {
        new Counter();
```

```
        new Counter();
        new Counter();

        System.out.println("Count: " + Counter.count);  // Output: 3
    }
}
```

Common Static Methods in Java

Math Class (java.lang.Math)

```
int max = Math.max(10, 20);      // Returns larger value
int abs = Math.abs(-25);          // Returns absolute value
double sqrt = Math.sqrt(16);     // Returns square root
int random = (int)(Math.random() * 100);  // Random number
```

System Class

```

```

System.out.println("Hello"); // static method call

System.exit(0); // terminates JVM

```
---

## Best Practices

1. **Use static for utilities**: Helper methods that don't need object state
2. **Use static for constants**: Values that don't change
3. **Don't overuse static**: Can make code harder to test
4. **Instance methods**: When behavior depends on object state

---

## Why This Matters for Angular Developers?

- Spring Boot uses static methods in utility classes
- Static factory methods are common in design patterns
- Understanding this helps read Java backend code
- Main method is static - entry point for Spring Boot apps

---

## Exercises
```

```
### Exercise 1: Static Calculator
Create a class with static methods for subtract, multiply, and divide.

### Exercise 2: Object Counter
Create a class that counts how many objects have been created.

### Exercise 3: Hybrid Class
Create a class with both static and instance methods demonstrating when each is appropriate.

---

## Summary

- Static methods belong to the class, instance methods to objects
- Static members are shared across all instances
- Cannot access instance members from static context
- Essential for utility methods and main entry points
```

Code Examples

AppConfigDemo.java

AppConfigDemo.java

```java
// AppConfigDemo - Demonstrates Static Config for Angular Integration
// Use case: Application-wide configuration settings

public class AppConfigDemo {
    // Static config - same for all users
    private static String API_BASE_URL = "https://api.example.com";
    private static int TIMEOUT_SECONDS = 30;
    private static boolean DEBUG_MODE = true;

    // Instance config - per user
    private String userToken;
    private String currentLanguage;

    public AppConfigDemo(String userToken, String language) {
        this.userToken = userToken;
        this.currentLanguage = language;
    }

    // Static getters
    public static String getApiBaseUrl() {
        return API_BASE_URL;
    }

    public static int getTimeout() {
        return TIMEOUT_SECONDS;
    }
```

```
    public static boolean isDebugMode() {
        return DEBUG_MODE;
    }

    // Static setter - modify global config
    public static void setApiBaseUrl(String url) {
        API_BASE_URL = url;
    }

    // Instance getters
    public String getUserToken() {
        return userToken;
    }

    public String getCurrentLanguage() {
        return currentLanguage;
    }

    public static void main(String[] args) {
        System.out.println("=== STATIC CONFIG FOR ANGULAR ===\n");

        // Access static config globally
        System.out.println("API URL: " + AppConfigDemo.getApiBaseUrl());
        System.out.println("Timeout: " + AppConfigDemo.getTimeout());
        System.out.println("Debug: " + AppConfigDemo.isDebugMode());

        // Create user-specific instances
        AppConfigDemo user1 = new AppConfigDemo("token-123", "en");
        AppConfigDemo user2 = new AppConfigDemo("token-456", "es");

        System.out.println("\n--- User 1 ---");
        System.out.println("Token: " + user1.getUserToken());
        System.out.println("Language: " + user1.getCurrentLanguage());

        System.out.println("\n--- User 2 ---");
        System.out.println("Token: " + user2.getUserToken());
        System.out.println("Language: " + user2.getCurrentLanguage());

        // Update global config
        AppConfigDemo.setApiBaseUrl("https://new-api.example.com");
        System.out.println("\n--- Updated Global Config ---");
        System.out.println("New API URL: " + AppConfigDemo.getApiBaseUrl());

        System.out.println("\n=== USE CASES ===");
        System.out.println("1. Global API endpoints");
        System.out.println("2. Feature flags");
        System.out.println("3. Environment settings");
        System.out.println("4. Per-user tokens (instance)");
    }
}
```

StaticCounterDemo.java

StaticCounterDemo.java

```
// StaticCounterDemo - Demonstrates Static Methods for Angular State Management
// Use case: Managing shared application state across components

public class StaticCounterDemo {
    // Static variable - shared across all instances
    private static int instanceCount = 0;

    // Instance variable - unique to each object
    private String instanceId;

    public StaticCounterDemo() {
        instanceCount++;
        instanceId = "Instance-" + instanceCount;
    }

    // Static method - called without creating instance
    public static int getInstanceCount() {
        return instanceCount;
    }

    // Static method for resetting counter
    public static void resetCounter() {
        instanceCount = 0;
    }

    // Instance method - requires object reference
    public String getInstanceId() {
        return instanceId;
    }

    public static void main(String[] args) {
        System.out.println("=== STATIC COUNTER FOR STATE MANAGEMENT ===\n");

        // Create multiple instances
        StaticCounterDemo obj1 = new StaticCounterDemo();
        StaticCounterDemo obj2 = new StaticCounterDemo();
        StaticCounterDemo obj3 = new StaticCounterDemo();

        // Access static method without instance
        System.out.println("Total instances: " + StaticCounterDemo.getInstanceCount());

        // Access instance method through object
        System.out.println("Object 1 ID: " + obj1.getInstanceId());
        System.out.println("Object 2 ID: " + obj2.getInstanceId());

        // Reset static counter
        StaticCounterDemo.resetCounter();
        System.out.println("\nAfter reset: " + StaticCounterDemo.getInstanceCount());

        System.out.println("\n=== ANGULAR USE CASE ===");
        System.out.println("Static methods are ideal for:");
```

```
        System.out.println("1. Global app state (user session)");
        System.out.println("2. API service counters");
        System.out.println("3. Configuration settings");
        System.out.println("4. Logging services");
    }
}
```

StaticVsInstanceDemo.java

StaticVsInstanceDemo.java

```
// StaticVsInstanceDemo - Demonstrates Static vs Instance methods
// Static methods belong to the class, instance methods belong to objects

public class StaticVsInstanceDemo {

    // Instance variable - each object has its own copy
    private String name;

    // Static variable - shared across all objects
    private static int instanceCount = 0;

    // Constructor
    public StaticVsInstanceDemo(String name) {
        this.name = name;
        instanceCount++;  // Increment for each new object
    }

    // Instance method - requires an object to call
    public void instanceMethod() {
        System.out.println("Instance method called by: " + name);
        System.out.println("Can access instance variable: " + this.name);
        System.out.println("Can access static variable: " + instanceCount);
    }

    // Static method - belongs to class, not object
    public static void staticMethod() {
        System.out.println("Static method called");
        // Cannot access instance variable directly:
        // System.out.println(name);  // ERROR!

        // Can access static variable:
        System.out.println("Instance count: " + instanceCount);
    }

    // Static factory method - alternative to constructor
    public static StaticVsInstanceDemo createInstance(String name) {
        return new StaticVsInstanceDemo(name);
    }

    // Main method - static, entry point
    public static void main(String[] args) {
```

```
        System.out.println("=== STATIC VS INSTANCE DEMO ===\n");

        // Calling static method (no object needed)
        System.out.println("--- Static Method Call ---");
        staticMethod();
        StaticVsInstanceDemo.staticMethod();

        // Calling instance method (requires object)
        System.out.println("\n--- Instance Method Call ---");
        StaticVsInstanceDemo obj1 = new StaticVsInstanceDemo("Object 1");
        obj1.instanceMethod();

        StaticVsInstanceDemo obj2 = new StaticVsInstanceDemo("Object 2");
        obj2.instanceMethod();

        // Static variable is shared
        System.out.println("\n--- Static Variable ---");
        System.out.println("Total instances: " + instanceCount);

        // Static factory method
        System.out.println("\n--- Static Factory Method ---");
        StaticVsInstanceDemo obj3 = createInstance("Factory Object");
        obj3.instanceMethod();

        // Key differences
        System.out.println("\n=== KEY DIFFERENCES ===");
        System.out.println("| Static                    | Instance                 |");
        System.out.println("|---------------------------|--------------------------|");
        System.out.println("| Class-level               | Object-level             |");
        System.out.println("| Shared across objects    | Unique per object        |");
        System.out.println("| Called via ClassName     | Called via object        |");
        System.out.println("| Cannot use 'this'        | Can use 'this'           |");
    }
}
```

UserFactoryDemo.java

UserFactoryDemo.java

```
// UserFactoryDemo - Demonstrates Static Factory Methods for Angular Services
// Use case: Creating service instances with predefined configurations

public class UserFactoryDemo {
    private String name;
    private String role;
    private boolean isActive;

    // Private constructor - must use factory
    private UserFactoryDemo(String name, String role, boolean isActive) {
        this.name = name;
        this.role = role;
        this.isActive = isActive;
```

```
    }

    // Static factory method - creates admin users
    public static UserFactoryDemo createAdmin(String name) {
        return new UserFactoryDemo(name, "ADMIN", true);
    }

    // Static factory method - creates regular users
    public static UserFactoryDemo createUser(String name) {
        return new UserFactoryDemo(name, "USER", true);
    }

    // Static factory method - creates guest users
    public static UserFactoryDemo createGuest() {
        return new UserFactoryDemo("Guest", "GUEST", false);
    }

    // Static factory method - creates user from JSON (Angular data)
    public static UserFactoryDemo fromJSON(String json) {
        // Simulate parsing JSON from Angular
        String[] parts = json.replace("{\"name\":\"", "").replace("\"}", "").split("\",\"role\":\...
        return new UserFactoryDemo(parts[0], parts[1], true);
    }

    public void display() {
        System.out.println("User: " + name + " | Role: " + role + " | Active: " + isActive);
    }

    public static void main(String[] args) {
        System.out.println("=== STATIC FACTORY METHODS FOR SERVICES ===\n");

        // Use factory methods
        UserFactoryDemo admin = UserFactoryDemo.createAdmin("Alice");
        UserFactoryDemo user = UserFactoryDemo.createUser("Bob");
        UserFactoryDemo guest = UserFactoryDemo.createGuest();

        admin.display();
        user.display();
        guest.display();

        // Create from Angular JSON
        System.out.println("\n--- From Angular JSON ---");
        UserFactoryDemo fromJson = UserFactoryDemo.fromJSON("{\"name\":\"Charlie\",\"role\":\"USE...
        fromJson.display();

        System.out.println("\n=== ANGULAR USE CASE ===");
        System.out.println("Factory methods are perfect for:");
        System.out.println("1. Creating standardized API responses");
        System.out.println("2. Building user roles (Admin, User, Guest)");
        System.out.println("3. Converting JSON to Java objects");
        System.out.println("4. Service layer object creation");
    }
}
```

11

Variables and DataTypes

11_Variables_and_DataTypes\examples\11_Variables_and_DataTypes.md

Variables and Data Types in Java

Short Definition

Variables are containers for storing data values. Java is a strongly-typed language with primitive data types and reference types. Understanding data types is essential for writing Java programs.

Key Bullet Points

- **Variables**: Named storage locations in memory
- **Data Types**: Define the type of data a variable can hold
- **Primitive Types**: 8 basic data types built into Java
- **Reference Types**: Objects, arrays, strings
- **Type Casting**: Converting between data types
- **Constants**: Variables that cannot be changed (final keyword)

Primitive Data Types

Integer Types

Type	Size	Range
`byte`	8-bit	-128 to 127
`short`	16-bit	-32,768 to 32,767
`int`	32-bit	-2,147,483,648 to 2,147,483,647
`long`	64-bit	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

Floating Point Types

Type	Size	Range
`float`	32-bit	Approximately ±3.4×10^38
`double`	64-bit	Approximately ±1.8×10^308

Other Types

Type	Size	Values
`char`	16-bit	Unicode characters (0 to 65,535)
`boolean`	1-bit	true or false

Variable Declaration

```
// Declaration
int age;
String name;
double salary;

// Declaration with initialization
int count = 10;
String message = "Hello";
boolean isActive = true;
```

Variable Types

1. Local Variables

```
public void method() {
    int localVar = 10;  // Local variable
    System.out.println(localVar);
}
```

2. Instance Variables

```
class Person {
    String name;  // Instance variable
    int age;
```

```
}
```

3. Static Variables

```
class Counter {
    static int count = 0;  // Static variable
}
```

Type Casting

Implicit Casting (Widening)

```
int num = 100;
long largeNum = num;    // int to long
double decimal = num;   // int to double
```

Explicit Casting (Narrowing)

```
double pi = 3.14159;
int intPi = (int) pi;  // double to int (truncates)
```

Constants

Use the `final` keyword to create constants:

```
final double PI = 3.14159;
final int MAX_SIZE = 100;

PI = 3.14;  // Error! Cannot reassign
```

Reference Types

Strings

```
String name = "John";
String greeting = new String("Hello");
```

Arrays

```
int[] numbers = {1, 2, 3, 4, 5};
String[] names = new String[10];
```

Objects

```
Person p = new Person("Alice", 25);
```

Naming Conventions

Type	Convention	Example
Variables	camelCase	`studentCount`
Constants	UPPER_SNAKE_CASE	`MAX_SIZE`
Classes	PascalCase	`StudentRecord`
Methods	camelCase	`calculateTotal()`

Why This Matters for Angular Developers?

- Java backend uses these data types for API responses
- JSON maps to Java objects with these types
- TypeScript interfaces mirror Java classes
- Understanding types helps with data transformation

Exercises

Exercise 1: Declare Variables

Create variables of each primitive type and print their values.

Exercise 2: Type Casting

Convert a double to int and observe the result.

Exercise 3: Constants

Create a class with constants for application settings.

Summary

- Java has 8 primitive data types and reference types
- Variables must be declared before use
- Type casting allows conversion between types
- Constants are declared with the `final` keyword
- Proper naming conventions improve code readability

Code Examples

ProductInventoryDemo.java

ProductInventoryDemo.java

```
// ProductInventoryDemo - Demonstrates Wrapper Classes and Null Handling
// Use case: E-commerce product data with nullable fields

public class ProductInventoryDemo {
    // Primitive wrapper types (can be null)
    private Integer productId;
    private Integer stockQuantity;
    private Double price;
    private Boolean isAvailable;

    // Reference type
    private String productName;
    private String category;

    // Constructor with null handling
    public ProductInventoryDemo(Integer productId, String productName, Double price, Integer stoc...
        this.productId = productId;
        this.productName = productName;
        this.price = price;
        this.stockQuantity = stock;
        this.isAvailable = (stock != null && stock > 0);
        this.category = "General";
    }

    // Getters with null-safe returns
    public Integer getProductId() {
        return productId;
    }

    public String getProductName() {
        return productName;
    }

    public Double getPrice() {
```

```
        return price != null ? price : 0.0;
    }

    public Integer getStockQuantity() {
        return stockQuantity != null ? stockQuantity : 0;
    }

    public Boolean isAvailable() {
        return isAvailable;
    }

    public String getCategory() {
        return category;
    }

    // Calculate total value
    public double getTotalValue() {
        return getPrice() * getStockQuantity();
    }

    // Check if product needs restock
    public boolean needsRestock() {
        return getStockQuantity() < 10;
    }

    public static void main(String[] args) {
        System.out.println("=== WRAPPER CLASSES FOR NULL SAFETY ===\n");

        // Product with full data
        ProductInventoryDemo product1 = new ProductInventoryDemo(1, "Laptop", 999.99, 50);
        System.out.println("Product: " + product1.getProductName());
        System.out.println("Price: $" + product1.getPrice());
        System.out.println("Stock: " + product1.getStockQuantity());
        System.out.println("Available: " + product1.isAvailable());
        System.out.println("Total Value: $" + product1.getTotalValue());
        System.out.println("Needs Restock: " + product1.needsRestock());

        // Product with null stock (out of stock)
        ProductInventoryDemo product2 = new ProductInventoryDemo(2, "Phone", 699.99, null);
        System.out.println("\n--- Product with null stock ---");
        System.out.println("Product: " + product2.getProductName());
        System.out.println("Stock: " + product2.getStockQuantity());
        System.out.println("Available: " + product2.isAvailable());

        System.out.println("\n=== ANGULAR USE CASE ===");
        System.out.println("Wrapper classes are essential for:");
        System.out.println("1. Handling optional API fields");
        System.out.println("2. Managing null from database");
        System.out.println("3. Form field validation");
        System.out.println("4. TypeScript 'number | null' compatibility");
    }
}
```

UserModelDemo.java

UserModelDemo.java

```
// UserModelDemo - Demonstrates Data Types for Angular User Models
// Use case: Type-safe user data handling

public class UserModelDemo {
    // Primitive types for simple data
    private int userId;
    private boolean isActive;
    private double accountBalance;

    // Reference types for complex data
    private String userName;
    private String email;
    private String[] roles;

    // Constructor
    public UserModelDemo(int userId, String userName, String email, boolean isActive) {
        this.userId = userId;
        this.userName = userName;
        this.email = email;
        this.isActive = isActive;
        this.accountBalance = 0.0;
        this.roles = new String[] {"USER"};
    }

    // Getters and Setters
    public int getUserId() { return userId; }
    public void setUserId(int id) { this.userId = id; }

    public String getUserName() { return userName; }
    public void setUserName(String name) { this.userName = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public boolean isActive() { return isActive; }
    public void setActive(boolean active) { this.isActive = active; }

    public double getAccountBalance() { return accountBalance; }
    public void setAccountBalance(double balance) { this.accountBalance = balance; }

    public String[] getRoles() { return roles; }
    public void addRole(String role) {
        String[] newRoles = new String[roles.length + 1];
        System.arraycopy(roles, 0, newRoles, 0, roles.length);
        newRoles[roles.length] = role;
        this.roles = newRoles;
    }

    // Convert to JSON-like string for Angular
```

```
    public String toJSON() {
        StringBuilder json = new StringBuilder();
        json.append("{");
        json.append("\"userId\":").append(userId).append(",");
        json.append("\"userName\":\"").append(userName).append("\",");
        json.append("\"email\":\"").append(email).append("\",");
        json.append("\"isActive\":").append(isActive).append(",");
        json.append("\"accountBalance\":").append(accountBalance);
        json.append("}");
        return json.toString();
    }

    public static void main(String[] args) {
        System.out.println("=== DATA TYPES FOR ANGULAR MODELS ===\n");

        // Create user
        UserModelDemo user = new UserModelDemo(1, "Alice", "alice@example.com", true);
        user.setAccountBalance(100.50);

        // Display user data
        System.out.println("User ID: " + user.getUserId());
        System.out.println("Name: " + user.getUserName());
        System.out.println("Email: " + user.getEmail());
        System.out.println("Active: " + user.isActive());
        System.out.println("Balance: $" + user.getAccountBalance());

        // Send to Angular
        System.out.println("\n--- JSON Output ---");
        System.out.println(user.toJSON());

        System.out.println("\n=== DATA TYPE MAPPING ===");
        System.out.println("int -> number (TypeScript)");
        System.out.println("boolean -> boolean");
        System.out.println("double -> number");
        System.out.println("String -> string");
        System.out.println("String[] -> string[]");
    }
}
```

VariablesDemo.java

VariablesDemo.java

```
// VariablesDemo - Demonstrates Java Variables and Data Types
// Java has primitive types and reference types

public class VariablesDemo {

    // ===== PRIMITIVE DATA TYPES =====
    byte byteVar = 127;
    short shortVar = 32767;
    int intVar = 2147483647;
```

```
    long longVar = 9223372036854775807L;
    float floatVar = 3.14159f;
    double doubleVar = 3.14159265358979;
    char charVar = 'A';
    boolean booleanVar = true;

    // Reference types
    String stringVar = "Hello Java";
    int[] arrayVar = {1, 2, 3};

    public static void main(String[] args) {
        VariablesDemo demo = new VariablesDemo();

        System.out.println("=== VARIABLES AND DATA TYPES ===\n");
        System.out.println("byte: " + demo.byteVar);
        System.out.println("short: " + demo.shortVar);
        System.out.println("int: " + demo.intVar);
        System.out.println("long: " + demo.longVar);
        System.out.println("float: " + demo.floatVar);
        System.out.println("double: " + demo.doubleVar);
        System.out.println("char: " + demo.charVar);
        System.out.println("boolean: " + demo.booleanVar);

        System.out.println("\n--- Type Casting ---");
        int i = 100;
        long l = i;
        double d = i;
        System.out.println("Implicit: int to long = " + l);

        double pi = 3.14159;
        int intPi = (int) pi;
        System.out.println("Explicit: double to int = " + intPi);

        System.out.println("\n--- Constants ---");
        final double PI = 3.14159;
        System.out.println("Constant: " + PI);

        System.out.println("\n--- Naming Conventions ---");
        int studentCount = 25;
        String firstName = "John";
        int MAX_SIZE = 100;
        System.out.println("camelCase: studentCount = " + studentCount);
        System.out.println("UPPER_SNAKE: MAX_SIZE = " + MAX_SIZE);
    }
}
```

12

Scanner Class Examples

12_Scanner_Class_Examples\examples\12_Scanner_Class_Examples.md

Scanner Class in Java

Short Definition

The Scanner class is a built-in Java utility that allows reading input from various sources like the console, files, or strings. It's the most common way to get user input in Java applications.

Key Bullet Points

- **Part of java.util package**: Must import to use
- **Multiple Input Sources**: Console, files, strings
- **Token-based Reading**: Reads tokens separated by whitespace
- **Various Methods**: next(), nextInt(), nextDouble(), etc.
- **Exception Handling**: Can throw InputMismatchException
- **Resource Management**: Should be closed after use

Importing Scanner

```
import java.util.Scanner;
```

Creating Scanner Objects

```
// Scanner for console input
Scanner scanner = new Scanner(System.in);

// Scanner for string input
Scanner stringScanner = new Scanner("Hello World 123");

// Scanner for file input
Scanner fileScanner = new Scanner(new File("input.txt"));
```

Scanner Methods

Reading Different Types

Method	Returns	Description
`next()`	String	Reads next token
`nextLine()`	String	Reads entire line
`nextInt()`	int	Reads next integer
`nextDouble()`	double	Reads next double
`nextBoolean()`	boolean	Reads next boolean
`nextByte()`	byte	Reads next byte
`nextShort()`	short	Reads next short
`nextLong()`	long	Reads next long
`nextFloat()`	float	Reads next float

Check Methods

Method	Returns	Description
`hasNext()`	boolean	Checks if more tokens
`hasNextInt()`	boolean	Checks if next is int
`hasNextDouble()`	boolean	Checks if next is double

Example: Basic Input

```
import java.util.Scanner;

class InputDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = scanner.nextLine();

        System.out.print("Enter your age: ");
        int age = scanner.nextInt();

        System.out.print("Enter your salary: ");
        double salary = scanner.nextDouble();
```

```
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Salary: $" + salary);

        scanner.close();
    }
}
```

Example: Using hasNext() Loop

```
import java.util.Scanner;

class TokenReader {
    public static void main(String[] args) {
        Scanner scanner = new Scanner("One Two Three Four Five");

        while (scanner.hasNext()) {
            System.out.println(scanner.next());
        }

        scanner.close();
    }
}
```

Example: File Input

```
import java.util.Scanner;
import java.io.File;
import java.io.FileNotFoundException;

class FileReader {
    public static void main(String[] args) {
        try {
            Scanner fileScanner = new Scanner(new File("data.txt"));

            while (fileScanner.hasNextLine()) {
                System.out.println(fileScanner.nextLine());
            }

            fileScanner.close();
        } catch (FileNotFoundException e) {
            System.out.println("File not found!");
        }
    }
```

```
}
```

Best Practices

1. **Close Scanner**: Always close when done
2. **Use Try-With-Resources**: Auto-closes
3. **Validate Input**: Check with hasNextInt() etc.
4. **Handle Exceptions**: Catch InputMismatchException

Modern Alternative: BufferedReader

For large inputs, BufferedReader is more efficient:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;

BufferedReader reader = new BufferedReader(
    new InputStreamReader(System.in)
);
String line = reader.readLine();
```

Why This Matters for Angular Developers?

- In web apps, forms replace Scanner
- Angular uses ngModel for two-way binding
- REST API receives input via @RequestBody
- Understanding input helps in testing Java code

Exercises

Exercise 1: Simple Calculator

Create a calculator that reads two numbers and an operator, then performs the operation.

Exercise 2: Student Information

Read and store student name, age, and grades using Scanner.

Exercise 3: File Reader

Create a program that reads and displays the contents of a text file.

Summary

- Scanner is used for reading user input
- Multiple methods for different data types
- Always close to prevent resource leaks
- Can read from console, files, or strings
- For web apps, forms replace console input

Code Examples

FormInputDemo.java

FormInputDemo.java

```
// FormInputDemo - Demonstrates Scanner for Angular Form Data Processing
// Use case: Processing user input from forms

import java.util.*;

public class FormInputDemo {
    private Scanner scanner;

    public FormInputDemo() {
        this.scanner = new Scanner(System.in);
    }

    // Simulate reading form data
    public Map<String, Object> readUserForm() {
        Map<String, Object> formData = new HashMap<>();

        System.out.println("=== ANGULAR FORM INPUT ===");

        // Read text input
        System.out.print("Enter name: ");
        String name = scanner.nextLine();
        formData.put("name", name);

        // Read email
        System.out.print("Enter email: ");
        String email = scanner.nextLine();
        formData.put("email", email);

        // Read age
        System.out.print("Enter age: ");
        int age = scanner.nextInt();
        formData.put("age", age);
```

```
        // Read subscription preference
        System.out.print("Subscribe to newsletter (true/false): ");
        boolean subscribed = scanner.nextBoolean();
        formData.put("subscribed", subscribed);

        return formData;
    }

    // Validate form data
    public boolean validateForm(Map<String, Object> formData) {
        String name = (String) formData.get("name");
        String email = (String) formData.get("email");
        int age = (int) formData.get("age");

        if (name == null || name.trim().isEmpty()) {
            System.out.println("Error: Name is required");
            return false;
        }

        if (email == null || !email.contains("@")) {
            System.out.println("Error: Valid email required");
            return false;
        }

        if (age < 0 || age > 150) {
            System.out.println("Error: Invalid age");
            return false;
        }

        return true;
    }

    // Convert to JSON for Angular
    public String toJSON(Map<String, Object> formData) {
        StringBuilder json = new StringBuilder();
        json.append("{");
        json.append("\"name\":\"").append(formData.get("name")).append("\",");
        json.append("\"email\":\"").append(formData.get("email")).append("\",");
        json.append("\"age\":").append(formData.get("age")).append(",");
        json.append("\"subscribed\":").append(formData.get("subscribed"));
        json.append("}");
        return json.toString();
    }

    public void close() {
        scanner.close();
    }

    public static void main(String[] args) {
        FormInputDemo demo = new FormInputDemo();

        System.out.println("\n=== FORM INPUT DEMO ===\n");
```

```
        // Simulated form data (in real use, uncomment readUserForm)
        Map<String, Object> formData = new HashMap<>();
        formData.put("name", "John Doe");
        formData.put("email", "john@example.com");
        formData.put("age", 25);
        formData.put("subscribed", true);

        // Validate
        if (demo.validateForm(formData)) {
            System.out.println("Form is valid!");

            // Convert to JSON
            String json = demo.toJSON(formData);
            System.out.println("\nJSON for Angular: " + json);
        }

        demo.close();
    }
}
```

ScannerDemo.java

ScannerDemo.java

```
// ScannerDemo - Demonstrates Java Scanner class for user input
// Scanner is used to read input from console, files, or strings

import java.util.Scanner;

public class ScannerDemo {

    public static void main(String[] args) {
        System.out.println("=== SCANNER CLASS DEMO ===\n");

        // Create Scanner object for console input
        Scanner scanner = new Scanner(System.in);

        // For demo purposes, we'll use predefined input
        // In real programs, uncomment the scanner.nextLine() calls

        System.out.println("--- Reading Different Types ---");

        // String input
        System.out.print("Enter your name: ");
        String name = "John Doe";  // Simulated input
        // String name = scanner.nextLine();  // Real input
        System.out.println("Name: " + name);

        // Integer input
        System.out.print("Enter your age: ");
        int age = 25;  // Simulated
```

```
        // int age = scanner.nextInt();
        System.out.println("Age: " + age);

        // Double input
        System.out.print("Enter salary: ");
        double salary = 50000.50;  // Simulated
        // double salary = scanner.nextDouble();
        System.out.println("Salary: $" + salary);

        // Boolean input
        System.out.print("Are you employed? (true/false): ");
        boolean employed = true;  // Simulated
        // boolean employed = scanner.nextBoolean();
        System.out.println("Employed: " + employed);

        // Character input
        System.out.print("Enter first letter: ");
        char grade = 'A';  // Simulated
        // String gradeStr = scanner.next();
        // char grade = gradeStr.charAt(0);
        System.out.println("Grade: " + grade);

        // Using Scanner methods
        System.out.println("\n--- Scanner Methods ---");
        String input = "Hello World 123 45.67";
        Scanner stringScanner = new Scanner(input);

        System.out.println("next(): " + stringScanner.next());
        System.out.println("next(): " + stringScanner.next());
        System.out.println("nextInt(): " + stringScanner.nextInt());
        System.out.println("nextDouble(): " + stringScanner.nextDouble());
        stringScanner.close();

        // Best practices
        System.out.println("\n--- Best Practices ---");
        System.out.println("1. Always close Scanner when done");
        System.out.println("2. Use hasNext() to check for input");
        System.out.println("3. Handle InputMismatchException");
        System.out.println("4. Use BufferedReader for large inputs");

        scanner.close();

        System.out.println("\n=== Angular Integration Note ===");
        System.out.println("In web apps, use forms instead of Scanner");
        System.out.println("Angular FormsModule provides ngModel for input");
    }
}
```

13

OOP Classes Objects

13_OOP_Classes_Objects\examples\13_OOP_Classes_Objects.md

Java Classes and Objects - OOP Fundamentals

Table of Contents

1 Introduction to OOP
2 Classes in Java
3 Objects in Java
4 Constructors
5 The `this` Keyword
6 Code Examples
7 Exercises
8 Solutions

---

1. Introduction to OOP

What is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a programming paradigm that uses "objects" to represent data and methods to manipulate that data. Java is a fully object-oriented language, meaning everything in Java is an object (except for primitive types).

Key OOP Concepts

Concept	Description
Encapsulation	Bundling data and methods that operate on that data into a single unit (class)
Inheritance	Mechanism where a class acquires the properties and behaviors of another class
Polymorphism	Ability of objects to take on many forms; method overloading and overriding
Abstraction	Hiding complex implementation details and showing only essential features

Why OOP Matters for Angular Developers?

When building full-stack applications with Angular and Java backend:

- Your Java backend will use classes to model data (entities, DTOs)
- REST APIs return objects that Angular consumes as JSON
- Understanding OOP helps you design better data models
- Spring Boot (popular Java framework) relies heavily on OOP principles

---

2. Classes in Java

What is a Class?

A **class** is a blueprint or template for creating objects. It defines:

- **Fields/Attributes** - Variables that hold data
- **Methods** - Functions that perform operations
- **Constructors** - Special methods for initializing objects

Class Syntax

```
// Class declaration
public class ClassName {
    // Fields (also called instance variables)
    fieldType fieldName;

    // Constructors
    ClassName() {
        // constructor body
    }

    // Methods
    returnType methodName(parameters) {
        // method body
        return value;
    }
}
```

Class Components

```
■■■
```

```
■  - state of the object                  ■
■■■
```

Type	Description	When to Use
Default Constructor	No parameters, provided by Java if no constructor defined	When default initialization is sufficient
Parameterized Constructor	Accepts parameters	When you need to initialize with specific values
Copy Constructor	Creates object as copy of another	When you need to clone objects

Constructor Examples

```
// Default Constructor
public class Student {
    String name;
    int age;

    // Default constructor (no arguments)
    public Student() {
        this.name = "Unknown";
        this.age = 0;
    }
}

// Parameterized Constructor
public class Student {
    String name;
    int age;

    // Parameterized constructor
    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Constructor Overloading

You can have multiple constructors with different parameters:

```
public class Student {
    String name;
    int age;
    String course;
```

```
    // Constructor 1: No parameters
    public Student() {
        this.name = "Unknown";
        this.age = 0;
        this.course = "None";
    }

    // Constructor 2: Two parameters
    public Student(String name, int age) {
        this.name = name;
        this.age = age;
        this.course = "None";
    }

    // Constructor 3: Three parameters
    public Student(String name, int age, String course) {
        this.name = name;
        this.age = age;
        this.course = course;
    }
}
```

---

5. The `this` Keyword

Uses of `this`

The `this` keyword refers to the current object. It has several uses:

1. **Distinguish instance variables from local variables**
2. **Call one constructor from another (constructor chaining)**
3. **Pass current object as parameter**
4. **Return current object from method**

Example: `this` to Avoid Variable Shadowing

```
public class Employee {
    String name;      // instance variable
    int salary;       // instance variable

    // Without 'this' - parameter names shadow instance variables
    public void setData(String name, int salary) {
        name = name;      // WRONG: assigns parameter to itself
        salary = salary;  // WRONG: assigns parameter to itself
    }

    // With 'this' - correctly assigns to instance variables
```

```
    public void setData(String name, int salary) {
        this.name = name;      // CORRECT: instance = parameter
        this.salary = salary;  // CORRECT: instance = parameter
    }
}
```

Example: Constructor Chaining with `this()`

```
public class User {
    String username;
    String email;
    String password;
    boolean active;

    // Constructor 1: Minimal info
    public User(String username) {
        this(username, "default@email.com");  // call constructor 2
    }

    // Constructor 2: Username and email
    public User(String username, String email) {
        this(username, email, "password123");  // call constructor 3
    }

    // Constructor 3: All parameters
    public User(String username, String email, String password) {
        this.username = username;
        this.email = email;
        this.password = password;
        this.active = true;
    }
}
```

6. Code Examples

Example 1: Basic Class and Object

```
/**
 * BankAccount - A simple class representing a bank account
 * This demonstrates fundamental class structure and object creation
 *
 * @author Java Learning
 * @version 1.0
 */
public class BankAccount {
    // ==================== FIELDS ====================
```

```
    // These represent the state/data of each BankAccount object
    private String accountNumber;    // Unique account identifier
    private String accountHolderName; // Name of the account owner
    private double balance;          // Current account balance

    // ==================== CONSTRUCTORS ====================

    /**
     * Default constructor - creates account with default values
     */
    public BankAccount() {
        this.accountNumber = "000000";
        this.accountHolderName = "Unknown";
        this.balance = 0.0;
    }

    /**
     * Parameterized constructor - creates account with specified values
     * @param accountNumber The unique account number
     * @param accountHolderName The name of the account holder
     * @param balance The initial balance
     */
    public BankAccount(String accountNumber, String accountHolderName, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
    }

    // ==================== GETTERS ====================
    // Public methods to access private fields (Encapsulation)

    public String getAccountNumber() {
        return this.accountNumber;
    }

    public String getAccountHolderName() {
        return this.accountHolderName;
    }

    public double getBalance() {
        return this.balance;
    }

    // ==================== SETTERS ====================
    // Public methods to modify private fields

    public void setAccountHolderName(String name) {
        this.accountHolderName = name;
    }

    public void setBalance(double balance) {
        if (balance >= 0) {
            this.balance = balance;
```

```
        } else {
            System.out.println("Error: Balance cannot be negative!");
        }
    }

    // ==================== METHODS ====================
    // These represent the behavior of BankAccount

    /**
     * Deposit money into the account
     * @param amount The amount to deposit
     * @return true if deposit was successful, false otherwise
     */
    public boolean deposit(double amount) {
        if (amount > 0) {
            this.balance += amount;
            System.out.println("Deposited: $" + amount);
            return true;
        } else {
            System.out.println("Error: Deposit amount must be positive!");
            return false;
        }
    }

    /**
     * Withdraw money from the account
     * @param amount The amount to withdraw
     * @return true if withdrawal was successful, false otherwise
     */
    public boolean withdraw(double amount) {
        if (amount > 0 && amount <= this.balance) {
            this.balance -= amount;
            System.out.println("Withdrawn: $" + amount);
            return true;
        } else {
            System.out.println("Error: Invalid withdrawal amount!");
            return false;
        }
    }

    /**
     * Display account information
     */
    public void displayAccountInfo() {
        System.out.println("■■■
```

```
    /**
     * Main method - demonstrates class usage
     */
    public static void main(String[] args) {
        System.out.println("=== BANK ACCOUNT DEMO ===\n");

        // Create first account using default constructor
        BankAccount account1 = new BankAccount();
        System.out.println("--- Account 1 (Default Constructor) ---");
        account1.displayAccountInfo();

        // Create second account using parameterized constructor
        BankAccount account2 = new BankAccount("ACC123456", "John Smith", 5000.00);
        System.out.println("\n--- Account 2 (Parameterized Constructor) ---");
        account2.displayAccountInfo();

        // Perform operations on account2
        System.out.println("\n--- Operations on Account 2 ---");
        account2.deposit(1000.00);    // Deposit $1000
        account2.withdraw(500.00);    // Withdraw $500
        account2.displayAccountInfo();

        // Test validation - try to set negative balance
        System.out.println("\n--- Testing Validation ---");
        account2.setBalance(-1000);   // This should fail
    }
}
```

Output:

```
=== BANK ACCOUNT DEMO ===

--- Account 1 (Default Constructor) ---
╔═══════════════════════════════════════╗
║        ACCOUNT INFORMATION          ║
╔═══════════════════════════════════════╗
║ Account Number: 000000
║ Holder Name:    Unknown
║ Balance:        $0.0
╔═══════════════════════════════════════╗

--- Account 2 (Parameterized Constructor) ---
╔═══════════════════════════════════════╗
║        ACCOUNT INFORMATION          ║
╔═══════════════════════════════════════╗
║ Account Number: ACC123456
║ Holder Name:    John Smith
║ Balance:        $5000.0
╔═══════════════════════════════════════╗

--- Operations on Account 2 ---
```

```
Deposited: $1000.0
Withdrawn: $500.0
■■■
```

```
    // ==================== GETTERS AND SETTERS ====================

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public int getQuantityInStock() {
        return quantityInStock;
    }

    public void setQuantityInStock(int quantityInStock) {
        this.quantityInStock = quantityInStock;
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }

    public boolean isActive() {
        return active;
```

```
    }

    public void setActive(boolean active) {
        this.active = active;
    }

    // ==================== BUSINESS METHODS ====================

    /**
     * Check if product is available for purchase
     */
    public boolean isAvailable() {
        return active && quantityInStock > 0;
    }

    /**
     * Calculate total value of this product in inventory
     */
    public double getInventoryValue() {
        return price * quantityInStock;
    }

    /**
     * Reduce stock after purchase
     * @param quantity Amount to reduce
     * @return true if successful, false if insufficient stock
     */
    public boolean reduceStock(int quantity) {
        if (quantity > 0 && quantity <= quantityInStock) {
            quantityInStock -= quantity;
            return true;
        }
        return false;
    }

    /**
     * Convert to JSON-like string (simplified representation)
     * In real apps, use Jackson ObjectMapper
     */
    @Override
    public String toString() {
        return "{" +
                "\"id\":" + id +
                ", \"name\":\"" + name + "\"" +
                ", \"description\":\"" + description + "\"" +
                ", \"price\":" + price +
                ", \"quantityInStock\":" + quantityInStock +
                ", \"category\":\"" + category + "\"" +
                ", \"active\":" + active +
                "}";
    }

    // ==================== MAIN METHOD FOR TESTING ====================
```

```
    public static void main(String[] args) {
        System.out.println("=== PRODUCT CLASS DEMO ===\n");

        // Create a product - like you'd do in a backend service
        Product laptop = new Product(
            1,
            "MacBook Pro 16\"",
            "Apple MacBook Pro with M3 chip, 16GB RAM",
            2499.99,
            50,
            "Electronics"
        );

        // Display product details
        System.out.println("Product Details:");
        System.out.println("  ID: " + laptop.getId());
        System.out.println("  Name: " + laptop.getName());
        System.out.println("  Price: $" + laptop.getPrice());
        System.out.println("  Stock: " + laptop.getQuantityInStock());
        System.out.println("  Category: " + laptop.getCategory());
        System.out.println("  Available: " + laptop.isAvailable());
        System.out.println("  Inventory Value: $" + laptop.getInventoryValue());

        // Show JSON representation (what Angular receives)
        System.out.println("\nJSON Representation (for Angular):");
        System.out.println(laptop.toString());

        // Simulate purchase
        System.out.println("\n--- Simulating Purchase ---");
        boolean success = laptop.reduceStock(5);
        System.out.println("Purchase of 5 laptops: " + (success ? "Successful" : "Failed"));
        System.out.println("Remaining stock: " + laptop.getQuantityInStock());
    }
}
```

Example 3: Student Management System

```
import java.util.ArrayList;
import java.util.List;

/**
 * Student - Represents a student in a college■■■■
 * Demonstrates more complex class design with multiple data types
 */
class Student {
    // ==================== FIELDS ====================
    private int studentId;
    private String firstName;
    private String lastName;
```

```
    private String email;
    private int age;
    private String major;
    private double gpa;
    private List<String> enrolledCourses;

    // ==================== CONSTRUCTORS ====================

    public Student() {
        this.enrolledCourses = new ArrayList<>();
    }

    public Student(int studentId, String firstName, String lastName, String email, int age) {
        this.studentId = studentId;
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
        this.age = age;
        this.major = "Undeclared";
        this.gpa = 0.0;
        this.enrolledCourses = new ArrayList<>();
    }

    // ==================== GETTERS AND SETTERS ====================

    public int getStudentId() { return studentId; }
    public void setStudentId(int studentId) { this.studentId = studentId; }

    public String getFirstName() { return firstName; }
    public void setFirstName(String firstName) { this.firstName = firstName; }

    public String getLastName() { return lastName; }
    public void setLastName(String lastName) { this.lastName = lastName; }

    public String getFullName() {
        return firstName + " " + lastName;
    }

    public String getEmail() { return email; }
    public void setEmail(String email) {
        if (email.contains("@")) {
            this.email = email;
        } else {
            System.out.println("Error: Invalid email format!");
        }
    }

    public int getAge() { return age; }
    public void setAge(int age) {
        if (age >= 0 && age <= 150) {
            this.age = age;
        }
    }
```

```
    public String getMajor() { return major; }
    public void setMajor(String major) { this.major = major; }

    public double getGpa() { return gpa; }
    public void setGpa(double gpa) {
        if (gpa >= 0.0 && gpa <= 4.0) {
            this.gpa = gpa;
        }
    }

    public List<String> getEnrolledCourses() { return enrolledCourses; }

    // ==================== METHODS ====================

    public void enrollCourse(String course) {
        if (!enrolledCourses.contains(course)) {
            enrolledCourses.add(course);
            System.out.println(getFullName() + " enrolled in: " + course);
        }
    }

    public void dropCourse(String course) {
        if (enrolledCourses.remove(course)) {
            System.out.println(getFullName() + " dropped: " + course);
        } else {
            System.out.println(getFullName() + " is not enrolled in: " + course);
        }
    }

    public void displayInfo() {
        System.out.println("■■■
```

```
        Student student1 = new Student(1001, "Alice", "Johnson", "alice@college.edu", 20);
        student1.setMajor("Computer Science");
        student1.setGpa(3.85);

        Student student2 = new Student(1002, "Bob", "Smith", "bob@college.edu", 22);
        student2.setMajor("Mathematics");
        student2.setGpa(3.5);

        // Display student info
        student1.displayInfo();
        System.out.println();
        student2.displayInfo();

        // Enroll students in courses
        System.out.println("\n=== ENROLLMENT ===");
        student1.enrollCourse("Data Structures");
        student1.enrollCourse("Algorithms");
        student1.enrollCourse("Web Development");  // This is relevant for Angular!

        student2.enrollCourse("Linear Algebra");
        student2.enrollCourse("Calculus III");

        // Display updated info
        System.out.println("\n=== UPDATED INFORMATION ===");
        student1.displayInfo();

        // Test email validation
        System.out.println("\n=== EMAIL VALIDATION TEST ===");
        student1.setEmail("newemail@college.edu");
        System.out.println("New email: " + student1.getEmail());

        student1.setEmail("invalid-email");  // Should show error
    }
}
```

---

7. Exercises

Exercise 1: Create a Rectangle Class

Objective: Create a `Rectangle` class that represents a rectangle with width and height.

Requirements:

1. Create fields for `width` and `height` (both private, double type)
2. Create a default constructor that sets width and height to 1.0
3. Create a parameterized constructor that accepts width and height
4. Create getter and setter methods for both fields
5. Create methods:

- `getArea()` - returns the area (width × height)
- `getPerimeter()` - returns the perimeter (2 × (width + height))
- `display()` - prints all rectangle information

Expected Output:

```
Rectangle 1 (Default):
Width: 1.0, Height: 1.0
Area: 1.0, Perimeter: 4.0

Rectangle 2 (Custom):
Width: 5.0, Height: 3.0
Area: 15.0, Perimeter: 16.0
```

Exercise 2: Create a Book Class for Library System

Objective: Create a `Book` class that models a book in a library system.

Requirements:

1. Fields: `isbn` (String), `title` (String), `author` (String), `year` (int), `isBorrowed` (boolean)
2. Default constructor and parameterized constructor
3. Getters and setters for all fields
4. Methods:
   - `borrowBook()` - marks book as borrowed if not already borrowed
   - `returnBook()` - marks book as returned if currently borrowed
   - `displayBookInfo()` - prints all book details in formatted way

Expected Output:

```
Book: The Great Gatsby
Author: F. Scott Fitzgerald
Year: 1925
ISBN: 978-0743273565
Status: Available

--- Borrowing book ---
Successfully borrowed: The Great Gatsby

--- Attempting to borrow again ---
Book already borrowed!
```

Exercise 3: Create a Calculator Class

Objective: Create a `Calculator` class that performs basic arithmetic operations.

Requirements:

1. Create methods for:
   - `add(double a, double b)` - returns sum
   - `subtract(double a, double b)` - returns difference
   - `multiply(double a, double b)` - returns product
   - `divide(double a, double b)` - returns quotient (handle division by zero)
   - `power(double base, double exponent)` - returns base^exponent
   - `sqrt(double number)` - returns square root
2. Create a main method that demonstrates all operations

Exercise 4: Design a Car Class (Angular Integration Context)

Objective: Create a `Car` class that could be part of a car rental backend API.

Context: This class would eventually be converted to JSON and sent to an Angular frontend.

Requirements:

1. Fields: `id`, `make`, `model`, `year`, `color`, `pricePerDay`, `isAvailable`
2. Implement constructor and all getters/setters
3. Methods:
   - `calculateRentalCost(int days)` - calculates total rental cost
   - `getCarDetails()` - returns formatted string (simulating JSON output)
   - `rentCar()` - marks car as rented
   - `returnCar()` - marks car as available

---

8. Solutions

Solution 1: Rectangle Class

```
/**
 * Rectangle - Solution to Exercise 1
 * Represents a rectangle with width and height
 */
public class Rectangle {
    // Private fields - encapsulation
    private double width;
    private double height;

    // Default constructor
    public Rectangle() {
        this.width = 1.0;
        this.height = 1.0;
```

```
    }

    // Parameterized constructor
    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    // Getters and Setters
    public double getWidth() {
        return width;
    }

    public void setWidth(double width) {
        this.width = width;
    }

    public double getHeight() {
        return height;
    }

    public void setHeight(double height) {
        this.height = height;
    }

    // Calculate area
    public double getArea() {
        return width * height;
    }

    // Calculate perimeter
    public double getPerimeter() {
        return 2 * (width + height);
    }

    // Display rectangle information
    public void display() {
        System.out.println("Width: " + width + ", Height: " + height);
        System.out.println("Area: " + getArea() + ", Perimeter: " + getPerimeter());
    }

    // Main method for testing
    public static void main(String[] args) {
        // Rectangle 1 - Default constructor
        System.out.println("Rectangle 1 (Default):");
        Rectangle rect1 = new Rectangle();
        rect1.display();

        System.out.println();

        // Rectangle 2 - Parameterized constructor
        System.out.println("Rectangle 2 (Custom):");
        Rectangle rect2 = new Rectangle(5.0, 3.0);
```

```
        rect2.display();
    }
}
```

Solution 2: Book Class

```
/**
 * Book - Solution to Exercise 2
 * Represents a book in a library system
 */
public class Book {
    // Fields
    private String isbn;
    private String title;
    private String author;
    private int year;
    private boolean isBorrowed;

    // Default constructor
    public Book() {
        this.isBorrowed = false;
    }

    // Parameterized constructor
    public Book(String isbn, String title, String author, int year) {
        this.isbn = isbn;
        this.title = title;
        this.author = author;
        this.year = year;
        this.isBorrowed = false;
    }

    // Getters and Setters
    public String getIsbn() { return isbn; }
    public void setIsbn(String isbn) { this.isbn = isbn; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public int getYear() { return year; }
    public void setYear(int year) { this.year = year; }

    public boolean isBorrowed() { return isBorrowed; }
    public void setBorrowed(boolean borrowed) { isBorrowed = borrowed; }

    // Borrow book method
    public boolean borrowBook() {
```

```
        if (!isBorrowed) {
            isBorrowed = true;
            return true;
        }
        return false;
    }

    // Return book method
    public boolean returnBook() {
        if (isBorrowed) {
            isBorrowed = false;
            return true;
        }
        return false;
    }

    // Display book information
    public void displayBookInfo() {
        System.out.println("Book: " + title);
        System.out.println("Author: " + author);
        System.out.println("Year: " + year);
        System.out.println("ISBN: " + isbn);
        System.out.println("Status: " + (isBorrowed ? "Borrowed" : "Available"));
    }

    // Main method for testing
    public static void main(String[] args) {
        // Create a book
        Book book = new Book("978-0743273565", "The Great Gatsby",
                            "F. Scott Fitzgerald", 1925);

        // Display book info
        book.displayBookInfo();

        // Test borrowing
        System.out.println("\n--- Borrowing book ---");
        if (book.borrowBook()) {
            System.out.println("Successfully borrowed: " + book.getTitle());
        }

        // Try to borrow again
        System.out.println("\n--- Attempting to borrow again ---");
        if (!book.borrowBook()) {
            System.out.println("Book already borrowed!");
        }

        // Display updated status
        System.out.println("\n--- Updated Status ---");
        book.displayBookInfo();
    }
}
```

Solution 3: Calculator Class

```
/**
 * Calculator - Solution to Exercise 3
 * Performs basic arithmetic operations
 */
public class Calculator {

    // Addition
    public double add(double a, double b) {
        return a + b;
    }

    // Subtraction
    public double subtract(double a, double b) {
        return a - b;
    }

    // Multiplication
    public double multiply(double a, double b) {
        return a * b;
    }

    // Division - handles division by zero
    public double divide(double a, double b) {
        if (b == 0) {
            System.out.println("Error: Division by zero!");
            return Double.NaN;  // Not a Number
        }
        return a / b;
    }

    // Power (a^b)
    public double power(double base, double exponent) {
        return Math.pow(base, exponent);
    }

    // Square root
    public double sqrt(double number) {
        if (number < 0) {
            System.out.println("Error: Cannot calculate square root of negative number!");
            return Double.NaN;
        }
        return Math.sqrt(number);
    }

    // Main method - demonstrates all operations
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        double a = 10;
        double b = 5;
```

```
        System.out.println("=== CALCULATOR OPERATIONS ===");
        System.out.println("a = " + a + ", b = " + b);
        System.out.println();

        System.out.println("Addition:        " + calc.add(a, b));
        System.out.println("Subtraction:    " + calc.subtract(a, b));
        System.out.println("Multiplication: " + calc.multiply(a, b));
        System.out.println("Division:       " + calc.divide(a, b));

        // Test power
        System.out.println("\nPower operations:");
        System.out.println("2^8 = " + calc.power(2, 8));
        System.out.println("3^4 = " + calc.power(3, 4));

        // Test square root
        System.out.println("\nSquare root operations:");
        System.out.println("sqrt(144) = " + calc.sqrt(144));
        System.out.println("sqrt(81) = " + calc.sqrt(81));

        // Test division by zero
        System.out.println("\nDivision by zero test:");
        double result = calc.divide(a, 0);
        System.out.println("Result: " + result);
    }
}
```

Solution 4: Car Class (Angular Integration)

```
/**
 * Car - Solution to Exercise 4
 * Represents a car in a rental system
 * This class demonstrates how Java objects map to JSON for Angular
 */
public class Car {
    // Fields
    private int id;
    private String make;
    private String model;
    private int year;
    private String color;
    private double pricePerDay;
    private boolean isAvailable;

    // Constructors
    public Car() {
        this.isAvailable = true;
    }

    public Car(int id, String make, String model, int year,
```

```
                String color, double pricePerDay) {
        this.id = id;
        this.make = make;
        this.model = model;
        this.year = year;
        this.color = color;
        this.pricePerDay = pricePerDay;
        this.isAvailable = true;
    }

    // Getters and Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getMake() { return make; }
    public void setMake(String make) { this.make = make; }

    public String getModel() { return model; }
    public void setModel(String model) { this.model = model; }

    public int getYear() { return year; }
    public void setYear(int year) { this.year = year; }

    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }

    public double getPricePerDay() { return pricePerDay; }
    public void setPricePerDay(double pricePerDay) { this.pricePerDay = pricePerDay; }

    public boolean isAvailable() { return isAvailable; }
    public void setAvailable(boolean available) { isAvailable = available; }

    // Calculate rental cost
    public double calculateRentalCost(int days) {
        if (days <= 0) {
            System.out.println("Error: Days must be positive!");
            return 0;
        }
        return pricePerDay * days;
    }

    // Get car details (simulates JSON output for Angular)
    public String getCarDetails() {
        return "{" +
                "\"id\":" + id +
                ", \"make\":\"" + make + "\"" +
                ", \"model\":\"" + model + "\"" +
                ", \"year\":" + year +
                ", \"color\":\"" + color + "\"" +
                ", \"pricePerDay\":" + pricePerDay +
                ", \"isAvailable\":" + isAvailable +
                "}";
    }
```

```
    // Rent car
    public boolean rentCar() {
        if (isAvailable) {
            isAvailable = false;
            System.out.println("Car rented successfully!");
            return true;
        }
        System.out.println("Car is not available for rent.");
        return false;
    }

    // Return car
    public boolean returnCar() {
        if (!isAvailable) {
            isAvailable = true;
            System.out.println("Car returned successfully!");
            return true;
        }
        System.out.println("Car was not rented.");
        return false;
    }

    // Display car info
    public void displayInfo() {
        System.out.println("■■■
```

```
        // Test rent/return
        System.out.println("\n--- Testing Rent/Return ---");
        car.rentCar();
        System.out.println("Available: " + car.isAvailable());

        car.returnCar();
        System.out.println("Available: " + car.isAvailable());
    }
}
```

Summary

Key Takeaways

1. **Classes are blueprints** - they define the structure and behavior of objects
2. **Objects are instances** - created from classes using the `new` keyword
3. **Constructors initialize** - objects and can be overloaded for flexibility
4. `this` **keyword** - refers to current object and helps avoid variable shadowing
5. **Encapsulation** - use private fields with public getters/setters
6. **For Angular Integration** - Java classes become JSON via serialization libraries

Next Steps

After mastering classes and objects, continue with:

- **Inheritance** - Creating class hierarchies
- **Polymorphism** - Method overloading and overriding
- **Abstraction** - Abstract classes and interfaces

These concepts will help you build more sophisticated backend systems that integrate seamlessly with Angular frontends!

Happy Coding! ■

Code Examples

AngularComponentDemo.java

AngularComponentDemo.java

```
// AngularComponentDemo - Demonstrates OOP for Angular Component Structure
// Use case: Modeling Angular components in Java backend

public class AngularComponentDemo {
```

```
    // Component properties (fields)
    private String componentName;
    private String template;
    private String[] inputs;
    private String[] outputs;
    private boolean isActive;

    // Constructor
    public AngularComponentDemo(String name) {
        this.componentName = name;
        this.template = "<div>Default Template</div>";
        this.inputs = new String[] {};
        this.outputs = new String[] {};
        this.isActive = true;
    }

    // Component initialization
    public void initialize(String template, String[] inputs, String[] outputs) {
        this.template = template;
        this.inputs = inputs;
        this.outputs = outputs;
        System.out.println("Component initialized: " + componentName);
    }

    // Input binding methods
    public void setInput(String key, String value) {
        System.out.println("Input bound: " + key + " = " + value);
    }

    // Output event methods
    public String emitEvent(String eventName, Object data) {
        System.out.println("Event emitted: " + eventName);
        return "Event(" + eventName + ", " + data + ")";
    }

    // Template rendering
    public String render() {
        return "Rendering: " + componentName + " with template:\n" + template;
    }

    // Lifecycle hook
    public void onInit() {
        System.out.println(componentName + " - ngOnInit called");
    }

    public void onDestroy() {
        System.out.println(componentName + " - ngOnDestroy called");
    }

    public static void main(String[] args) {
        System.out.println("=== OOP FOR ANGULAR COMPONENTS ===\n");

        // Create component
```

```
        AngularComponentDemo userList = new AngularComponentDemo("UserListComponent");

        // Initialize with inputs/outputs
        userList.initialize(
            "<ul><li *ngFor='let user of users'>{{user.name}}</li></ul>",
            new String[] {"users", "title"},
            new String[] {"userSelected", "userDeleted"}
        );

        // Simulate lifecycle
        userList.onInit();

        // Bind inputs
        userList.setInput("title", "User List");

        // Emit events
        userList.emitEvent("userSelected", "{id: 1}");

        // Render
        System.out.println("\n" + userList.render());

        // Cleanup
        userList.onDestroy();

        System.out.println("\n=== OOP MAPPING ===");
        System.out.println("Java Class -> TypeScript Class");
        System.out.println("Constructor -> Component constructor");
        System.out.println("Methods -> Component methods");
        System.out.println("Fields -> Component properties");
    }
}
```

BankAccount.java

BankAccount.java

```
// BankAccount - A simple class representing a bank account
// Demonstrates fundamental class structure, fields, constructors, and methods
// This class is used to demonstrate OOP concepts in Java
// and shows how Java objects map to JSON for Angular frontend

public class BankAccount {

    // ==================== FIELDS ====================
    private String accountNumber;
    private String accountHolderName;
    private double balance;

    // ==================== CONSTRUCTORS ====================

    // Default constructor
    public BankAccount() {
```

```
        this.accountNumber = "000000";
        this.accountHolderName = "Unknown";
        this.balance = 0.0;
    }

    // Parameterized constructor
    public BankAccount(String accountNumber, String accountHolderName, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
    }

    // ==================== GETTERS ====================
    public String getAccountNumber() { return this.accountNumber; }
    public String getAccountHolderName() { return this.accountHolderName; }
    public double getBalance() { return this.balance; }

    // ==================== SETTERS ====================
    public void setAccountHolderName(String name) { this.accountHolderName = name; }

    public void setBalance(double balance) {
        if (balance >= 0) {
            this.balance = balance;
        } else {
            System.out.println("Error: Balance cannot be negative!");
        }
    }

    // ==================== METHODS ====================
    public boolean deposit(double amount) {
        if (amount > 0) {
            this.balance += amount;
            System.out.println("Deposited: $" + amount);
            return true;
        } else {
            System.out.println("Error: Deposit amount must be positive!");
            return false;
        }
    }

    public boolean withdraw(double amount) {
        if (amount > 0 && amount <= this.balance) {
            this.balance -= amount;
            System.out.println("Withdrawn: $" + amount);
            return true;
        } else {
            System.out.println("Error: Invalid withdrawal amount!");
            return false;
        }
    }

    public void displayAccountInfo() {
        System.out.println("=== ACCOUNT INFORMATION ===");
```

```
        System.out.println("Account Number: " + this.accountNumber);
        System.out.println("Holder Name:    " + this.accountHolderName);
        System.out.println("Balance:        $" + this.balance);
    }

    // Main method for demonstration
    public static void main(String[] args) {
        System.out.println("=== BANK ACCOUNT DEMO ===\n");

        BankAccount account1 = new BankAccount();
        System.out.println("--- Account 1 (Default) ---");
        account1.displayAccountInfo();

        BankAccount account2 = new BankAccount("ACC123456", "John Smith", 5000.00);
        System.out.println("\n--- Account 2 ---");
        account2.displayAccountInfo();

        System.out.println("\n--- Operations ---");
        account2.deposit(1000.00);
        account2.withdraw(500.00);
        account2.displayAccountInfo();
    }
}
```

Product.java

Product.java

```
// Product - E-commerce product class demonstrating OOP
// Used for building product catalogs in Java backend
// Maps directly to Angular Product interface

public class Product {
    // Fields
    private int productId;
    private String productName;
    private String category;
    private double price;
    private int stockQuantity;
    private boolean inStock;

    // Default constructor
    public Product() {
        this.productId = 0;
        this.productName = "Unknown";
        this.category = "General";
        this.price = 0.0;
        this.stockQuantity = 0;
        this.inStock = false;
    }

    // Parameterized constructor
```

```
    public Product(int productId, String productName, String category,
                   double price, int stockQuantity) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
        this.price = price;
        this.stockQuantity = stockQuantity;
        this.inStock = stockQuantity > 0;
    }

    // Getters
    public int getProductId() { return productId; }
    public String getProductName() { return productName; }
    public String getCategory() { return category; }
    public double getPrice() { return price; }
    public int getStockQuantity() { return stockQuantity; }
    public boolean isInStock() { return inStock; }

    // Setters
    public void setPrice(double price) {
        if (price > 0) this.price = price;
    }

    public void setStockQuantity(int quantity) {
        this.stockQuantity = quantity;
        this.inStock = quantity > 0;
    }

    // Business methods
    public double calculateDiscount(double discountPercent) {
        return price * (1 - discountPercent / 100);
    }

    public boolean isAvailable(int requiredQuantity) {
        return stockQuantity >= requiredQuantity;
    }

    public void displayProduct() {
        System.out.println("=== PRODUCT DETAILS ===");
        System.out.println("ID: " + productId);
        System.out.println("Name: " + productName);
        System.out.println("Category: " + category);
        System.out.println("Price: $" + price);
        System.out.println("Stock: " + stockQuantity);
        System.out.println("In Stock: " + (inStock ? "Yes" : "No"));
    }

    public static void main(String[] args) {
        System.out.println("=== PRODUCT CLASS DEMO ===\n");

        Product laptop = new Product(101, "MacBook Pro", "Electronics", 1299.99, 50);
        laptop.displayProduct();
```

```
        System.out.println("\nDiscounted Price: $" + laptop.calculateDiscount(10));
        System.out.println("Available (5 units): " + laptop.isAvailable(5));

        Product outOfStock = new Product(102, "Vintage Camera", "Antiques", 500.00, 0);
        System.out.println("\n" + outOfStock.getProductName() + " in stock: " + outOfStock.isInSt...
    }
}
```

Student.java

Student.java

```
// Student - Student class demonstrating OOP
// Used for student management systems in Java backend
// Maps to Angular Student interface

public class Student {
    // Fields - private for encapsulation
    private int studentId;
    private String name;
    private String email;
    private int age;
    private String major;
    private double gpa;

    // Default constructor
    public Student() {
        this.studentId = 0;
        this.name = "Unknown";
        this.email = "";
        this.age = 0;
        this.major = "Undeclared";
        this.gpa = 0.0;
    }

    // Parameterized constructor
    public Student(int studentId, String name, String email,
                   int age, String major, double gpa) {
        this.studentId = studentId;
        this.name = name;
        this.email = email;
        this.age = age;
        this.major = major;
        this.gpa = gpa;
    }

    // Getters
    public int getStudentId() { return studentId; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public int getAge() { return age; }
    public String getMajor() { return major; }
```

```
    public double getGpa() { return gpa; }

    // Setters
    public void setName(String name) { this.name = name; }
    public void setEmail(String email) { this.email = email; }
    public void setMajor(String major) { this.major = major; }

    public void setGpa(double gpa) {
        if (gpa >= 0.0 && gpa <= 4.0) {
            this.gpa = gpa;
        }
    }

    // Business methods
    public String getGradeLevel() {
        if (gpa >= 3.5) return "Dean's List";
        if (gpa >= 3.0) return "Good Standing";
        if (gpa >= 2.0) return "Satisfactory";
        return "Probation";
    }

    public boolean isHonorStudent() {
        return gpa >= 3.5;
    }

    public void displayInfo() {
        System.out.println("=== STUDENT INFORMATION ===");
        System.out.println("ID: " + studentId);
        System.out.println("Name: " + name);
        System.out.println("Email: " + email);
        System.out.println("Age: " + age);
        System.out.println("Major: " + major);
        System.out.println("GPA: " + gpa);
        System.out.println("Status: " + getGradeLevel());
    }

    public static void main(String[] args) {
        System.out.println("=== STUDENT CLASS DEMO ===\n");

        Student student1 = new Student(1001, "Alice Johnson",
            "alice@school.edu", 20, "Computer Science", 3.8);
        student1.displayInfo();

        System.out.println("\n--- Second Student ---");
        Student student2 = new Student(1002, "Bob Smith",
            "bob@school.edu", 19, "Mathematics", 2.5);
        student2.displayInfo();
        System.out.println("Honor Student: " + student2.isHonorStudent());
    }
}
```

14

Inheritance Polymorphism

14_Inheritance_Polymorphism\examples\14_Inheritance_Polymorphism.md

Java Inheritance and Polymorphism

Table of Contents

1 Introduction to Inheritance
2 Types of Inheritance
3 The `extends` Keyword
4 The `super` Keyword
5 Method Overriding
6 Polymorphism
7 Code Examples
8 Exercises
9 Solutions

---

1. Introduction to Inheritance

What is Inheritance?

Inheritance is one of the fundamental concepts of Object-Oriented Programming (OOP) that allows a class to inherit properties and behaviors from another class. It promotes code reusability and establishes a natural hierarchy between classes.

Key Terminology

Term	Description
Parent Class (Superclass)	The class whose properties are inherited
Child Class (Subclass)	The class that inherits from another class
IS-A Relationship	Inheritance relationship (e.g., Dog IS-A Animal)
Reusability	Ability to use existing code without modification

Why Inheritance Matters for Angular Developers?

When building full-stack applications:

- Backend entities often share common properties (ID, timestamps)
- REST APIs use inheritance for DTOs and entities
- Spring Data JPA uses inheritance for entities
- Understanding inheritance helps map Angular models to Java backend

2. Types of Inheritance

Types of Inheritance in Java

```
■■■
```

Basic Syntax

```
// Parent class (Superclass)
public class Animal {
    // Fields and methods
}

// Child class (Subclass)
public class Dog extends Animal {
    // Inherits all non-private members from Animal
    // Can add its own fields and methods
}
```

What Gets Inherited?

Inherited	Not Inherited
Public fields	Private fields
Public methods	Private methods
Protected fields	Constructors
Protected methods	Static members
Package-private members	

---

4. The `super` Keyword

Uses of `super`

The `super` keyword is used to:

1. **Call parent class constructors**
2. **Access parent class methods**
3. **Access parent class fields**

Constructor Chaining with `super()`

```
/**
 * Demonstrates super() for constructor chaining
 */
class Parent {
    String name;
```

```
    Parent() {
        System.out.println("Parent constructor called");
        this.name = "Parent";
    }

    Parent(String name) {
        System.out.println("Parent parameterized constructor called");
        this.name = name;
    }
}

class Child extends Parent {
    int age;

    // Automatically calls Parent() first
    Child() {
        super();  // Explicit call to parent constructor
        System.out.println("Child constructor called");
        this.age = 0;
    }

    // Calls Parent(String name) with "ChildName"
    Child(String name, int age) {
        super(name);  // Call parent constructor with parameter
        System.out.println("Child parameterized constructor called");
        this.age = age;
    }
}
```

5. Method Overriding

What is Method Overriding?

Method Overriding occurs when a subclass provides a specific implementation of a method already defined in its parent class.

Rules for Method Overriding

1. Method must have the **same name** as in parent class
2. Method must have the **same parameters** as in parent class
3. Method must have the **same return type** (or covariant return type)
4. Cannot override `static` methods
5. Cannot override `final` methods
6. Override method cannot be less accessible

Syntax

```
// Parent class
class Animal {
    public void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

// Child class
class Dog extends Animal {
    @Override  // Annotation - good practice to include
    public void makeSound() {
        System.out.println("Dog barks: Woof! Woof!");
    }
}
```

---

6. Polymorphism

What is Polymorphism?

Polymorphism allows objects of different classes to be treated as objects of a common type. It enables one interface to be used for a general class of actions.

Types of Polymorphism

```
■■■
```

```
/**
 * Method Overloading - Same name, different parameters
 */
class Calculator {
    // Overloaded add() methods
    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }

    public String add(String a, String b) {
        return a + b;
    }
}
```

Runtime Polymorphism with Method Overriding

```
/**
 * Runtime Polymorphism through method overriding
 */
class Animal {
    public void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    @Override
    public void sound() {
        System.out.println("Cat meows");
    }
}

// Main
public class PolymorphismDemo {
    public static void main(String[] args) {
```

```
        // Reference of type Animal, object of Dog
        Animal myAnimal = new Dog();
        myAnimal.sound();  // Outputs: Dog barks

        myAnimal = new Cat();
        myAnimal.sound();  // Outputs: Cat meows
    }
}
```

7. Code Examples

Example 1: Employee Hierarchy (Angular Backend Context)

```
/**
 * Employee - Base class for all employee types
 * This demonstrates how backend systems model hierarchical data
 */
public class Employee {
    // ==================== FIELDS ====================
    protected int id;                    // Protected: accessible by subclasses
    protected String name;
    protected String department;
    protected double salary;
    protected String email;

    // ==================== CONSTRUCTORS ====================

    public Employee() {
    }

    public Employee(int id, String name, String department, double salary) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.salary = salary;
        this.email = name.toLowerCase().replace(" ", ".") + "@company.com";
    }

    // ==================== GETTERS AND SETTERS ====================

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }
```

```
    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    // ==================== METHODS ====================

    /**
     * Calculate annual salary
     */
    public double calculateAnnualSalary() {
        return salary * 12;
    }

    /**
     * Display employee information
     */
    public void displayInfo() {
        System.out.println("■■■
```

```
        this.teamSize = teamSize;
        this.bonus = 0.0;
    }

    // Getter and Setter for new field
    public int getTeamSize() { return teamSize; }
    public void setTeamSize(int teamSize) { this.teamSize = teamSize; }

    public double getBonus() { return bonus; }
    public void setBonus(double bonus) { this.bonus = bonus; }

    /**
     * Override: Calculate annual salary including bonus
     */
    @Override
    public double calculateAnnualSalary() {
        return (salary * 12) + bonus;
    }

    /**
     * Override: Manager-specific work
     */
    @Override
    public void work() {
        System.out.println(name + " is managing a team of " + teamSize + " people");
    }

    /**
     * Additional method specific to Manager
     */
    public void conductMeeting() {
        System.out.println(name + " is conducting a team meeting");
    }

    @Override
    public void displayInfo() {
        super.displayInfo();  // Call parent displayInfo
        System.out.println("Team Size: " + teamSize);
        System.out.println("Bonus:     $" + bonus);
    }
}

/**
 * Developer - Extends Employee
 * Has programming language specialization
 */
class Developer extends Employee {
    private String programmingLanguage;
    private boolean isRemote;

    public Developer() {
        super();
    }
```

```
    public Developer(int id, String name, String department, double salary,
                     String programmingLanguage, boolean isRemote) {
        super(id, name, department, salary);
        this.programmingLanguage = programmingLanguage;
        this.isRemote = isRemote;
    }

    // Getters and Setters
    public String getProgrammingLanguage() { return programmingLanguage; }
    public void setProgrammingLanguage(String programmingLanguage) {
        this.programmingLanguage = programmingLanguage;
    }

    public boolean isRemote() { return isRemote; }
    public void setRemote(boolean remote) { isRemote = remote; }

    /**
     * Override: Developer-specific work
     */
    @Override
    public void work() {
        System.out.println(name + " is coding in " + programmingLanguage +
                           (isRemote ? " (Remote)" : " (On-site)"));
    }

    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Language:  " + programmingLanguage);
        System.out.println("Work Mode: " + (isRemote ? "Remote" : "On-site"));
    }
}

/**
 * InheritanceDemo - Main class to demonstrate inheritance
 */
public class InheritanceDemo {
    public static void main(String[] args) {
        System.out.println("=== INHERITANCE DEMO: EMPLOYEE HIERARCHY ===\n");

        // Create Employee
        System.out.println("--- Creating Employee ---");
        Employee emp1 = new Employee(101, "John Doe", "HR", 5000);
        emp1.displayInfo();
        emp1.work();

        // Create Manager
        System.out.println("\n--- Creating Manager ---");
        Manager mgr1 = new Manager(201, "Jane Smith", "Engineering", 8000, 10);
        mgr1.setBonus(5000);
        mgr1.displayInfo();
        mgr1.work();
```

```
        mgr1.conductMeeting();

        // Create Developer
        System.out.println("\n--- Creating Developer ---");
        Developer dev1 = new Developer(301, "Mike Johnson", "IT", 7000, "Java", true);
        dev1.displayInfo();
        dev1.work();

        // Polymorphism Demo
        System.out.println("\n=== POLYMORPHISM DEMO ===");
        Employee[] employees = new Employee[3];
        employees[0] = new Employee(101, "Emp1", "General", 5000);
        employees[1] = new Manager(201, "Mgr1", "Sales", 7000, 5);
        employees[2] = new Developer(301, "Dev1", "Tech", 6500, "Python", false);

        System.out.println("\nCalling work() on each (Runtime Polymorphism):");
        for (Employee emp : employees) {
            emp.work();  // Different behavior based on actual object type
        }

        // Calculate total annual salary
        System.out.println("\n--- Annual Salary Calculation ---");
        System.out.println("Employee Annual: $" + emp1.calculateAnnualSalary());
        System.out.println("Manager Annual: $" + mgr1.calculateAnnualSalary());
        System.out.println("Developer Annual: $" + dev1.calculateAnnualSalary());
    }
}
```

Example 2: Vehicle Inheritance Hierarchy

```
/**
 * Vehicle - Base class for all vehicles
 */
class Vehicle {
    protected String brand;
    protected String model;
    protected int year;
    protected double price;

    // Constructor
    public Vehicle(String brand, String model, int year, double price) {
        this.brand = brand;
        this.model = model;
        this.year = year;
        this.price = price;
    }

    // Getter methods
    public String getBrand() { return brand; }
    public String getModel() { return model; }
```

```
    public int getYear() { return year; }
    public double getPrice() { return price; }

    // Common method
    public void displayDetails() {
        System.out.println("Brand: " + brand + ", Model: " + model +
                           ", Year: " + year + ", Price: $" + price);
    }

    // Method to be overridden
    public void start() {
        System.out.println(brand + " " + model + " is starting...");
    }

    public void stop() {
        System.out.println(brand + " " + model + " is stopping...");
    }
}

/**
 * Car - Extends Vehicle
 */
class Car extends Vehicle {
    private int numberOfDoors;
    private String fuelType;

    public Car(String brand, String model, int year, double price,
               int numberOfDoors, String fuelType) {
        super(brand, model, year, price);  // Call parent constructor
        this.numberOfDoors = numberOfDoors;
        this.fuelType = fuelType;
    }

    // Getter methods
    public int getNumberOfDoors() { return numberOfDoors; }
    public String getFuelType() { return fuelType; }

    // Override start method
    @Override
    public void start() {
        System.out.println(brand + " " + model + " car is starting with " +
                           fuelType + " engine...");
    }

    // Car-specific method
    public void honk() {
        System.out.println("Car horn: Beep! Beep!");
    }

    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Doors: " + numberOfDoors + ", Fuel: " + fuelType);
```

```
    }
}

/**
 * Motorcycle - Extends Vehicle
 */
class Motorcycle extends Vehicle {
    private int engineCC;
    private boolean hasSidecar;

    public Motorcycle(String brand, String model, int year, double price,
                      int engineCC, boolean hasSidecar) {
        super(brand, model, year, price);
        this.engineCC = engineCC;
        this.hasSidecar = hasSidecar;
    }

    // Getter methods
    public int getEngineCC() { return engineCC; }
    public boolean hasSidecar() { return hasSidecar; }

    // Override start method
    @Override
    public void start() {
        System.out.println(brand + " " + model + " motorcycle engine is starting...");
    }

    // Motorcycle-specific method
    public void wheelie() {
        System.out.println("Doing a wheelie!");
    }

    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Engine: " + engineCC + "cc, Sidecar: " + (hasSidecar ? "Yes" : "No"));
    }
}

/**
 * Truck - Extends Vehicle (for Angular backend demo)
 */
class Truck extends Vehicle {
    private double payloadCapacity;
    private int numberOfWheels;

    public Truck(String brand, String model, int year, double price,
                 double payloadCapacity, int numberOfWheels) {
        super(brand, model, year, price);
        this.payloadCapacity = payloadCapacity;
        this.numberOfWheels = numberOfWheels;
    }
```

```
    // Getter methods
    public double getPayloadCapacity() { return payloadCapacity; }
    public int getNumberOfWheels() { return numberOfWheels; }

    // Override start method
    @Override
    public void start() {
        System.out.println(brand + " " + model + " truck is starting its engine...");
    }

    // Truck-specific method
    public void loadCargo(double weight) {
        if (weight <= payloadCapacity) {
            System.out.println("Loading " + weight + " tons of cargo...");
        } else {
            System.out.println("Warning: Weight exceeds payload capacity!");
        }
    }

    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Payload: " + payloadCapacity + " tons, Wheels: " + numberOfWheels);
    }
}

/**
 * VehicleDemo - Main demonstration class
 */
public class VehicleDemo {
    public static void main(String[] args) {
        System.out.println("=== VEHICLE INHERITANCE HIERARCHY ===\n");

        // Create different vehicles
        Car car = new Car("Toyota", "Camry", 2023, 28000, 4, "Gasoline");
        Motorcycle bike = new Motorcycle("Harley-Davidson", "Sportster", 2022, 12000, 883, false);
        Truck truck = new Ford("F-150", 2023, 35000, 2.5, 4);

        // Display details
        System.out.println("--- CAR ---");
        car.displayDetails();
        car.start();
        car.honk();

        System.out.println("\n--- MOTORCYCLE ---");
        bike.displayDetails();
        bike.start();
        bike.wheelie();

        System.out.println("\n--- TRUCK ---");
        truck.displayDetails();
        truck.start();
        truck.loadCargo(2.0);
```

```
        // Polymorphism with array
        System.out.println("\n=== POLYMORPHISM DEMO ===");
        Vehicle[] vehicles = {car, bike, truck};

        for (Vehicle v : vehicles) {
            v.start();  // Calls overridden method based on actual type
            System.out.println("---");
        }
    }
}

// Note: Ford class for truck example - simple subclass
class Ford extends Truck {
    public Ford(String model, int year, double price, double payload, int wheels) {
        super("Ford", model, year, price, payload, wheels);
    }
}
```

Example 3: Shape Hierarchy (with Method Overloading)

```
/**
 * Shape - Abstract base class (we'll cover abstraction later)
 * Demonstrates polymorphism with different shapes
 */
class Shape {
    protected String color;

    public Shape() {
        this.color = "white";
    }

    public Shape(String color) {
        this.color = color;
    }

    // Methods to be overridden by subclasses
    public double getArea() {
        return 0.0;
    }

    public double getPerimeter() {
        return 0.0;
    }

    public void display() {
        System.out.println("Color: " + color);
    }
}
```

```
/**
 * Circle - Extends Shape
 */
class Circle extends Shape {
    private double radius;

    // Multiple constructors (overloading)
    public Circle() {
        super();
        this.radius = 1.0;
    }

    public Circle(double radius) {
        super();
        this.radius = radius;
    }

    public Circle(double radius, String color) {
        super(color);
        this.radius = radius;
    }

    // Getter and Setter
    public double getRadius() { return radius; }
    public void setRadius(double radius) { this.radius = radius; }

    // Override methods
    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }

    @Override
    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Shape: Circle");
        System.out.println("Radius: " + radius);
        System.out.println("Area: " + getArea());
        System.out.println("Perimeter: " + getPerimeter());
    }
}

/**
 * Rectangle - Extends Shape
 */
class Rectangle extends Shape {
    private double width;
    private double height;
```

```
    // Multiple constructors (overloading)
    public Rectangle() {
        super();
        this.width = 1.0;
        this.height = 1.0;
    }

    public Rectangle(double width, double height) {
        super();
        this.width = width;
        this.height = height;
    }

    public Rectangle(double width, double height, String color) {
        super(color);
        this.width = width;
        this.height = height;
    }

    // Getters and Setters
    public double getWidth() { return width; }
    public void setWidth(double width) { this.width = width; }
    public double getHeight() { return height; }
    public void setHeight(double height) { this.height = height; }

    // Override methods
    @Override
    public double getArea() {
        return width * height;
    }

    @Override
    public double getPerimeter() {
        return 2 * (width + height);
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Shape: Rectangle");
        System.out.println("Width: " + width + ", Height: " + height);
        System.out.println("Area: " + getArea());
        System.out.println("Perimeter: " + getPerimeter());
    }
}

/**
 * Triangle - Extends Shape
 */
class Triangle extends Shape {
    private double side1;
    private double side2;
```

```
    private double side3;

    public Triangle() {
        super();
        this.side1 = 1.0;
        this.side2 = 1.0;
        this.side3 = 1.0;
    }

    public Triangle(double side1, double side2, double side3) {
        super();
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
    }

    public Triangle(double side1, double side2, double side3, String color) {
        super(color);
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
    }

    // Override methods
    @Override
    public double getArea() {
        // Heron's formula
        double s = (side1 + side2 + side3) / 2;
        return Math.sqrt(s * (s - side1) * (s - side2) * (s - side3));
    }

    @Override
    public double getPerimeter() {
        return side1 + side2 + side3;
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Shape: Triangle");
        System.out.println("Sides: " + side1 + ", " + side2 + ", " + side3);
        System.out.println("Area: " + getArea());
        System.out.println("Perimeter: " + getPerimeter());
    }
}

/**
 * ShapeDemo - Main demonstration class
 */
public class ShapeDemo {
    public static void main(String[] args) {
        System.out.println("=== SHAPE HIERARCHY DEMO ===\n");
```

```
        // Create different shapes
        Circle circle = new Circle(5.0, "Red");
        Rectangle rectangle = new Rectangle(4.0, 6.0, "Blue");
        Triangle triangle = new Triangle(3.0, 4.0, 5.0, "Green");

        // Display each shape
        System.out.println("--- CIRCLE ---");
        circle.display();

        System.out.println("\n--- RECTANGLE ---");
        rectangle.display();

        System.out.println("\n--- TRIANGLE ---");
        triangle.display();

        // Polymorphism: Treat all shapes uniformly
        System.out.println("\n=== POLYMORPHISM: CALCULATE AREAS ===");
        Shape[] shapes = {circle, rectangle, triangle};

        double totalArea = 0;
        for (Shape shape : shapes) {
            System.out.println("Area of " + shape.getClass().getSimpleName() +
                               ": " + shape.getArea());
            totalArea += shape.getArea();
        }
        System.out.println("Total Area: " + totalArea);
    }
}
```

---

8. Exercises

Exercise 1: Banking System Inheritance

Objective: Create a banking system with different account types.

Requirements:

1. Create a `BankAccount` class with fields: accountNumber, holderName, balance
2. Create `SavingsAccount` that extends `BankAccount` with field: interestRate
3. Create `CheckingAccount` that extends `BankAccount` with field: overdraftLimit
4. Each account type should have its own `calculateInterest()` or `calculateFee()` method
5. Demonstrate polymorphism by creating an array of different account types

Expected Output:

```
Account: 1001, Holder: John, Balance: $5000.0
Savings Interest: $250.0

Account: 1002, Holder: Jane, Balance: $3000.0
Checking Fee: $15.0
```

Exercise 2: University Person Hierarchy

Objective: Model a university system with different person types.

Requirements:

1. Create `Person` class with: name, age, email
2. Create `Student` that extends Person with: studentId, major, gpa
3. Create `Teacher` that extends Person with: teacherId, department, salary
4. Create `Staff` that extends Person with: staffId, position, department
5. Each subclass should override `displayInfo()` to show specific details

---

Exercise 3: E-commerce Product Hierarchy

Objective: Model products for an e-commerce backend.

Context: This would be sent to an Angular frontend as JSON.

Requirements:

1. Create `Product` base class with: id, name, price, category
2. Create `Electronics` with: brand, warrantyMonths
3. Create `Clothing` with: size, color, material
4. Create `Book` with: author, isbn, pages
5. Each should have appropriate getters/setters and toString() method

---

9. Solutions

Solution 1: Banking System

```
/**
 * BankAccount - Base class for bank accounts
 */
class BankAccount {
    protected String accountNumber;
    protected String holderName;
    protected double balance;
```

```
    public BankAccount() {
    }

    public BankAccount(String accountNumber, String holderName, double balance) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
    }

    // Getters and Setters
    public String getAccountNumber() { return accountNumber; }
    public void setAccountNumber(String accountNumber) { this.accountNumber = accountNumber; }

    public String getHolderName() { return holderName; }
    public void setHolderName(String holderName) { this.holderName = holderName; }

    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; }

    public void displayAccount() {
        System.out.println("Account: " + accountNumber + ", Holder: " +
                           holderName + ", Balance: $" + balance);
    }

    // Method to be overridden
    public double calculateInterest() {
        return 0.0;
    }
}

/**
 * SavingsAccount - Extends BankAccount with interest
 */
class SavingsAccount extends BankAccount {
    private double interestRate;  // Annual interest rate (e.g., 0.05 = 5%)

    public SavingsAccount() {
        super();
    }

    public SavingsAccount(String accountNumber, String holderName,
                          double balance, double interestRate) {
        super(accountNumber, holderName, balance);
        this.interestRate = interestRate;
    }

    public double getInterestRate() { return interestRate; }
    public void setInterestRate(double interestRate) { this.interestRate = interestRate; }

    @Override
    public double calculateInterest() {
        return balance * interestRate;
```

```
    }

    @Override
    public void displayAccount() {
        super.displayAccount();
        System.out.println("Savings Interest: $" + calculateInterest());
    }
}

/**
 * CheckingAccount - Extends BankAccount with overdraft
 */
class CheckingAccount extends BankAccount {
    private double overdraftLimit;
    private double monthlyFee;

    public CheckingAccount() {
        super();
        this.monthlyFee = 15.0;
    }

    public CheckingAccount(String accountNumber, String holderName,
                           double balance, double overdraftLimit) {
        super(accountNumber, holderName, balance);
        this.overdraftLimit = overdraftLimit;
        this.monthlyFee = 15.0;
    }

    public double getOverdraftLimit() { return overdraftLimit; }
    public void setOverdraftLimit(double overdraftLimit) { this.overdraftLimit = overdraftLimit; }

    public double getMonthlyFee() { return monthlyFee; }

    public double calculateFee() {
        return monthlyFee;
    }

    @Override
    public void displayAccount() {
        super.displayAccount();
        System.out.println("Checking Fee: $" + calculateFee());
    }
}

/**
 * BankingDemo - Main demonstration
 */
public class BankingDemo {
    public static void main(String[] args) {
        System.out.println("=== BANKING SYSTEM DEMO ===\n");

        // Create accounts
        SavingsAccount savings = new SavingsAccount("1001", "John", 5000.0, 0.05);
```

```
        CheckingAccount checking = new CheckingAccount("1002", "Jane", 3000.0, 500.0);

        // Display account info
        savings.displayAccount();
        System.out.println();
        checking.displayAccount();

        // Polymorphism with array
        System.out.println("\n=== POLYMORPHISM DEMO ===");
        BankAccount[] accounts = {savings, checking};

        for (BankAccount account : accounts) {
            System.out.println("Account: " + account.getAccountNumber() +
                               ", Balance: $" + account.getBalance());
        }
    }
}
```

Solution 2: University Person Hierarchy

```
/**
 * Person - Base class
 */
class Person {
    protected String name;
    protected int age;
    protected String email;

    public Person() {
    }

    public Person(String name, int age, String email) {
        this.name = name;
        this.age = age;
        this.email = email;
    }

    // Getters and Setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
```

```
        System.out.println("Email: " + email);
    }
}

/**
 * Student - Extends Person
 */
class Student extends Person {
    private String studentId;
    private String major;
    private double gpa;

    public Student() {
        super();
    }

    public Student(String name, int age, String email,
                   String studentId, String major, double gpa) {
        super(name, age, email);
        this.studentId = studentId;
        this.major = major;
        this.gpa = gpa;
    }

    // Getters and Setters
    public String getStudentId() { return studentId; }
    public void setStudentId(String studentId) { this.studentId = studentId; }

    public String getMajor() { return major; }
    public void setMajor(String major) { this.major = major; }

    public double getGpa() { return gpa; }
    public void setGpa(double gpa) { this.gpa = gpa; }

    @Override
    public void displayInfo() {
        System.out.println("=== STUDENT INFO ===");
        super.displayInfo();
        System.out.println("Student ID: " + studentId);
        System.out.println("Major: " + major);
        System.out.println("GPA: " + gpa);
    }
}

/**
 * Teacher - Extends Person
 */
class Teacher extends Person {
    private String teacherId;
    private String department;
    private double salary;

    public Teacher() {
```

```
        super();
    }

    public Teacher(String name, int age, String email,
                   String teacherId, String department, double salary) {
        super(name, age, email);
        this.teacherId = teacherId;
        this.department = department;
        this.salary = salary;
    }

    // Getters and Setters
    public String getTeacherId() { return teacherId; }
    public void setTeacherId(String teacherId) { this.teacherId = teacherId; }

    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }

    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }

    @Override
    public void displayInfo() {
        System.out.println("=== TEACHER INFO ===");
        super.displayInfo();
        System.out.println("Teacher ID: " + teacherId);
        System.out.println("Department: " + department);
        System.out.println("Salary: $" + salary);
    }
}

/**
 * Staff - Extends Person
 */
class Staff extends Person {
    private String staffId;
    private String position;
    private String department;

    public Staff() {
        super();
    }

    public Staff(String name, int age, String email,
                 String staffId, String position, String department) {
        super(name, age, email);
        this.staffId = staffId;
        this.position = position;
        this.department = department;
    }

    // Getters and Setters
    public String getStaffId() { return staffId; }
```

```
    public void setStaffId(String staffId) { this.staffId = staffId; }

    public String getPosition() { return position; }
    public void setPosition(String position) { this.position = position; }

    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }

    @Override
    public void displayInfo() {
        System.out.println("=== STAFF INFO ===");
        super.displayInfo();
        System.out.println("Staff ID: " + staffId);
        System.out.println("Position: " + position);
        System.out.println("Department: " + department);
    }
}

/**
 * UniversityDemo - Main demonstration
 */
public class UniversityDemo {
    public static void main(String[] args) {
        System.out.println("=== UNIVERSITY SYSTEM DEMO ===\n");

        Student student = new Student("Alice", 20, "alice@uni.edu",
                                      "S001", "Computer Science", 3.8);
        Teacher teacher = new Teacher("Dr. Smith", 45, "smith@uni.edu",
                                     "T001", "Computer Science", 80000);
        Staff staff = new Staff("Bob", 35, "bob@uni.edu",
                               "ST001", "Administrative", "Registrar");

        student.displayInfo();
        System.out.println();
        teacher.displayInfo();
        System.out.println();
        staff.displayInfo();
    }
}
```

Solution 3: E-commerce Product Hierarchy

```
/**
 * Product - Base class for all products
 * This maps directly to what Angular would receive via REST API
 */
class Product {
    protected int id;
    protected String name;
    protected double price;
```

```
    protected String category;

    public Product() {
    }

    public Product(int id, String name, double price, String category) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.category = category;
    }

    // Getters and Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }

    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }

    // Simulates JSON output for Angular
    @Override
    public String toString() {
        return "{" +
                "\"id\":" + id +
                ", \"name\":\"" + name + "\"" +
                ", \"price\":" + price +
                ", \"category\":\"" + category + "\"" +
                "}";
    }

    public void displayInfo() {
        System.out.println("ID: " + id + ", Name: " + name +
                           ", Price: $" + price + ", Category: " + category);
    }
}

/**
 * Electronics - Extends Product
 */
class Electronics extends Product {
    private String brand;
    private int warrantyMonths;

    public Electronics() {
        super();
    }
```

```
    public Electronics(int id, String name, double price, String category,
                       String brand, int warrantyMonths) {
        super(id, name, price, category);
        this.brand = brand;
        this.warrantyMonths = warrantyMonths;
    }

    public String getBrand() { return brand; }
    public void setBrand(String brand) { this.brand = brand; }

    public int getWarrantyMonths() { return warrantyMonths; }
    public void setWarrantyMonths(int warrantyMonths) { this.warrantyMonths = warrantyMonths; }

    @Override
    public String toString() {
        return "{" +
                super.toString().substring(0, super.toString().length() - 1) +
                ", \"brand\":\"" + brand + "\"" +
                ", \"warrantyMonths\":" + warrantyMonths +
                "}";
    }

    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Brand: " + brand + ", Warranty: " + warrantyMonths + " months");
    }
}

/**
 * Clothing - Extends Product
 */
class Clothing extends Product {
    private String size;
    private String color;
    private String material;

    public Clothing() {
        super();
    }

    public Clothing(int id, String name, double price, String category,
                    String size, String color, String material) {
        super(id, name, price, category);
        this.size = size;
        this.color = color;
        this.material = material;
    }

    public String getSize() { return size; }
    public void setSize(String size) { this.size = size; }

    public String getColor() { return color; }
```

```
    public void setColor(String color) { this.color = color; }

    public String getMaterial() { return material; }
    public void setMaterial(String material) { this.material = material; }

    @Override
    public String toString() {
        return "{" +
                super.toString().substring(0, super.toString().length() - 1) +
                ", \"size\":\"" + size + "\"" +
                ", \"color\":\"" + color + "\"" +
                ", \"material\":\"" + material + "\"" +
                "}";
    }

    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Size: " + size + ", Color: " + color + ", Material: " + material);
    }
}

/**
 * Book - Extends Product
 */
class Book extends Product {
    private String author;
    private String isbn;
    private int pages;

    public Book() {
        super();
    }

    public Book(int id, String name, double price, String category,
                String author, String isbn, int pages) {
        super(id, name, price, category);
        this.author = author;
        this.isbn = isbn;
        this.pages = pages;
    }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public String getIsbn() { return isbn; }
    public void setIsbn(String isbn) { this.isbn = isbn; }

    public int getPages() { return pages; }
    public void setPages(int pages) { this.pages = pages; }

    @Override
    public String toString() {
```

```
        return "{" +
                super.toString().substring(0, super.toString().length() - 1) +
                ", \"author\":\"" + author + "\"" +
                ", \"isbn\":\"" + isbn + "\"" +
                ", \"pages\":" + pages +
                "}";
    }

    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Author: " + author + ", ISBN: " + isbn + ", Pages: " + pages);
    }
}

/**
 * EcommerceDemo - Main demonstration
 */
public class EcommerceDemo {
    public static void main(String[] args) {
        System.out.println("=== E-COMMERCE PRODUCT HIERARCHY ===\n");

        // Create products
        Electronics laptop = new Electronics(1, "MacBook Pro", 2499.99,
                                             "Electronics", "Apple", 24);
        Clothing tshirt = new Clothing(2, "T-Shirt", 29.99, "Clothing",
                                       "M", "Blue", "Cotton");
        Book book = new Book(3, "Java Programming", 49.99, "Books",
                             "John Doe", "978-0134685991", 1200);

        // Display info
        System.out.println("--- ELECTRONICS ---");
        laptop.displayInfo();
        System.out.println("JSON: " + laptop);

        System.out.println("\n--- CLOTHING ---");
        tshirt.displayInfo();
        System.out.println("JSON: " + tshirt);

        System.out.println("\n--- BOOK ---");
        book.displayInfo();
        System.out.println("JSON: " + book);

        // Polymorphism: Process all products uniformly
        System.out.println("\n=== POLYMORPHISM: ALL PRODUCTS ===");
        Product[] products = {laptop, tshirt, book};

        double totalValue = 0;
        for (Product p : products) {
            System.out.println(p.getName() + ": $" + p.getPrice());
            totalValue += p.getPrice();
        }
        System.out.println("Total Inventory Value: $" + totalValue);
```

```
    }
}
```

Summary

Key Takeaways

1. **Inheritance promotes reusability** - Child classes inherit parent class members
2. `extends` **keyword** - Used to create inheritance relationship
3. `super` **keyword** - Used to access parent class members
4. **Method Overriding** - Child class provides specific implementation
5. **Polymorphism** - Objects of different types can be treated uniformly
6. **Method Overloading** - Same method name, different parameters

Angular Backend Integration Tips

- Java classes map to JSON via serialization (Jackson library)
- Inheritance hierarchies become nested JSON objects
- Use `@JsonIgnoreProperties` to control JSON output
- REST APIs often return polymorphic responses

Next Steps

Continue learning about:

- **Abstraction** - Abstract classes and interfaces
- **Encapsulation** - Data hiding best practices
- **Generics** - Type-safe collections

Happy Coding! ■

Code Examples

AngularServiceDemo.java

AngularServiceDemo.java

```
// AngularServiceDemo - Demonstrates Inheritance for Angular Services
// Use case: Service hierarchy with inheritance

// Base Service
class BaseService {
```

```
    protected String baseUrl;
    protected int timeout;

    public BaseService() {
        this.baseUrl = "https://api.example.com";
        this.timeout = 30000;
    }

    public String getBaseUrl() {
        return baseUrl;
    }

    public int getTimeout() {
        return timeout;
    }

    // Common method for all services
    public void logRequest(String endpoint) {
        System.out.println("Request to: " + baseUrl + endpoint);
    }
}

// UserService inherits from BaseService
class UserService extends BaseService {
    private String[] userEndpoints;

    public UserService() {
        super(); // Call parent constructor
        this.userEndpoints = new String[] {"/users", "/users/:id", "/users/profile"};
    }

    public String getAllUsers() {
        logRequest("/users");
        return "GET /users - Returns all users";
    }

    public String getUserById(int id) {
        logRequest("/users/" + id);
        return "GET /users/" + id + " - Returns user " + id;
    }

    public String createUser(String userData) {
        logRequest("/users");
        return "POST /users - Creates user: " + userData;
    }
}

// AdminService extends UserService with more functionality
class AdminService extends UserService {
    private boolean isAdminMode;

    public AdminService() {
        super(); // Call parent constructor
```

```
        this.isAdminMode = true;
    }

    // Override parent method
    @Override
    public String getAllUsers() {
        logRequest("/admin/users");
        return "GET /admin/users - Returns all users (admin view)";
    }

    public String deleteUser(int id) {
        logRequest("/admin/users/" + id);
        return "DELETE /admin/users/" + id + " - Deletes user " + id;
    }

    public String getSystemStats() {
        return "System stats: 1000 users, 50 admins, 10K requests";
    }
}

public class AngularServiceDemo {
    public static void main(String[] args) {
        System.out.println("=== INHERITANCE FOR ANGULAR SERVICES ===\n");

        // Base service
        BaseService base = new BaseService();
        System.out.println("Base URL: " + base.getBaseUrl());

        // User service
        UserService userService = new UserService();
        System.out.println("\n--- UserService ---");
        System.out.println(userService.getAllUsers());
        System.out.println(userService.getUserById(1));

        // Admin service - inherits from UserService
        AdminService adminService = new AdminService();
        System.out.println("\n--- AdminService ---");
        System.out.println(adminService.getAllUsers()); // Overridden
        System.out.println(adminService.getUserById(1)); // Inherited
        System.out.println(adminService.deleteUser(5));
        System.out.println(adminService.getSystemStats());

        // Polymorphism
        System.out.println("\n--- Polymorphism ---");
        BaseService service = new AdminService();
        service.logRequest("/users"); // Calls overridden method

        System.out.println("\n=== USE CASES ===");
        System.out.println("1. BaseService -> HttpService");
        System.out.println("2. UserService extends BaseService");
        System.out.println("3. AdminService extends UserService");
        System.out.println("4. Override methods for custom behavior");
    }
```

```
}
```

InheritanceDemo.java

InheritanceDemo.java

```
// EmployeeInheritanceDemo - Demonstrates inheritance and polymorphism
// Contains both parent and child classes in one file for easy compilation

// ===== PARENT CLASS =====
class Person {
    protected String name;
    protected int age;
    protected String address;
    protected String phoneNumber;

    public Person(String name, int age, String address, String phoneNumber) {
        this.name = name;
        this.age = age;
        this.address = address;
        this.phoneNumber = phoneNumber;
    }

    public Person() {
        this.name = "Unknown";
        this.age = 0;
        this.address = "";
        this.phoneNumber = "";
    }

    public String getName() { return name; }
    public int getAge() { return age; }
    public String getAddress() { return address; }
    public String getPhoneNumber() { return phoneNumber; }

    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }
    public void setAddress(String address) { this.address = address; }
    public void setPhoneNumber(String phoneNumber) { this.phoneNumber = phoneNumber; }

    public void displayInfo() {
        System.out.println("Name: " + name + ", Age: " + age +
                           ", Address: " + address + ", Phone: " + phoneNumber);
    }

    public void introduce() {
        System.out.println("Hello, I'm " + name + " and I'm " + age + " years old.");
    }
}

// ===== CHILD CLASS 1: Employee =====
class Employee extends Person {
```

```
    private int employeeId;
    private String department;
    private String jobTitle;
    private double salary;

    public Employee(String name, int age, String address, String phoneNumber,
                    int employeeId, String department, String jobTitle, double salary) {
        super(name, age, address, phoneNumber);
        this.employeeId = employeeId;
        this.department = department;
        this.jobTitle = jobTitle;
        this.salary = salary;
    }

    public int getEmployeeId() { return employeeId; }
    public String getDepartment() { return department; }
    public String getJobTitle() { return jobTitle; }
    public double getSalary() { return salary; }

    public double calculateAnnualSalary() {
        return salary * 12;
    }

    // Method overriding - demonstrates polymorphism
    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("  -> Employee ID: " + employeeId + ", Dept: " +
                           department + ", Title: " + jobTitle + ", Salary: $" + salary);
    }

    @Override
    public void introduce() {
        System.out.println("Hi, I'm " + name + ", a " + jobTitle + " in " + department);
    }
}

// ===== CHILD CLASS 2: Student =====
class Student extends Person {
    private int studentId;
    private String major;
    private double gpa;

    public Student(String name, int age, String address, String phoneNumber,
                   int studentId, String major, double gpa) {
        super(name, age, address, phoneNumber);
        this.studentId = studentId;
        this.major = major;
        this.gpa = gpa;
    }

    public int getStudentId() { return studentId; }
    public String getMajor() { return major; }
```

```
    public double getGpa() { return gpa; }

    public boolean isHonorStudent() {
        return gpa >= 3.5;
    }

    // Method overriding
    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("  -> Student ID: " + studentId + ", Major: " +
                           major + ", GPA: " + gpa);
    }

    @Override
    public void introduce() {
        System.out.println("Hey, I'm " + name + ", studying " + major + " at university");
    }
}

// ===== MAIN CLASS =====
public class InheritanceDemo {
    public static void main(String[] args) {
        System.out.println("=== INHERITANCE & POLYMORPHISM DEMO ===\n");

        // Create Employee object
        Employee emp = new Employee("Alice Smith", 28, "456 Oak Ave", "555-5678",
                                    1001, "Engineering", "Software Engineer", 7500.00);
        System.out.println("--- Employee ---");
        emp.displayInfo();
        emp.introduce();
        System.out.println("Annual Salary: $" + emp.calculateAnnualSalary());

        // Create Student object
        Student student = new Student("Bob Johnson", 20, "789 Pine St", "555-9999",
                                      2001, "Computer Science", 3.8);
        System.out.println("\n--- Student ---");
        student.displayInfo();
        student.introduce();
        System.out.println("Honor Student: " + student.isHonorStudent());

        // Demonstrate polymorphism - same method, different behavior
        System.out.println("\n=== POLYMORPHISM DEMO ===");
        Person[] people = {emp, student};
        for (Person p : people) {
            p.introduce();  // Different output based on actual type
        }
    }
}
```

Person.java

Person.java

```
// Person - Base class demonstrating inheritance
// This is the parent/superclass that other classes will inherit from

public class Person {
    // Protected fields - accessible by subclasses
    protected String name;
    protected int age;
    protected String address;
    protected String phoneNumber;

    // Constructor
    public Person(String name, int age, String address, String phoneNumber) {
        this.name = name;
        this.age = age;
        this.address = address;
        this.phoneNumber = phoneNumber;
    }

    // Default constructor
    public Person() {
        this.name = "Unknown";
        this.age = 0;
        this.address = "";
        this.phoneNumber = "";
    }

    // Getters
    public String getName() { return name; }
    public int getAge() { return age; }
    public String getAddress() { return address; }
    public String getPhoneNumber() { return phoneNumber; }

    // Setters
    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }
    public void setAddress(String address) { this.address = address; }
    public void setPhoneNumber(String phoneNumber) { this.phoneNumber = phoneNumber; }

    // Common method - can be overridden
    public void displayInfo() {
        System.out.println("=== PERSON INFORMATION ===");
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Address: " + address);
        System.out.println("Phone: " + phoneNumber);
    }

    // Common behavior
    public void introduce() {
        System.out.println("Hello, my name is " + name + " and I am " + age + " years old.");
    }
```

```
    // Main for testing
    public static void main(String[] args) {
        System.out.println("=== PERSON CLASS DEMO ===\n");

        Person person = new Person("John Doe", 30, "123 Main St", "555-1234");
        person.displayInfo();
        person.introduce();
    }
}
```

15

Encapsulation Abstraction

15_Encapsulation_Abstraction\examples\15_Encapsulation_Abstraction.md

Java Encapsulation and Abstraction

Table of Contents

1 Encapsulation
2 Access Modifiers
3 Getters and Setters
4 Data Validation
5 Immutability
6 Abstraction
7 Abstract Classes
8 Interfaces
9 Code Examples
10 Exercises
11 Solutions

---

1. Encapsulation

What is Encapsulation?

Encapsulation is the bundling of data (fields) and methods that operate on that data into a single unit (class). It restricts direct access to some of an object's components to prevent accidental interference and misuse.

Benefits of Encapsulation

```
■■■
```

```
■   ■                                          ■           ■
■   ■  PRIVATE ■■■■■■■ PUBLIC API                ■           ■
■   ■  (Hidden)      (Controlled Access)         ■           ■
■   ■■■■■■■
```

```
account.balance = -1000000;  // Invalid! But allowed

/**
 * Example: With encapsulation (GOOD)
 */
class BankAccountGood {
    private double balance;  // Private - protected

    // Getter - read-only access
    public double getBalance() {
        return balance;
    }

    // Setter - with validation
    public void setBalance(double balance) {
        if (balance >= 0) {
            this.balance = balance;
        } else {
            System.out.println("Invalid balance!");
        }
    }
}
```

---

4. Data Validation

Validation in Setters

```
/**
 * Validation in setters prevents invalid data
 */
class Employee {
    private String name;
    private int age;
    private double salary;

    public void setName(String name) {
        if (name != null && !name.trim().isEmpty()) {
            this.name = name;
        } else {
            throw new IllegalArgumentException("Name cannot be empty");
        }
    }

    public void setAge(int age) {
        if (age >= 0 && age <= 150) {
            this.age = age;
        } else {
            throw new IllegalArgumentException("Invalid age");
```

```
        }
    }

    public void setSalary(double salary) {
        if (salary >= 0) {
            this.salary = salary;
        } else {
            throw new IllegalArgumentException("Salary cannot be negative");
        }
    }
}
```

5. Immutability

Creating Immutable Classes

```
/**
 * Immutable class - cannot be modified after creation
 */
public final class Person {
    private final String name;
    private final int age;
    private final String email;

    public Person(String name, int age, String email) {
        this.name = name;
        this.age = age;
        this.email = email;
    }

    // Only getters - no setters!
    public String getName() { return name; }
    public int getAge() { return age; }
    public String getEmail() { return email; }

    // To "modify", create a new object
    public Person withName(String newName) {
        return new Person(newName, this.age, this.email);
    }
}
```

6. Abstraction

What is Abstraction?

Abstraction hides implementation details and shows only functionality to users.

```
■■■
```

An **interface** is a contract that defines what a class must do but not how.

```
/**
 * Interface definition
 */
interface Drawable {
    void draw();  // Abstract method - no implementation

    // Java 8+ allows default methods
    default void display() {
        System.out.println("Displaying...");
    }

    // Static method
    static void printInfo() {
        System.out.println("Drawable interface");
    }
}

/**
 * Class implements interface
 */
class Circle implements Drawable {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public void draw() {
        System.out.println("Drawing circle with radius " + radius);
    }
}
```

Interface vs Abstract Class

Feature	Abstract Class	Interface
Methods	Abstract + concrete	Abstract + default (Java 8+)
Fields	Can have any type	Public static final only
Constructor	Can have	Cannot have
Inheritance	extends (one)	implements (multiple)
Access modifiers	Any	Public only

9. Code Examples

Example 1: Bank Account with Full Encapsulation

```
/**
 * BankAccount - Fully encapsulated class
 * Demonstrates data protection and validation
 */
public class BankAccount {
    // Private fields - hidden from outside
    private String accountNumber;
    private String holderName;
    private double balance;
    private String accountType;
    private boolean isActive;

    // ==================== CONSTRUCTORS ====================

    public BankAccount() {
        this.accountNumber = "000000";
        this.holderName = "Unknown";
        this.balance = 0.0;
        this.accountType = "Basic";
        this.isActive = true;
    }

    public BankAccount(String accountNumber, String holderName,
                       double balance, String accountType) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
        this.accountType = accountType;
        this.isActive = true;
    }

    // ==================== GETTERS ====================

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getHolderName() {
        return holderName;
    }

    public double getBalance() {
        return balance;
    }

    public String getAccountType() {
        return accountType;
```

```
    }

    public boolean isActive() {
        return isActive;
    }

    // ==================== SETTERS WITH VALIDATION ====================

    public void setHolderName(String holderName) {
        if (holderName != null && !holderName.trim().isEmpty()) {
            this.holderName = holderName;
        } else {
            System.out.println("Error: Invalid holder name!");
        }
    }

    public void setAccountType(String accountType) {
        if (accountType != null &&
           (accountType.equals("Basic") || accountType.equals("Premium"))) {
            this.accountType = accountType;
        } else {
            System.out.println("Error: Invalid account type!");
        }
    }

    // ==================== BUSINESS METHODS ====================

    public void deposit(double amount) {
        if (amount > 0 && isActive) {
            balance += amount;
            System.out.println("Deposited: $" + amount);
        } else {
            System.out.println("Error: Cannot deposit!");
        }
    }

    public boolean withdraw(double amount) {
        if (amount > 0 && amount <= balance && isActive) {
            balance -= amount;
            System.out.println("Withdrawn: $" + amount);
            return true;
        } else {
            System.out.println("Error: Cannot withdraw!");
            return false;
        }
    }

    public void displayInfo() {
        System.out.println("╔═══════════════════════════════════════════╗");
        System.out.println("║      BANK ACCOUNT INFO             ║");
        System.out.println("╠═══════════════════════════════════════════╣");
        System.out.println("║ Account #: " + accountNumber);
        System.out.println("║ Holder:   " + holderName);
```

```
        System.out.println("■ Balance:  $" + balance);
        System.out.println("■ Type:     " + accountType);
        System.out.println("■ Status:   " + (isActive ? "Active" : "Inactive"));
        System.out.println("■■■
```

```
    public Shape(String color) {
        this.color = color;
    }

    // Concrete method
    public String getColor() {
        return color;
    }

    public void setColor(String color) {
        this.color = color;
    }

    // Abstract methods - no implementation
    public abstract double getArea();
    public abstract double getPerimeter();

    // Concrete method using abstract methods
    public void displayDetails() {
        System.out.println("Color: " + color);
        System.out.println("Area: " + getArea());
        System.out.println("Perimeter: " + getPerimeter());
    }
}

/**
 * Circle - Concrete implementation
 */
class Circle extends Shape {
    private double radius;

    public Circle(double radius, String color) {
        super(color);
        this.radius = radius;
    }

    public double getRadius() { return radius; }
    public void setRadius(double radius) { this.radius = radius; }

    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }

    @Override
    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }
}

/**
 * Rectangle - Concrete implementation
 */
```

```
class Rectangle extends Shape {
    private double width;
    private double height;

    public Rectangle(double width, double height, String color) {
        super(color);
        this.width = width;
        this.height = height;
    }

    public double getWidth() { return width; }
    public void setWidth(double width) { this.width = width; }
    public double getHeight() { return height; }
    public void setHeight(double height) { this.height = height; }

    @Override
    public double getArea() {
        return width * height;
    }

    @Override
    public double getPerimeter() {
        return 2 * (width + height);
    }
}

/**
 * Triangle - Concrete implementation
 */
class Triangle extends Shape {
    private double side1, side2, side3;

    public Triangle(double side1, double side2, double side3, String color) {
        super(color);
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
    }

    @Override
    public double getArea() {
        double s = (side1 + side2 + side3) / 2;
        return Math.sqrt(s * (s - side1) * (s - side2) * (s - side3));
    }

    @Override
    public double getPerimeter() {
        return side1 + side2 + side3;
    }
}

/**
 * ShapeDemo - Test abstraction
```

```
 */
public class ShapeDemo {
    public static void main(String[] args) {
        System.out.println("=== ABSTRACTION DEMO: SHAPES ===\n");

        // Cannot instantiate abstract class
        // Shape shape = new Shape("Red");  // ERROR!

        // Create concrete implementations
        Circle circle = new Circle(5, "Red");
        Rectangle rectangle = new Rectangle(4, 6, "Blue");
        Triangle triangle = new Triangle(3, 4, 5, "Green");

        // Polymorphism: treat all as Shape
        Shape[] shapes = {circle, rectangle, triangle};

        double totalArea = 0;
        for (Shape shape : shapes) {
            System.out.println("--- " + shape.getClass().getSimpleName() + " ---");
            shape.displayDetails();
            totalArea += shape.getArea();
            System.out.println();
        }

        System.out.println("Total Area: " + totalArea);
    }
}
```

Example 3: Interfaces for Payment System

```
/**
 * PaymentProcessor - Interface defining payment operations
 * This is common in Angular + Java backends
 */
interface PaymentProcessor {
    void processPayment(double amount);
    void refundPayment(double amount);
    boolean validateCard(String cardNumber);
}

/**
 * CreditCardPayment - Implements PaymentProcessor
 */
class CreditCardPayment implements PaymentProcessor {
    private String cardNumber;
    private String cardHolder;
    private double balance;

    public CreditCardPayment(String cardNumber, String cardHolder, double balance) {
        this.cardNumber = cardNumber;
```

```
        this.cardHolder = cardHolder;
        this.balance = balance;
    }

    @Override
    public void processPayment(double amount) {
        if (validateCard(cardNumber) && amount <= balance) {
            balance -= amount;
            System.out.println("Credit card payment of $" + amount + " processed");
        } else {
            System.out.println("Payment failed!");
        }
    }

    @Override
    public void refundPayment(double amount) {
        balance += amount;
        System.out.println("Refunded $" + amount + " to card");
    }

    @Override
    public boolean validateCard(String cardNumber) {
        return cardNumber != null && cardNumber.length() == 16;
    }

    public double getBalance() { return balance; }
}

/**
 * PayPalPayment - Another implementation
 */
class PayPalPayment implements PaymentProcessor {
    private String email;
    private double walletBalance;

    public PayPalPayment(String email, double walletBalance) {
        this.email = email;
        this.walletBalance = walletBalance;
    }

    @Override
    public void processPayment(double amount) {
        if (amount <= walletBalance) {
            walletBalance -= amount;
            System.out.println("PayPal payment of $" + amount + " processed");
        } else {
            System.out.println("Insufficient PayPal balance!");
        }
    }

    @Override
    public void refundPayment(double amount) {
        walletBalance += amount;
```

```
        System.out.println("Refunded $" + amount + " to PayPal");
    }

    @Override
    public boolean validateCard(String cardNumber) {
        // PayPal doesn't use card numbers directly
        return email != null && email.contains("@");
    }
}

/**
 * CryptoPayment - Cryptocurrency implementation
 */
class CryptoPayment implements PaymentProcessor {
    private String walletAddress;
    private double cryptoBalance;

    public CryptoPayment(String walletAddress, double cryptoBalance) {
        this.walletAddress = walletAddress;
        this.cryptoBalance = cryptoBalance;
    }

    @Override
    public void processPayment(double amount) {
        if (amount <= cryptoBalance) {
            cryptoBalance -= amount;
            System.out.println("Crypto payment of $" + amount + " processed");
        } else {
            System.out.println("Insufficient crypto balance!");
        }
    }

    @Override
    public void refundPayment(double amount) {
        cryptoBalance += amount;
        System.out.println("Refunded $" + amount + " to crypto wallet");
    }

    @Override
    public boolean validateCard(String cardNumber) {
        // Crypto wallets validated by address
        return walletAddress != null && walletAddress.length() > 10;
    }
}

/**
 * PaymentDemo - Test interface implementations
 */
public class PaymentDemo {
    public static void main(String[] args) {
        System.out.println("=== INTERFACE DEMO: PAYMENT PROCESSORS ===\n");

        // Different payment methods implementing same interface
```

```
        PaymentProcessor[] processors = {
            new CreditCardPayment("1234567890123456", "John Doe", 5000),
            new PayPalPayment("john@example.com", 1000),
            new CryptoPayment("0x1234567890abcdef", 10)
        };

        double paymentAmount = 100;

        // Polymorphism: same interface, different implementations
        for (PaymentProcessor processor : processors) {
            System.out.println("Processing payment with: " +
                               processor.getClass().getSimpleName());
            processor.processPayment(paymentAmount);
            System.out.println();
        }
    }
}
```

---

10. Exercises

Exercise 1: Student Class with Encapsulation

Requirements:

1. Create a `Student` class with private fields: id, name, age, gpa, email
2. Implement getters and setters with validation:
   - Name: not empty
   - Age: 0-150
   - GPA: 0.0-4.0
   - Email: must contain @
1. Create displayInfo() method

Exercise 2: Abstract Shape Calculator

Requirements:

1. Create abstract class `Shape3D` with abstract methods: getVolume(), getSurfaceArea()
2. Create `Sphere` class extending Shape3D
3. Create `Cube` class extending Shape3D
4. Create `Cylinder` class extending Shape3D
5. Test all with a main method

Exercise 3: Interface for Data Repository

Requirements:

1. Create `Repository<T>` interface with methods: save(T), findById(id), findAll(), delete(id)
2. Implement `UserRepository` implementing Repository for User objects
3. Create simple User class and test CRUD operations

---

11. Solutions

Solution 1: Encapsulated Student Class

```
/**
 * Student - Fully encapsulated class
 */
class Student {
    private int id;
    private String name;
    private int age;
    private double gpa;
    private String email;

    // Constructors
    public Student() {}

    public Student(int id, String name, int age, double gpa, String email) {
        this.id = id;
        this.setName(name);
        this.setAge(age);
        this.setGpa(gpa);
        this.setEmail(email);
    }

    // Getters
    public int getId() { return id; }
    public String getName() { return name; }
    public int getAge() { return age; }
    public double getGpa() { return gpa; }
    public String getEmail() { return email; }

    // Setters with validation
    public void setName(String name) {
        if (name != null && !name.trim().isEmpty()) {
            this.name = name;
        } else {
            throw new IllegalArgumentException("Name cannot be empty");
        }
    }

    public void setAge(int age) {
        if (age >= 0 && age <= 150) {
```

```
            this.age = age;
        } else {
            throw new IllegalArgumentException("Age must be between 0 and 150");
        }
    }

    public void setGpa(double gpa) {
        if (gpa >= 0.0 && gpa <= 4.0) {
            this.gpa = gpa;
        } else {
            throw new IllegalArgumentException("GPA must be between 0.0 and 4.0");
        }
    }

    public void setEmail(String email) {
        if (email != null && email.contains("@")) {
            this.email = email;
        } else {
            throw new IllegalArgumentException("Invalid email format");
        }
    }

    public void displayInfo() {
        System.out.println("ID: " + id + ", Name: " + name +
                           ", Age: " + age + ", GPA: " + gpa + ", Email: " + email);
    }
}

public class StudentEncapsulationDemo {
    public static void main(String[] args) {
        Student s = new Student(1, "Alice", 20, 3.8, "alice@email.com");
        s.displayInfo();

        // Test validation
        try {
            s.setGpa(5.0);  // Invalid
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Solution 2: Abstract Shape Classes

```
abstract class Shape3D {
    protected String name;

    public Shape3D(String name) {
        this.name = name;
    }
```

```
    public abstract double getVolume();
    public abstract double getSurfaceArea();

    public void display() {
        System.out.println(name + " - Volume: " + getVolume() +
                           ", Surface Area: " + getSurfaceArea());
    }
}

class Sphere extends Shape3D {
    private double radius;

    public Sphere(double radius) {
        super("Sphere");
        this.radius = radius;
    }

    @Override
    public double getVolume() {
        return (4.0/3) * Math.PI * Math.pow(radius, 3);
    }

    @Override
    public double getSurfaceArea() {
        return 4 * Math.PI * radius * radius;
    }
}

class Cube extends Shape3D {
    private double side;

    public Cube(double side) {
        super("Cube");
        this.side = side;
    }

    @Override
    public double getVolume() {
        return Math.pow(side, 3);
    }

    @Override
    public double getSurfaceArea() {
        return 6 * side * side;
    }
}

class Cylinder extends Shape3D {
    private double radius;
    private double height;

    public Cylinder(double radius, double height) {
```

```
        super("Cylinder");
        this.radius = radius;
        this.height = height;
    }

    @Override
    public double getVolume() {
        return Math.PI * radius * radius * height;
    }

    @Override
    public double getSurfaceArea() {
        return 2 * Math.PI * radius * (radius + height);
    }
}

public class Shape3DDemo {
    public static void main(String[] args) {
        Shape3D[] shapes = {
            new Sphere(5),
            new Cube(4),
            new Cylinder(3, 7)
        };

        for (Shape3D shape : shapes) {
            shape.display();
        }
    }
}
```

Solution 3: Repository Interface

```
/**
 * Generic Repository interface
 */
interface Repository<T> {
    void save(T entity);
    T findById(int id);
    java.util.List<T> findAll();
    void delete(int id);
}

/**
 * User class
 */
class User {
    private int id;
    private String name;
    private String email;
```

```
    public User(int id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }

    @Override
    public String toString() {
        return "User{id=" + id + ", name='" + name + "', email='" + email + "'}";
    }
}

/**
 * UserRepository implementation
 */
class UserRepository implements Repository<User> {
    private java.util.Map<Integer, User> database = new java.util.HashMap<>();

    @Override
    public void save(User user) {
        database.put(user.getId(), user);
        System.out.println("Saved: " + user);
    }

    @Override
    public User findById(int id) {
        return database.get(id);
    }

    @Override
    public java.util.List<User> findAll() {
        return new java.util.ArrayList<>(database.values());
    }

    @Override
    public void delete(int id) {
        database.remove(id);
        System.out.println("Deleted user with id: " + id);
    }
}

public class RepositoryDemo {
    public static void main(String[] args) {
        UserRepository repo = new UserRepository();

        // CRUD Operations
        repo.save(new User(1, "Alice", "alice@email.com"));
        repo.save(new User(2, "Bob", "bob@email.com"));
```

```
        System.out.println("All users: " + repo.findAll());
        System.out.println("Find by id 1: " + repo.findById(1));

        repo.delete(1);
        System.out.println("After delete: " + repo.findAll());
    }
}
```

Summary

Key Takeaways

1. **Encapsulation** protects data using private fields + public methods
2. **Access modifiers** control visibility (private, protected, public)
3. **Getters/Setters** provide controlled access to fields
4. **Validation** in setters prevents invalid data
5. **Immutable classes** cannot be modified after creation
6. **Abstraction** hides complexity using abstract classes/interfaces
7. **Interfaces** define contracts (multiple inheritance possible)

Angular Backend Connection

- Java entities use encapsulation for data protection
- Abstract classes and interfaces model service contracts
- REST APIs often return abstract types as JSON
- Spring Data uses repository interfaces

Next Topics

- Control Flow Statements
- Strings and String Handling
- Arrays

Happy Coding! ■

Code Examples

EncapsulationDemo.java

EncapsulationDemo.java

```
// EncapsulationDemo - Demonstrates Encapsulation and Abstraction
// Encapsulation: hiding data and methods | Abstraction: hiding complexity

public class EncapsulationDemo {

    // ===== ENCAPSULATION =====
    // Private fields - hidden from outside
    private String name;
    private int age;
    private double salary;

    // Public getters and setters - controlled access
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getAge() { return age; }
    public void setAge(int age) {
        if (age > 0 && age < 150) {
            this.age = age;
        }
    }

    public double getSalary() { return salary; }
    public void setSalary(double salary) {
        if (salary > 0) {
            this.salary = salary;
        }
    }

    // ===== ABSTRACTION =====
    // Abstract concept - user doesn't know internal details
    public void displayInfo() {
        System.out.println("Name: " + name + ", Age: " + age);
    }

    public static void main(String[] args) {
        System.out.println("=== ENCAPSULATION & ABSTRACTION ===\n");

        EncapsulationDemo emp = new EncapsulationDemo();

        // Set values through setters (controlled access)
        emp.setName("John Doe");
        emp.setAge(30);
        emp.setSalary(50000);

        // Get values through getters
        System.out.println("Name: " + emp.getName());
        System.out.println("Age: " + emp.getAge());
        System.out.println("Salary: $" + emp.getSalary());

        // Validation in setter
        System.out.println("\n--- Validation ---");
```

```
        emp.setAge(-5);  // Won't set due to validation
        System.out.println("After invalid age: " + emp.getAge());

        System.out.println("\n--- Benefits ---");
        System.out.println("1. Data hiding - private fields");
        System.out.println("2. Validation - in setters");
        System.out.println("3. Flexibility - can change internal code");
        System.out.println("4. Reusability - reusable components");

        System.out.println("\n--- Angular Note ---");
        System.out.println("Java classes map to TypeScript interfaces/classes");
        System.out.println("Encapsulation in Java = private fields + getters");
    }
}
```

16

Control Flow

16_Control_Flow\examples\16_Control_Flow.md

Java Control Flow Statements

Table of Contents

1 Introduction
2 Decision Making Statements
3 Loops
4 Jump Statements
5 Switch Expressions (Java 14+)
6 Code Examples
7 Exercises
8 Solutions

1. Introduction

What are Control Flow Statements?

Control flow statements control the order in which statements are executed in a program. They allow you to make decisions, repeat actions, and transfer control.

```
■■■
```

```
■   ■■■
```

```
    default:
        // default code
}
```

3. Loops

3.1 for Loop

```
// Traditional for loop
for (int i = 0; i < 10; i++) {
    System.out.println(i);
}

// For loop with multiple variables
for (int i = 0, j = 10; i < j; i++, j--) {
    System.out.println(i + " " + j);
}
```

3.2 Enhanced for-each Loop

```
// For-each loop (used with arrays and collections)
int[] numbers = {1, 2, 3, 4, 5};

for (int num : numbers) {
    System.out.println(num);
}
```

3.3 while Loop

```
// While loop - checks condition first
while (condition) {
    // code to repeat
}
```

3.4 do-while Loop

```
// do-while - executes at least once
do {
    // code to repeat
} while (condition);
```

4. Jump Statements

4.1 break Statement

- Used to exit from a loop or switch statement
- Terminates the nearest enclosing loop or switch

4.2 continue Statement

- Skips the current iteration of a loop
- Continues with the next iteration

4.3 return Statement

- Exits from a method
- Can return a value

---

5. Switch Expressions (Java 14+)

Traditional vs Modern Switch

```
// Traditional switch (old way)
int day = 3;
String dayName;

switch (day) {
    case 1:
        dayName = "Monday";
        break;
    case 2:
        dayName = "Tuesday";
        break;
    default:
        dayName = "Unknown";
}

// Modern switch expression (Java 14+)
String dayName = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    case 3 -> "Wednesday";
    default -> "Unknown";
};

// With multiple values
```

```
String weekend = switch (day) {
    case 1, 2, 3, 4, 5 -> "Weekday";
    case 6, 7 -> "Weekend";
    default -> "Invalid";
};
```

6. Code Examples

Example 1: Grade Calculator with if-else

```
import java.util.Scanner;

/**
 * GradeCalculator - Demonstrates if-else and switch
 * Shows how to handle different grade scenarios
 */
public class GradeCalculator {

    /**
     * Calculate grade using if-else
     */
    public static String getGradeIfElse(int score) {
        if (score >= 90 && score <= 100) {
            return "A - Excellent";
        } else if (score >= 80 && score < 90) {
            return "B - Good";
        } else if (score >= 70 && score < 80) {
            return "C - Average";
        } else if (score >= 60 && score < 70) {
            return "D - Below Average";
        } else if (score >= 0 && score < 60) {
            return "F - Fail";
        } else {
            return "Invalid score";
        }
    }

    /**
     * Calculate grade using switch expression (Java 14+)
     */
    public static String getGradeSwitch(int score) {
        // Convert score to grade range
        int gradeLevel = score / 10;

        return switch (gradeLevel) {
            case 10, 9 -> "A - Excellent";
            case 8 -> "B - Good";
            case 7 -> "C - Average";
```

```
            case 6 -> "D - Below Average";
            case 5, 4, 3, 2, 1, 0 -> "F - Fail";
            default -> "Invalid score";
        };
    }

    public static void main(String[] args) {
        System.out.println("=== GRADE CALCULATOR ===\n");

        // Test with different scores
        int[] testScores = {95, 87, 78, 65, 55, 105, -5};

        for (int score : testScores) {
            System.out.println("Score: " + score + " -> " + getGradeIfElse(score));
        }

        System.out.println("\n--- Using Switch Expression ---");
        for (int score : testScores) {
            System.out.println("Score: " + score + " -> " + getGradeSwitch(score));
        }
    }
}
```

Example 2: Number Patterns with Loops

```
/**
 * PatternPrinter - Demonstrates various loop patterns
 * Useful for understanding loop control
 */
public class PatternPrinter {

    /**
     * Print a simple triangle pattern
     * *
     * **
     * ***
     * ****
     * *****
     */
    public static void printTriangle(int rows) {
        System.out.println("=== Triangle Pattern ===");
        for (int i = 1; i <= rows; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print("* ");
            }
            System.out.println();
        }
    }

    /**
```

```
     * Print inverted triangle
     * *****
     * ****
     * ***
     * **
     * *
     */
    public static void printInvertedTriangle(int rows) {
        System.out.println("\n=== Inverted Triangle ===");
        for (int i = rows; i >= 1; i--) {
            for (int j = 1; j <= i; j++) {
                System.out.print("* ");
            }
            System.out.println();
        }
    }

    /**
     * Print number pyramid
     *     1
     *    2 2
     *   3 3 3
     *  4 4 4 4
     * 5 5 5 5 5
     */
    public static void printNumberPyramid(int rows) {
        System.out.println("\n=== Number Pyramid ===");
        for (int i = 1; i <= rows; i++) {
            // Print spaces
            for (int s = 1; s <= rows - i; s++) {
                System.out.print("  ");
            }
            // Print numbers
            for (int j = 1; j <= i; j++) {
                System.out.print(i + " ");
            }
            System.out.println();
        }
    }

    /**
     * Calculate factorial using while loop
     */
    public static long factorial(int number) {
        if (number < 0) {
            throw new IllegalArgumentException("Number must be non-negative");
        }

        long result = 1;
        int counter = number;

        while (counter > 0) {
            result *= counter;
```

```
            counter--;
        }

        return result;
    }

    /**
     * Calculate factorial using for loop
     */
    public static long factorialFor(int number) {
        if (number < 0) {
            throw new IllegalArgumentException("Number must be non-negative");
        }

        long result = 1;
        for (int i = number; i > 0; i--) {
            result *= i;
        }

        return result;
    }

    /**
     * Print Fibonacci series
     */
    public static void printFibonacci(int count) {
        System.out.println("\n=== Fibonacci Series (" + count + " terms) ===");
        int first = 0, second = 1;

        System.out.print(first + " " + second);

        for (int i = 2; i < count; i++) {
            int next = first + second;
            System.out.print(" " + next);
            first = second;
            second = next;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // Print patterns
        printTriangle(5);
        printInvertedTriangle(5);
        printNumberPyramid(5);

        // Calculate factorials
        System.out.println("\n=== Factorial Calculations ===");
        for (int i = 0; i <= 10; i++) {
            System.out.println(i + "! = " + factorial(i));
        }

        // Print Fibonacci
```

```
        printFibonacci(10);
    }
}
```

Example 3: Menu-Driven Calculator with Switch

```
import java.util.Scanner;

/**
 * Calculator - Menu-driven calculator using switch
 * Demonstrates switch statement and loop control
 */
public class Calculator {

    /**
     * Add two numbers
     */
    public static double add(double a, double b) {
        return a + b;
    }

    /**
     * Subtract two numbers
     */
    public static double subtract(double a, double b) {
        return a - b;
    }

    /**
     * Multiply two numbers
     */
    public static double multiply(double a, double b) {
        return a * b;
    }

    /**
     * Divide two numbers
     */
    public static double divide(double a, double b) {
        if (b == 0) {
            System.out.println("Error: Division by zero!");
            return Double.NaN;
        }
        return a / b;
    }

    /**
     * Calculate power
     */
    public static double power(double base, double exponent) {
```

```
        return Math.pow(base, exponent);
    }

    /**
     * Calculate square root
     */
    public static double squareRoot(double number) {
        if (number < 0) {
            System.out.println("Error: Cannot calculate square root of negative!");
            return Double.NaN;
        }
        return Math.sqrt(number);
    }

    /**
     * Perform calculation based on operator
     */
    public static double calculate(double num1, double num2, char operator) {
        return switch (operator) {
            case '+' -> add(num1, num2);
            case '-' -> subtract(num1, num2);
            case '*' -> multiply(num1, num2);
            case '/' -> divide(num1, num2);
            case '^' -> power(num1, num2);
            default -> {
                System.out.println("Invalid operator!");
                yield Double.NaN;
            }
        };
    }

    /**
     * Display menu
     */
    public static void displayMenu() {
        System.out.println("\n╔═══════════════════════════════╗");
        System.out.println("║   CALCULATOR MENU    ║");
        System.out.println("╠═══════════════════════════════╣");
        System.out.println("║ 1. Addition (+)      ║");
        System.out.println("║ 2. Subtraction (-)   ║");
        System.out.println("║ 3. Multiplication (*)║");
        System.out.println("║ 4. Division (/)      ║");
        System.out.println("║ 5. Power (^)        ║");
        System.out.println("║ 6. Square Root      ║");
        System.out.println("║ 7. Exit             ║");
        System.out.println("╚═══════════════════════════════╝");
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int choice;

        System.out.println("=== MENU-DRIVEN CALCULATOR ===");
```

```
        do {
            displayMenu();
            System.out.print("Enter your choice (1-7): ");
            choice = scanner.nextInt();

            switch (choice) {
                case 1 -> {
                    System.out.print("Enter first number: ");
                    double a = scanner.nextDouble();
                    System.out.print("Enter second number: ");
                    double b = scanner.nextDouble();
                    System.out.println("Result: " + add(a, b));
                }
                case 2 -> {
                    System.out.print("Enter first number: ");
                    double a = scanner.nextDouble();
                    System.out.print("Enter second number: ");
                    double b = scanner.nextDouble();
                    System.out.println("Result: " + subtract(a, b));
                }
                case 3 -> {
                    System.out.print("Enter first number: ");
                    double a = scanner.nextDouble();
                    System.out.print("Enter second number: ");
                    double b = scanner.nextDouble();
                    System.out.println("Result: " + multiply(a, b));
                }
                case 4 -> {
                    System.out.print("Enter first number: ");
                    double a = scanner.nextDouble();
                    System.out.print("Enter second number: ");
                    double b = scanner.nextDouble();
                    System.out.println("Result: " + divide(a, b));
                }
                case 5 -> {
                    System.out.print("Enter base: ");
                    double base = scanner.nextDouble();
                    System.out.print("Enter exponent: ");
                    double exp = scanner.nextDouble();
                    System.out.println("Result: " + power(base, exp));
                }
                case 6 -> {
                    System.out.print("Enter number: ");
                    double num = scanner.nextDouble();
                    System.out.println("Result: " + squareRoot(num));
                }
                case 7 -> System.out.println("Exiting calculator. Goodbye!");
                default -> System.out.println("Invalid choice! Please try again.");
            }
        } while (choice != 7);

        scanner.close();
```

```
    }
}
```

Example 4: Find Prime Numbers with break and continue

```
/**
 * PrimeNumberDemo - Demonstrates break and continue
 * Shows efficient prime number checking
 */
public class PrimeNumberDemo {

    /**
     * Check if a number is prime using break
     */
    public static boolean isPrime(int number) {
        if (number <= 1) {
            return false;
        }

        for (int i = 2; i <= Math.sqrt(number); i++) {
            if (number % i == 0) {
                return false;  // Found divisor, not prime
            }
        }

        return true;
    }

    /**
     * Print all prime numbers up to n using continue
     */
    public static void printPrimesUpTo(int n) {
        System.out.println("Prime numbers up to " + n + ":");

        for (int i = 2; i <= n; i++) {
            if (!isPrime(i)) {
                continue;  // Skip non-prime numbers
            }
            System.out.print(i + " ");
        }
        System.out.println();
    }

    /**
     * Find first n prime numbers
     */
    public static void findFirstNPrimes(int count) {
        System.out.println("\nFirst " + count + " prime numbers:");
        int found = 0;
        int number = 2;
```

```
        while (found < count) {
            if (isPrime(number)) {
                System.out.print(number + " ");
                found++;
            }
            number++;
        }
        System.out.println();
    }

    /**
     * Count primes in a range
     */
    public static int countPrimesInRange(int start, int end) {
        int count = 0;

        for (int i = start; i <= end; i++) {
            if (isPrime(i)) {
                count++;
            }
        }

        return count;
    }

    /**
     * Print prime factors of a number
     */
    public static void printPrimeFactors(int number) {
        if (number <= 1) {
            System.out.println("No prime factors");
            return;
        }

        System.out.print("Prime factors of " + number + ": ");

        // Handle factor of 2
        while (number % 2 == 0) {
            System.out.print(2 + " ");
            number /= 2;
        }

        // Handle odd factors
        for (int i = 3; i <= Math.sqrt(number); i += 2) {
            while (number % i == 0) {
                System.out.print(i + " ");
                number /= i;
            }
        }

        // If remaining number is greater than 1
        if (number > 1) {
```

```
            System.out.print(number);
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // Test isPrime
        System.out.println("=== Prime Number Tests ===");
        int[] testNumbers = {1, 2, 3, 4, 5, 17, 18, 19, 100};

        for (int num : testNumbers) {
            System.out.println(num + " is " + (isPrime(num) ? "prime" : "not prime"));
        }

        // Print primes
        printPrimesUpTo(50);

        // Find first primes
        findFirstNPrimes(15);

        // Count primes
        System.out.println("\nPrimes between 1 and 100: " +
                           countPrimesInRange(1, 100));

        // Prime factors
        printPrimeFactors(100);
        printPrimeFactors(17);
    }
}
```

7. Exercises

Exercise 1: Number Classification

Requirements:

1 Create a program that classifies a number as:
- Positive, Negative, or Zero
- Even or Odd
- Prime or Composite

1 Use if-else statements
2 Test with multiple numbers

Exercise 2: Multiplication Table

Requirements:

1. Print multiplication table (1-10) using nested loops
2. Format as proper table
3. Use for loops

Exercise 3: ATM Simulator

Requirements:

1. Create menu with options: Check Balance, Deposit, Withdraw, Exit
2. Use switch statement for menu
3. Implement validation for withdrawal (cannot exceed balance)
4. Use do-while for menu loop

---

8. Solutions

Solution 1: Number Classification

```
public class NumberClassifier {

    public static void classify(int number) {
        System.out.println("\n=== Classifying " + number + " ===");

        // Positive/Negative/Zero
        if (number > 0) {
            System.out.println("Type: Positive");
        } else if (number < 0) {
            System.out.println("Type: Negative");
        } else {
            System.out.println("Type: Zero");
        }

        // Even/Odd (skip for zero)
        if (number != 0) {
            if (number % 2 == 0) {
                System.out.println("Parity: Even");
            } else {
                System.out.println("Parity: Odd");
            }
        }

        // Prime/Composite (only for positive)
        if (number > 1) {
            if (isPrime(number)) {
                System.out.println("Classification: Prime");
            } else {
                System.out.println("Classification: Composite");
            }
```

```
        } else if (number <= 1) {
            System.out.println("Classification: Not prime");
        }
    }

    public static boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        int[] numbers = {0, 1, 2, 17, 18, 19, 100, -5};

        for (int num : numbers) {
            classify(num);
        }
    }
}
```

Solution 2: Multiplication Table

```
public class MultiplicationTable {

    public static void main(String[] args) {
        System.out.println("=== MULTIPLICATION TABLE (1-10) ===\n");

        // Print header
        System.out.print("     ");
        for (int i = 1; i <= 10; i++) {
            System.out.printf("%4d", i);
        }
        System.out.println();
        System.out.println("     " + "----".repeat(10));

        // Print table
        for (int i = 1; i <= 10; i++) {
            System.out.printf("%4d|", i);
            for (int j = 1; j <= 10; j++) {
                System.out.printf("%4d", i * j);
            }
            System.out.println();
        }
    }
}
```

Solution 3: ATM Simulator

```
import java.util.Scanner;

public class ATMSimulator {
    private double balance = 1000.0;

    public void checkBalance() {
        System.out.println("Your current balance: $" + balance);
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: $" + amount);
            System.out.println("New balance: $" + balance);
        } else {
            System.out.println("Invalid deposit amount!");
        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: $" + amount);
            System.out.println("Remaining balance: $" + balance);
        } else if (amount > balance) {
            System.out.println("Insufficient funds!");
        } else {
            System.out.println("Invalid withdrawal amount!");
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ATMSimulator atm = new ATMSimulator();
        int choice;

        do {
            System.out.println("\n■■■■■■■■■■■■■■■■■■■■");
            System.out.println("■   ATM MENU    ■");
            System.out.println("■■■■■■■■■■■■■■■■■■■■");
            System.out.println("■ 1. Balance    ■");
            System.out.println("■ 2. Deposit   ■");
            System.out.println("■ 3. Withdraw  ■");
            System.out.println("■ 4. Exit       ■");
            System.out.println("■■■■■■■■■■■■■■■■■■■■");

            System.out.print("Choice: ");
            choice = scanner.nextInt();

            switch (choice) {
```

```
                case 1 -> atm.checkBalance();
                case 2 -> {
                    System.out.print("Enter amount: ");
                    double dep = scanner.nextDouble();
                    atm.deposit(dep);
                }
                case 3 -> {
                    System.out.print("Enter amount: ");
                    double wd = scanner.nextDouble();
                    atm.withdraw(wd);
                }
                case 4 -> System.out.println("Thank you!");
                default -> System.out.println("Invalid choice!");
            }
        } while (choice != 4);

        scanner.close();
    }
}
```

Summary

Key Takeaways

1. **if-else** - Basic conditional execution
2. **switch** - Multi-way branching (use modern switch expressions!)
3. **for loop** - Known iteration count
4. **while loop** - Condition-based iteration
5. **do-while** - At least one execution
6. **for-each** - Iterate collections/arrays
7. **break** - Exit loops/switch
8. **continue** - Skip iteration
9. **return** - Exit methods

Next Topics

- Strings and String Handling
- Arrays
- Exception Handling

Happy Coding! ■

Code Examples

ControlFlowDemo.java

ControlFlowDemo.java

```
// ControlFlowDemo - Demonstrates Java Control Flow Statements
// If-else, switch, loops, and branching statements

public class ControlFlowDemo {

    public static void main(String[] args) {
        System.out.println("=== CONTROL FLOW DEMO ===\n");

        // IF-ELSE
        System.out.println("--- If-Else ---");
        int age = 20;
        if (age >= 18) {
            System.out.println("Adult");
        } else if (age >= 13) {
            System.out.println("Teenager");
        } else {
            System.out.println("Child");
        }

        // Ternary operator
        System.out.println("\n--- Ternary Operator ---");
        String status = age >= 18 ? "Adult" : "Minor";
        System.out.println("Status: " + status);

        // SWITCH
        System.out.println("\n--- Switch Statement ---");
        int day = 3;
        String dayName;
        switch (day) {
            case 1: dayName = "Monday"; break;
            case 2: dayName = "Tuesday"; break;
            case 3: dayName = "Wednesday"; break;
            case 4: dayName = "Thursday"; break;
            case 5: dayName = "Friday"; break;
            case 6: dayName = "Saturday"; break;
            case 7: dayName = "Sunday"; break;
            default: dayName = "Invalid";
        }
        System.out.println("Day: " + dayName);

        // Switch expression (Java 14+)
        System.out.println("\n--- Switch Expression ---");
        String result = switch (age) {
            case 18 -> "Exactly 18";
            case 19, 20 -> "Young adult";
            default -> "Other age";
        };
```

```
        System.out.println(result);

        // FOR LOOP
        System.out.println("\n--- For Loop ---");
        for (int i = 1; i <= 5; i++) {
            System.out.print(i + " ");
        }
        System.out.println();

        // FOR-EACH
        System.out.println("\n--- For-Each Loop ---");
        String[] fruits = {"Apple", "Banana", "Cherry"};
        for (String fruit : fruits) {
            System.out.println(fruit);
        }

        // WHILE
        System.out.println("\n--- While Loop ---");
        int count = 0;
        while (count < 3) {
            System.out.println("Count: " + count);
            count++;
        }

        // DO-WHILE
        System.out.println("\n--- Do-While Loop ---");
        int num = 0;
        do {
            System.out.println("Number: " + num);
            num++;
        } while (num < 3);

        // BREAK and CONTINUE
        System.out.println("\n--- Break ---");
        for (int i = 1; i <= 10; i++) {
            if (i == 5) break;
            System.out.print(i + " ");
        }
        System.out.println();

        System.out.println("\n--- Continue ---");
        for (int i = 1; i <= 5; i++) {
            if (i == 3) continue;
            System.out.print(i + " ");
        }
        System.out.println();

        // LABELED BREAK
        System.out.println("\n--- Labeled Break ---");
        outer: for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 3; j++) {
                if (i == 2 && j == 2) break outer;
                System.out.println("i=" + i + ", j=" + j);
```

```
            }
        }
    }
}
```

UserAccessControlDemo.java

UserAccessControlDemo.java

```
// UserAccessControlDemo - Demonstrates Control Flow for Access Control
// Use case: Role-based access control for Angular apps

public class UserAccessControlDemo {

    // Enum for user roles
    enum Role { ADMIN, USER, GUEST, MODERATOR }

    // Method demonstrating control flow for access control
    public static String checkAccess(Role role, String resource) {
        System.out.println("\n=== ACCESS CONTROL CHECK ===");
        System.out.println("User Role: " + role);
        System.out.println("Requested Resource: " + resource);

        // Nested if-else for access levels
        if (role == Role.ADMIN) {
            // Admin has full access
            if (resource.equals("settings") || resource.equals("users") || resource.equals("repor...
                return "ACCESS GRANTED - Full permissions";
            } else {
                return "ACCESS GRANTED - All resources";
            }
        } else if (role == Role.MODERATOR) {
            // Moderator has limited access
            if (resource.equals("users")) {
                return "ACCESS GRANTED - View and edit users";
            } else if (resource.equals("reports")) {
                return "ACCESS GRANTED - View reports only";
            } else {
                return "ACCESS DENIED - Insufficient permissions";
            }
        } else if (role == Role.USER) {
            // Regular user access
            if (resource.equals("profile") || resource.equals("dashboard")) {
                return "ACCESS GRANTED - Own data only";
            } else {
                return "ACCESS DENIED - User cannot access " + resource;
            }
        } else {
            // Guest has minimal access
            if (resource.equals("public")) {
                return "ACCESS GRANTED - Public content";
            } else {
```

```
                return "ACCESS DENIED - Login required";
            }
        }
    }

    // Switch statement for feature flags
    public static String getFeatureLevel(Role role) {
        System.out.println("\n=== FEATURE FLAGS ===");

        switch (role) {
            case ADMIN:
                return "ALL_FEATURES";
            case MODERATOR:
                return "MODERATOR_FEATURES";
            case USER:
                return "STANDARD_FEATURES";
            case GUEST:
                return "BASIC_FEATURES";
            default:
                return "NO_FEATURES";
        }
    }

    // Loop example for processing permissions
    public static String[] getPermissions(Role role) {
        System.out.println("\n=== PERMISSION LIST ===");

        String[] permissions;

        switch (role) {
            case ADMIN:
                permissions = new String[] {"read", "write", "delete", "admin", "audit"};
                break;
            case MODERATOR:
                permissions = new String[] {"read", "write", "edit_users", "view_reports"};
                break;
            case USER:
                permissions = new String[] {"read", "write_own", "edit_profile"};
                break;
            default:
                permissions = new String[] {"read_public"};
        }

        // Enhanced for loop
        for (String perm : permissions) {
            System.out.println("- " + perm);
        }

        return permissions;
    }

    public static void main(String[] args) {
        System.out.println("=== CONTROL FLOW FOR ACCESS CONTROL ===\n");
```

```
        // Test different roles
        Role[] roles = {Role.ADMIN, Role.MODERATOR, Role.USER, Role.GUEST};
        String[] resources = {"settings", "users", "dashboard", "public"};

        // Nested loops for testing
        for (Role role : roles) {
            for (String resource : resources) {
                String result = checkAccess(role, resource);
                System.out.println("Result: " + result);
            }
        }

        // Feature flags
        for (Role role : roles) {
            System.out.println("\nRole: " + role + " -> Features: " + getFeatureLevel(role));
        }

        // Permissions list
        getPermissions(Role.ADMIN);

        System.out.println("\n=== ANGULAR USE CASE ===");
        System.out.println("1. Route guards (*ngIf based on roles)");
        System.out.println("2. Button visibility (*ngIf='canEdit')");
        System.out.println("3. Feature flags for A/B testing");
        System.out.println("4. Permission-based UI rendering");
    }
}
```

17

Strings

17_Strings\examples\17_Strings.md

Java Strings and String Handling

Table of Contents

1 Introduction to Strings
2 String Creation
3 String Immutability
4 String Methods
5 StringBuilder and StringBuffer
6 String Formatting
7 Regular Expressions
8 Code Examples
9 Exercises
10 Solutions

---

1. Introduction to Strings

What is a String?

In Java, a **String** is a sequence of characters. It's one of the most commonly used data types.

```
╔═══════════════════════════════════════════════════════════════════════╗
║                      STRING CONCEPT                                  ║
╔═══════════════════════════════════════════════════════════════════════╗
║                                                                      ║
║   String: "Hello World"                                             ║
║                                                                      ║
║   Index:    [0] [1] [2] [3] [4] [5] [6] [7] [8] [9] [10]  ║
║   Char:     H  e  l  l  o     W  o  r  l  d               ║
║                                                                      ║
║   Length: 11 characters                                             ║
║                                                                      ║
╔═══════════════════════════════════════════════════════════════════════╗
```

---

2. String Creation

Methods of Creating Strings

```
// Method 1: String literal (most common)
String s1 = "Hello";

// Method 2: Using new keyword
String s2 = new String("Hello");

// Method 3: From character array
char[] chars = {'H', 'e', 'l', 'l', 'o'};
String s3 = new String(chars);

// Method 4: From byte array
byte[] bytes = {72, 101, 108, 108, 111};
String s4 = new String(bytes);
```

3. String Immutability

What is Immutability?

Strings in Java are **immutable** - once created, they cannot be changed.

```
■■■
```

```
■   s1 == s3    → false  (different objects)                ■
■   s1.equals(s3) → true   (same content)                    ■
■                                                            ■
■■■
```

Class	Thread Safe	Performance
String	N/A (immutable)	Slow for many modifications
StringBuilder	No	Fast
StringBuffer	Yes	Moderate

```
// String - slow for many modifications
String result = "";
for (int i = 0; i < 1000; i++) {
    result += i;  // Creates new object each time!
}

// StringBuilder - fast
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append(i);  // Modifies in place
}
String result = sb.toString();

// StringBuffer - thread-safe version
StringBuffer sbf = new StringBuffer();
sbf.append("Hello").append(" World");
```

6. String Formatting

printf-style Formatting

```
// Using printf
System.out.printf("Name: %s, Age: %d, Price: %.2f%n", "John", 25, 99.99);

// Using String.format
String formatted = String.format("Name: %s, Age: %d", "John", 25);
```

Format Specifiers

Specifier	Description	Example
`%s`	String	"Hello"
`%d`	Integer	42
`%f`	Float/Double	3.14

Specifier	Description	Example
`%.2f`	Float with 2 decimals	3.14
`%n`	Newline	(line break)
`%b`	Boolean	true
`%c`	Character	'A'

7. Regular Expressions

Pattern Matching

```
import java.util.regex.*;

String email = "user@example.com";
String pattern = "^[A-Za-z0-9+_.-]+@(.+)$";

if (email.matches(pattern)) {
    System.out.println("Valid email");
}
```

Common Regex Patterns

Pattern	Matches
`\d`	Any digit
`\w`	Word character
`\s`	Whitespace
`.`	Any character
`*`	Zero or more
`+`	One or more
`?`	Zero or one
`^`	Start of string
`$`	End of string

8. Code Examples

Example 1: String Operations Demo

```
/**
 * StringOperations - Demonstrates common string operations
 * Essential for Angular backend data processing
 */
public class StringOperations {

    public static void main(String[] args) {
        System.out.println("=== STRING OPERATIONS DEMO ===\n");

        String text = "  Hello, World! Welcome to Java Programming  ";

        // Basic operations
        System.out.println("Original: \"" + text + "\"");
        System.out.println("Length: " + text.length());
        System.out.println("Char at 0: " + text.charAt(0));
        System.out.println("Uppercase: " + text.toUpperCase());
        System.out.println("Lowercase: " + text.toLowerCase());
        System.out.println("Trimmed: \"" + text.trim() + "\"");

        // Substring
        System.out.println("\n--- Substring Examples ---");
        System.out.println("Substring(0,5): " + text.substring(0, 5));
        System.out.println("Substring(7): " + text.substring(7));

        // Search
        System.out.println("\n--- Search Examples ---");
        System.out.println("Index of 'World': " + text.indexOf("World"));
        System.out.println("Contains 'Java': " + text.contains("Java"));
        System.out.println("Starts with '  Hello': " + text.startsWith("  Hello"));
        System.out.println("Ends with 'ming  ': " + text.endsWith("ming  "));

        // Replace
        System.out.println("\n--- Replace Examples ---");
        System.out.println("Replace 'Java' with 'Python': " +
                           text.replace("Java", "Python"));
        System.out.println("Replace all digits with '#': " +
                           text.replaceAll("\\d", "#"));

        // Split
        System.out.println("\n--- Split Examples ---");
        String csv = "apple,banana,cherry,date";
        String[] fruits = csv.split(",");
        System.out.print("Split by comma: ");
        for (String fruit : fruits) {
            System.out.print(fruit + " | ");
        }
        System.out.println();
```

```
        // Concatenation
        System.out.println("\n--- Concatenation ---");
        String s1 = "Hello";
        String s2 = "World";
        System.out.println("concat: " + s1.concat(" ").concat(s2));
        System.out.println("+ operator: " + s1 + " " + s2);

        // Comparison
        System.out.println("\n--- Comparison ---");
        String a = "hello";
        String b = "hello";
        String c = new String("hello");
        System.out.println("a == b: " + (a == b));  // true (same reference)
        System.out.println("a == c: " + (a == c));  // false (different objects)
        System.out.println("a.equals(c): " + a.equals(c));  // true (same content)
        System.out.println("equalsIgnoreCase: " +
                           "HELLO".equalsIgnoreCase("hello"));
    }
}
```

Example 2: User Input Validation

```
import java.util.Scanner;

/**
 * UserInputValidator - Validates user input using strings
 * Common in Angular + Java backends for form validation
 */
public class UserInputValidator {

    /**
     * Validate email address
     */
    public static boolean isValidEmail(String email) {
        // Simple email validation
        String regex = "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$";
        return email != null && email.matches(regex);
    }

    /**
     * Validate phone number (10 digits)
     */
    public static boolean isValidPhone(String phone) {
        String regex = "^\\d{10}$";
        return phone != null && phone.matches(regex);
    }

    /**
     * Validate username (alphanumeric, 5-15 chars)
```

```
     */
    public static boolean isValidUsername(String username) {
        String regex = "^[A-Za-z0-9_]{5,15}$";
        return username != null && username.matches(regex);
    }

    /**
     * Validate password (at least 8 chars, 1 digit, 1 letter)
     */
    public static boolean isValidPassword(String password) {
        if (password == null || password.length() < 8) {
            return false;
        }

        boolean hasDigit = false;
        boolean hasLetter = false;

        for (char c : password.toCharArray()) {
            if (Character.isDigit(c)) hasDigit = true;
            if (Character.isLetter(c)) hasLetter = true;
        }

        return hasDigit && hasLetter;
    }

    /**
     * Check if string contains only alphabets
     */
    public static boolean isAlphabetic(String str) {
        return str != null && str.matches("^[A-Za-z]+$");
    }

    /**
     * Check if string contains only digits
     */
    public static boolean isNumeric(String str) {
        return str != null && str.matches("^\\d+$");
    }

    /**
     * Mask sensitive data (like passwords)
     */
    public static String maskData(String data, int visibleChars) {
        if (data == null) return null;
        if (data.length() <= visibleChars) return data;

        String visible = data.substring(0, visibleChars);
        String masked = "*".repeat(data.length() - visibleChars);
        return visible + masked;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
```

```
        System.out.println("=== USER INPUT VALIDATOR ===\n");

        // Email validation
        System.out.print("Enter email: ");
        String email = scanner.nextLine();
        System.out.println("Valid email? " + isValidEmail(email));

        // Phone validation
        System.out.print("Enter phone (10 digits): ");
        String phone = scanner.nextLine();
        System.out.println("Valid phone? " + isValidPhone(phone));

        // Username validation
        System.out.print("Enter username: ");
        String username = scanner.nextLine();
        System.out.println("Valid username? " + isValidUsername(username));

        // Password validation
        System.out.print("Enter password: ");
        String password = scanner.nextLine();
        System.out.println("Valid password? " + isValidPassword(password));

        // Mask password
        System.out.println("Masked password: " + maskData(password, 2));

        scanner.close();
    }
}
```

Example 3: StringBuilder Demo

```
/**
 * StringBuilderDemo - Shows efficient string manipulation
 */
public class StringBuilderDemo {

    public static void main(String[] args) {
        System.out.println("=== STRINGBUILDER DEMO ===\n");

        // Create StringBuilder
        StringBuilder sb = new StringBuilder();

        // Append
        sb.append("Hello");
        sb.append(" ");
        sb.append("World");
        System.out.println("After append: " + sb);

        // Insert
```

```
        sb.insert(5, ",");
        System.out.println("After insert: " + sb);

        // Replace
        sb.replace(0, 5, "Hi");
        System.out.println("After replace: " + sb);

        // Delete
        sb.delete(0, 2);
        System.out.println("After delete: " + sb);

        // Reverse
        sb.reverse();
        System.out.println("After reverse: " + sb);

        // Chain operations
        StringBuilder builder = new StringBuilder();
        String result = builder
            .append("Java")
            .append(" ")
            .append("is")
            .append(" ")
            .append("awesome")
            .toString();

        System.out.println("\nChained result: " + result);

        // Performance comparison
        System.out.println("\n=== PERFORMANCE TEST ===");
        int iterations = 10000;

        // String concatenation (slow)
        long start = System.currentTimeMillis();
        String s = "";
        for (int i = 0; i < iterations; i++) {
            s += "a";
        }
        long stringTime = System.currentTimeMillis() - start;
        System.out.println("String time: " + stringTime + "ms");

        // StringBuilder (fast)
        start = System.currentTimeMillis();
        StringBuilder sb2 = new StringBuilder();
        for (int i = 0; i < iterations; i++) {
            sb2.append("a");
        }
        long builderTime = System.currentTimeMillis() - start;
        System.out.println("StringBuilder time: " + builderTime + "ms");
    }
}
```

Example 4: CSV Parser

```
/**
 * CSVParser - Parse CSV data (common for Angular backend integration)
 */
public class CSVParser {

    /**
     * Parse CSV line into array
     */
    public static String[] parseCSVLine(String line) {
        return line.split(",");
    }

    /**
     * Parse CSV with quotes handling
     */
    public static String[] parseCSVWithQuotes(String line) {
        // Simple implementation - split by comma not inside quotes
        java.util.List<String> result = new java.util.ArrayList<>();
        StringBuilder current = new StringBuilder();
        boolean inQuotes = false;

        for (char c : line.toCharArray()) {
            if (c == '"') {
                inQuotes = !inQuotes;
            } else if (c == ',' && !inQuotes) {
                result.add(current.toString().trim());
                current = new StringBuilder();
            } else {
                current.append(c);
            }
        }
        result.add(current.toString().trim());

        return result.toArray(new String[0]);
    }

    /**
     * Create CSV line from array
     */
    public static String createCSVLine(String[] data) {
        return String.join(",", data);
    }

    /**
     * Parse entire CSV file
     */
    public static String[][] parseCSV(String csvData) {
        String[] lines = csvData.split("\n");
        String[][] result = new String[lines.length][];
```

```
        for (int i = 0; i < lines.length; i++) {
            result[i] = parseCSVLine(lines[i]);
        }

        return result;
    }

    public static void main(String[] args) {
        System.out.println("=== CSV PARSER DEMO ===\n");

        // Sample CSV data
        String csvLine = "John,Doe,25,john@example.com";
        System.out.println("Input: " + csvLine);

        String[] parsed = parseCSVLine(csvLine);
        System.out.println("Parsed:");
        for (int i = 0; i < parsed.length; i++) {
            System.out.println("  [" + i + "]: " + parsed[i]);
        }

        // Create CSV line
        String[] data = {"Alice", "Smith", "30", "alice@example.com"};
        System.out.println("\nCreated CSV: " + createCSVLine(data));

        // Multi-line CSV
        String csvData = "Name,Age,Email\nJohn,25,john@test.com\nJane,30,jane@test.com";
        System.out.println("\n--- Multi-line CSV ---");
        String[][] multi = parseCSV(csvData);
        for (String[] row : multi) {
            for (String cell : row) {
                System.out.print(cell + " | ");
            }
            System.out.println();
        }
    }
}
```

---

9. Exercises

Exercise 1: Palindrome Checker

Requirements:

1. Create a method to check if a string is a palindrome
2. Handle case-insensitivity
3. Ignore spaces and punctuation
4. Test with: "racecar", "A man a plan a canal Panama", "hello"

Exercise 2: Word Counter

Requirements:

1. Count total words in a sentence
2. Count occurrences of each word
3. Find the most frequent word
4. Use String methods: split, toLowerCase, trim

Exercise 3: Password Strength Checker

Requirements:

1. Check minimum 8 characters
2. At least one uppercase letter
3. At least one lowercase letter
4. At least one digit
5. At least one special character

---

10. Solutions

Solution 1: Palindrome Checker

```
public class PalindromeChecker {

    public static boolean isPalindrome(String str) {
        // Remove spaces and convert to lowercase
        String cleaned = str.toLowerCase().replaceAll("[^a-z0-9]", "");

        // Reverse and compare
        String reversed = new StringBuilder(cleaned).reverse().toString();

        return cleaned.equals(reversed);
    }

    public static void main(String[] args) {
        String[] testStrings = {
            "racecar",
            "A man a plan a canal Panama",
            "hello",
            "Was it a car or a cat I saw"
        };

        for (String s : testStrings) {
            System.out.println("\"" + s + "\" is palindrome: " + isPalindrome(s));
        }
```

```
    }
}
```

Solution 2: Word Counter

```
import java.util.*;

public class WordCounter {

    public static void countWords(String sentence) {
        // Clean and split
        String[] words = sentence.toLowerCase().trim().split("\\s+");

        // Count occurrences
        Map<String, Integer> wordCount = new HashMap<>();
        for (String word : words) {
            wordCount.put(word, wordCount.getOrDefault(word, 0) + 1);
        }

        // Find most frequent
        String mostFrequent = "";
        int maxCount = 0;
        for (Map.Entry<String, Integer> entry : wordCount.entrySet()) {
            if (entry.getValue() > maxCount) {
                maxCount = entry.getValue();
                mostFrequent = entry.getKey();
            }
        }

        // Output
        System.out.println("Total words: " + words.length);
        System.out.println("Unique words: " + wordCount.size());
        System.out.println("Most frequent: \"" + mostFrequent + "\" (" + maxCount + " times)");
        System.out.println("Word frequencies: " + wordCount);
    }

    public static void main(String[] args) {
        String sentence = "the quick brown fox jumps over the lazy dog the dog was not amused";
        countWords(sentence);
    }
}
```

Solution 3: Password Strength Checker

```
public class PasswordStrengthChecker {

    public static String checkStrength(String password) {
```

```
        if (password == null || password.length() < 8) {
            return "Weak - Too short";
        }

        boolean hasUpper = false, hasLower = false, hasDigit = false, hasSpecial = false;

        for (char c : password.toCharArray()) {
            if (Character.isUpperCase(c)) hasUpper = true;
            else if (Character.isLowerCase(c)) hasLower = true;
            else if (Character.isDigit(c)) hasDigit = true;
            else hasSpecial = true;
        }

        int strength = 0;
        if (hasUpper) strength++;
        if (hasLower) strength++;
        if (hasDigit) strength++;
        if (hasSpecial) strength++;

        return switch (strength) {
            case 4 -> "Strong";
            case 3 -> "Medium";
            default -> "Weak";
        };
    }

    public static void main(String[] args) {
        String[] passwords = {"pass", "password", "Password1", "Pass@word1", "Str0ng!Pass"};

        for (String p : passwords) {
            System.out.println(p + ": " + checkStrength(p));
        }
    }
}
```

Summary

Key Takeaways

1. **Strings are immutable** - Use StringBuilder for modifications
2. **String pool** - Literals are reused for efficiency
3. **Common methods** - length, charAt, substring, indexOf, replace, split
4. **StringBuilder** - Efficient for string concatenation
5. **Regex** - Powerful pattern matching with Pattern/Matcher

Angular Backend Connection

- JSON uses string data extensively
- Input validation in Java mirrors Angular validation
- String formatting for API responses

Happy Coding! ■

Code Examples

StringMethodsDemo.java

StringMethodsDemo.java

```
// StringMethodsDemo - Demonstrates String operations in Java
// Important for text processing in backend applications

public class StringMethodsDemo {

    public static void main(String[] args) {
        System.out.println("=== STRING METHODS DEMO ===\n");

        String str = "Hello, World!";
        String empty = "";
        String spaces = "   Trim me   ";
        String numbers = "12345";
        String mixed = "JaVaScRiPt";

        // Basic methods
        System.out.println("--- Basic Methods ---");
        System.out.println("Original: " + str);
        System.out.println("Length: " + str.length());
        System.out.println("Char at 0: " + str.charAt(0));
        System.out.println("Substring (0,5): " + str.substring(0, 5));
        System.out.println("Uppercase: " + str.toUpperCase());
        System.out.println("Lowercase: " + str.toLowerCase());

        // Search methods
        System.out.println("\n--- Search Methods ---");
        System.out.println("Contains 'World': " + str.contains("World"));
        System.out.println("Index of 'o': " + str.indexOf('o'));
        System.out.println("Last index of 'o': " + str.lastIndexOf('o'));
        System.out.println("Starts with 'Hello': " + str.startsWith("Hello"));
        System.out.println("Ends with '!': " + str.endsWith("!"));

        // Modification methods
        System.out.println("\n--- Modification Methods ---");
        System.out.println("Replace 'World' with 'Java': " + str.replace("World", "Java"));
        System.out.println("Trimmed: '" + spaces.trim() + "'");
        System.out.println("Concatenate: " + str.concat(" - Added"));

        // Comparison methods
        System.out.println("\n--- Comparison Methods ---");
```

```
        String s1 = "hello";
        String s2 = "hello";
        String s3 = "Hello";
        System.out.println("s1 == s2: " + (s1 == s2));
        System.out.println("s1.equals(s2): " + s1.equals(s2));
        System.out.println("s1.equalsIgnoreCase(s3): " + s1.equalsIgnoreCase(s3));
        System.out.println("CompareTo: " + s1.compareTo(s3));

        // StringBuilder (mutable)
        System.out.println("\n--- StringBuilder ---");
        StringBuilder sb = new StringBuilder("Hello");
        sb.append(" World");
        sb.insert(5, ",");
        sb.reverse();
        System.out.println("StringBuilder result: " + sb.toString());

        // String splitting
        System.out.println("\n--- String Split ---");
        String csv = "apple,banana,cherry,date";
        String[] fruits = csv.split(",");
        System.out.print("Split result: ");
        for (String f : fruits) {
            System.out.print(f + " ");
        }
        System.out.println();

        // Angular integration note
        System.out.println("\n=== ANGULAR INTEGRATION ===");
        System.out.println("String -> JSON string");
        System.out.println("String methods map to JavaScript String methods");
    }
}
```

UserInputValidationDemo.java

UserInputValidationDemo.java

```
// UserInputValidationDemo - Demonstrates String Methods for Input Validation
// Use case: Validating Angular form inputs on backend

public class UserInputValidationDemo {

    // Validate email format
    public static boolean validateEmail(String email) {
        if (email == null || email.isEmpty()) {
            return false;
        }

        // Check basic email format
        boolean hasAt = email.contains("@");
        boolean hasDot = email.contains(".");
        boolean hasSpace = email.contains(" ");
```

```
        // More sophisticated validation
        int atIndex = email.indexOf("@");
        int dotIndex = email.lastIndexOf(".");

        return hasAt && hasDot && !hasSpace
            && atIndex > 0 && dotIndex > atIndex + 1
            && dotIndex < email.length() - 1;
    }

    // Validate password strength
    public static String validatePassword(String password) {
        if (password == null) {
            return "Password is required";
        }

        if (password.length() < 8) {
            return "Password must be at least 8 characters";
        }

        boolean hasUpper = false, hasLower = false, hasDigit = false, hasSpecial = false;

        for (char c : password.toCharArray()) {
            if (Character.isUpperCase(c)) hasUpper = true;
            else if (Character.isLowerCase(c)) hasLower = true;
            else if (Character.isDigit(c)) hasDigit = true;
            else hasSpecial = true;
        }

        if (!hasUpper) return "Password must contain uppercase letter";
        if (!hasLower) return "Password must contain lowercase letter";
        if (!hasDigit) return "Password must contain number";
        if (!hasSpecial) return "Password must contain special character";

        return "Password is strong";
    }

    // Sanitize user input (prevent XSS)
    public static String sanitizeInput(String input) {
        if (input == null) {
            return "";
        }

        // Remove script tags and dangerous patterns
        String result = input;
        result = removePattern(result, "<script>");
        result = removePattern(result, "</script>");
        result = removePattern(result, "javascript:");
        result = removePattern(result, "onerror=");
        return result.trim();
    }

    private static String removePattern(String text, String pattern) {
```

```
        return text.replace(pattern, "");
    }

    // Format phone number
    public static String formatPhoneNumber(String phone) {
        if (phone == null) return "";

        // Remove all non-digits
        String digits = phone.replaceAll("\\D", "");

        if (digits.length() == 10) {
            return String.format("(%s) %s-%s",
                digits.substring(0, 3),
                digits.substring(3, 6),
                digits.substring(6));
        }

        return phone;
    }

    // Format username (lowercase, no spaces)
    public static String formatUsername(String username) {
        if (username == null) return "";

        return username
            .toLowerCase()
            .replaceAll("\\s+", "_")
            .replaceAll("[^a-z0-9_]", "");
    }

    public static void main(String[] args) {
        System.out.println("=== STRING VALIDATION FOR ANGULAR FORMS ===\n");

        // Email validation
        String[] emails = {"test@example.com", "invalid.email", "user@domain", ""};
        System.out.println("--- Email Validation ---");
        for (String email : emails) {
            System.out.println("'" + email + "': " + validateEmail(email));
        }

        // Password validation
        System.out.println("\n--- Password Validation ---");
        System.out.println("'Pass123': " + validatePassword("Pass123"));
        System.out.println("'StrongP@ss1': " + validatePassword("StrongP@ss1"));

        // Input sanitization
        System.out.println("\n--- Input Sanitization ---");
        String malicious = "<script>alert('xss')</script>";
        System.out.println("Original: " + malicious);
        System.out.println("Sanitized: " + sanitizeInput(malicious));

        // Phone formatting
        System.out.println("\n--- Phone Formatting ---");
```

```
        System.out.println("1234567890 -> " + formatPhoneNumber("1234567890"));
        System.out.println("(123) 456-7890 -> " + formatPhoneNumber("(123) 456-7890"));

        // Username formatting
        System.out.println("\n--- Username Formatting ---");
        System.out.println("John Doe -> " + formatUsername("John Doe"));
        System.out.println("User@123 -> " + formatUsername("User@123"));

        System.out.println("\n=== USE CASES ===");
        System.out.println("1. Form input validation (Reactive Forms)");
        System.out.println("2. XSS prevention");
        System.out.println("3. Data formatting for display");
        System.out.println("4. Username normalization");
    }
}
```

18

Arrays

18_Arrays\examples\18_Arrays.md

Java Arrays

Table of Contents

1 Introduction to Arrays
2 Array Declaration and Initialization
3 Array Types
4 Array Operations
5 Arrays Utility Class
6 Code Examples
7 Exercises
8 Solutions

---

1. Introduction to Arrays

What is an Array?

An **array** is a container object that holds a fixed number of values of a single type.

```
■■■
```

```
// Method 1: Declaration with size
int[] numbers = new int[5];

// Method 2: Declaration with initialization
int[] numbers = {1, 2, 3, 4, 5};

// Method 3: Declare then initialize
int[] numbers;
numbers = new int[]{1, 2, 3, 4, 5};

// Method 4: Array of objects
String[] names = new String[3];
names[0] = "John";
names[1] = "Jane";
names[2] = "Bob";
```

---

3. Array Types

Single-Dimensional Arrays

```
int[] intArray = {1, 2, 3, 4, 5};
double[] doubleArray = {1.1, 2.2, 3.3};
char[] charArray = {'a', 'b', 'c'};
boolean[] boolArray = {true, false, true};
String[] stringArray = {"Apple", "Banana", "Cherry"};
```

Multi-Dimensional Arrays

```
// 2D Array
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// 3D Array
int[][][] cube = new int[2][3][4];
```

---

4. Array Operations

Common Operations

Operation	Description
`array.length`	Get array length
`array[index]`	Access element
`Arrays.sort()`	Sort array
`Arrays.fill()`	Fill array
`Arrays.toString()`	Convert to string
`Arrays.deepToString()`	2D array to string

5. Arrays Utility Class

java.util.Arrays Methods

```
import java.util.Arrays;

// Sort
Arrays.sort(array);

// Binary Search (array must be sorted)
int index = Arrays.binarySearch(array, value);

// Fill
Arrays.fill(array, value);

// CopyOf
int[] copy = Arrays.copyOf(original, newLength);

// CopyOfRange
int[] copy = Arrays.copyOfRange(original, from, to);

// Equals
boolean equal = Arrays.equals(array1, array2);

// toString
String str = Arrays.toString(array);
```

6. Code Examples

Example 1: Basic Array Operations

```java
import java.util.Arrays;

/**
 * BasicArrayOperations - Fundamental array operations
 */
public class BasicArrayOperations {

    public static void main(String[] args) {
        System.out.println("=== BASIC ARRAY OPERATIONS ===\n");

        // Create array
        int[] numbers = {5, 2, 8, 1, 9, 3};

        // Print original
        System.out.println("Original array: " + Arrays.toString(numbers));

        // Length
        System.out.println("Length: " + numbers.length);

        // Access elements
        System.out.println("First element: " + numbers[0]);
        System.out.println("Last element: " + numbers[numbers.length - 1]);

        // Modify element
        numbers[0] = 10;
        System.out.println("After modifying first: " + Arrays.toString(numbers));

        // Sort
        Arrays.sort(numbers);
        System.out.println("After sorting: " + Arrays.toString(numbers));

        // Search (binary search - array must be sorted!)
        int index = Arrays.binarySearch(numbers, 5);
        System.out.println("Index of 5: " + index);

        // Fill
        int[] filled = new int[5];
        Arrays.fill(filled, 7);
        System.out.println("Filled array: " + Arrays.toString(filled));

        // Copy
        int[] copy = Arrays.copyOf(numbers, numbers.length + 2);
        System.out.println("Copied with extra space: " + Arrays.toString(copy));

        // CopyOfRange
        int[] range = Arrays.copyOfRange(numbers, 1, 4);
        System.out.println("Copy of range [1-4): " + Arrays.toString(range));
    }
}
```

Example 2: 2D Array - Matrix Operations

```
import java.util.Arrays;

/**
 * MatrixOperations - 2D array operations
 */
public class MatrixOperations {

    public static void main(String[] args) {
        System.out.println("=== MATRIX OPERATIONS ===\n");

        // Create 3x3 matrix
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        // Print matrix
        System.out.println("Matrix:");
        printMatrix(matrix);

        // Sum of all elements
        int sum = sumMatrix(matrix);
        System.out.println("\nSum of all elements: " + sum);

        // Sum of rows
        System.out.println("\nRow sums:");
        for (int i = 0; i < matrix.length; i++) {
            int rowSum = 0;
            for (int j = 0; j < matrix[0].length; j++) {
                rowSum += matrix[i][j];
            }
            System.out.println("Row " + i + ": " + rowSum);
        }

        // Sum of columns
        System.out.println("\nColumn sums:");
        for (int j = 0; j < matrix[0].length; j++) {
            int colSum = 0;
            for (int i = 0; i < matrix.length; i++) {
                colSum += matrix[i][j];
            }
            System.out.println("Column " + j + ": " + colSum);
        }

        // Find largest element
        int max = findMax(matrix);
        System.out.println("\nLargest element: " + max);

        // Transpose matrix
        System.out.println("\nTranspose:");
        int[][] transpose = transpose(matrix);
```

```
        printMatrix(transpose);
    }

    public static void printMatrix(int[][] matrix) {
        for (int[] row : matrix) {
            System.out.println(Arrays.toString(row));
        }
    }

    public static int sumMatrix(int[][] matrix) {
        int sum = 0;
        for (int[] row : matrix) {
            for (int num : row) {
                sum += num;
            }
        }
        return sum;
    }

    public static int findMax(int[][] matrix) {
        int max = matrix[0][0];
        for (int[] row : matrix) {
            for (int num : row) {
                if (num > max) max = num;
            }
        }
        return max;
    }

    public static int[][] transpose(int[][] matrix) {
        int rows = matrix.length;
        int cols = matrix[0].length;
        int[][] result = new int[cols][rows];

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                result[j][i] = matrix[i][j];
            }
        }
        return result;
    }
}
```

Example 3: Array of Objects

```
/**
 * Student class for array of objects example
 */
class Student {
    private int id;
```

```
    private String name;
    private double gpa;

    public Student(int id, String name, double gpa) {
        this.id = id;
        this.name = name;
        this.gpa = gpa;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public double getGpa() { return gpa; }

    @Override
    public String toString() {
        return "Student{id=" + id + ", name='" + name + "', gpa=" + gpa + "}";
    }
}

/**
 * ArrayOfObjects - Working with arrays of objects
 */
public class ArrayOfObjects {

    public static void main(String[] args) {
        System.out.println("=== ARRAY OF OBJECTS ===\n");

        // Create array of students
        Student[] students = new Student[5];

        // Initialize students
        students[0] = new Student(1, "Alice", 3.8);
        students[1] = new Student(2, "Bob", 3.5);
        students[2] = new Student(3, "Charlie", 3.9);
        students[3] = new Student(4, "Diana", 3.7);
        students[4] = new Student(5, "Eve", 3.6);

        // Print all students
        System.out.println("All students:");
        for (Student s : students) {
            System.out.println("  " + s);
        }

        // Find top student
        Student topStudent = students[0];
        for (Student s : students) {
            if (s.getGpa() > topStudent.getGpa()) {
                topStudent = s;
            }
        }
        System.out.println("\nTop student: " + topStudent.getName());

        // Calculate average GPA
```

```
        double totalGpa = 0;
        for (Student s : students) {
            totalGpa += s.getGpa();
        }
        double avgGpa = totalGpa / students.length;
        System.out.println("Average GPA: " + avgGpa);
    }
}
```

Example 4: Searching and Sorting

```
import java.util.Arrays;

/**
 * SearchAndSort - Various search and sort algorithms
 */
public class SearchAndSort {

    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};

        System.out.println("=== SEARCH AND SORT DEMO ===\n");
        System.out.println("Original: " + Arrays.toString(arr));

        // Bubble Sort
        int[] bubble = arr.clone();
        bubbleSort(bubble);
        System.out.println("Bubble Sort: " + Arrays.toString(bubble));

        // Selection Sort
        int[] selection = arr.clone();
        selectionSort(selection);
        System.out.println("Selection Sort: " + Arrays.toString(selection));

        // Insertion Sort
        int[] insertion = arr.clone();
        insertionSort(insertion);
        System.out.println("Insertion Sort: " + Arrays.toString(insertion));

        // Linear Search
        System.out.println("\nLinear Search for 25: " + linearSearch(arr, 25));

        // Binary Search (array must be sorted!)
        Arrays.sort(arr);
        System.out.println("Sorted array: " + Arrays.toString(arr));
        System.out.println("Binary Search for 25: " + binarySearch(arr, 25));
    }

    public static void bubbleSort(int[] arr) {
        int n = arr.length;
```

```
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    // Swap
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }

    public static void selectionSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[minIdx]) {
                    minIdx = j;
                }
            }
            // Swap
            int temp = arr[minIdx];
            arr[minIdx] = arr[i];
            arr[i] = temp;
        }
    }

    public static void insertionSort(int[] arr) {
        int n = arr.length;
        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }

    public static int linearSearch(int[] arr, int target) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                return i;
            }
        }
        return -1;
    }

    public static int binarySearch(int[] arr, int target) {
        int left = 0;
        int right = arr.length - 1;
```

```
        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (arr[mid] == target) {
                return mid;
            } else if (arr[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return -1;
    }
}
```

---

7. Exercises

Exercise 1: Array Statistics

Requirements:

1. Find minimum, maximum, and average of array
2. Count even and odd numbers
3. Return results as formatted output

Exercise 2: Remove Duplicates

Requirements:

1. Remove duplicate elements from sorted array
2. Return new length
3. Do it in-place (without extra collection)

---

8. Solutions

Solution 1: Array Statistics

```
import java.util.Arrays;

public class ArrayStatistics {

    public static void main(String[] args) {
        int[] numbers = {5, 10, 15, 20, 25, 30, 35, 40};
```

```
        // Min
        int min = numbers[0];
        for (int n : numbers) {
            if (n < min) min = n;
        }

        // Max
        int max = numbers[0];
        for (int n : numbers) {
            if (n > max) max = n;
        }

        // Average
        double sum = 0;
        for (int n : numbers) {
            sum += n;
        }
        double avg = sum / numbers.length;

        // Count even/odd
        int even = 0, odd = 0;
        for (int n : numbers) {
            if (n % 2 == 0) even++;
            else odd++;
        }

        System.out.println("Array: " + Arrays.toString(numbers));
        System.out.println("Min: " + min);
        System.out.println("Max: " + max);
        System.out.println("Average: " + avg);
        System.out.println("Even count: " + even);
        System.out.println("Odd count: " + odd);
    }
}
```

Solution 2: Remove Duplicates

```
import java.util.Arrays;

public class RemoveDuplicates {

    public static int removeDuplicates(int[] sorted) {
        if (sorted.length == 0) return 0;

        int unique = 1;
        for (int i = 1; i < sorted.length; i++) {
            if (sorted[i] != sorted[unique - 1]) {
                sorted[unique] = sorted[i];
                unique++;
```

```
            }
        }
        return unique;
    }

    public static void main(String[] args) {
        int[] arr = {1, 1, 2, 2, 2, 3, 4, 4, 5};
        System.out.println("Original: " + Arrays.toString(arr));

        int newLength = removeDuplicates(arr);
        System.out.println("New length: " + newLength);
        System.out.println("Array after: " + Arrays.toString(arr));
    }
}
```

Summary

Key Takeaways

1. Arrays store fixed-size sequential elements
2. Index starts at 0
3. Use Arrays utility class for operations
4. For dynamic sizing, use ArrayList
5. Multi-dimensional arrays are arrays of arrays

Happy Coding! ■

Code Examples

ArraysDemo.java

ArraysDemo.java

```
// ArraysDemo - Demonstrates Java Arrays
// Arrays are fundamental for storing collections of data

public class ArraysDemo {

    public static void main(String[] args) {
        System.out.println("=== ARRAYS DEMO ===\n");

        // Declare and initialize arrays
        int[] numbers = {1, 2, 3, 4, 5};
        String[] names = new String[3];
        names[0] = "Alice";
        names[1] = "Bob";
```

```
        names[2] = "Charlie";

        // Basic operations
        System.out.println("--- Basic Operations ---");
        System.out.println("First number: " + numbers[0]);
        System.out.println("Array length: " + numbers.length);

        // Loop through array
        System.out.println("\n--- For Loop ---");
        for (int i = 0; i < numbers.length; i++) {
            System.out.println("numbers[" + i + "] = " + numbers[i]);
        }

        // Enhanced for loop
        System.out.println("\n--- Enhanced For Loop ---");
        for (String name : names) {
            System.out.println("Name: " + name);
        }

        // Multidimensional array
        System.out.println("\n--- Multidimensional Array ---");
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        for (int row = 0; row < matrix.length; row++) {
            for (int col = 0; col < matrix[row].length; col++) {
                System.out.print(matrix[row][col] + " ");
            }
            System.out.println();
        }

        // Array utility methods
        System.out.println("\n--- Arrays Utility Methods ---");
        int[] toSort = {5, 2, 8, 1, 9};
        System.out.println("Before sort: ");
        printArray(toSort);

        java.util.Arrays.sort(toSort);
        System.out.println("After sort: ");
        printArray(toSort);

        // Binary search (array must be sorted)
        int index = java.util.Arrays.binarySearch(toSort, 5);
        System.out.println("Index of 5: " + index);

        // Fill array
        int[] filled = new int[5];
        java.util.Arrays.fill(filled, 10);
        System.out.println("Filled array: ");
        printArray(filled);
```

```
        // Copy array
        int[] copied = java.util.Arrays.copyOf(numbers, numbers.length * 2);
        System.out.println("Copied array length: " + copied.length);
    }

    static void printArray(int[] arr) {
        for (int num : arr) {
            System.out.print(num + " ");
        }
        System.out.println();
    }
}
```

ProductCatalogDemo.java

ProductCatalogDemo.java

```
// ProductCatalogDemo - Demonstrates Array Operations for Product Management
// Use case: Managing product listings in Angular e-commerce app backend

public class ProductCatalogDemo {

    // Product class
    static class Product {
        private int id;
        private String name;
        private double price;

        public Product(int id, String name, double price) {
            this.id = id;
            this.name = name;
            this.price = price;
        }

        public int getId() { return id; }
        public String getName() { return name; }
        public double getPrice() { return price; }

        @Override
        public String toString() {
            return "Product{id=" + id + ", name='" + name + "', price=" + price + "}";
        }
    }

    // Initialize product array
    public static Product[] initializeProducts() {
        Product[] products = new Product[5];
        products[0] = new Product(1, "Laptop", 999.99);
        products[1] = new Product(2, "Smartphone", 699.99);
        products[2] = new Product(3, "Tablet", 449.99);
        products[3] = new Product(4, "Headphones", 199.99);
```

```
        products[4] = new Product(5, "Smartwatch", 299.99);
        return products;
    }

    // Linear search for product
    public static Product findProductById(Product[] products, int id) {
        for (Product p : products) {
            if (p.getId() == id) {
                return p;
            }
        }
        return null;
    }

    // Find products in price range
    public static Product[] findByPriceRange(Product[] products, double min, double max) {
        // First pass: count matching products
        int count = 0;
        for (Product p : products) {
            if (p.getPrice() >= min && p.getPrice() <= max) {
                count++;
            }
        }

        // Second pass: populate result array
        Product[] result = new Product[count];
        int index = 0;
        for (Product p : products) {
            if (p.getPrice() >= min && p.getPrice() <= max) {
                result[index++] = p;
            }
        }

        return result;
    }

    // Sort products by price (bubble sort)
    public static Product[] sortByPrice(Product[] products) {
        Product[] sorted = products.clone();

        for (int i = 0; i < sorted.length - 1; i++) {
            for (int j = 0; j < sorted.length - i - 1; j++) {
                if (sorted[j].getPrice() > sorted[j + 1].getPrice()) {
                    // Swap
                    Product temp = sorted[j];
                    sorted[j] = sorted[j + 1];
                    sorted[j + 1] = temp;
                }
            }
        }

        return sorted;
    }
```

```
    // Calculate total inventory value
    public static double calculateTotalValue(Product[] products) {
        double total = 0;
        for (Product p : products) {
            total += p.getPrice();
        }
        return total;
    }

    // Apply discount to all products
    public static void applyDiscount(Product[] products, double discountPercent) {
        for (int i = 0; i < products.length; i++) {
            double newPrice = products[i].getPrice() * (1 - discountPercent / 100);
            products[i] = new Product(
                products[i].getId(),
                products[i].getName(),
                newPrice
            );
        }
    }

    public static void main(String[] args) {
        System.out.println("=== PRODUCT CATALOG WITH ARRAYS ===\n");

        // Initialize products
        Product[] products = initializeProducts();

        // Display all products
        System.out.println("--- All Products ---");
        for (Product p : products) {
            System.out.println(p);
        }

        // Search by ID
        System.out.println("\n--- Search by ID ---");
        Product found = findProductById(products, 3);
        System.out.println("Found: " + found);

        // Price range search
        System.out.println("\n--- Price Range ($200-$500) ---");
        Product[] inRange = findByPriceRange(products, 200, 500);
        for (Product p : inRange) {
            System.out.println(p);
        }

        // Sort by price
        System.out.println("\n--- Sorted by Price ---");
        Product[] sorted = sortByPrice(products);
        for (Product p : sorted) {
            System.out.println(p);
        }
```

```
        // Calculate total
        System.out.println("\n--- Total Inventory Value ---");
        System.out.println("Total: $" + calculateTotalValue(products));

        // Apply discount
        System.out.println("\n--- After 10% Discount ---");
        applyDiscount(products, 10);
        for (Product p : products) {
            System.out.println(p);
        }

        System.out.println("\n=== ANGULAR USE CASES ===");
        System.out.println("1. Product listing component (*ngFor)");
        System.out.println("2. Product filtering (price range slider)");
        System.out.println("3. Sorting (price, name, popularity)");
        System.out.println("4. Search functionality");
        System.out.println("5. Shopping cart calculations");
    }
}
```

19

Exception Handling

19_Exception_Handling\examples\19_Exception_Handling.md

Java Exception Handling

Table of Contents

1 Introduction to Exceptions
2 Exception Types
3 Try-Catch-Finally
4 Throwing Exceptions
5 Custom Exceptions
6 Best Practices
7 Code Examples
8 Exercises
9 Solutions

---

1. Introduction to Exceptions

What is an Exception?

An **exception** is an event that disrupts the normal flow of program execution.

```
■■■
```

Type	Description	Examples
Checked	Compile-time checked	IOException, SQLException
Unchecked	Runtime exceptions	NullPointerException, ArrayIndexOutOfBoundsException

Common Exceptions

```
// Arithmetic
ArithmeticException: / by zero

// Null Pointer
NullPointerException: accessing null object

// Array
ArrayIndexOutOfBoundsException
ArrayStoreException

// I/O
FileNotFoundException
IOException

// Number
NumberFormatException
```

---

3. Try-Catch-Finally

Basic Syntax

```
try {
    // Code that might throw exception
} catch (ExceptionType1 e) {
    // Handle ExceptionType1
} catch (ExceptionType2 e) {
    // Handle ExceptionType2
} finally {
    // Always executes (cleanup)
}
```

---

4. Throwing Exceptions

Using throw and throws

```
public void method() throws IOException {
    // throws declares exceptions

    if (somethingWrong) {
        throw new IOException("Error message");  // throw throws the exception
    }
}
```

5. Custom Exceptions

Creating Custom Exceptions

```
// Custom unchecked exception
class InvalidAgeException extends RuntimeException {
    public InvalidAgeException(String message) {
        super(message);
    }
}

// Custom checked exception
class BusinessException extends Exception {
    public BusinessException(String message) {
        super(message);
    }
}
```

6. Best Practices

1. Catch specific exceptions, not Exception
2. Don't swallow exceptions
3. Use finally for cleanup
4. Throw early, catch late
5. Document exceptions with throws

7. Code Examples

Example 1: Basic Exception Handling

```java
import java.util.Scanner;

/**
 * BasicExceptionHandling - Core exception handling patterns
 */
public class BasicExceptionHandling {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("=== BASIC EXCEPTION HANDLING ===\n");

        // Division with try-catch
        System.out.print("Enter dividend: ");
        int dividend = scanner.nextInt();
        System.out.print("Enter divisor: ");
        int divisor = scanner.nextInt();

        try {
            int result = dividend / divisor;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero!");
        }

        // Array access
        System.out.println("\n--- Array Access ---");
        int[] numbers = {1, 2, 3, 4, 5};

        try {
            System.out.println("Element at index 10: " + numbers[10]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Index out of bounds!");
        }

        // Multiple catch blocks
        System.out.println("\n--- Multiple Catch ---");
        String[] inputs = {"5", "abc", "null"};

        for (String input : inputs) {
            try {
                int num = Integer.parseInt(input);
                System.out.println("Parsed: " + num);
            } catch (NumberFormatException e) {
                System.out.println("Cannot parse '" + input + "': " + e.getMessage());
            }
        }

        scanner.close();
    }
}
```

Example 2: Try-With-Resources

```
import java.io.*;

/**
 * TryWithResources - Automatic resource management
 * (Java 7+)
 */
public class TryWithResources {

    public static void main(String[] args) {
        System.out.println("=== TRY WITH RESOURCES ===\n");

        // Read file with try-with-resources
        readFile("nonexistent.txt");

        // Create and read temp file
        createAndReadTempFile();
    }

    public static void readFile(String filename) {
        // AutoCloseable resources
        try (FileReader reader = new FileReader(filename)) {
            int ch;
            while ((ch = reader.read()) != -1) {
                System.out.print((char) ch);
            }
        } catch (FileNotFoundException e) {
            System.out.println("File not found: " + filename);
        } catch (IOException e) {
            System.out.println("IO Error: " + e.getMessage());
        }
    }

    public static void createAndReadTempFile() {
        File tempFile = new File("temp.txt");

        // Write to file
        try (FileWriter writer = new FileWriter(tempFile)) {
            writer.write("Hello, World!\n");
            writer.write("Java Exception Handling");
            System.out.println("Written to file successfully");
        } catch (IOException e) {
            System.out.println("Write error: " + e.getMessage());
        }

        // Read from file
        try (BufferedReader reader = new BufferedReader(new FileReader(tempFile))) {
            String line;
            System.out.println("\nFile contents:");
            while ((line = reader.readLine()) != null) {
                System.out.println("  " + line);
```

```
            }
        } catch (IOException e) {
            System.out.println("Read error: " + e.getMessage());
        } finally {
            // Cleanup
            if (tempFile.exists()) {
                tempFile.delete();
                System.out.println("\nTemp file deleted");
            }
        }
    }
}
```

Example 3: Custom Exceptions

```
/**
 * Custom exception for validation
 */
class InvalidInputException extends Exception {
    public InvalidInputException(String message) {
        super(message);
    }
}

/**
 * Custom unchecked exception
 */
class ValidationException extends RuntimeException {
    public ValidationException(String message) {
        super(message);
    }
}

/**
 * BankAccount with exception handling
 */
class BankAccount {
    private double balance;
    private String accountNumber;

    public BankAccount(String accountNumber, double initialBalance) {
        this.accountNumber = accountNumber;
        if (initialBalance < 0) {
            throw new ValidationException("Initial balance cannot be negative");
        }
        this.balance = initialBalance;
    }

    public void withdraw(double amount) throws InvalidInputException {
        if (amount <= 0) {
```

```
            throw new InvalidInputException("Withdrawal amount must be positive");
        }
        if (amount > balance) {
            throw new InvalidInputException("Insufficient balance. Available: " + balance);
        }
        balance -= amount;
        System.out.println("Withdrawn: $" + amount);
    }

    public void deposit(double amount) throws InvalidInputException {
        if (amount <= 0) {
            throw new InvalidInputException("Deposit amount must be positive");
        }
        balance += amount;
        System.out.println("Deposited: $" + amount);
    }

    public double getBalance() {
        return balance;
    }
}

/**
 * CustomExceptionDemo - Demonstrates custom exceptions
 */
public class CustomExceptionDemo {

    public static void main(String[] args) {
        System.out.println("=== CUSTOM EXCEPTIONS DEMO ===\n");

        BankAccount account = new BankAccount("ACC123", 1000);
        System.out.println("Initial balance: $" + account.getBalance());

        // Valid deposit
        try {
            account.deposit(500);
        } catch (InvalidInputException e) {
            System.out.println("Error: " + e.getMessage());
        }

        // Invalid deposit
        try {
            account.deposit(-100);
        } catch (InvalidInputException e) {
            System.out.println("Error: " + e.getMessage());
        }

        // Valid withdrawal
        try {
            account.withdraw(300);
        } catch (InvalidInputException e) {
            System.out.println("Error: " + e.getMessage());
        }
```

```
        // Overdraft attempt
        try {
            account.withdraw(5000);
        } catch (InvalidInputException e) {
            System.out.println("Error: " + e.getMessage());
        }

        System.out.println("\nFinal balance: $" + account.getBalance());

        // Test validation exception (unchecked)
        System.out.println("\n--- Unchecked Exception ---");
        try {
            BankAccount badAccount = new BankAccount("ACC999", -100);
        } catch (ValidationException e) {
            System.out.println("Caught validation error: " + e.getMessage());
        }
    }
}
```

---

8. Exercises

Exercise 1: Calculator with Exception Handling

Requirements:

1. Create calculator with divide operation
2. Handle division by zero
3. Handle invalid input
4. Use custom exception

Exercise 2: Student Grade Validator

Requirements:

1. Validate grade (0-100)
2. Throw custom exception for invalid grades
3. Handle and display appropriate messages

---

9. Solutions

Solution 1: Calculator with Exception Handling

```
class CalculatorException extends Exception {
    public CalculatorException(String message) {
        super(message);
    }
}

class Calculator {
    public double divide(double a, double b) throws CalculatorException {
        if (b == 0) {
            throw new CalculatorException("Cannot divide by zero");
        }
        return a / b;
    }
}

public class CalculatorDemo {
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        try {
            System.out.println("10 / 2 = " + calc.divide(10, 2));
            System.out.println("10 / 0 = " + calc.divide(10, 0));
        } catch (CalculatorException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Solution 2: Student Grade Validator

```
class InvalidGradeException extends Exception {
    public InvalidGradeException(String message) {
        super(message);
    }
}

class GradeValidator {
    public void validateGrade(int grade) throws InvalidGradeException {
        if (grade < 0 || grade > 100) {
            throw new InvalidGradeException("Grade must be between 0 and 100, got: " + grade);
        }
        System.out.println("Valid grade: " + grade);
    }
}

public class GradeDemo {
    public static void main(String[] args) {
        GradeValidator validator = new GradeValidator();
        int[] grades = {85, -5, 100, 150};
```

```
        for (int grade : grades) {
            try {
                validator.validateGrade(grade);
            } catch (InvalidGradeException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
}
```

Summary

Key Takeaways

1. **Throwable** is the parent of all exceptions
2. **Checked** exceptions must be declared or caught
3. **Unchecked** exceptions occur at runtime
4. **try-with-resources** auto-closes resources (Java 7+)
5. **Custom exceptions** for domain-specific errors

Happy Coding! ■

Code Examples

ExceptionHandlingDemo.java

ExceptionHandlingDemo.java

```
// ExceptionHandlingDemo - Demonstrates Java Exception Handling
// Essential for building robust Java backends that Angular can communicate with

import java.util.Scanner;

public class ExceptionHandlingDemo {

    // ===== BASIC TRY-CATCH =====
    public static void basicTryCatch() {
        System.out.println("=== BASIC TRY-CATCH ===");

        try {
            int[] numbers = {1, 2, 3};
            System.out.println("Accessing index 5: " + numbers[5]);  // ArrayIndexOutOfBoundsExce...
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
```

```
        System.out.println("Program continues after exception!\n");
    }

    // ===== MULTIPLE CATCH BLOCKS =====
    public static void multipleCatch() {
        System.out.println("=== MULTIPLE CATCH BLOCKS ===");

        try {
            int result = 10 / 0;  // ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic error: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("General exception: " + e.getMessage());
        }

        System.out.println();
    }

    // ===== TRY-CATCH-FINALLY =====
    public static void tryCatchFinally() {
        System.out.println("=== TRY-CATCH-FINALLY ===");

        try {
            String str = null;
            System.out.println("Length: " + str.length());  // NullPointerException
        } catch (NullPointerException e) {
            System.out.println("Null pointer exception caught");
        } finally {
            System.out.println("Finally block - always executes!");
        }

        System.out.println();
    }

    // ===== THROW AND THROWS =====
    public static void throwAndThrows() throws IllegalArgumentException {
        System.out.println("=== THROW AND THROWS ===");

        int age = -5;

        if (age < 0) {
            throw new IllegalArgumentException("Age cannot be negative!");
        }
    }

    // ===== CUSTOM EXCEPTION =====
    public static void customExceptionDemo() {
        System.out.println("=== CUSTOM EXCEPTION ===");

        try {
            validateAge(-25);
        } catch (InvalidAgeException e) {
            System.out.println("Custom Exception: " + e.getMessage());
```

```
        }
    }

    public static void validateAge(int age) throws InvalidAgeException {
        if (age < 0 || age > 150) {
            throw new InvalidAgeException("Invalid age: " + age);
        }
        System.out.println("Valid age: " + age);
    }

    // ===== TRY-WITH-RESOURCES (Java 7+) =====
    public static void tryWithResourcesDemo() {
        System.out.println("\n=== TRY-WITH-RESOURCES ===");

        // This automatically closes resources
        try (Scanner scanner = new Scanner(System.in)) {
            System.out.println("Enter a number: ");
            // int num = scanner.nextInt();  // Uncomment to test
            System.out.println("Resource automatically closed!");
        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }

    // ===== EXCEPTION HIERARCHY =====
    public static void exceptionHierarchy() {
        System.out.println("=== EXCEPTION HIERARCHY ===");
        System.out.println("Throwable");
        System.out.println("  ■■■ Error (unchecked) - System errors");
        System.out.println("  ■■■ Exception");
        System.out.println("        ■■■ RuntimeException (unchecked)");
        System.out.println("        ■     ■■■ NullPointerException");
        System.out.println("        ■     ■■■ ArrayIndexOutOfBoundsException");
        System.out.println("        ■     ■■■ ArithmeticException");
        System.out.println("        ■■■ Checked Exceptions");
        System.out.println("              ■■■ IOException");
        System.out.println("              ■■■ SQLException");
    }

    // Main method
    public static void main(String[] args) {
        System.out.println("=== EXCEPTION HANDLING DEMO ===\n");

        basicTryCatch();
        multipleCatch();
        tryCatchFinally();

        try {
            throwAndThrows();
        } catch (IllegalArgumentException e) {
            System.out.println("Caught: " + e.getMessage());
        }
```

```
        customExceptionDemo();
        exceptionHierarchy();

        System.out.println("\n=== BEST PRACTICES ===");
        System.out.println("1. Catch specific exceptions first");
        System.out.println("2. Always use finally for cleanup");
        System.out.println("3. Use try-with-resources for AutoCloseable");
        System.out.println("4. Don't swallow exceptions - at least log them");
        System.out.println("5. Create custom exceptions for business rules");
    }
}

// Custom Exception Class
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}
```

19

Methods

Code Examples

AngularServiceDemo.java

AngularServiceDemo.java

```
// AngularServiceDemo - Demonstrates Method Types for Angular Service Layer
// Use case: Backend service methods that Angular services call

public class AngularServiceDemo {

    // Service class representing an Angular service
    static class UserService {

        // Static method - Configuration/factory
        public static UserService createService() {
            System.out.println("UserService created");
            return new UserService();
        }

        // Instance method - Regular operation
        public String getUserById(int id) {
            return "User-" + id;
        }

        // Instance method - Data processing
        public String[] getAllUsers() {
            return new String[]{"Alice", "Bob", "Charlie"};
        }

        // Method with parameters
        public boolean validateUser(String username, String password) {
            return username.length() >= 3 && password.length() >= 6;
        }

        // Method with return value
        public int getUserCount() {
            return 42;
        }

        // Method with multiple parameters
        public String createUser(String name, String email, String role) {
            return "Created: " + name + " (" + email + ") as " + role;
        }

        // Method overloading
        public String getUser(int id) { return "User-" + id; }
        public String getUser(int id, boolean withDetails) {
            return withDetails ? "User-" + id + " [detailed]" : "User-" + id;
        }
    }
```

```
    // Utility class with static methods
    static class ValidationUtils {

        public static boolean isValidEmail(String email) {
            return email != null && email.contains("@");
        }

        public static boolean isValidPassword(String password) {
            return password != null && password.length() >= 8;
        }

        public static String sanitize(String input) {
            return input == null ? "" : input.trim();
        }
    }

    public static void main(String[] args) {
        System.out.println("=== METHOD TYPES FOR ANGULAR SERVICES ===\n");

        // Static method usage
        System.out.println("--- Static Method (Factory) ---");
        UserService service = UserService.createService();

        // Instance methods
        System.out.println("\n--- Instance Methods ---");
        System.out.println("getUserById(1): " + service.getUserById(1));

        String[] users = service.getAllUsers();
        for (String user : users) {
            System.out.println("User: " + user);
        }

        // Parameter methods
        System.out.println("\n--- Parameter Methods ---");
        System.out.println("validateUser: " + service.validateUser("john", "pass123"));
        System.out.println("createUser: " + service.createUser("John", "john@email.com", "USER"));

        // Method overloading
        System.out.println("\n--- Method Overloading ---");
        System.out.println("getUser(1): " + service.getUser(1));
        System.out.println("getUser(1, true): " + service.getUser(1, true));

        // Static utilities
        System.out.println("\n--- Static Utilities ---");
        System.out.println("isValidEmail: " + ValidationUtils.isValidEmail("test@email.com"));
        System.out.println("isValidPassword: " + ValidationUtils.isValidPassword("password123"));
        System.out.println("sanitize: '" + ValidationUtils.sanitize("  hello  ") + "'");

        System.out.println("\n=== ANGULAR SERVICE PARALLELS ===");
        System.out.println("1. HTTP calls (getUser, getAllUsers)");
        System.out.println("2. Form validation (validateUser)");
        System.out.println("3. Data transformation (sanitize)");
```

```
        System.out.println("4. Factory methods (createService)");
        System.out.println("5. Utility functions (static methods)");
    }
}
```

20

Collections Framework

20_Collections_Framework\examples\20_Collections_Framework.md

Java Collections Framework

Table of Contents

1 Introduction
2 Collection Hierarchy
3 List Implementations
4 Set Implementations
5 Map Implementations
6 Queue Implementations
7 Collections Utility Class
8 Code Examples
9 Exercises
10 Solutions

---

1. Introduction

What is the Collections Framework?

The **Java Collections Framework** provides a set of interfaces and classes for storing and manipulating groups of objects.

```
■■■
```

2. Collection Hierarchy

Core Interfaces

Interface	Description	Key Methods
`Collection`	Root interface	add, remove, size, iterator
`List`	Ordered, indexed	get(i), set(i), add(i), remove(i)
`Set`	No duplicates	add, remove, contains
`Queue`	FIFO operations	offer, poll, peek
`Map`	Key-value pairs	put, get, keySet, values

---

3. List Implementations

ArrayList

```
// ArrayList - Dynamic array, most common
List<String> arrayList = new ArrayList<>();
arrayList.add("Apple");
arrayList.add("Banana");
arrayList.get(0);  // "Apple"
```

LinkedList

```
// LinkedList - Doubly linked list
List<String> linkedList = new LinkedList<>();
linkedList.add("Apple");
linkedList.addFirst("Banana");  // Add at beginning
linkedList.addLast("Cherry");   // Add at end
```

Vector

```
// Vector - Synchronized (thread-safe) ArrayList
List<String> vector = new Vector<>();
```

Comparison

Feature	ArrayList	LinkedList	Vector
Access	O(1)	O(n)	O(1)
Insert/Delete	O(n)	O(1)	O(n)
Thread-safe	No	No	Yes
Growth	50%	Double	100%

4. Set Implementations

HashSet

```
// HashSet - Unordered, no duplicates
Set<String> hashSet = new HashSet<>();
hashSet.add("Apple");
hashSet.add("Apple");  // Won't be added (duplicate)
```

LinkedHashSet

```
// LinkedHashSet - Maintains insertion order
Set<String> linkedHashSet = new LinkedHashSet<>();
```

TreeSet

```
// TreeSet - Sorted order, implements NavigableSet
Set<String> treeSet = new TreeSet<>();
```

5. Map Implementations

HashMap

```
// HashMap - Key-value, no order
Map<String, Integer> hashMap = new HashMap<>();
hashMap.put("Apple", 10);
hashMap.put("Banana", 20);
hashMap.get("Apple");  // 10
```

LinkedHashMap

```
// LinkedHashMap - Maintains insertion order
Map<String, Integer> linkedHashMap = new LinkedHashMap<>();
```

TreeMap

```
// TreeMap - Sorted by keys
Map<String, Integer> treeMap = new TreeMap<>();
```

Hashtable

```
// Hashtable - Synchronized, legacy
Map<String, Integer> hashtable = new Hashtable<>();
```

---

6. Queue Implementations

PriorityQueue

```
// PriorityQueue - Elements ordered by priority
Queue<Integer> priorityQueue = new PriorityQueue<>();
priorityQueue.offer(5);
priorityQueue.offer(1);
priorityQueue.poll();  // Returns 1 (highest priority)
```

ArrayDeque

```
// ArrayDeque - Double-ended queue
Deque<String> deque = new ArrayDeque<>();
deque.addFirst("A");
deque.addLast("B");
deque.removeFirst();  // "A"
```

---

7. Collections Utility Class

Common Methods

```
import java.util.Collections;
import java.util.List;
import java.util.ArrayList;

// Sorting
Collections.sort(list);

// Reverse
Collections.reverse(list);

// Shuffle
Collections.shuffle(list);

// Synchronized collection
List<String> syncList = Collections.synchronizedList(new ArrayList<>());

// Unmodifiable collection
List<String> unmodifiable = Collections.unmodifiableList(list);

// Binary search (list must be sorted!)
int index = Collections.binarySearch(sortedList, target);
```

---

8. Code Examples

Example 1: List Operations

```
import java.util.*;

/**
 * ListOperationsDemo - Comprehensive List demonstrations
 */
public class ListOperationsDemo {

    public static void main(String[] args) {
        System.out.println("=== LIST OPERATIONS DEMO ===\n");

        // ArrayList
        List<String> fruits = new ArrayList<>();

        // Add elements
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Cherry");
        fruits.add(1, "Avocado");  // Insert at index

        System.out.println("Fruits: " + fruits);

        // Access elements
```

```
        System.out.println("First: " + fruits.get(0));
        System.out.println("Size: " + fruits.size());

        // Modify
        fruits.set(0, "Apricot");
        System.out.println("After set: " + fruits);

        // Remove
        fruits.remove("Banana");
        fruits.remove(0);  // By index
        System.out.println("After remove: " + fruits);

        // Search
        System.out.println("Contains Cherry? " + fruits.contains("Cherry"));
        System.out.println("Index of Cherry: " + fruits.indexOf("Cherry"));

        // Iterate
        System.out.println("\n--- Iteration ---");
        for (String fruit : fruits) {
            System.out.println("  " + fruit);
        }

        // Sublist
        List<String> sublist = fruits.subList(0, 2);
        System.out.println("Sublist: " + sublist);

        // Array to List and vice versa
        String[] arr = {"X", "Y", "Z"};
        List<String> listFromArray = Arrays.asList(arr);
        System.out.println("List from array: " + listFromArray);
    }
}
```

Example 2: Set Operations

```
import java.util.*;

/**
 * SetOperationsDemo - Comprehensive Set demonstrations
 */
public class SetOperationsDemo {

    public static void main(String[] args) {
        System.out.println("=== SET OPERATIONS DEMO ===\n");

        // HashSet - Unordered
        Set<String> hashSet = new HashSet<>();
        hashSet.add("Apple");
        hashSet.add("Banana");
        hashSet.add("Apple");  // Duplicate - ignored
```

```
        hashSet.add(null);     // Allows one null

        System.out.println("HashSet: " + hashSet);

        // LinkedHashSet - Insertion order
        Set<String> linkedSet = new LinkedHashSet<>();
        linkedSet.add("First");
        linkedSet.add("Second");
        linkedSet.add("Third");
        System.out.println("LinkedHashSet: " + linkedSet);

        // TreeSet - Sorted order
        Set<String> treeSet = new TreeSet<>();
        treeSet.add("Zebra");
        treeSet.add("Apple");
        treeSet.add("Monkey");
        System.out.println("TreeSet: " + treeSet);

        // Set operations
        Set<Integer> setA = new HashSet<>(Arrays.asList(1, 2, 3, 4, 5));
        Set<Integer> setB = new HashSet<>(Arrays.asList(4, 5, 6, 7, 8));

        System.out.println("\n--- Set Operations ---");
        System.out.println("Set A: " + setA);
        System.out.println("Set B: " + setB);

        // Union
        Set<Integer> union = new HashSet<>(setA);
        union.addAll(setB);
        System.out.println("Union: " + union);

        // Intersection
        Set<Integer> intersection = new HashSet<>(setA);
        intersection.retainAll(setB);
        System.out.println("Intersection: " + intersection);

        // Difference
        Set<Integer> difference = new HashSet<>(setA);
        difference.removeAll(setB);
        System.out.println("A - B: " + difference);
    }
}
```

Example 3: Map Operations

```
import java.util.*;

/**
 * MapOperationsDemo - Comprehensive Map demonstrations
 */
```

```java
public class MapOperationsDemo {

    public static void main(String[] args) {
        System.out.println("=== MAP OPERATIONS DEMO ===\n");

        // Create map
        Map<String, Integer> products = new HashMap<>();

        // Put entries
        products.put("Laptop", 999);
        products.put("Phone", 699);
        products.put("Tablet", 449);
        products.put("Laptop", 1099);  // Updates existing key

        System.out.println("Products: " + products);

        // Get value
        System.out.println("Laptop price: $" + products.get("Laptop"));
        System.out.println("Non-existent: " + products.getOrDefault("TV", 0));

        // Check existence
        System.out.println("Contains 'Phone'? " + products.containsKey("Phone"));
        System.out.println("Contains value 699? " + products.containsValue(699));

        // Remove
        products.remove("Tablet");
        System.out.println("After remove: " + products);

        // Iterate
        System.out.println("\n--- Iteration ---");
        for (Map.Entry<String, Integer> entry : products.entrySet()) {
            System.out.println(entry.getKey() + " = $" + entry.getValue());
        }

        // Keys only
        System.out.println("\nKeys: " + products.keySet());

        // Values only
        System.out.println("Values: " + products.values());

        // TreeMap - Sorted by keys
        System.out.println("\n--- TreeMap (Sorted) ---");
        Map<String, Integer> sortedMap = new TreeMap<>(products);
        System.out.println(sortedMap);

        // Get methods
        System.out.println("\nFirst key: " + sortedMap.firstKey());
        System.out.println("Last key: " + sortedMap.lastKey());
    }
}
```

Example 4: Queue and Stack

```
import java.util.*;

/**
 * QueueStackDemo - Queue and Stack demonstrations
 */
public class QueueStackDemo {

    public static void main(String[] args) {
        System.out.println("=== QUEUE AND STACK DEMO ===\n");

        // PriorityQueue - Smallest element first (by default)
        System.out.println("--- PriorityQueue ---");
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        pq.offer(30);
        pq.offer(10);
        pq.offer(50);
        pq.offer(20);

        System.out.println("Added: 30, 10, 50, 20");
        System.out.println("Poll (removes smallest): " + pq.poll());
        System.out.println("Poll: " + pq.poll());
        System.out.println("Poll: " + pq.poll());

        // ArrayDeque - Can be used as Queue or Stack
        System.out.println("\n--- ArrayDeque ---");
        Deque<String> deque = new ArrayDeque<>();

        // Queue operations (FIFO)
        deque.offer("First");
        deque.offer("Second");
        deque.offer("Third");

        System.out.println("Queue poll: " + deque.poll());
        System.out.println("Queue poll: " + deque.poll());

        // Stack operations (LIFO)
        deque.push("A");
        deque.push("B");
        deque.push("C");

        System.out.println("\nStack pop: " + deque.pop());
        System.out.println("Stack pop: " + deque.pop());
        System.out.println("Stack pop: " + deque.pop());
    }
}
```

Example 5: Collections Utility

```java
import java.util.*;

/**
 * CollectionsUtilityDemo - Collections utility methods
 */
public class CollectionsUtilityDemo {

    public static void main(String[] args) {
        System.out.println("=== COLLECTIONS UTILITY DEMO ===\n");

        List<Integer> numbers = new ArrayList<>(Arrays.asList(5, 2, 8, 1, 9, 3, 7, 4, 6));

        System.out.println("Original: " + numbers);

        // Sort
        Collections.sort(numbers);
        System.out.println("Sorted: " + numbers);

        // Reverse
        Collections.reverse(numbers);
        System.out.println("Reversed: " + numbers);

        // Shuffle
        Collections.shuffle(numbers);
        System.out.println("Shuffled: " + numbers);

        // Binary search (list must be sorted!)
        Collections.sort(numbers);
        int index = Collections.binarySearch(numbers, 5);
        System.out.println("Index of 5: " + index);

        // Min/Max
        System.out.println("Min: " + Collections.min(numbers));
        System.out.println("Max: " + Collections.max(numbers));

        // Fill
        List<String> list = new ArrayList<>(Arrays.asList("a", "b", "c"));
        Collections.fill(list, "X");
        System.out.println("After fill: " + list);

        // Frequency
        List<String> words = Arrays.asList("apple", "banana", "apple", "cherry", "apple");
        System.out.println("'apple' count: " + Collections.frequency(words, "apple"));

        // Synchronized collection
        List<String> syncList = Collections.synchronizedList(new ArrayList<>());
        System.out.println("Synchronized list created");

        // Unmodifiable collection
        List<String> unmodifiable = Collections.unmodifiableList(Arrays.asList("a", "b"));
        System.out.println("Unmodifiable list: " + unmodifiable);
    }
}
```

9. Exercises

Exercise 1: Student Management System

Requirements:

1. Use ArrayList to store Student objects
2. Add, remove, search students by ID
3. Sort students by name and GPA

Exercise 2: Word Frequency Counter

Requirements:

1. Count occurrences of each word in text
2. Use HashMap for counting
3. Display words sorted alphabetically

10. Solutions

Solution 1: Student Management System

```
import java.util.*;

class Student {
    private int id;
    private String name;
    private double gpa;

    public Student(int id, String name, double gpa) {
        this.id = id;
        this.name = name;
        this.gpa = gpa;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public double getGpa() { return gpa; }

    @Override
    public String toString() {
        return "Student{id=" + id + ", name='" + name + "', gpa=" + gpa + "}";
    }
```

```
}

public class StudentManagement {
    public static void main(String[] args) {
        List<Student> students = new ArrayList<>();

        students.add(new Student(1, "Alice", 3.8));
        students.add(new Student(2, "Bob", 3.5));
        students.add(new Student(3, "Charlie", 3.9));

        // Sort by name
        students.sort(Comparator.comparing(Student::getName));
        System.out.println("Sorted by name: " + students);

        // Sort by GPA descending
        students.sort(Comparator.comparing(Student::getGpa).reversed());
        System.out.println("Sorted by GPA: " + students);

        // Search by ID
        int searchId = 2;
        for (Student s : students) {
            if (s.getId() == searchId) {
                System.out.println("Found: " + s);
            }
        }
    }
}
```

Solution 2: Word Frequency Counter

```
import java.util.*;

public class WordFrequency {
    public static void main(String[] args) {
        String text = "the quick brown fox jumps over the lazy dog the dog was not amused";
        String[] words = text.toLowerCase().split("\\s+");

        Map<String, Integer> frequency = new HashMap<>();

        for (String word : words) {
            frequency.put(word, frequency.getOrDefault(word, 0) + 1);
        }

        // Sort by alphabetical order
        TreeMap<String, Integer> sorted = new TreeMap<>(frequency);

        System.out.println("Word frequencies:");
        for (Map.Entry<String, Integer> entry : sorted.entrySet()) {
            System.out.println(entry.getKey() + ": " + entry.getValue());
        }
```

```
    }
}
```

Summary

Key Takeaways

1. **List** - Ordered, indexed, allows duplicates (ArrayList, LinkedList)
2. **Set** - No duplicates (HashSet, LinkedHashSet, TreeSet)
3. **Map** - Key-value pairs (HashMap, TreeMap, LinkedHashMap)
4. **Queue** - FIFO operations (PriorityQueue, ArrayDeque)
5. **Collections utility** - Provides sorting, searching, synchronization

Angular Backend Connection

- Collections map to JSON arrays/objects
- Maps become JSON objects with key-value pairs
- Spring Data JPA uses collections for database operations

Happy Coding! ■

Code Examples

ArrayListDemo.java

ArrayListDemo.java

```
// ArrayListDemo - Demonstrates ArrayList in Java Collections Framework
// Important for Angular developers to understand dynamic lists

import java.util.*;

public class ArrayListDemo {

    public static void main(String[] args) {
        System.out.println("=== ARRAYLIST DEMO ===");

        // Create ArrayList
        ArrayList<String> fruits = new ArrayList<>();

        // Add elements
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Orange");
```

```
        fruits.add(1, "Mango");  // Insert at index 1

        System.out.println("Fruits: " + fruits);
        System.out.println("Size: " + fruits.size());
        System.out.println("First item: " + fruits.get(0));
        System.out.println("Contains 'Banana': " + fruits.contains("Banana"));

        // Modify
        fruits.set(0, "Green Apple");
        System.out.println("After set: " + fruits);

        // Remove
        fruits.remove("Banana");
        fruits.remove(0);
        System.out.println("After remove: " + fruits);

        // Iterate
        System.out.println("Using for-each:");
        for (String fruit : fruits) {
            System.out.println("  - " + fruit);
        }

        System.out.println("\n=== KEY OPERATIONS ===");
        System.out.println("add() - Add element");
        System.out.println("get() - Get element by index");
        System.out.println("set() - Update element");
        System.out.println("remove() - Remove element");
        System.out.println("size() - Get list size");
    }
}
```

HashMapDemo.java

HashMapDemo.java

```
// HashMapDemo - Demonstrates HashMap in Java Collections Framework
// Important for Angular developers - maps to JSON objects

import java.util.*;

public class HashMapDemo {

    public static void main(String[] args) {
        System.out.println("=== HASHMAP DEMO ===");

        // Create HashMap
        HashMap<Integer, String> studentMap = new HashMap<>();

        // Put elements
        studentMap.put(101, "Alice");
        studentMap.put(102, "Bob");
        studentMap.put(103, "Charlie");
```

```
        studentMap.put(104, "Diana");

        System.out.println("Students: " + studentMap);

        // Get by key
        System.out.println("Student 102: " + studentMap.get(102));

        // Check existence
        System.out.println("Contains key 105: " + studentMap.containsKey(105));
        System.out.println("Contains value 'Alice': " + studentMap.containsValue("Alice"));

        // Iterate
        System.out.println("All entries:");
        for (Map.Entry<Integer, String> entry : studentMap.entrySet()) {
            System.out.println("  ID: " + entry.getKey() + ", Name: " + entry.getValue());
        }

        // Remove
        studentMap.remove(103);
        System.out.println("After removing 103: " + studentMap);

        System.out.println("\n=== USE CASES ===");
        System.out.println("1. Store user data by ID");
        System.out.println("2. Cache frequently accessed data");
        System.out.println("3. Maps directly to JSON objects");
    }
}
```

HashSetDemo.java

HashSetDemo.java

```
// HashSetDemo - Demonstrates HashSet in Java Collections Framework
// Important for storing unique elements

import java.util.*;

public class HashSetDemo {

    public static void main(String[] args) {
        System.out.println("=== HASHSET DEMO ===");

        HashSet<Integer> numbers = new HashSet<>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(10);  // Duplicate - ignored

        System.out.println("Numbers: " + numbers);
        System.out.println("Size: " + numbers.size());
        System.out.println("Contains 20: " + numbers.contains(20));
```

```
        numbers.remove(20);
        System.out.println("After remove: " + numbers);

        System.out.println("\n=== KEY FEATURES ===");
        System.out.println("1. No duplicate elements");
        System.out.println("2. Fast lookup O(1)");
        System.out.println("3. Unordered collection");
    }
}
```

LinkedListDemo.java

LinkedListDemo.java

```
// LinkedListDemo - Demonstrates LinkedList in Java Collections Framework
// Important for queue and deque operations

import java.util.*;

public class LinkedListDemo {

    public static void main(String[] args) {
        System.out.println("=== LINKEDLIST DEMO ===");

        LinkedList<String> queue = new LinkedList<>();

        queue.add("First");
        queue.add("Second");
        queue.add("Third");

        System.out.println("Queue: " + queue);

        // Queue operations
        System.out.println("Peek: " + queue.peek());
        System.out.println("Poll: " + queue.poll());
        System.out.println("After poll: " + queue);

        // Add to front/back
        queue.addFirst("New First");
        queue.addLast("New Last");
        System.out.println("After additions: " + queue);

        System.out.println("\n=== KEY OPERATIONS ===");
        System.out.println("addFirst(), addLast() - Add elements");
        System.out.println("removeFirst(), removeLast() - Remove elements");
        System.out.println("peek() - Get without removing");
        System.out.println("poll() - Get and remove");
    }
}
```

PriorityQueueDemo.java

PriorityQueueDemo.java

```
// PriorityQueueDemo - Demonstrates PriorityQueue in Java Collections Framework
// Important for priority-based processing

import java.util.*;

public class PriorityQueueDemo {

    public static void main(String[] args) {
        System.out.println("=== PRIORITYQUEUE DEMO ===");

        PriorityQueue<Integer> pq = new PriorityQueue<>();

        pq.add(30);
        pq.add(10);
        pq.add(50);
        pq.add(20);

        System.out.println("Elements (in priority order):");
        while (!pq.isEmpty()) {
            System.out.print(pq.poll() + " ");
        }
        System.out.println();

        System.out.println("\n=== USE CASES ===");
        System.out.println("1. Task scheduling");
        System.out.println("2. Priority-based processing");
        System.out.println("3. Finding kth largest/smallest element");
    }
}
```

StackDemo.java

StackDemo.java

```
// StackDemo - Demonstrates Stack in Java Collections Framework
// Important for LIFO (Last In First Out) operations

import java.util.*;

public class StackDemo {

    public static void main(String[] args) {
        System.out.println("=== STACK DEMO ===");

        Stack<String> stack = new Stack<>();

        stack.push("First");
        stack.push("Second");
        stack.push("Third");
```

```
        System.out.println("Stack: " + stack);
        System.out.println("Peek: " + stack.peek());
        System.out.println("Pop: " + stack.pop());
        System.out.println("After pop: " + stack);

        System.out.println("\n=== USE CASES ===");
        System.out.println("1. Undo operations in editors");
        System.out.println("2. Browser back button");
        System.out.println("3. Expression evaluation");
    }
}
```

21

Generics

21_Generics\examples\21_Generics.md

Java Advanced Collections

Table of Contents

1 Advanced Set Operations
2 Advanced Map Operations
3 WeakHashMap and IdentityHashMap
4 Collections Utility Methods
5 Code Examples

---

1. Advanced Set Operations

NavigableSet and SortedSet

```
// TreeSet with NavigableSet methods
NavigableSet<Integer> treeSet = new TreeSet<>();

treeSet.add(5);
treeSet.add(2);
treeSet.add(8);
treeSet.add(1);

treeSet.lower(5);  // 2 (greatest < 5)
treeSet.higher(5); // 8 (least > 5)
treeSet.floor(5);  // 5 (greatest <= 5)
treeSet.ceiling(5); // 5 (least >= 5)
```

---

2. Advanced Map Operations

Compute Operations

```
Map<String, Integer> map = new HashMap<>();

// compute - compute value for key
map.compute("A", (k, v) -> (v == null) ? 1 : v + 1);

// computeIfAbsent - if key missing, compute and put
map.computeIfAbsent("B", k -> 100);
```

```
// computeIfPresent - if key exists, compute new value
map.computeIfPresent("A", (k, v) -> v + 10);

// merge - merge values
map.merge("C", 50, (v1, v2) -> v1 + v2);
```

3. WeakHashMap and IdentityHashMap

WeakHashMap

```
// Keys can be garbage collected when no references
WeakHashMap<Key, String> weakMap = new WeakHashMap<>();
```

IdentityHashMap

```
// Uses == instead of equals() for key comparison
IdentityHashMap<String, String> identityMap = new IdentityHashMap<>();
```

4. Collections Utility Methods

Singleton Collections

```
// Singleton list (one element)
List<String> singleList = Collections.singletonList("Only one");

// Singleton set
Set<String> singleSet = Collections.singleton("Only one");

// Singleton map
Map<String, Integer> singleMap = Collections.singletonMap("Key", 1);
```

Empty Collections

```
List<String> emptyList = Collections.emptyList();
Set<String> emptySet = Collections.emptySet();
Map<String, Integer> emptyMap = Collections.emptyMap();
```

5. Code Examples

AdvancedCollectionsDemo

```
import java.util.*;

public class AdvancedCollectionsDemo {
    public static void main(String[] args) {
        System.out.println("=== ADVANCED COLLECTIONS DEMO ===\n");

        // NavigableSet
        NavigableSet<Integer> numbers = new TreeSet<>();
        numbers.addAll(Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10));

        System.out.println("NavigableSet: " + numbers);
        System.out.println("lower(5): " + numbers.lower(5));
        System.out.println("higher(5): " + numbers.higher(5));
        System.out.println("floor(5): " + numbers.floor(5));
        System.out.println("ceiling(5): " + numbers.ceiling(5));

        // SubSet
        System.out.println("subSet(3, 7): " + numbers.subSet(3, 7));

        // Map compute
        System.out.println("\n--- Map Compute Operations ---");
        Map<String, Integer> scores = new HashMap<>();
        scores.put("A", 10);

        scores.compute("A", (k, v) -> v + 5);
        System.out.println("After compute: " + scores);

        scores.computeIfAbsent("B", k -> 100);
        System.out.println("After computeIfAbsent: " + scores);

        // Merge
        scores.merge("C", 50, (v1, v2) -> v1 + v2);
        System.out.println("After merge: " + scores);
    }
}
```

Advanced Collections Complete!

Code Examples

AdvancedGenericsDemo.java

AdvancedGenericsDemo.java

```
// AdvancedGenericsDemo - More Generics Examples
// Generic classes, methods, and wildcards

// Generic class
class Container<T> {
    private T value;

    public void set(T value) { this.value = value; }
    public T get() { return value; }
}

// Bounded type - only accepts Number or subclasses
class NumberBox<T extends Number> {
    private T number;

    public void set(T number) { this.number = number; }
    public T get() { return number; }

    public double getDoubleValue() { return number.doubleValue(); }
}

// Generic method
class Utils {
    public static <T> void printArray(T[] array) {
        for (T item : array) {
            System.out.print(item + " ");
        }
        System.out.println();
    }

    public static <T extends Comparable<T>> T findMax(T a, T b) {
        return a.compareTo(b) > 0 ? a : b;
    }
}

// Multiple type parameters
class Pair<K, V> {
    private K key;
    private V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() { return key; }
    public V getValue() { return value; }
}

public class AdvancedGenericsDemo {

    public static void main(String[] args) {
```

```
        System.out.println("=== ADVANCED GENERICS ===\n");

        // Generic class
        System.out.println("--- Generic Class ---");
        Container<String> strContainer = new Container<>();
        strContainer.set("Hello Generics");
        System.out.println("String: " + strContainer.get());

        Container<Integer> intContainer = new Container<>();
        intContainer.set(42);
        System.out.println("Integer: " + intContainer.get());

        // Bounded type
        System.out.println("\n--- Bounded Type ---");
        NumberBox<Integer> numBox = new NumberBox<>();
        numBox.set(100);
        System.out.println("Double value: " + numBox.getDoubleValue());

        // Generic method
        System.out.println("\n--- Generic Method ---");
        String[] names = {"Alice", "Bob", "Charlie"};
        Utils.printArray(names);

        Integer[] nums = {1, 2, 3, 4, 5};
        Utils.printArray(nums);

        // Multiple type parameters
        System.out.println("\n--- Multiple Type Parameters ---");
        Pair<String, Integer> pair = new Pair<>("Age", 25);
        System.out.println("Key: " + pair.getKey() + ", Value: " + pair.getValue());

        // Wildcards
        System.out.println("\n--- Wildcards ---");
        System.out.println("? extends Number - Upper bound");
        System.out.println("? super Integer - Lower bound");
        System.out.println("? - Unbounded");
    }
}
```

21

Inheritance

Code Examples

AngularServiceHierarchyDemo.java

AngularServiceHierarchyDemo.java

```
// AngularServiceHierarchyDemo - Demonstrates Inheritance for Service Hierarchy
// Use case: Angular service inheritance patterns with Java backend

public class AngularServiceHierarchyDemo {

    // Base service class
    static class BaseService {
        protected String baseUrl;

        public BaseService() {
            this.baseUrl = "https://api.example.com";
        }

        protected String getFullUrl(String endpoint) {
            return baseUrl + endpoint;
        }

        public void log(String message) {
            System.out.println("[BASE] " + message);
        }
    }

    // User service extending base
    static class UserService extends BaseService {

        public UserService() {
            super(); // Call parent constructor
        }

        public String getUsers() {
            log("Fetching users from: " + getFullUrl("/users"));
            return "User list data";
        }

        public String getUserById(int id) {
            log("Fetching user " + id);
            return "User-" + id;
        }

        // Method overriding
        public void log(String message) {
            System.out.println("[USER] " + message);
        }
    }
```

```
    // Product service extending base
    static class ProductService extends BaseService {

        public ProductService() {
            this.baseUrl = "https://api.example.com/products";
        }

        public String getProducts() {
            log("Fetching products from: " + getFullUrl("/all"));
            return "Product list data";
        }

        public String getProductById(int id) {
            log("Fetching product " + id);
            return "Product-" + id;
        }

        @Override
        public void log(String message) {
            System.out.println("[PRODUCT] " + message);
        }
    }

    // Admin service extending UserService
    static class AdminService extends UserService {

        public AdminService() {
            this.baseUrl = "https://admin.api.example.com";
        }

        public String getAllData() {
            log("Admin access to all data");
            return "All sensitive data";
        }

        public String deleteUser(int id) {
            log("Deleting user " + id);
            return "Deleted";
        }
    }

    public static void main(String[] args) {
        System.out.println("=== INHERITANCE FOR SERVICE HIERARCHY ===\n");

        // Regular user service
        System.out.println("--- User Service ---");
        UserService userService = new UserService();
        userService.getUsers();
        userService.getUserById(1);

        // Product service
        System.out.println("\n--- Product Service ---");
        ProductService productService = new ProductService();
```

```
        productService.getProducts();
        productService.getProductById(1);

        // Admin service (multi-level inheritance)
        System.out.println("\n--- Admin Service (Multi-level) ---");
        AdminService adminService = new AdminService();
        adminService.getUsers();  // Inherited from UserService
        adminService.getAllData(); // Own method
        adminService.deleteUser(5); // Admin specific

        // Demonstrate inheritance chain
        System.out.println("\n--- Inheritance Chain ---");
        System.out.println("AdminService extends UserService extends BaseService");
        System.out.println("AdminService has access to:");
        System.out.println("- BaseService methods: getFullUrl()");
        System.out.println("- UserService methods: getUsers(), getUserById()");
        System.out.println("- AdminService methods: getAllData(), deleteUser()");

        System.out.println("\n=== ANGULAR PARALLELS ===");
        System.out.println("1. UserService extends BaseHttpService");
        System.out.println("2. ProductService extends BaseHttpService");
        System.out.println("3. AdminService extends UserService (privileges)");
        System.out.println("4. Shared authentication logic in base");
        System.out.println("5. Override HTTP interceptors");
    }
}
```

22

Generics Multithreading FileIO

22_Generics_Multithreading_FileIO\examples\22_Generics_Multithreading_FileIO.md

Java Advanced Topics: Generics, Multithreading, and File I/O

Table of Contents

1 Generics
2 Multithreading
3 File I/O Operations
4 Code Examples
5 Exercises
6 Solutions

Part 1: Generics

What are Generics?

Generics enable types (classes and interfaces) to be parameters when defining classes, interfaces, and methods.

```
■■■
```

```
// Generic class
class Box<T> {
    private T content;

    public void set(T content) { this.content = content; }
    public T get() { return content; }
}

// Usage
Box<String> stringBox = new Box<>();
stringBox.set("Hello");
String s = stringBox.get();  // No cast needed

Box<Integer> intBox = new Box<>();
intBox.set(42);
int i = intBox.get();
```

Generic Methods

```
public static <T> void printArray(T[] array) {
    for (T element : array) {
        System.out.println(element);
    }
}

// Usage
String[] strings = {"A", "B", "C"};
Integer[] numbers = {1, 2, 3};
printArray(strings);
printArray(numbers);
```

Bounded Type Parameters

```
// Only accept Number or its subclasses
class Calculator<T extends Number> {
    T value;

    public double doubleValue() {
        return value.doubleValue();
    }
}

// Usage
Calculator<Integer> intCalc = new Calculator<>();  // OK
Calculator<String> strCalc = new Calculator<>();   // Error!
```

Part 2: Multithreading

What is Multithreading?

Multithreading allows concurrent execution of multiple parts of a program.

```
■■■
```

```
t1.start();
t2.start();
```

Thread Lifecycle

```
    NEW ■■■ RUNNABLE ■■■ TERMINATED
              ■
              ▼
           BLOCKED/WAITING ■■■ RUNNABLE
```

Thread Synchronization

```
class Counter {
    private int count = 0;

    // Synchronized method
    public synchronized void increment() {
        count++;
    }

    public synchronized int getCount() {
        return count;
    }
}
```

Executor Service

```
import java.util.concurrent.*;

// Create thread pool
ExecutorService executor = Executors.newFixedThreadPool(3);

// Submit tasks
executor.submit(() -> System.out.println("Task 1"));
executor.submit(() -> System.out.println("Task 2"));

// Shutdown
executor.shutdown();
```

Part 3: File I/O Operations

File I/O Overview

```
■■■
```

```
import java.nio.file.*;

// Check if file exists
boolean exists = Files.exists(Path.of("file.txt"));

// Create file
Files.createFile(Path.of("newfile.txt"));

// Delete file
Files.delete(Path.of("file.txt"));

// Copy file
Files.copy(Path.of("source.txt"), Path.of("dest.txt"));

// Get file info
System.out.println(Files.size(Path.of("file.txt")));
System.out.println(Files.getLastModifiedTime(Path.of("file.txt")));
```

4. Code Examples

Example 1: Generic Repository

```
/**
 * Generic Repository - Pattern for data access
 */
class Repository<T> {
    private java.util.Map<Integer, T> database = new java.util.HashMap<>();
    private int idCounter = 0;

    public T save(T entity) {
        int id = ++idCounter;
        database.put(id, entity);
        System.out.println("Saved with id: " + id);
        return entity;
    }

    public T findById(int id) {
        return database.get(id);
    }

    public java.util.List<T> findAll() {
        return new java.util.ArrayList<>(database.values());
    }

    public void delete(int id) {
        database.remove(id);
    }
}
```

```
class User {
    private String name;
    public User(String name) { this.name = name; }
    @Override public String toString() { return "User: " + name; }
}

public class GenericRepositoryDemo {
    public static void main(String[] args) {
        Repository<User> userRepo = new Repository<>();

        userRepo.save(new User("Alice"));
        userRepo.save(new User("Bob"));

        System.out.println(userRepo.findAll());
    }
}
```

Example 2: Thread Synchronization

```
/**
 * BankAccount with thread-safe operations
 */
class BankAccount {
    private double balance;

    public BankAccount(double initialBalance) {
        this.balance = initialBalance;
    }

    public synchronized void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: $" + amount + ", Balance: $" + balance);
        }
    }

    public synchronized void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: $" + amount + ", Balance: $" + balance);
        }
    }

    public synchronized double getBalance() {
        return balance;
    }
}

public class ThreadSynchronizationDemo {
```

```
    public static void main(String[] args) throws InterruptedException {
        BankAccount account = new BankAccount(1000);

        // Create multiple threads
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                account.deposit(100);
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                account.withdraw(50);
            }
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final balance: $" + account.getBalance());
    }
}
```

Example 3: File Operations

```
import java.nio.file.*;
import java.io.*;

/**
 * FileOperationsDemo - Comprehensive file handling
 */
public class FileOperationsDemo {

    public static void main(String[] args) throws IOException {
        Path filePath = Paths.get("demo.txt");

        // Write to file
        String content = "Hello, Java File I/O!\nWelcome to the course.";
        Files.writeString(filePath, content);
        System.out.println("Written to file");

        // Read from file
        String readContent = Files.readString(filePath);
        System.out.println("\nFile content:\n" + readContent);

        // Read lines
        System.out.println("\nLine by line:");
```

```
        for (String line : Files.readAllLines(filePath)) {
            System.out.println("  " + line);
        }

        // Append to file
        Files.writeString(filePath, "\nAppended line",
            StandardOpenOptionAPPEND);

        // File info
        System.out.println("\nFile info:");
        System.out.println("  Exists: " + Files.exists(filePath));
        System.out.println("  Size: " + Files.size(filePath) + " bytes");
        System.out.println("  ReadOnly: " + Files.isReadOnly(filePath));

        // Delete file
        Files.delete(filePath);
        System.out.println("\nFile deleted");
    }
}
```

Example 4: Executor Service Demo

```
import java.util.concurrent.*;

/**
 * ExecutorServiceDemo - Thread pool example
 */
public class ExecutorServiceDemo {

    public static void main(String[] args) {
        // Create fixed thread pool
        ExecutorService executor = Executors.newFixedThreadPool(3);

        // Submit tasks
        for (int i = 1; i <= 10; i++) {
            final int taskId = i;
            executor.submit(() -> {
                System.out.println("Task " + taskId + " executing in " +
                    Thread.currentThread().getName());
                try {
                    Thread.sleep(500);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            });
        }

        // Shutdown
        executor.shutdown();
        try {
```

```
            if (!executor.awaitTermination(10, TimeUnit.SECONDS)) {
                executor.shutdownNow();
            }
        } catch (InterruptedException e) {
            executor.shutdownNow();
        }

        System.out.println("All tasks completed");
    }
}
```

---

5. Exercises

Exercise 1: Generic Stack

Requirements:

1. Create generic Stack class with push/pop operations
2. Use bounded type to accept only Numbers

Exercise 2: Producer-Consumer

Requirements:

1. Create producer thread that adds items
2. Create consumer thread that removes items
3. Use synchronized blocks

Exercise 3: Copy File

Requirements:

1. Copy content from one file to another
2. Use both traditional I/O and NIO

---

6. Solutions

Solution 1: Generic Stack

```
class Stack<T extends Number> {
    private java.util.Stack<T> stack = new java.util.Stack<>();
```

```
    public void push(T item) { stack.push(item); }
    public T pop() { return stack.pop(); }
    public boolean isEmpty() { return stack.isEmpty(); }
}

public class StackDemo {
    public static void main(String[] args) {
        Stack<Integer> intStack = new Stack<>();
        intStack.push(10);
        intStack.push(20);
        System.out.println("Pop: " + intStack.pop());
    }
}
```

Solution 2: Producer-Consumer

```
class SharedQueue {
    private java.util.Queue<Integer> queue = new java.util.LinkedList<>();
    private int maxSize = 5;

    public synchronized void produce(int value) {
        while (queue.size() >= maxSize) {
            try { wait(); } catch (InterruptedException e) {}
        }
        queue.add(value);
        System.out.println("Produced: " + value);
        notify();
    }

    public synchronized int consume() {
        while (queue.isEmpty()) {
            try { wait(); } catch (InterruptedException e) {}
        }
        int value = queue.poll();
        System.out.println("Consumed: " + value);
        notify();
        return value;
    }
}
```

Solution 3: Copy File

```
import java.nio.file.*;

public class CopyFile {
    public static void main(String[] args) throws Exception {
        // Using NIO
```

```
        Files.copy(Paths.get("source.txt"), Paths.get("dest.txt"));
        System.out.println("File copied successfully");
    }
}
```

Summary

Key Takeaways

1. **Generics** - Type-safe reusable code, bounded types restrict type parameters
2. **Multithreading** - Thread class, Runnable interface, synchronized keyword
3. **File I/O** - Byte/Character streams, NIO (Path, Files), try-with-resources

Happy Coding! ■

Code Examples

FileIODemo.java

FileIODemo.java

```
// FileIODemo - Demonstrates Java File I/O Operations
// Important for reading/writing files in backend

import java.io.*;
import java.nio.file.*;
import java.util.Scanner;

public class FileIODemo {

    public static void main(String[] args) {
        System.out.println("=== FILE I/O DEMO ===\n");

        String filename = "test.txt";

        // Write to file (try-with-resources)
        System.out.println("--- Writing to File ---");
        try (FileWriter writer = new FileWriter(filename)) {
            writer.write("Hello from Java File I/O!\n");
            writer.write("Line 2: This is a test.\n");
            writer.write("Line 3: End of file.");
            System.out.println("Written successfully!");
        } catch (IOException e) {
            System.out.println("Error writing: " + e.getMessage());
        }
```

```
        // Read from file
        System.out.println("\n--- Reading from File ---");
        try (FileReader reader = new FileReader(filename)) {
            int ch;
            while ((ch = reader.read()) != -1) {
                System.out.print((char) ch);
            }
        } catch (IOException e) {
            System.out.println("Error reading: " + e.getMessage());
        }

        // Using BufferedReader (more efficient)
        System.out.println("\n--- BufferedReader ---");
        try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println("> " + line);
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }

        // Using Scanner
        System.out.println("\n--- Using Scanner ---");
        try (Scanner scanner = new Scanner(new File(filename))) {
            scanner.useDelimiter("\\n");
            while (scanner.hasNext()) {
                System.out.println("~ " + scanner.next());
            }
        } catch (FileNotFoundException e) {
            System.out.println("File not found: " + e.getMessage());
        }

        // Using Files class (Java NIO)
        System.out.println("\n--- Java NIO Files ---");
        try {
            String content = Files.readString(Path.of(filename));
            System.out.println("Content:\n" + content);

            // Create new file
            String newFile = "output.txt";
            Files.writeString(Path.of(newFile), "Written via NIO\n");
            System.out.println("NIO write complete!");

            // List directory
            System.out.println("\nDirectory contents:");
            Files.list(Paths.get(".")).forEach(p -> System.out.println(p));
        } catch (IOException e) {
            System.out.println("NIO Error: " + e.getMessage());
        }

        // File operations
        System.out.println("\n--- File Operations ---");
```

```
        File file = new File(filename);
        System.out.println("Exists: " + file.exists());
        System.out.println("Can read: " + file.canRead());
        System.out.println("Can write: " + file.canWrite());
        System.out.println("Size: " + file.length() + " bytes");
    }
}
```

GenericsDemo.java

GenericsDemo.java

```
// GenericsDemo - Demonstrates Java Generics
// Generics enable type-safe collections and classes

public class GenericsDemo {

    // Generic class
    static class Box<T> {
        private T content;

        public void set(T content) { this.content = content; }
        public T get() { return content; }
    }

    // Generic method
    public static <T> void printArray(T[] array) {
        for (T element : array) {
            System.out.print(element + " ");
        }
        System.out.println();
    }

    // Bounded type parameter
    public static <T extends Number> double sum(T a, T b) {
        return a.doubleValue() + b.doubleValue();
    }

    public static void main(String[] args) {
        System.out.println("=== GENERICS DEMO ===\n");

        // Generic Box
        System.out.println("--- Generic Box ---");
        Box<String> stringBox = new Box<>();
        stringBox.set("Hello Generics");
        System.out.println("String box: " + stringBox.get());

        Box<Integer> intBox = new Box<>();
        intBox.set(42);
        System.out.println("Integer box: " + intBox.get());

        // Generic method
```

```
        System.out.println("\n--- Generic Method ---");
        Integer[] intArray = {1, 2, 3, 4, 5};
        String[] strArray = {"A", "B", "C"};

        System.out.print("Integer array: ");
        printArray(intArray);
        System.out.print("String array: ");
        printArray(strArray);

        // Bounded type
        System.out.println("\n--- Bounded Type ---");
        System.out.println("Sum of 5 and 10: " + sum(5, 10));
        System.out.println("Sum of 3.5 and 2.5: " + sum(3.5, 2.5));

        // Multiple type parameters
        System.out.println("\n--- Multiple Type Parameters ---");
        Pair<String, Integer> pair = new Pair<>("Age", 25);
        System.out.println("Key: " + pair.getKey() + ", Value: " + pair.getValue());

        // Wildcards
        System.out.println("\n--- Wildcards ---");
        java.util.List<Number> numbers = java.util.Arrays.asList(1, 2.5, 3);
        printList(numbers);
    }

    // Wildcard example
    public static void printList(java.util.List<?> list) {
        System.out.print("List: ");
        for (Object obj : list) {
            System.out.print(obj + " ");
        }
        System.out.println();
    }
}

// Pair class with two type parameters
class Pair<K, V> {
    private K key;
    private V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() { return key; }
    public V getValue() { return value; }
}
```

MultithreadingDemo.java

MultithreadingDemo.java

```
// MultithreadingDemo - Demonstrates Java Multithreading
// Important for concurrent backend operations

public class MultithreadingDemo {

    // Thread using Runnable interface
    static class MyRunnable implements Runnable {
        private String name;

        public MyRunnable(String name) {
            this.name = name;
        }

        public void run() {
            for (int i = 1; i <= 3; i++) {
                System.out.println(name + ": " + i);
                try {
                    Thread.sleep(500);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }
    }

    // Thread using Thread class
    static class MyThread extends Thread {
        private String name;

        public MyThread(String name) {
            this.name = name;
        }

        public void run() {
            for (int i = 1; i <= 3; i++) {
                System.out.println(name + ": " + i);
                try {
                    Thread.sleep(500);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }
    }

    public static void main(String[] args) {
        System.out.println("=== MULTITHREADING DEMO ===\n");

        // Method 1: Using Runnable
        System.out.println("--- Using Runnable ---");
        Runnable r1 = new MyRunnable("Thread-1");
        Thread t1 = new Thread(r1);
```

```
        t1.start();

        // Method 2: Using Thread class
        System.out.println("--- Using Thread Class ---");
        Thread t2 = new MyThread("Thread-2");
        t2.start();

        // Lambda (Java 8+)
        System.out.println("--- Using Lambda ---");
        Thread t3 = new Thread(() -> {
            for (int i = 1; i <= 3; i++) {
                System.out.println("Thread-3: " + i);
                try { Thread.sleep(500); } catch (Exception e) {}
            }
        });
        t3.start();

        // Join threads
        try {
            t1.join();
            t2.join();
            t3.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("\n--- Thread Methods ---");
        Thread mainThread = Thread.currentThread();
        System.out.println("Main thread: " + mainThread.getName());
        System.out.println("Thread count: " + Thread.activeCount());

        // Synchronization
        System.out.println("\n--- Synchronization ---");
        Counter counter = new Counter();
        Thread t4 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) counter.increment();
        });
        Thread t5 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) counter.increment();
        });
        t4.start();
        t5.start();
        try {
            t4.join();
            t5.join();
        } catch (InterruptedException e) {}
        System.out.println("Counter: " + counter.getCount());
    }
}

// Counter with synchronization
class Counter {
    private int count = 0;
```

```
    public synchronized void increment() {
        count++;
    }

    public synchronized int getCount() {
        return count;
    }
}
```

22

Polymorphism

Code Examples

PaymentProcessorDemo.java

PaymentProcessorDemo.java

```
// PaymentProcessorDemo - Demonstrates Polymorphism for Payment Processing
// Use case: Multiple payment methods in Angular e-commerce app

public class PaymentProcessorDemo {

    // Base payment class
    static abstract class Payment {
        protected double amount;

        public Payment(double amount) {
            this.amount = amount;
        }

        // Abstract method - must be implemented
        public abstract boolean process();

        // Concrete method
        public double getAmount() {
            return amount;
        }
    }

    // Credit card payment
    static class CreditCardPayment extends Payment {
        private String cardNumber;

        public CreditCardPayment(double amount, String cardNumber) {
            super(amount);
            this.cardNumber = cardNumber;
        }

        @Override
        public boolean process() {
            System.out.println("Processing Credit Card: $" + amount);
            System.out.println("Card: ****" + cardNumber.substring(cardNumber.length() - 4));
            return true;
        }
    }

    // PayPal payment
    static class PayPalPayment extends Payment {
        private String email;

        public PayPalPayment(double amount, String email) {
            super(amount);
```

```
            this.email = email;
        }

        @Override
        public boolean process() {
            System.out.println("Processing PayPal: $" + amount);
            System.out.println("Email: " + email);
            return true;
        }
    }

    // Crypto payment
    static class CryptoPayment extends Payment {
        private String walletAddress;

        public CryptoPayment(double amount, String walletAddress) {
            super(amount);
            this.walletAddress = walletAddress;
        }

        @Override
        public boolean process() {
            System.out.println("Processing Crypto: $" + amount);
            System.out.println("Wallet: " + walletAddress.substring(0, 6) + "...");
            return true;
        }
    }

    // Payment processor - demonstrates polymorphism
    public static boolean processPayment(Payment payment) {
        System.out.println("\n--- Processing Payment ---");
        return payment.process();
    }

    // Process multiple payments
    public static void processAll(Payment[] payments) {
        System.out.println("\n=== PROCESSING MULTIPLE PAYMENTS ===");
        for (Payment p : payments) {
            processPayment(p);
            System.out.println("Status: Success\n");
        }
    }

    public static void main(String[] args) {
        System.out.println("=== POLYMORPHISM IN PAYMENT PROCESSING ===\n");

        // Create different payment types
        Payment[] payments = new Payment[3];
        payments[0] = new CreditCardPayment(99.99, "4111111111111111");
        payments[1] = new PayPalPayment(49.99, "user@email.com");
        payments[2] = new CryptoPayment(0.005, "0x1234567890abcdef");

        // Process all with polymorphism
```

```
        processAll(payments);

        // Direct processing
        System.out.println("=== DIRECT POLYMORPHISM ===");
        Payment card = new CreditCardPayment(150.00, "5555555555554444");
        Payment paypal = new PayPalPayment(75.00, "paypal@user.com");

        processPayment(card);
        processPayment(paypal);

        System.out.println("\n=== ANGULAR USE CASES ===");
        System.out.println("1. Multiple payment options (credit, PayPal, crypto)");
        System.out.println("2. Different shipping methods");
        System.out.println("3. Various discount types");
        System.out.println("4. Multiple notification channels");
        System.out.println("5. Dynamic component rendering");
    }
}
```

23

Encapsulation

Code Examples

UserModelDemo.java

UserModelDemo.java

```
// UserModelDemo - Demonstrates Encapsulation for Angular Data Models
// Use case: Protected data models for Angular frontend

public class UserModelDemo {

    // Fully encapsulated User class
    static class User {
        // Private fields (data hiding)
        private int id;
        private String name;
        private String email;
        private String password;
        private boolean active;

        // Public constructor
        public User(int id, String name, String email) {
            this.id = id;
            this.name = name;
            this.email = email;
            this.active = true;
        }

        // Public getters (read access)
        public int getId() { return id; }
        public String getName() { return name; }
        public String getEmail() { return email; }
        public boolean isActive() { return active; }

        // Setters with validation
        public void setName(String name) {
            if (name != null && !name.trim().isEmpty()) {
                this.name = name;
            }
        }

        public void setEmail(String email) {
            if (email != null && email.contains("@")) {
                this.email = email;
            }
        }

        public void setActive(boolean active) {
            this.active = active;
        }
```

```
        // Secure password setter
        public void setPassword(String password) {
            // Could add encryption here
            this.password = password;
        }

        // No getter for password (security)
        public boolean verifyPassword(String password) {
            return this.password != null && this.password.equals(password);
        }
    }

    // User DTO for API responses (without sensitive data)
    static class UserDTO {
        private int id;
        private String name;
        private String email;

        public UserDTO(int id, String name, String email) {
            this.id = id;
            this.name = name;
            this.email = email;
        }

        public int getId() { return id; }
        public String getName() { return name; }
        public String getEmail() { return email; }

        // Factory method to create DTO from entity
        public static UserDTO fromUser(User user) {
            return new UserDTO(user.getId(), user.getName(), user.getEmail());
        }
    }

    public static void main(String[] args) {
        System.out.println("=== ENCAPSULATION FOR DATA MODELS ===\n");

        // Create user with constructor
        User user = new User(1, "John", "john@email.com");
        user.setPassword("secret123");

        // Access through public methods
        System.out.println("--- Reading Data ---");
        System.out.println("ID: " + user.getId());
        System.out.println("Name: " + user.getName());
        System.out.println("Email: " + user.getEmail());
        System.out.println("Active: " + user.isActive());

        // Update through setters with validation
        System.out.println("\n--- Updating Data ---");
        user.setName("John Doe");
        System.out.println("Updated name: " + user.getName());
```

```
        // Invalid update (empty name)
        user.setName("");
        System.out.println("After empty name: " + user.getName());

        // Password handling (no getter!)
        System.out.println("\n--- Password Handling ---");
        System.out.println("Password verify (correct): " + user.verifyPassword("secret123"));
        System.out.println("Password verify (wrong): " + user.verifyPassword("wrong"));

        // Create DTO from entity (for API response)
        System.out.println("\n--- DTO for Angular ---");
        UserDTO dto = UserDTO.fromUser(user);
        System.out.println("DTO ID: " + dto.getId());
        System.out.println("DTO Name: " + dto.getName());

        // Trying to access password would fail - not available!
        // user.getPassword() - compile error

        System.out.println("\n=== ANGULAR BENEFITS ===");
        System.out.println("1. Protected API responses (no passwords)");
        System.out.println("2. Validation before updates");
        System.out.println("3. Data integrity (private fields)");
        System.out.println("4. Controlled access to sensitive data");
        System.out.println("5. Type safety with DTOs");
    }
}
```

23

JDBC Database Connectivity

23_JDBC_Database_Connectivity\examples\23_JDBC_Database_Connectivity.md

Java JDBC and Database Connectivity

Table of Contents

1 Introduction to JDBC
2 JDBC Drivers
3 Database Connections
4 CRUD Operations
5 PreparedStatement
6 Transaction Management
7 Code Examples

---

1. Introduction to JDBC

What is JDBC?

JDBC (Java Database Connectivity) is an API for connecting and executing queries with databases.

```
■■■
```

2. JDBC Drivers

Types of JDBC Drivers

Type	Description	Performance
Type 1	JDBC-ODBC Bridge	Slow
Type 2	Native-API	Medium
Type 3	Network Protocol	Fast
Type 4	Native Protocol (Pure Java)	Fastest

---

3. Database Connections

Basic Connection Steps

```
// 1. Load driver
Class.forName("com.mysql.cj.jdbc.Driver");

// 2. Create connection
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/mydb",
    "username",
    "password"
);

// 3. Create statement
Statement stmt = conn.createStatement();

// 4. Execute query
ResultSet rs = stmt.executeQuery("SELECT * FROM users");

// 5. Process results
while (rs.next()) {
    System.out.println(rs.getString("name"));
}

// 6. Close resources
rs.close();
stmt.close();
conn.close();
```

---

4. CRUD Operations

Create (INSERT)

```
String sql = "INSERT INTO users (name, email) VALUES ('John', 'john@example.com')";
int rows = stmt.executeUpdate(sql);
```

Read (SELECT)

```
String sql = "SELECT * FROM users WHERE id = 1";
ResultSet rs = stmt.executeQuery(sql);
if (rs.next()) {
    String name = rs.getString("name");
}
```

Update

```
String sql = "UPDATE users SET name = 'Jane' WHERE id = 1";
int rows = stmt.executeUpdate(sql);
```

Delete

```
String sql = "DELETE FROM users WHERE id = 1";
int rows = stmt.executeUpdate(sql);
```

---

5. PreparedStatement

Why Use PreparedStatement?

- **Prevents SQL Injection**
- **Better Performance** (compiled query)
- **Parameter Binding**

```
String sql = "INSERT INTO users (name, email) VALUES (?, ?)";
PreparedStatement pstmt = conn.prepareStatement(sql);

pstmt.setString(1, "John");
pstmt.setString(2, "john@example.com");
pstmt.executeUpdate();
```

6. Transaction Management

ACID Properties

```
conn.setAutoCommit(false);

try {
    // Multiple operations
    stmt.executeUpdate("UPDATE accounts SET balance = balance - 100 WHERE id = 1");
    stmt.executeUpdate("UPDATE accounts SET balance = balance + 100 WHERE id = 2");

    conn.commit();  // Save changes
} catch (Exception e) {
    conn.rollback();  // Undo changes
    e.printStackTrace();
}
```

7. Code Examples

JDBCBasicDemo

```
import java.sql.*;

/**
 * Simulated JDBC Demo (without actual database)
 * Shows the pattern of JDBC operations
 */
public class JDBCBasicDemo {

    // Simulated database in memory
    private static String[][] users = {
        {"1", "John Doe", "john@example.com"},
        {"2", "Jane Smith", "jane@example.com"},
        {"3", "Bob Wilson", "bob@example.com"}
    };

    public static void main(String[] args) {
        System.out.println("=== JDBC CONCEPTS DEMO ===\n");

        System.out.println("In real JDBC, you would:");

        // 1. Load Driver
        System.out.println("1. Load Driver: Class.forName(\"com.mysql.cj.jdbc.Driver\")");
```

```
        // 2. Get Connection
        System.out.println("\n2. Get Connection:");
        System.out.println("   Connection conn = DriverManager.getConnection(");
        System.out.println("       \"jdbc:mysql://localhost:3306/mydb\",");
        System.out.println("       \"username\", \"password\");");

        // 3. Create Statement
        System.out.println("\n3. Create Statement:");
        System.out.println("   Statement stmt = conn.createStatement();");

        // 4. Execute Query
        System.out.println("\n4. Execute Query:");
        System.out.println("   ResultSet rs = stmt.executeQuery(\"SELECT * FROM users\");");

        // 5. Process Results
        System.out.println("\n5. Process Results:");
        System.out.println("   while (rs.next()) {");
        System.out.println("       String name = rs.getString(\"name\");");
        System.out.println("   }");

        // Demo with simulated data
        System.out.println("\n=== Simulated Results ===");
        System.out.println("ID | Name           | Email");
        System.out.println("---|----------------|----------------");
        for (String[] user : users) {
            System.out.println(user[0] + " | " + user[1] + " | " + user[2]);
        }

        // PreparedStatement example
        System.out.println("\n=== PreparedStatement Example ===");
        System.out.println("String sql = \"INSERT INTO users (name, email) VALUES (?, ?)\";");
        System.out.println("PreparedStatement pstmt = conn.prepareStatement(sql);");
        System.out.println("pstmt.setString(1, \"New User\");");
        System.out.println("pstmt.setString(2, \"new@example.com\");");

        // Transaction example
        System.out.println("\n=== Transaction Example ===");
        System.out.println("conn.setAutoCommit(false);");
        System.out.println("try {");
        System.out.println("    stmt.executeUpdate(\"UPDATE accounts SET balance = balance - 100\...
        System.out.println("    stmt.executeUpdate(\"UPDATE accounts SET balance = balance + 100\...
        System.out.println("    conn.commit();");
        System.out.println("} catch (Exception e) {");
        System.out.println("    conn.rollback();");
        System.out.println("}");
    }
}
```

Summary

Key Takeaways

1. **JDBC API** - Standard Java API for database access
2. **Driver Manager** - Manages JDBC drivers
3. **Connection** - Represents database connection
4. **Statement/PreparedStatement** - Executes SQL
5. **ResultSet** - Contains query results
6. **Transactions** - Group operations with commit/rollback

JDBC Complete!

Code Examples

JDBCDemo.java

JDBCDemo.java

```
// JDBCDemo - Java Database Connectivity
// Demonstrates database connections in Java

public class JDBCDemo {

    public static void main(String[] args) {
        System.out.println("=== JDBC DEMO ===\n");

        // JDBC Steps
        System.out.println("--- JDBC Steps ---");
        System.out.println("1. Load driver");
        System.out.println("2. Establish connection");
        System.out.println("3. Create statement");
        System.out.println("4. Execute query");
        System.out.println("5. Process results");
        System.out.println("6. Close connection");

        // Connection URL format
        System.out.println("\n--- Connection URL ---");
        System.out.println("MySQL: jdbc:mysql://localhost:3306/mydb");
        System.out.println("PostgreSQL: jdbc:postgresql://localhost:5432/mydb");
        System.out.println("Oracle: jdbc:oracle:thin:@localhost:1521:mydb");

        // SQL Operations
        System.out.println("\n--- SQL Operations ---");
        System.out.println("SELECT -> ResultSet");
        System.out.println("INSERT/UPDATE/DELETE -> int (affected rows)");

        // Example queries (pseudo-code)
        System.out.println("\n--- Example Queries ---");
        System.out.println("SELECT * FROM users WHERE id = ?");
        System.out.println("INSERT INTO users (name, email) VALUES (?, ?)");
```

```
        System.out.println("UPDATE users SET name = ? WHERE id = ?");
        System.out.println("DELETE FROM users WHERE id = ?");

        // Try-with-resources (recommended)
        System.out.println("\n--- Modern Approach ---");
        System.out.println("try (Connection conn = DriverManager.getConnection(url)) {");
        System.out.println("    // code");
        System.out.println("} // Auto-closes resources");

        // Angular Integration
        System.out.println("\n=== ANGULAR INTEGRATION ===");
        System.out.println("Java JDBC -> Repository Layer");
        System.out.println("Angular HTTP calls -> REST API endpoints");
        System.out.println("Database <-> JDBC <-> Java <-> REST <-> Angular");
    }
}
```

24

Interfaces

Code Examples

DataProviderDemo.java

DataProviderDemo.java

```
// DataProviderDemo - Demonstrates Interfaces for Data Provider Pattern
// Use case: Multiple data sources for Angular app

public class DataProviderDemo {

    // Interface for data providers
    interface DataProvider {
        String getData();
        String getName();
    }

    // Interface with default method
    interface AdvancedDataProvider extends DataProvider {
        default String processData() {
            return "Processing: " + getData();
        }

        // Static method in interface
        static DataProvider create() {
            return new RESTDataProvider();
        }
    }

    // REST API implementation
    static class RESTDataProvider implements DataProvider {
        private String endpoint;

        public RESTDataProvider() {
            this.endpoint = "/api/data";
        }

        @Override
        public String getData() {
            return "REST Data from " + endpoint;
        }

        @Override
        public String getName() {
            return "REST Provider";
        }
    }

    // Database implementation
    static class DatabaseDataProvider implements DataProvider {
        private String query;
```

```
    public DatabaseDataProvider() {
        this.query = "SELECT * FROM users";
    }

    @Override
    public String getData() {
        return "DB Data: " + query;
    }

    @Override
    public String getName() {
        return "Database Provider";
    }
}

// Cache implementation
static class CacheDataProvider implements AdvancedDataProvider {
    private String cachedData;

    public CacheDataProvider() {
        this.cachedData = "Cached User Data";
    }

    @Override
    public String getData() {
        return cachedData;
    }

    @Override
    public String getName() {
        return "Cache Provider";
    }

    @Override
    public String processData() {
        return "Cache processed: " + cachedData;
    }
}

// Service using provider
static class DataService {
    private DataProvider provider;

    public DataService(DataProvider provider) {
        this.provider = provider;
    }

    public void fetchData() {
        System.out.println("Provider: " + provider.getName());
        System.out.println("Data: " + provider.getData());
    }
}
```

```
    public static void main(String[] args) {
        System.out.println("=== INTERFACES FOR DATA PROVIDERS ===\n");

        // Use REST provider
        System.out.println("--- REST Provider ---");
        DataProvider rest = new RESTDataProvider();
        DataService service1 = new DataService(rest);
        service1.fetchData();

        // Use Database provider
        System.out.println("\n--- Database Provider ---");
        DataProvider db = new DatabaseDataProvider();
        DataService service2 = new DataService(db);
        service2.fetchData();

        // Use Cache provider (implements AdvancedDataProvider)
        System.out.println("\n--- Cache Provider (Advanced) ---");
        AdvancedDataProvider cache = new CacheDataProvider();
        System.out.println("Name: " + cache.getName());
        System.out.println("Data: " + cache.getData());
        System.out.println("Processed: " + cache.processData());

        // Use static factory
        System.out.println("\n--- Static Factory ---");
        DataProvider created = AdvancedDataProvider.create();
        System.out.println("Created: " + created.getName());

        // Multiple providers in list
        System.out.println("\n--- All Providers ---");
        DataProvider[] providers = {rest, db, cache};
        for (DataProvider p : providers) {
            System.out.println("- " + p.getName() + ": " + p.getData());
        }

        System.out.println("\n=== ANGULAR USE CASES ===");
        System.out.println("1. REST API data service");
        System.out.println("2. Local storage service");
        System.out.println("3. WebSocket data service");
        System.out.println("4. Mock data for testing");
        System.out.println("5. Fallback data providers");
    }
}
```

24

Serialization ObjectIO

24_Serialization_ObjectIO\examples\24_Serialization_ObjectIO.md

Java Serialization and Object I/O

Table of Contents

1 Object Serialization
2 Serializable Interface
3 ObjectInputStream and ObjectOutputStream
4 Externalizable Interface
5 Code Examples

---

1. Object Serialization

What is Serialization?

Serialization converts Java objects to a format that can be stored or transmitted.

```
■■■
```

```
import java.io.Serializable;

class User implements Serializable {
    private static final long serialVersionUID = 1L;

    private String name;
    private int age;
    // transient fields are not serialized
    private transient String password;
}
```

3. ObjectInputStream and ObjectOutputStream

Writing Objects

```
// Serialize (write object to file)
FileOutputStream fileOut = new FileOutputStream("user.ser");
ObjectOutputStream out = new ObjectOutputStream(fileOut);
out.writeObject(user);
out.close();
```

Reading Objects

```
// Deserialize (read object from file)
FileInputStream fileIn = new FileInputStream("user.ser");
ObjectInputStream in = new ObjectInputStream(fileIn);
User user = (User) in.readObject();
in.close();
```

4. Externalizable Interface

Custom Serialization

```
class User implements Externalizable {
    private String name;
    private int age;

    @Override
    public void writeExternal(ObjectOutput out) throws IOException {
        out.writeObject(name);
        out.writeInt(age);
```

```
    }

    @Override
    public void readExternal(ObjectInput in) throws IOException,
                                                    ClassNotFoundException {
        name = (String) in.readObject();
        age = in.readInt();
    }
}
```

---

5. Code Examples

SerializationDemo

```
import java.io.*;

class Person implements Serializable {
    private static final long serialVersionUID = 1L;

    private String name;
    private int age;
    private String email;

    public Person(String name, int age, String email) {
        this.name = name;
        this.age = age;
        this.email = email;
    }

    @Override
    public String toString() {
        return "Person{name='" + name + "', age=" + age + ", email='" + email + "'}";
    }
}

public class SerializationDemo {
    public static void main(String[] args) {
        System.out.println("=== SERIALIZATION DEMO ===\n");

        // Create object
        Person person = new Person("John", 30, "john@example.com");
        String filename = "person.ser";

        // Serialize (write to file)
        try (ObjectOutputStream out = new ObjectOutputStream(
                new FileOutputStream(filename))) {
            out.writeObject(person);
            System.out.println("Object serialized: " + person);
```

```
        } catch (IOException e) {
            e.printStackTrace();
        }

        // Deserialize (read from file)
        try (ObjectInputStream in = new ObjectInputStream(
                new FileInputStream(filename))) {
            Person loadedPerson = (Person) in.readObject();
            System.out.println("Object deserialized: " + loadedPerson);
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }

        // Clean up
        new File(filename).delete();
        System.out.println("\nDemo complete!");
    }
}
```

Summary

Key Takeaways

1. **Serializable** - Marker interface for serialization
2. **serialVersionUID** - Version control for serialized objects
3. **transient** - Fields not serialized
4. **ObjectOutputStream** - Write objects
5. **ObjectInputStream** - Read objects

Serialization Complete!

Code Examples

SerializationDemo.java

SerializationDemo.java

```
// SerializationDemo - Object Serialization in Java
// Converting objects to byte streams and vice versa

import java.io.*;

class Person implements Serializable {
    private static final long serialVersionUID = 1L;

    private String name;
```

```
    private int age;
    private transient String password;  // Won't be serialized

    public Person(String name, int age, String password) {
        this.name = name;
        this.age = age;
        this.password = password;
    }

    public void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class SerializationDemo {

    public static void main(String[] args) {
        System.out.println("=== SERIALIZATION DEMO ===\n");

        // What is serialization?
        System.out.println("--- What is Serialization? ---");
        System.out.println("Converting object state to byte stream");
        System.out.println("Used for: File storage, Network transfer, Session storage");

        // Serializable interface
        System.out.println("\n--- Requirements ---");
        System.out.println("1. Class must implement Serializable");
        System.out.println("2. All fields must be serializable");
        System.out.println("3. Use 'transient' to skip fields");

        // Object to file
        System.out.println("\n--- Serialization Process ---");
        System.out.println("Object -> ObjectOutputStream -> File");

        // Object from file
        System.out.println("\n--- Deserialization Process ---");
        System.out.println("File -> ObjectInputStream -> Object");

        // Use cases
        System.out.println("\n--- Use Cases ---");
        System.out.println("1. Storing objects to files");
        System.out.println("2. Sending objects over network");
        System.out.println("3. HTTP session replication");
        System.out.println("4. Cache persistence");

        // JSON vs Serialization
        System.out.println("\n--- JSON vs Serialization ---");
        System.out.println("JSON: Human-readable, language-independent");
        System.out.println("Java Serialization: Binary, Java-only, faster");
        System.out.println("For web apps: JSON is preferred (Angular compatibility)");
    }
}
```

25

Lambda Stream API

25_Lambda_Stream_API\examples\25_Lambda_Stream_API.md

Java Lambda Expressions and Stream API

Table of Contents

1 Lambda Expressions
2 Functional Interfaces
3 Method References
4 Stream API
5 Intermediate Operations
6 Terminal Operations
7 Code Examples
8 Exercises
9 Solutions

---

1. Lambda Expressions

What are Lambda Expressions?

A **lambda expression** is a concise way to represent an anonymous function (a function without a name).

```
■■■
```

```
// No parameters
() -> System.out.println("Hello")

// One parameter
x -> x * 2

// Multiple parameters
(int a, int b) -> a + b

// With body
(x, y) -> {
    int sum = x + y;
    return sum;
}
```

2. Functional Interfaces

What is a Functional Interface?

A functional interface has exactly one abstract method.

```
■■■
```

```
// Function - transforms values
Function<String, Integer> length = s -> s.length();

// Predicate - tests conditions
Predicate<Integer> isEven = n -> n % 2 == 0;
```

3. Method References

What are Method References?

Method references are a shorthand notation for lambda expressions.

```
// Method reference types:

// Static method: ClassName::staticMethod
Function<String, Integer> parser = Integer::parseInt;

// Instance method of object: object::instanceMethod
String str = "Hello";
Supplier<Integer> len = str::length;

// Instance method of type: ClassName::instanceMethod
Function<String, String> upper = String::toUpperCase;

// Constructor: ClassName::new
Supplier<ArrayList<String>> listSupplier = ArrayList::new;
```

4. Stream API

What is Stream API?

The Stream API provides a functional approach to processing collections.

```
■■■
```

```
■       .map(x -> x * 2)        // Intermediate              ■
■       .collect(Collectors.toList())  // Terminal            ■
■                                                             ■
■■■
```

Operation	Description	Returns
`allMatch(Predicate)`	All match?	boolean
`noneMatch(Predicate)`	None match?	boolean
`findFirst()`	First element	Optional
`min(Comparator)`	Minimum	Optional
`max(Comparator)`	Maximum	Optional

7. Code Examples

Example 1: Lambda Basics

```
import java.util.*;
import java.util.function.*;

/**
 * LambdaBasicsDemo - Lambda expression fundamentals
 */
public class LambdaBasicsDemo {

    public static void main(String[] args) {
        System.out.println("=== LAMBDA BASICS DEMO ===\n");

        // Runnable
        Runnable r = () -> System.out.println("Hello from lambda!");
        r.run();

        // Consumer
        Consumer<String> printer = s -> System.out.println("Printing: " + s);
        printer.accept("Hello");

        // Supplier
        Supplier<Double> random = () -> Math.random();
        System.out.println("Random: " + random.get());

        // Function
        Function<String, Integer> stringLength = s -> s.length();
        System.out.println("Length of 'Hello': " + stringLength.apply("Hello"));

        // Predicate
        Predicate<Integer> isPositive = n -> n > 0;
        System.out.println("Is 5 positive? " + isPositive.test(5));
        System.out.println("Is -3 positive? " + isPositive.test(-3));
```

```
        // BinaryOperator
        BinaryOperator<Integer> add = (a, b) -> a + b;
        System.out.println("5 + 3 = " + add.apply(5, 3));
    }
}
```

Example 2: Stream Operations

```
import java.util.*;
import java.util.stream.*;

/**
 * StreamOperationsDemo - Comprehensive stream operations
 */
public class StreamOperationsDemo {

    public static void main(String[] args) {
        System.out.println("=== STREAM OPERATIONS DEMO ===\n");

        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

        // Filter - keep even numbers
        List<Integer> evens = numbers.stream()
            .filter(n -> n % 2 == 0)
            .collect(Collectors.toList());
        System.out.println("Even numbers: " + evens);

        // Map - double each number
        List<Integer> doubled = numbers.stream()
            .map(n -> n * 2)
            .collect(Collectors.toList());
        System.out.println("Doubled: " + doubled);

        // Filter + Map - square of odd numbers
        List<Integer> oddSquares = numbers.stream()
            .filter(n -> n % 2 != 0)
            .map(n -> n * n)
            .collect(Collectors.toList());
        System.out.println("Odd squares: " + oddSquares);

        // Distinct
        List<Integer> withDuplicates = Arrays.asList(1, 2, 2, 3, 3, 3, 4);
        List<Integer> distinct = withDuplicates.stream()
            .distinct()
            .collect(Collectors.toList());
        System.out.println("Distinct: " + distinct);

        // Sorted
        List<Integer> unsorted = Arrays.asList(5, 2, 8, 1, 9, 3);
        List<Integer> sorted = unsorted.stream()
```

```
            .sorted()
            .collect(Collectors.toList());
        System.out.println("Sorted: " + sorted);

        // Limit and Skip
        System.out.println("First 3: " + numbers.stream().limit(3).collect(Collectors.toList()));
        System.out.println("Skip first 5: " + numbers.stream().skip(5).collect(Collectors.toList(...

        // Terminal operations
        long count = numbers.stream().count();
        System.out.println("\nCount: " + count);

        int sum = numbers.stream().mapToInt(Integer::intValue).sum();
        System.out.println("Sum: " + sum);

        OptionalDouble avg = numbers.stream().mapToInt(Integer::intValue).average();
        System.out.println("Average: " + avg.orElse(0));

        Optional<Integer> min = numbers.stream().min(Integer::compareTo);
        System.out.println("Min: " + min.orElse(0));

        Optional<Integer> max = numbers.stream().max(Integer::compareTo);
        System.out.println("Max: " + max.orElse(0));

        // Find operations
        System.out.println("\nFirst even: " + numbers.stream().filter(n -> n % 2 == 0).findFirst(...
        System.out.println("Any > 5: " + numbers.stream().anyMatch(n -> n > 5));
        System.out.println("All > 0: " + numbers.stream().allMatch(n -> n > 0));
        System.out.println("None > 100: " + numbers.stream().noneMatch(n -> n > 100));
    }
}
```

Example 3: Collectors

```
import java.util.*;
import java.util.stream.*;

/**
 * CollectorsDemo - Various collector usages
 */
public class CollectorsDemo {

    public static void main(String[] args) {
        System.out.println("=== COLLECTORS DEMO ===\n");

        List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "David", "Eve");

        // toList
        List<String> list = names.stream().collect(Collectors.toList());
        System.out.println("To List: " + list);
```

```
        // toSet
        Set<String> set = names.stream().collect(Collectors.toSet());
        System.out.println("To Set: " + set);

        // toMap
        Map<String, Integer> nameLengths = names.stream()
            .collect(Collectors.toMap(name -> name, name -> name.length()));
        System.out.println("To Map: " + nameLengths);

        // joining
        String joined = names.stream().collect(Collectors.joining(", "));
        System.out.println("Joined: " + joined);

        // counting
        Long count = names.stream().collect(Collectors.counting());
        System.out.println("Count: " + count);

        // summing
        List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
        Integer sum = nums.stream().collect(Collectors.summingInt(n -> n));
        System.out.println("Sum: " + sum);

        // averaging
        Double avg = nums.stream().collect(Collectors.averagingDouble(n -> n));
        System.out.println("Average: " + avg);

        // groupingBy
        Map<String, List<String>> byFirstLetter = names.stream()
            .collect(Collectors.groupingBy(name -> name.substring(0, 1)));
        System.out.println("Grouped: " + byFirstLetter);

        // partitioningBy
        Map<Boolean, List<Integer>> partitioned = nums.stream()
            .collect(Collectors.partitioningBy(n -> n % 2 == 0));
        System.out.println("Partitioned: " + partitioned);
    }
}
```

Example 4: Method References

```
import java.util.*;

/**
 * MethodReferenceDemo - Method reference examples
 */
public class MethodReferenceDemo {

    public static void main(String[] args) {
        System.out.println("=== METHOD REFERENCE DEMO ===\n");
```

```
        List<String> names = Arrays.asList("alice", "bob", "charlie");

        // Static method reference
        System.out.println("Static method:");
        names.stream()
            .map(String::toUpperCase)
            .forEach(System.out::println);

        // Instance method of object
        System.out.println("\nInstance method of object:");
        String prefix = "Mr. ";
        names.stream()
            .map(prefix::concat)
            .forEach(System.out::println);

        // Instance method of type
        System.out.println("\nInstance method of type:");
        names.stream()
            .map(String::toUpperCase)
            .map(String::toLowerCase)
            .forEach(System.out::println);

        // Constructor reference
        System.out.println("\nConstructor reference:");
        List<String> strings = Arrays.asList("1", "2", "3");
        List<Integer> integers = strings.stream()
            .map(Integer::new)
            .collect(java.util.stream.Collectors.toList());
        System.out.println("Converted: " + integers);
    }
}
```

Example 5: Real-World Example

```
import java.util.*;
import java.util.stream.*;

/**
 * Employee class for real-world example
 */
class Employee {
    private String name;
    private String department;
    private double salary;

    public Employee(String name, String department, double salary) {
        this.name = name;
        this.department = department;
        this.salary = salary;
```

```
    }

    public String getName() { return name; }
    public String getDepartment() { return department; }
    public double getSalary() { return salary; }

    @Override
    public String toString() {
        return name + " (" + department + ", $" + salary + ")";
    }
}

/**
 * RealWorldStreamDemo - Practical stream usage
 */
public class RealWorldStreamDemo {

    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("Alice", "IT", 75000),
            new Employee("Bob", "HR", 55000),
            new Employee("Charlie", "IT", 80000),
            new Employee("Diana", "Marketing", 60000),
            new Employee("Eve", "IT", 70000),
            new Employee("Frank", "HR", 58000)
        );

        System.out.println("=== REAL-WORLD STREAM DEMO ===\n");

        // Get all IT employees
        System.out.println("IT Department:");
        employees.stream()
            .filter(e -> e.getDepartment().equals("IT"))
            .forEach(System.out::println);

        // Average salary in IT
        double itAvg = employees.stream()
            .filter(e -> e.getDepartment().equals("IT"))
            .mapToDouble(Employee::getSalary)
            .average()
            .orElse(0);
        System.out.println("\nIT Average Salary: $" + itAvg);

        // Sum of all salaries
        double totalSalary = employees.stream()
            .mapToDouble(Employee::getSalary)
            .sum();
        System.out.println("Total Salary: $" + totalSalary);

        // Highest paid employee
        Employee highestPaid = employees.stream()
            .max(Comparator.comparingDouble(Employee::getSalary))
            .orElse(null);
```

```
        System.out.println("Highest Paid: " + highestPaid);

        // Group by department
        System.out.println("\nEmployees by Department:");
        employees.stream()
            .collect(Collectors.groupingBy(Employee::getDepartment))
            .forEach((dept, emps) -> {
                System.out.println(dept + ": " + emps);
            });

        // Top 2 highest paid
        System.out.println("\nTop 2 Highest Paid:");
        employees.stream()
            .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
            .limit(2)
            .forEach(System.out::println);
    }
}
```

---

8. Exercises

Exercise 1: Filter and Transform

Requirements:

1. Filter a list to keep only positive numbers
2. Square each number
3. Sum the results

Exercise 2: Grouping

Requirements:

1. Group words by first letter
2. Count words in each group

Exercise 3: Find and Match

Requirements:

1. Find first word starting with 'a'
2. Check if any word has length > 5

---

9. Solutions

Solution 1

```
List<Integer> numbers = Arrays.asList(-3, -2, -1, 0, 1, 2, 3, 4, 5);

int result = numbers.stream()
    .filter(n -> n > 0)
    .map(n -> n * n)
    .mapToInt(Integer::intValue)
    .sum();

System.out.println("Sum of squares: " + result);
```

Solution 2

```
List<String> words = Arrays.asList("apple", "banana", "apricot", "blueberry", "cherry");

Map<Character, Long> grouped = words.stream()
    .collect(Collectors.groupingBy(
        w -> w.charAt(0),
        Collectors.counting()
    ));

System.out.println(grouped);
```

Solution 3

```
List<String> words = Arrays.asList("hello", "world", "apple", "beautiful");

String firstA = words.stream()
    .filter(w -> w.startsWith("a"))
    .findFirst()
    .orElse("Not found");

boolean hasLong = words.stream()
    .anyMatch(w -> w.length() > 5);

System.out.println("First 'a' word: " + firstA);
System.out.println("Has length > 5: " + hasLong);
```

Summary

Key Takeaways

1. **Lambda expressions** - Concise anonymous functions
2. **Functional interfaces** - Single abstract method interfaces
3. **Method references** - Shorthand for lambdas
4. **Stream API** - Functional collection processing
5. **Intermediate operations** - Transform streams (filter, map, sorted)
6. **Terminal operations** - Produce results (collect, forEach, reduce)

Happy Coding! ■

Code Examples

AngularIntegrationDemo.java

AngularIntegrationDemo.java

```
// AngularIntegrationDemo - Demonstrates Stream API for Angular Data Processing
// Shows how to process JSON-like data from Angular frontends

import java.util.*;
import java.util.stream.*;

public class AngularIntegrationDemo {

    public static void main(String[] args) {
        System.out.println("=== ANGULAR INTEGRATION DEMO ===\n");

        // Simulating JSON data from Angular/API
        List<Map<String, Object>> users = new ArrayList<>();

        Map<String, Object> user1 = new HashMap<>();
        user1.put("id", 1);
        user1.put("name", "Alice");
        user1.put("age", 25);
        user1.put("active", true);
        users.add(user1);

        Map<String, Object> user2 = new HashMap<>();
        user2.put("id", 2);
        user2.put("name", "Bob");
        user2.put("age", 17);
        user2.put("active", true);
        users.add(user2);

        Map<String, Object> user3 = new HashMap<>();
        user3.put("id", 3);
        user3.put("name", "Charlie");
        user3.put("age", 30);
        user3.put("active", false);
        users.add(user3);
```

```
        // Get active users above 18
        List<Map<String, Object>> activeAdults = users.stream()
            .filter(u -> (Boolean)u.get("active"))
            .filter(u -> (Integer)u.get("age") >= 18)
            .collect(Collectors.toList());

        System.out.println("Active adults: " + activeAdults);

        // Get names only
        List<String> names = users.stream()
            .map(u -> (String)u.get("name"))
            .collect(Collectors.toList());
        System.out.println("All names: " + names);

        System.out.println("\n=== USE CASES ===");
        System.out.println("1. Filter API responses based on user criteria");
        System.out.println("2. Transform data for Angular components");
        System.out.println("3. Process form submissions");
        System.out.println("4. Aggregate data for dashboards");
    }
}
```

LambdaExpressionsDemo.java

LambdaExpressionsDemo.java

```
// LambdaExpressionsDemo - Demonstrates Lambda Expressions in Java
// Essential for modern Java backend development
// Maps to functional programming concepts in TypeScript

import java.util.*;

public class LambdaExpressionsDemo {

    public static void main(String[] args) {
        System.out.println("=== LAMBDA EXPRESSIONS DEMO ===\n");

        // Traditional way
        Comparator<String> comparator1 = new Comparator<String>() {
            @Override
            public int compare(String a, String b) {
                return a.compareTo(b);
            }
        };

        // Lambda way - more concise
        Comparator<String> comparator2 = (a, b) -> a.compareTo(b);

        // More lambda examples
        Runnable runnable = () -> System.out.println("Hello from lambda!");
        runnable.run();
```

```
        // Function with parameters
        Calculator add = (a, b) -> a + b;
        Calculator multiply = (a, b) -> a * b;

        System.out.println("5 + 3 = " + add.calculate(5, 3));
        System.out.println("5 * 3 = " + multiply.calculate(5, 3));

        // Method references
        List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
        names.forEach(System.out::println);

        System.out.println("\n=== KEY BENEFITS ===");
        System.out.println("1. Concise and readable code");
        System.out.println("2. Better support for functional programming");
        System.out.println("3. Maps well to JavaScript arrow functions");
    }
}

// Functional Interface
interface Calculator {
    int calculate(int a, int b);
}
```

StreamBasicDemo.java

StreamBasicDemo.java

```
// StreamBasicDemo - Demonstrates Stream API Basic Operations
// Essential for processing collections in modern Java

import java.util.*;
import java.util.stream.*;

public class StreamBasicDemo {

    public static void main(String[] args) {
        System.out.println("=== STREAM BASIC OPERATIONS DEMO ===\n");

        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

        // Filter - keep only even numbers
        List<Integer> evens = numbers.stream()
            .filter(n -> n % 2 == 0)
            .collect(Collectors.toList());
        System.out.println("Even numbers: " + evens);

        // Map - transform each element
        List<Integer> squares = numbers.stream()
            .map(n -> n * n)
            .collect(Collectors.toList());
        System.out.println("Squares: " + squares);
```

```
        // Filter + Map combined
        List<Integer> doubleOfEvens = numbers.stream()
            .filter(n -> n % 2 == 0)
            .map(n -> n * 2)
            .collect(Collectors.toList());
        System.out.println("Double of evens: " + doubleOfEvens);

        System.out.println("\n=== KEY CONCEPTS ===");
        System.out.println("filter() - Select elements based on condition");
        System.out.println("map() - Transform each element");
        System.out.println("collect() - Convert stream back to collection");
    }
}
```

StreamIntermediateDemo.java

StreamIntermediateDemo.java

```
// StreamIntermediateDemo - Demonstrates Stream API Intermediate Operations
// Essential for transforming and filtering streams

import java.util.*;
import java.util.stream.*;

public class StreamIntermediateDemo {

    public static void main(String[] args) {
        System.out.println("=== STREAM INTERMEDIATE OPERATIONS DEMO ===\n");

        // distinct - remove duplicates
        List<Integer> withDupes = Arrays.asList(1, 2, 2, 3, 3, 3, 4);
        List<Integer> distinct = withDupes.stream().distinct().collect(Collectors.toList());
        System.out.println("Distinct: " + distinct);

        // sorted
        List<String> unsorted = Arrays.asList("Banana", "Apple", "Cherry");
        List<String> sorted = unsorted.stream().sorted().collect(Collectors.toList());
        System.out.println("Sorted: " + sorted);

        // limit and skip
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
        List<Integer> limited = numbers.stream().limit(5).collect(Collectors.toList());
        List<Integer> skipped = numbers.stream().skip(5).collect(Collectors.toList());
        System.out.println("Limit 5: " + limited);
        System.out.println("Skip 5: " + skipped);

        // flatMap - flatten nested collections
        List<List<Integer>> nested = Arrays.asList(
            Arrays.asList(1, 2),
            Arrays.asList(3, 4),
            Arrays.asList(5, 6)
        );
```

```
        List<Integer> flat = nested.stream()
            .flatMap(list -> list.stream())
            .collect(Collectors.toList());
        System.out.println("FlatMap: " + flat);

        System.out.println("\n=== KEY INTERMEDIATE OPERATIONS ===");
        System.out.println("distinct() - Remove duplicate elements");
        System.out.println("sorted() - Sort elements");
        System.out.println("limit(n) - Take first n elements");
        System.out.println("skip(n) - Skip first n elements");
        System.out.println("flatMap() - Flatten nested collections");
    }
}
```

StreamTerminalDemo.java

StreamTerminalDemo.java

```
// StreamTerminalDemo - Demonstrates Stream API Terminal Operations
// Essential for processing collections and producing results

import java.util.*;
import java.util.stream.*;

public class StreamTerminalDemo {

    public static void main(String[] args) {
        System.out.println("=== STREAM TERMINAL OPERATIONS DEMO ===\n");

        List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "David", "Eve");

        // forEach
        System.out.print("forEach: ");
        names.stream().forEach(name -> System.out.print(name + " "));
        System.out.println();

        // collect
        List<String> filtered = names.stream()
            .filter(name -> name.length() > 3)
            .collect(Collectors.toList());
        System.out.println("Names > 3 chars: " + filtered);

        // count
        long count = names.stream()
            .filter(name -> name.startsWith("A"))
            .count();
        System.out.println("Names starting with A: " + count);

        // reduce - combine elements
        List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
        int sum = nums.stream().reduce(0, (a, b) -> a + b);
        System.out.println("Sum: " + sum);
```

```
        // anyMatch, allMatch, noneMatch
        System.out.println("Any name starts with 'C': " +
            names.stream().anyMatch(n -> n.startsWith("C")));
        System.out.println("All names > 2 chars: " +
            names.stream().allMatch(n -> n.length() > 2));

        // findFirst, findAny
        Optional<String> first = names.stream().findFirst();
        System.out.println("First: " + first.orElse("None"));

        System.out.println("\n=== KEY TERMINAL OPERATIONS ===");
        System.out.println("forEach() - Execute action for each element");
        System.out.println("collect() - Gather elements into collection");
        System.out.println("reduce() - Combine elements into single value");
        System.out.println("anyMatch/allMatch/noneMatch - Boolean checks");
    }
}
```

26

Advanced Stream Operations

26_Advanced_Stream_Operations\examples\26_Advanced_Stream_Operations.md

Java Advanced Stream Operations

Table of Contents

1 Parallel Streams
2 Stream Reduction
3 Collecting Results
4 FlatMap Operations
5 Stream Pipeline Internals

---

1. Parallel Streams

Using Parallel Streams

```
// Sequential stream
list.stream()
    .map(x -> x * 2)
    .collect(Collectors.toList());

// Parallel stream - uses multiple cores
list.parallelStream()
    .map(x -> x * 2)
    .collect(Collectors.toList());
```

When to Use Parallel Streams

- Large datasets
- Independent operations
- Non-blocking operations

---

2. Stream Reduction

reduce() Method

```
// Sum using reduce
Integer sum = numbers.stream()
    .reduce(0, (a, b) -> a + b);
```

```
// Using method reference
Integer sum2 = numbers.stream()
    .reduce(0, Integer::sum);

// Max/Min with reduce
numbers.stream()
    .reduce(Integer::max)
    .orElse(0);
```

3. Collecting Results

Advanced Collectors

```
// Grouping by
Map<String, List<Person>> byCity = people.stream()
    .collect(Collectors.groupingBy(Person::getCity));

// Partitioning
Map<Boolean, List<Integer>> partitioned = numbers.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));

// Summarizing
IntSummaryStatistics stats = numbers.stream()
    .collect(Collectors.summarizingInt(Integer::intValue));
```

4. FlatMap Operations

Using FlatMap

```
// Flatten lists
List<List<Integer>> listOfLists = Arrays.asList(
    Arrays.asList(1, 2),
    Arrays.asList(3, 4)
);

List<Integer> flatList = listOfLists.stream()
    .flatMap(list -> list.stream())
    .collect(Collectors.toList());
```

5. Code Examples

AdvancedStreamDemo

```
import java.util.*;
import java.util.stream.*;

public class AdvancedStreamDemo {
    public static void main(String[] args) {
        System.out.println("=== ADVANCED STREAM DEMO ===\n");

        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

        // Reduce
        System.out.println("--- Reduce Operations ---");
        int sum = numbers.stream().reduce(0, Integer::sum);
        System.out.println("Sum: " + sum);

        int product = numbers.stream().reduce(1, (a, b) -> a * b);
        System.out.println("Product: " + product);

        // Collect - groupingBy
        System.out.println("\n--- Grouping ---");
        List<String> words = Arrays.asList("apple", "banana", "apricot", "blue", "cherry");
        Map<Character, List<String>> grouped = words.stream()
            .collect(Collectors.groupingBy(w -> w.charAt(0)));
        System.out.println("Grouped by first letter: " + grouped);

        // Partitioning
        System.out.println("\n--- Partitioning ---");
        Map<Boolean, List<Integer>> evenOdd = numbers.stream()
            .collect(Collectors.partitioningBy(n -> n % 2 == 0));
        System.out.println("Even: " + evenOdd.get(true));
        System.out.println("Odd: " + evenOdd.get(false));

        // FlatMap
        System.out.println("\n--- FlatMap ---");
        List<List<Integer>> nested = Arrays.asList(
            Arrays.asList(1, 2, 3),
            Arrays.asList(4, 5, 6),
            Arrays.asList(7, 8, 9)
        );

        List<Integer> flat = nested.stream()
            .flatMap(List::stream)
            .collect(Collectors.toList());
        System.out.println("Flattened: " + flat);

        // Statistics
        System.out.println("\n--- Statistics ---");
        IntSummaryStatistics stats = numbers.stream()
            .collect(Collectors.summarizingInt(Integer::intValue));
        System.out.println("Count: " + stats.getCount());
        System.out.println("Sum: " + stats.getSum());
```

```
        System.out.println("Min: " + stats.getMin());
        System.out.println("Max: " + stats.getMax());
        System.out.println("Average: " + stats.getAverage());
    }
}
```

Summary

Key Takeaways

1. **Parallel streams** - Multi-core processing
2. **reduce()** - Combine elements into single value
3. **Collectors** - Grouping, partitioning, statistics
4. **flatMap()** - Flatten nested structures

Advanced Streams Complete!

Code Examples

AdvancedStreamDemo.java

AdvancedStreamDemo.java

```
// AdvancedStreamDemo - Advanced Stream API Operations
// Collectors, grouping, partitioning, and parallel streams

import java.util.*;
import java.util.stream.*;

public class AdvancedStreamDemo {

    public static void main(String[] args) {
        System.out.println("=== ADVANCED STREAM OPERATIONS ===\n");

        List<Person> people = Arrays.asList(
            new Person("Alice", 25, "NYC"),
            new Person("Bob", 30, "LA"),
            new Person("Charlie", 25, "NYC"),
            new Person("Diana", 30, "LA"),
            new Person("Eve", 25, "NYC")
        );

        // Grouping by
        System.out.println("--- Grouping By ---");
        Map<Integer, List<Person>> byAge = people.stream()
            .collect(Collectors.groupingBy(Person::getAge));
```

```
        System.out.println("By age: " + byAge);

        // Grouping by with count
        Map<String, Long> countByCity = people.stream()
            .collect(Collectors.groupingBy(Person::getCity, Collectors.counting()));
        System.out.println("Count by city: " + countByCity);

        // Partitioning
        System.out.println("\n--- Partitioning ---");
        Map<Boolean, List<Person>> partitioned = people.stream()
            .collect(Collectors.partitioningBy(p -> p.getAge() >= 25));
        System.out.println("Age >= 25: " + partitioned);

        // Summarizing
        System.out.println("\n--- Summarizing ---");
        IntSummaryStatistics stats = people.stream()
            .collect(Collectors.summarizingInt(Person::getAge));
        System.out.println("Age stats: " + stats);

        // Parallel streams
        System.out.println("\n--- Parallel Streams ---");
        long start = System.currentTimeMillis();
        long sum = people.parallelStream()
            .mapToInt(Person::getAge)
            .sum();
        long time = System.currentTimeMillis() - start;
        System.out.println("Parallel sum: " + sum + " (took " + time + "ms)");

        // Mapping and collecting
        System.out.println("\n--- Mapping and Collecting ---");
        Set<String> names = people.stream()
            .map(Person::getName)
            .collect(Collectors.toSet());
        System.out.println("Names set: " + names);

        // Joining
        String allNames = people.stream()
            .map(Person::getName)
            .collect(Collectors.joining(", "));
        System.out.println("Joined: " + allNames);
    }
}

class Person {
    private String name;
    private int age;
    private String city;

    public Person(String name, int age, String city) {
        this.name = name;
        this.age = age;
        this.city = city;
    }
```

```
    public String getName() { return name; }
    public int getAge() { return age; }
    public String getCity() { return city; }

    public String toString() { return name + "(" + age + ")"; }
}
```

26

Lambda Streams

Code Examples

AngularPipesDemo.java

AngularPipesDemo.java

```
// AngularPipesDemo - Demonstrates Stream Operations as Angular Pipe Equivalents
// Use case: Data transformation in Java backend (like Angular pipes)

import java.util.*;
import java.util.stream.*;

public class AngularPipesDemo {

    static class User {
        private String name;
        private int age;
        private String department;
        private boolean active;

        public User(String name, int age, String department, boolean active) {
            this.name = name;
            this.age = age;
            this.department = department;
            this.active = active;
        }

        public String getName() { return name; }
        public int getAge() { return age; }
        public String getDepartment() { return department; }
        public boolean isActive() { return active; }

        @Override
        public String toString() {
            return name + " (" + age + ") - " + department;
        }
    }

    public static void main(String[] args) {
        // Sample data
        List<User> users = Arrays.asList(
            new User("Alice", 25, "Engineering", true),
            new User("Bob", 30, "Marketing", true),
            new User("Charlie", 35, "Engineering", false),
            new User("Diana", 28, "Sales", true),
            new User("Eve", 32, "Engineering", true)
        );

        System.out.println("=== STREAMS AS ANGULAR PIPES ===\n");

        // Filter (like Angular async pipe + filter)
```

```
        System.out.println("--- Filter (ngIf + filter) ---");
        List<User> activeUsers = users.stream()
            .filter(User::isActive)
            .collect(Collectors.toList());
        activeUsers.forEach(u -> System.out.println("Active: " + u));

        // Map (like Angular map pipe)
        System.out.println("\n--- Map (map pipe) ---");
        List<String> names = users.stream()
            .map(User::getName)
            .collect(Collectors.toList());
        System.out.println("Names: " + names);

        // Map with transformation
        System.out.println("\n--- Map with Transform ---");
        List<String> upperNames = users.stream()
            .map(u -> u.getName().toUpperCase())
            .collect(Collectors.toList());
        System.out.println("Upper: " + upperNames);

        // Sort (like Angular orderBy)
        System.out.println("\n--- Sort (orderBy pipe) ---");
        List<User> sortedByAge = users.stream()
            .sorted(Comparator.comparingInt(User::getAge))
            .collect(Collectors.toList());
        sortedByAge.forEach(u -> System.out.println("Age: " + u.getAge() + " - " + u.getName()));

        // Reduce (like Angular reduce pipe)
        System.out.println("\n--- Reduce (reduce pipe) ---");
        int totalAge = users.stream()
            .mapToInt(User::getAge)
            .sum();
        System.out.println("Total age: " + totalAge);

        double averageAge = users.stream()
            .mapToInt(User::getAge)
            .average()
            .orElse(0);
        System.out.println("Average age: " + averageAge);

        // Find (like Angular first/last)
        System.out.println("\n--- Find (first) ---");
        User firstEngineer = users.stream()
            .filter(u -> u.getDepartment().equals("Engineering"))
            .findFirst()
            .orElse(null);
        System.out.println("First engineer: " + firstEngineer);

        // Group by (like Angular groupBy)
        System.out.println("\n--- Group By ---");
        Map<String, List<User>> byDepartment = users.stream()
            .collect(Collectors.groupingBy(User::getDepartment));
        byDepartment.forEach((dept, deptUsers) -> {
```

```
            System.out.println(dept + ": " + deptUsers.size() + " users");
        });

        // Chaining operations (like pipe chain)
        System.out.println("\n--- Chain (pipe chain) ---");
        List<String> result = users.stream()
            .filter(User::isActive)
            .filter(u -> u.getAge() > 25)
            .sorted(Comparator.comparing(User::getName))
            .map(User::getName)
            .collect(Collectors.toList());
        System.out.println("Active over 25: " + result);

        System.out.println("\n=== ANGULAR PIPE EQUIVALENTS ===");
        System.out.println("1. filter -> .filter()");
        System.out.println("2. map -> .map()");
        System.out.println("3. async -> .findFirst()/findAny()");
        System.out.println("4. orderBy -> .sorted()");
        System.out.println("5. json -> .collect()");
        System.out.println("6. reduce -> .reduce()/.sum()");
        System.out.println("7. groupBy -> .groupingBy()");
    }
}
```

27

Annotations Java8 DesignPatterns

27_Annotations_Java8_DesignPatterns\examples\27_Annotations_Java8_DesignPatterns.md

Java Annotations, Java 8+ Features, and Design Patterns

Table of Contents

1 Annotations
2 Java 8+ Features
3 Design Patterns
4 Code Examples
5 Solutions

---

1. Annotations

What are Annotations?

Annotations provide metadata about code and are used by the compiler and runtime.

```
■■■
```

```
public void oldMethod() { }

@SuppressWarnings("unchecked")  // Suppress warnings
public void legacyCode() { }

@FunctionalInterface  // Single abstract method
interface MyFunction {
    void apply();
}
```

Custom Annotations

```
// Define annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface MyAnnotation {
    String value() default "";
    int priority() default 1;
}

// Use annotation
@MyAnnotation(value = "Test", priority = 5)
public void testMethod() { }
```

---

2. Java 8+ Features

New Features Overview

Feature	Description
Lambda Expressions	Anonymous functions
Stream API	Functional collection processing
Optional	Null-safe container
Date/Time API	Improved date handling
Default Methods	Interfaces with implementations
Static Methods in Interfaces	Utility methods in interfaces
Method References	Short lambda syntax
CompletableFuture	Async programming

Date/Time API

```
import java.time.*;

// Current date/time
LocalDate today = LocalDate.now();
LocalTime now = LocalTime.now();
LocalDateTime now2 = LocalDateTime.now();

// Create specific date
LocalDate date = LocalDate.of(2024, 1, 15);

// Operations
LocalDate tomorrow = today.plusDays(1);
LocalDate lastMonth = today.minusMonths(1);

// Formatting
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");
String formatted = today.format(formatter);
```

Optional

```
import java.util.Optional;

// Create Optional
Optional<String> optional = Optional.of("Hello");
Optional<String> empty = Optional.empty();
Optional<String> nullable = Optional.ofNullable(null);

// Use Optional
String value = optional.orElse("Default");
String value2 = optional.map(String::toUpperCase).orElseGet(() -> "N/A");
```

Default Methods

```
interface Drawable {
    void draw();

    // Default method
    default void display() {
        System.out.println("Displaying...");
    }

    // Static method
    static void info() {
        System.out.println("Drawable interface");
    }
}
```

3. Design Patterns

Common Design Patterns

Pattern	Category	Description
Singleton	Creational	One instance only
Factory	Creational	Object creation via factory
Builder	Creational	Step-by-step object creation
Observer	Behavioral	Event notification
Repository	Architectural	Data access abstraction
DAO	Architectural	Data Access Object

Singleton Pattern

```
class Singleton {
    private static Singleton instance;

    private Singleton() { }  // Private constructor

    public static synchronized Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

Factory Pattern

```
interface Shape {
    void draw();
}

class Circle implements Shape {
    public void draw() { System.out.println("Circle"); }
}

class Rectangle implements Shape {
```

```
    public void draw() { System.out.println("Rectangle"); }
}

class ShapeFactory {
    public Shape getShape(String type) {
        if (type.equals("CIRCLE")) return new Circle();
        if (type.equals("RECTANGLE")) return new Rectangle();
        return null;
    }
}
```

Observer Pattern

```
interface Observer {
    void update(String message);
}

class Subscriber implements Observer {
    public void update(String message) {
        System.out.println("Received: " + message);
    }
}

class Publisher {
    private List<Observer> observers = new ArrayList<>();

    public void subscribe(Observer o) { observers.add(o); }

    public void notifyAll(String message) {
        for (Observer o : observers) {
            o.update(message);
        }
    }
}
```

---

4. Code Examples

Example 1: Custom Annotation

```
import java.lang.annotation.*;
import java.lang.reflect.*;

// Define annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Test {
```

```
    String description() default "No description";
    int priority() default 1;
}

// Class using annotation
class Calculator {
    @Test(description = "Addition test", priority = 1)
    public void testAdd() {
        System.out.println("Testing addition");
    }

    @Test(description = "Division test", priority = 2)
    public void testDivide() {
        System.out.println("Testing division");
    }
}

// Process annotations
public class AnnotationDemo {
    public static void main(String[] args) throws Exception {
        for (Method m : Calculator.class.getDeclaredMethods()) {
            if (m.isAnnotationPresent(Test.class)) {
                Test test = m.getAnnotation(Test.class);
                System.out.println("Method: " + m.getName());
                System.out.println("Description: " + test.description());
                System.out.println("Priority: " + test.priority());
            }
        }
    }
}
```

Example 2: Date/Time API

```
import java.time.*;
import java.time.format.*;

/**
 * DateTimeAPIDemo - Modern date/time handling
 */
public class DateTimeAPIDemo {
    public static void main(String[] args) {
        System.out.println("=== DATE/TIME API DEMO ===\n");

        // Current date/time
        System.out.println("Current: " + LocalDate.now());
        System.out.println("Time: " + LocalTime.now());
        System.out.println("DateTime: " + LocalDateTime.now());

        // Specific date
        LocalDate date = LocalDate.of(2024, 1, 15);
```

```
        System.out.println("\nSpecific date: " + date);

        // Operations
        System.out.println("Plus 10 days: " + date.plusDays(10));
        System.out.println("Minus 1 month: " + date.minusMonths(1));

        // Formatting
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("MMMM dd, yyyy");
        System.out.println("Formatted: " + date.format(formatter));

        // Duration
        LocalTime start = LocalTime.of(9, 0);
        LocalTime end = LocalTime.of(17, 30);
        Duration duration = Duration.between(start, end);
        System.out.println("\nWork hours: " + duration.toHours() + " hours");

        // Period
        LocalDate birthday = LocalDate.of(2000, 5, 15);
        Period age = Period.between(birthday, LocalDate.now());
        System.out.println("Age: " + age.getYears() + " years");
    }
}
```

Example 3: Design Patterns

```
/**
 * Singleton - Single instance
 */
class DatabaseConnection {
    private static DatabaseConnection instance;

    private DatabaseConnection() {
        System.out.println("Database connected");
    }

    public static DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }
}

/**
 * Factory - Object creation
 */
interface Payment {
    void pay(double amount);
}
```

```
class CreditCardPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Paid $" + amount + " via Credit Card");
    }
}

class PayPalPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Paid $" + amount + " via PayPal");
    }
}

class PaymentFactory {
    public static Payment getPayment(String type) {
        if (type.equalsIgnoreCase("CREDIT")) return new CreditCardPayment();
        if (type.equalsIgnoreCase("PAYPAL")) return new PayPalPayment();
        return null;
    }
}

/**
 * Builder - Step-by-step construction
 */
class User {
    private String name;
    private String email;
    private int age;

    private User(Builder builder) {
        this.name = builder.name;
        this.email = builder.email;
        this.age = builder.age;
    }

    static class Builder {
        private String name;
        private String email;
        private int age;

        public Builder name(String name) { this.name = name; return this; }
        public Builder email(String email) { this.email = email; return this; }
        public Builder age(int age) { this.age = age; return this; }
        public User build() { return new User(this); }
    }
}

/**
 * DesignPatternsDemo
 */
public class DesignPatternsDemo {
    public static void main(String[] args) {
        System.out.println("=== DESIGN PATTERNS DEMO ===\n");
```

```
        // Singleton
        DatabaseConnection db1 = DatabaseConnection.getInstance();
        DatabaseConnection db2 = DatabaseConnection.getInstance();
        System.out.println("Same instance? " + (db1 == db2));

        // Factory
        Payment payment = PaymentFactory.getPayment("CREDIT");
        payment.pay(100);

        // Builder
        User user = new User.Builder()
            .name("John")
            .email("john@example.com")
            .age(25)
            .build();
        System.out.println("\nUser created: " + user);
    }
}
```

---

5. Exercises

Exercise 1: Create Custom Annotation

Requirements:

1. Create @Author annotation with name and date
2. Apply to a class
3. Read annotation using reflection

Exercise 2: Implement Observer Pattern

Requirements:

1. Create Subject class with observers
2. Notify all observers when state changes

---

6. Solutions

Solution 1: Custom Annotation

```
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
```

```
@interface Author {
    String name();
    String date();
}

@Author(name = "John", date = "2024-01-15")
class MyClass { }

public class AuthorDemo {
    public static void main(String[] args) {
        if (MyClass.class.isAnnotationPresent(Author.class)) {
            Author a = MyClass.class.getAnnotation(Author.class);
            System.out.println("Author: " + a.name() + ", Date: " + a.date());
        }
    }
}
```

Solution 2: Observer Pattern

```
interface Observer {
    void update(String message);
}

class Subscriber implements Observer {
    private String name;
    Subscriber(String name) { this.name = name; }
    public void update(String message) {
        System.out.println(name + " received: " + message);
    }
}

class Publisher {
    private List<Observer> list = new java.util.ArrayList<>();

    public void subscribe(Observer o) { list.add(o); }
    public void notifyAllObservers(String msg) {
        for (Observer o : list) o.update(msg);
    }
}

public class ObserverDemo {
    public static void main(String[] args) {
        Publisher p = new Publisher();
        p.subscribe(new Subscriber("A"));
        p.subscribe(new Subscriber("B"));
        p.notifyAllObservers("Hello!");
    }
}
```

Summary

Key Takeaways

1. **Annotations** - Metadata for code (overrides, custom annotations)
2. **Java 8+ Features** - Lambda, Streams, Optional, Date/Time API
3. **Design Patterns** - Singleton, Factory, Builder, Observer

---

Happy Coding! ■

Code Examples

AnnotationsDemo.java

AnnotationsDemo.java

```
// AnnotationsDemo - Demonstrates Java Annotations
// Annotations are metadata that provide information about the program

import java.lang.annotation.*;
import java.lang.reflect.*;

// Custom annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface MyAnnotation {
    String value() default "";
    int priority() default 1;
}

// Marker annotation
@Retention(RetentionPolicy.RUNTIME)
@interface Deprecated {
}

// Custom annotation with multiple fields
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@interface Entity {
    String tableName();
    String schema() default "default";
}

// Usage of annotations
@Entity(tableName = "users", schema = "app")
class User {
    private int id;
    private String name;
```

```
    @MyAnnotation(value = "Getter for ID")
    public int getId() { return id; }

    @Deprecated
    public void oldMethod() {
        System.out.println("This method is deprecated!");
    }
}

public class AnnotationsDemo {

    @MyAnnotation(value = "Main method", priority = 10)
    public static void main(String[] args) {
        System.out.println("=== ANNOTATIONS DEMO ===\n");

        // Read annotations via reflection
        System.out.println("--- Reading Annotations ---");

        // Class-level annotation
        Entity entity = User.class.getAnnotation(Entity.class);
        if (entity != null) {
            System.out.println("Table: " + entity.tableName());
            System.out.println("Schema: " + entity.schema());
        }

        // Method-level annotation
        try {
            Method method = User.class.getMethod("getId");
            MyAnnotation annotation = method.getAnnotation(MyAnnotation.class);
            if (annotation != null) {
                System.out.println("Method annotation: " + annotation.value());
                System.out.println("Priority: " + annotation.priority());
            }
        } catch (NoSuchMethodException e) {
            e.printStackTrace();
        }

        // Built-in annotations
        System.out.println("\n--- Built-in Annotations ---");
        System.out.println("@Override - Marks method override");
        System.out.println("@Deprecated - Marks deprecated code");
        System.out.println("@SuppressWarnings - Suppresses warnings");
        System.out.println("@FunctionalInterface - Marks functional interface");

        // Angular/Spring annotations (conceptual)
        System.out.println("\n--- Spring Annotations (Conceptual) ---");
        System.out.println("@Controller - REST controller");
        System.out.println("@Service - Business logic");
        System.out.println("@Repository - Data access");
        System.out.println("@Autowired - Dependency injection");
        System.out.println("@GetMapping, @PostMapping - HTTP methods");
        System.out.println("@RequestBody - Request deserialization");
        System.out.println("@ResponseBody - Response serialization");
```

```
    }
}
```

27

Exception Handling

Code Examples

APIFaultHandlerDemo.java

APIFaultHandlerDemo.java

```
// APIFaultHandlerDemo - Demonstrates Exception Handling for Angular API Calls
// Use case: Proper error handling for Angular HTTP requests

public class APIFaultHandlerDemo {

    // Custom exception for API errors
    static class APIException extends Exception {
        private int statusCode;

        public APIException(String message, int statusCode) {
            super(message);
            this.statusCode = statusCode;
        }

        public int getStatusCode() {
            return statusCode;
        }
    }

    // Custom exception for validation
    static class ValidationException extends Exception {
        public ValidationException(String message) {
            super(message);
        }
    }

    // Simulate user service
    static class UserService {

        public String getUser(int id) throws ValidationException, APIException {
            if (id <= 0) {
                throw new ValidationException("Invalid user ID");
            }
            if (id > 1000) {
                throw new APIException("User not found", 404);
            }
            return "User-" + id;
        }

        public String createUser(String name) throws ValidationException, APIException {
            if (name == null || name.trim().isEmpty()) {
                throw new ValidationException("Name is required");
            }
            if (name.length() < 2) {
                throw new ValidationException("Name too short");
```

```
            }
            if (name.equals("error")) {
                throw new APIException("Server error", 500);
            }
            return "Created: " + name;
        }
    }

    // Handler with try-catch-finally
    public static void handleGetUser(int id) {
        UserService service = new UserService();

        try {
            String user = service.getUser(id);
            System.out.println("SUCCESS: " + user);
        } catch (ValidationException e) {
            System.out.println("VALIDATION ERROR: " + e.getMessage());
            // Send 400 to Angular
        } catch (APIException e) {
            System.out.println("API ERROR [" + e.getStatusCode() + "]: " + e.getMessage());
            // Send appropriate status to Angular
        } catch (Exception e) {
            System.out.println("UNKNOWN ERROR: " + e.getMessage());
            // Send 500 to Angular
        } finally {
            System.out.println("Request completed\n");
        }
    }

    // Handle create user
    public static void handleCreateUser(String name) {
        UserService service = new UserService();

        try {
            String result = service.createUser(name);
            System.out.println("SUCCESS: " + result);
        } catch (ValidationException e) {
            System.out.println("VALIDATION ERROR: " + e.getMessage());
        } catch (APIException e) {
            System.out.println("API ERROR [" + e.getStatusCode() + "]: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("UNKNOWN ERROR: " + e.getMessage());
        } finally {
            System.out.println("Request completed\n");
        }
    }

    // Method with throws
    public static String getUserSafe(int id) throws ValidationException, APIException {
        UserService service = new UserService();
        return service.getUser(id);
    }
```

```
    public static void main(String[] args) {
        System.out.println("=== EXCEPTION HANDLING FOR ANGULAR API ===\n");

        // Test various scenarios
        System.out.println("--- Valid ID ---");
        handleGetUser(1);

        System.out.println("--- Invalid ID (0) ---");
        handleGetUser(0);

        System.out.println("--- Not Found (ID > 1000) ---");
        handleGetUser(1001);

        System.out.println("--- Create Valid User ---");
        handleCreateUser("John");

        System.out.println("--- Create Empty User ---");
        handleCreateUser("");

        System.out.println("--- Create User 'error' ---");
        handleCreateUser("error");

        // Using throws
        System.out.println("--- Using throws keyword ---");
        try {
            String user = getUserSafe(5);
            System.out.println("Got: " + user);
        } catch (ValidationException e) {
            System.out.println("Validation Error: " + e.getMessage());
        } catch (APIException e) {
            System.out.println("Error: " + e.getMessage());
        }

        System.out.println("\n=== ANGULAR ERROR HANDLING ===");
        System.out.println("1. HttpErrorResponse handling");
        System.out.println("2. Form validation errors");
        System.out.println("3. 400/401/403/404/500 status codes");
        System.out.println("4. User-friendly error messages");
        System.out.println("5. Error logging and reporting");
    }
}
```

28

Java 9 10 11 12 Features

28_Java_9_10_11_12_Features\examples\28_Java_9_10_11_12_Features.md

Java 9-17 New Features

Table of Contents

1 Java 9 Features
2 Java 10 Features
3 Java 11 Features
4 Java 14-17 Features

---

1. Java 9 Features

New Methods

```
// List.of() - Immutable lists
List<String> list = List.of("a", "b", "c");

// Set.of() - Immutable sets
Set<Integer> set = Set.of(1, 2, 3);

// Map.of() - Immutable maps
Map<String, Integer> map = Map.of("a", 1, "b", 2);

// Stream takeWhile, dropWhile
stream.takeWhile(x -> x < 5)
stream.dropWhile(x -> x < 5)

// Optional orElseThrow
Optional<String> opt = Optional.empty();
String value = opt.orElseThrow();
```

Private Methods in Interfaces

```
interface MyInterface {
    default void method() {
        helper();
    }

    private void helper() {
        // Private implementation
    }
}
```

---

2. Java 10 Features

Local Variable Type Inference

```
// var keyword (Java 10+)
var message = "Hello";
var numbers = new ArrayList<Integer>();
var map = new HashMap<String, Integer>();
```

---

3. Java 11 Features

String Methods

```
// New String methods (Java 11+)
"  Hello  ".isBlank();           // true
"Hello".lines();                  // Stream of lines
"Hello".repeat(3);                // "HelloHelloHello"
"  Hello  ".strip();              // "Hello" (Unicode aware)
"  Hello  ".stripLeading();       // "Hello  "
"  Hello  ".stripTrailing();      // "  Hello"
```

Files Methods

```
// New Files methods
String content = Files.readString(Path.of("file.txt"));
Files.writeString(Path.of("file.txt"), "content");
boolean isBlank = Files.isSameFile(Path.of("a"), Path.of("b"));
```

Run Source File

```
// Java 11 can run single-file programs directly
// java MyProgram.java
```

---

4. Java 14-17 Features

Switch Expressions (Java 14+)

```
// Traditional switch
int result = switch (day) {
    case 1, 2, 3, 4, 5 -> 1;  // Weekday
    case 6, 7 -> 0;            // Weekend
    default -> -1;
};

// With blocks
int result = switch (day) {
    case 1, 2, 3, 4, 5 -> {
        yield 1;
    }
    case 6, 7 -> {
        yield 0;
    }
    default -> -1;
};
```

Records (Java 14+)

```
// Record - immutable data carrier
public record Person(String name, int age) {
    // Auto-generates:
    // - Private final fields
    // - Constructor
    // - equals(), hashCode(), toString()
    // - Getters: name(), age()
}

// Usage
Person p = new Person("John", 30);
System.out.println(p.name());  // John
```

Pattern Matching (Java 16+)

```
// Pattern matching for instanceof
if (obj instanceof String s) {
    // s is automatically typed as String here
    System.out.println(s.length());
}
```

Text Blocks (Java 15+)

```
// Multi-line strings
String json = """
    {
```

```
        "name": "John",
        "age": 30
    }
    """;
```

Code Examples

Java9PlusFeatures

```
public class Java9PlusFeatures {
    public static void main(String[] args) {
        System.out.println("=== JAVA 9+ FEATURES DEMO ===\n");

        // Immutable collections (Java 9+)
        var immutableList = List.of(1, 2, 3);
        var immutableMap = Map.of("a", 1, "b", 2);
        System.out.println("Immutable List: " + immutableList);

        // var keyword (Java 10+)
        var message = "Hello, Java!";
        var number = 42;
        System.out.println("var: " + message + " " + number);

        // New String methods (Java 11+)
        var blank = "   ".isBlank();
        var repeated = "Hi".repeat(3);
        var lines = "Hello\nWorld".lines().toList();
        System.out.println("\nisBlank: " + blank);
        System.out.println("repeat: " + repeated);
        System.out.println("lines: " + lines);

        // Switch expressions (Java 14+)
        var day = 3;
        var dayType = switch (day) {
            case 1, 2, 3, 4, 5 -> "Weekday";
            case 6, 7 -> "Weekend";
            default -> "Invalid";
        };
        System.out.println("\nSwitch expression: Day " + day + " is " + dayType);
    }
}
```

Summary

Quick Reference

Version	Key Features
Java 9	Immutable collections, Private interface methods
Java 10	Local variable type inference (var)
Java 11	New String methods, Run source files
Java 14	Switch expressions
Java 15	Text blocks
Java 16	Pattern matching for instanceof
Java 17	Sealed classes, Records

Java 9-17 Features Complete!

Code Examples

DateTimeAPIDemo.java

DateTimeAPIDemo.java

```
// DateTimeAPIDemo - Demonstrates Java Date/Time API (java.time)
// Important for handling dates in backend applications

import java.time.*;
import java.time.format.*;

public class DateTimeAPIDemo {

    public static void main(String[] args) {
        System.out.println("=== DATE/TIME API DEMO ===\n");

        // LocalDate - date only
        System.out.println("--- LocalDate ---");
        LocalDate today = LocalDate.now();
        System.out.println("Today: " + today);
        System.out.println("Year: " + today.getYear());
        System.out.println("Month: " + today.getMonth());
        System.out.println("Day of month: " + today.getDayOfMonth());

        // Specific date
        LocalDate birthday = LocalDate.of(1990, 5, 15);
        System.out.println("Birthday: " + birthday);

        // LocalTime - time only
```

```
        System.out.println("\n--- LocalTime ---");
        LocalTime now = LocalTime.now();
        System.out.println("Now: " + now);
        System.out.println("Hour: " + now.getHour());
        System.out.println("Minute: " + now.getMinute());

        // LocalDateTime - date and time
        System.out.println("\n--- LocalDateTime ---");
        LocalDateTime dateTime = LocalDateTime.now();
        System.out.println("DateTime: " + dateTime);

        // ZonedDateTime - with timezone
        System.out.println("\n--- ZonedDateTime ---");
        ZonedDateTime zonedNow = ZonedDateTime.now();
        System.out.println("Zoned: " + zonedNow);

        ZonedDateTime tokyo = ZonedDateTime.now(ZoneId.of("Asia/Tokyo"));
        System.out.println("Tokyo: " + tokyo);

        // Date manipulation
        System.out.println("\n--- Date Manipulation ---");
        LocalDate nextWeek = today.plusDays(7);
        System.out.println("Next week: " + nextWeek);

        LocalDate lastMonth = today.minusMonths(1);
        System.out.println("Last month: " + lastMonth);

        // Period - difference between dates
        System.out.println("\n--- Period ---");
        LocalDate start = LocalDate.of(2024, 1, 1);
        LocalDate end = LocalDate.now();
        Period period = Period.between(start, end);
        System.out.println("Days since Jan 1: " + period.getDays());
        System.out.println("Months: " + period.getMonths());

        // Duration - difference between times
        System.out.println("\n--- Duration ---");
        LocalTime startTime = LocalTime.of(9, 0);
        LocalTime endTime = LocalTime.of(17, 30);
        Duration duration = Duration.between(startTime, endTime);
        System.out.println("Work hours: " + duration.toHours());

        // Formatting
        System.out.println("\n--- Formatting ---");
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
        System.out.println("Formatted: " + today.format(formatter));

        DateTimeFormatter custom = DateTimeFormatter.ofPattern("EEEE, MMMM dd, yyyy");
        System.out.println("Full date: " + today.format(custom));

        // Parsing
        System.out.println("\n--- Parsing ---");
        String dateStr = "2024-12-25";
```

```
        LocalDate parsed = LocalDate.parse(dateStr);
        System.out.println("Parsed: " + parsed);

        // Instant - timestamp
        System.out.println("\n--- Instant ---");
        Instant instant = Instant.now();
        System.out.println("Current instant: " + instant);
        System.out.println("Epoch millis: " + instant.toEpochMilli());

        // Angular Integration
        System.out.println("\n=== ANGULAR INTEGRATION ===");
        System.out.println("LocalDateTime -> ISO 8601 String");
        System.out.println("ZonedDateTime -> ISO 8601 with timezone");
        System.out.println("JSON format: '2024-03-15T10:30:00'");
    }
}
```

28

Wrapper Classes

Code Examples

AngularDataTypeDemo.java

AngularDataTypeDemo.java

```
// AngularDataTypeDemo - Demonstrates Wrapper Classes for Type Mappings
// Use case: Java wrapper types mapping to TypeScript types

public class AngularDataTypeDemo {

    public static void main(String[] args) {
        System.out.println("=== WRAPPER CLASSES FOR ANGULAR TYPES ===\n");

        // Integer (maps to number in TypeScript)
        System.out.println("--- Integer -> number ---");
        Integer userId = 42;
        int primitiveId = userId; // Auto-unboxing
        System.out.println("User ID: " + userId);
        System.out.println("Primitive: " + primitiveId);

        // Double (maps to number in TypeScript)
        System.out.println("\n--- Double -> number ---");
        Double price = 19.99;
        double primitivePrice = price;
        System.out.println("Price: " + price);
        System.out.println("Formatted: $" + String.format("%.2f", price));

        // Boolean (maps to boolean in TypeScript)
        System.out.println("\n--- Boolean -> boolean ---");
        Boolean isActive = true;
        boolean primitiveActive = isActive;
        System.out.println("Active: " + isActive);
        System.out.println("Primitive: " + primitiveActive);

        // String (maps to string in TypeScript)
        System.out.println("\n--- String -> string ---");
        String name = "John Doe";
        System.out.println("Name: " + name);
        System.out.println("Upper: " + name.toUpperCase());

        // Character (maps to string in TypeScript)
        System.out.println("\n--- Character -> string ---");
        Character grade = 'A';
        char primitiveGrade = grade;
        System.out.println("Grade: " + grade);

        // Null handling (important for Angular)
        System.out.println("\n--- Null Handling ---");
        Integer nullValue = null;
        // int unboxed = nullValue; // NullPointerException!
```

```
        System.out.println("Null value: " + nullValue);

        // Default values for Angular forms
        System.out.println("\n--- Default Values ---");
        Integer defaultId = 0;
        String defaultName = "";
        Boolean defaultActive = false;
        System.out.println("Default ID: " + defaultId);
        System.out.println("Default Name: '" + defaultName + "'");
        System.out.println("Default Active: " + defaultActive);

        // Parsing (from string input)
        System.out.println("\n--- Parsing Strings ---");
        String numStr = "123";
        int parsed = Integer.parseInt(numStr);
        System.out.println("Parsed int: " + parsed);

        String boolStr = "true";
        boolean parsedBool = Boolean.parseBoolean(boolStr);
        System.out.println("Parsed bool: " + parsedBool);

        // Conversion for JSON
        System.out.println("\n--- JSON Conversion ---");
        Integer[] ids = {1, 2, 3};
        System.out.println("Array length: " + ids.length);

        // Array to list
        java.util.List<Integer> idList = java.util.Arrays.asList(ids);
        System.out.println("List size: " + idList.size());

        System.out.println("\n=== TYPE SCRIPT MAPPINGS ===");
        System.out.println("Java          -> TypeScript");
        System.out.println("---------------------------");
        System.out.println("Integer       -> number");
        System.out.println("Double        -> number");
        System.out.println("Boolean       -> boolean");
        System.out.println("String        -> string");
        System.out.println("Character     -> string");
        System.out.println("List<T>       -> T[]");
        System.out.println("Map<K,V>      -> {[key: K]: V}");
    }
}
```

29

Backend Design Patterns

29_Backend_Design_Patterns\examples\29_Backend_Design_Patterns.md

Backend Design Patterns for Java

Table of Contents

1 Repository Pattern
2 DAO Pattern
3 DTO Pattern
4 Service Layer Pattern
5 MVC Pattern
6 Dependency Injection

---

1. Repository Pattern

What is the Repository Pattern?

An abstraction layer between business logic and data access.

```
■■■
```

```
    List<T> findAll();
    void save(T entity);
    void delete(int id);
}

// Concrete implementation
class UserRepository implements Repository<User> {
    @Override
    public User findById(int id) {
        // Database query
        return new User();
    }

    @Override
    public List<User> findAll() {
        // Database query
        return new ArrayList<>();
    }
}
```

---

2. DAO Pattern

Data Access Object

```
// DAO Interface
interface UserDAO {
    User getUserById(int id);
    List<User> getAllUsers();
    void insertUser(User user);
    void updateUser(User user);
    void deleteUser(int id);
}

// DAO Implementation
class UserDAOImpl implements UserDAO {
    private Connection conn;

    @Override
    public User getUserById(int id) {
        // JDBC code
        return new User();
    }
    // ... other methods
}
```

---

3. DTO Pattern

Data Transfer Object

```
// DTO - Transfer data between layers
class UserDTO {
    private int id;
    private String name;
    private String email;

    // Getters and setters
}

// Entity - Database representation
@Entity
class UserEntity {
    @Id
    private int id;
    private String name;
    private String password;
    // ...
}

// Conversion
class UserMapper {
    public static UserDTO toDTO(UserEntity entity) {
        // Map entity to DTO
    }

    public static UserEntity toEntity(UserDTO dto) {
        // Map DTO to entity
    }
}
```

4. Service Layer Pattern

Business Logic Layer

```
// Service Interface
interface UserService {
    UserDTO getUserById(int id);
    List<UserDTO> getAllUsers();
    void createUser(UserDTO userDTO);
}

// Service Implementation
class UserServiceImpl implements UserService {
    private UserDAO userDAO;
```

```
    public UserServiceImpl(UserDAO userDAO) {
        this.userDAO = userDAO;
    }

    @Override
    public UserDTO getUserById(int id) {
        UserEntity entity = userDAO.getUserById(id);
        return UserMapper.toDTO(entity);
    }
}
```

---

5. MVC Pattern

Model-View-Controller

```
■■■
```

```
}

// Usage
UserRepository repository = new UserRepository();
UserService service = new UserService(repository);
```

Code Examples

BackendPatternsDemo

```
// Entity - Database model
class User {
    private int id;
    private String username;
    private String email;
    private String password;

    // Getters and setters
}

// DTO - For API responses
class UserDTO {
    private int id;
    private String username;
    private String email;

    public static UserDTO fromEntity(User user) {
        UserDTO dto = new UserDTO();
        dto.setId(user.getId());
        dto.setUsername(user.getUsername());
        dto.setEmail(user.getEmail());
        return dto;
    }
}

// DAO - Data access
class UserDAO {
    public User findById(int id) {
        // Simulated DB call
        return new User();
    }
}

// Service - Business logic
class UserService {
    private UserDAO userDAO;

    public UserService(UserDAO userDAO) {
```

```
        this.userDAO = userDAO;
    }

    public UserDTO getUserById(int id) {
        User user = userDAO.findById(id);
        return UserDTO.fromEntity(user);
    }
}

// Controller - API endpoint
class UserController {
    private UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    // Simulating GET /users/1
    public String getUser(int id) {
        UserDTO user = userService.getUserById(id);
        return user.toJson();
    }
}

// Demo
public class BackendPatternsDemo {
    public static void main(String[] args) {
        System.out.println("=== BACKEND DESIGN PATTERNS DEMO ===\n");

        // Build dependency chain
        UserDAO userDAO = new UserDAO();
        UserService userService = new UserService(userDAO);
        UserController userController = new UserController(userService);

        // Simulate API call
        System.out.println("GET /api/users/1");
        String response = userController.getUser(1);
        System.out.println("Response: " + response);
    }
}
```

Summary

Pattern Summary

Pattern	Purpose
Repository	Abstract data access

Pattern	Purpose
DAO	Database operations
DTO	Data transfer between layers
Service	Business logic
MVC	Request handling
DI	Dependency management

Backend Patterns Complete!

Code Examples

DesignPatternsDemo.java

DesignPatternsDemo.java

```
// DesignPatternsDemo - Common Design Patterns for Backend Development
// Important for building maintainable Java applications

import java.util.*;

public class DesignPatternsDemo {

    public static void main(String[] args) {
        System.out.println("=== DESIGN PATTERNS DEMO ===\n");

        // Singleton Pattern
        System.out.println("--- Singleton Pattern ---");
        DatabaseConnection conn1 = DatabaseConnection.getInstance();
        DatabaseConnection conn2 = DatabaseConnection.getInstance();
        System.out.println("Same instance: " + (conn1 == conn2));

        // Factory Pattern
        System.out.println("\n--- Factory Pattern ---");
        Notification email = NotificationFactory.create("EMAIL");
        email.send("Welcome!");
        Notification sms = NotificationFactory.create("SMS");
        sms.send("Your code is 1234");

        // Observer Pattern
        System.out.println("\n--- Observer Pattern ---");
        NewsAgency agency = new NewsAgency();
        NewsChannel channel1 = new NewsChannel("Channel 1");
        NewsChannel channel2 = new NewsChannel("Channel 2");
        agency.addObserver(channel1);
        agency.addObserver(channel2);
```

```
        agency.publishNews("Breaking: Java 21 Released!");

        // Repository Pattern
        System.out.println("\n--- Repository Pattern ---");
        UserRepository repo = new UserRepository();
        User user = new User(1, "john@email.com", "John");
        repo.save(user);
        User found = repo.findById(1);
        System.out.println("Found: " + found.getName());

        // Strategy Pattern
        System.out.println("\n--- Strategy Pattern ---");
        PaymentContext context = new PaymentContext();
        context.setStrategy(new CreditCardPayment());
        context.pay(100);
        context.setStrategy(new PayPalPayment());
        context.pay(50);
    }
}

// ===== SINGLETON PATTERN =====
class DatabaseConnection {
    private static DatabaseConnection instance;

    private DatabaseConnection() {}

    public static synchronized DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }

    public void query(String sql) {
        System.out.println("Executing: " + sql);
    }
}

// ===== FACTORY PATTERN =====
interface Notification {
    void send(String message);
}

class EmailNotification implements Notification {
    public void send(String message) {
        System.out.println("Email sent: " + message);
    }
}

class SMSNotification implements Notification {
    public void send(String message) {
        System.out.println("SMS sent: " + message);
    }
```

```
}

class NotificationFactory {
    public static Notification create(String type) {
        switch (type) {
            case "EMAIL": return new EmailNotification();
            case "SMS": return new SMSNotification();
            default: throw new IllegalArgumentException("Unknown type");
        }
    }
}

// ===== OBSERVER PATTERN =====

interface Observer {
    void update(String news);
}

class NewsAgency {
    private List<Observer> observers = new ArrayList<>();

    public void addObserver(Observer o) { observers.add(o); }
    public void removeObserver(Observer o) { observers.remove(o); }

    public void publishNews(String news) {
        System.out.println("Agency publishes: " + news);
        for (Observer o : observers) {
            o.update(news);
        }
    }
}

class NewsChannel implements Observer {
    private String name;

    public NewsChannel(String name) { this.name = name; }

    public void update(String news) {
        System.out.println(name + " received: " + news);
    }
}

// ===== REPOSITORY PATTERN =====
class User {
    private int id;
    private String email;
    private String name;

    public User(int id, String email, String name) {
        this.id = id;
        this.email = email;
        this.name = name;
    }
```

```
    public int getId() { return id; }
    public String getEmail() { return email; }
    public String getName() { return name; }
}

class UserRepository {
    private Map<Integer, User> users = new HashMap<>();

    public void save(User user) {
        users.put(user.getId(), user);
        System.out.println("Saved user: " + user.getName());
    }

    public User findById(int id) {
        return users.get(id);
    }

    public List<User> findAll() {
        return new ArrayList<>(users.values());
    }
}

// ===== STRATEGY PATTERN =====
interface PaymentStrategy {
    void pay(int amount);
}

class CreditCardPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid $" + amount + " via Credit Card");
    }
}

class PayPalPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid $" + amount + " via PayPal");
    }
}

class PaymentContext {
    private PaymentStrategy strategy;

    public void setStrategy(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public void pay(int amount) {
        strategy.pay(amount);
    }
}
```

ServiceFactoryDemo.java

ServiceFactoryDemo.java

```
// ServiceFactoryDemo - Demonstrates Design Patterns for Angular Services
// Use case: Backend design patterns that Angular developers should understand

public class ServiceFactoryDemo {

    // Singleton pattern
    static class DatabaseService {
        private static DatabaseService instance;

        private DatabaseService() {
            System.out.println("Database connected");
        }

        public static DatabaseService getInstance() {
            if (instance == null) {
                instance = new DatabaseService();
            }
            return instance;
        }

        public String query(String sql) {
            return "Result: " + sql;
        }
    }

    // Factory pattern
    interface DataService {
        String fetch();
    }

    static class UserDataService implements DataService {
        public String fetch() { return "User data"; }
    }

    static class ProductDataService implements DataService {
        public String fetch() { return "Product data"; }
    }

    static class DataServiceFactory {
        public static DataService getService(String type) {
            switch (type) {
                case "user": return new UserDataService();
                case "product": return new ProductDataService();
                default: return null;
            }
        }
    }

    // Builder pattern
    static class UserBuilder {
        private String name = "";
```

```
        private String email = "";
        private String role = "USER";

        public UserBuilder name(String name) {
            this.name = name;
            return this;
        }

        public UserBuilder email(String email) {
            this.email = email;
            return this;
        }

        public UserBuilder role(String role) {
            this.role = role;
            return this;
        }

        public String build() {
            return "User: " + name + ", " + email + ", " + role;
        }
    }

    // Observer pattern
    interface Observer {
        void update(String message);
    }

    static class UserObserver implements Observer {
        private String name;

        public UserObserver(String name) {
            this.name = name;
        }

        public void update(String message) {
            System.out.println(name + " received: " + message);
        }
    }

    static class NotificationService {
        private java.util.List<Observer> observers = new java.util.ArrayList<>();

        public void addObserver(Observer o) {
            observers.add(o);
        }

        public void notifyAll(String message) {
            for (Observer o : observers) {
                o.update(message);
            }
        }
    }
```

```
    public static void main(String[] args) {
        System.out.println("=== DESIGN PATTERNS FOR BACKEND ===\n");

        // Singleton
        System.out.println("--- Singleton ---");
        DatabaseService db1 = DatabaseService.getInstance();
        DatabaseService db2 = DatabaseService.getInstance();
        System.out.println("Same instance: " + (db1 == db2));

        // Factory
        System.out.println("\n--- Factory ---");
        DataService userService = DataServiceFactory.getService("user");
        DataService productService = DataServiceFactory.getService("product");
        System.out.println("User: " + userService.fetch());
        System.out.println("Product: " + productService.fetch());

        // Builder
        System.out.println("\n--- Builder ---");
        String user1 = new UserBuilder()
            .name("Alice")
            .email("alice@email.com")
            .role("ADMIN")
            .build();
        System.out.println(user1);

        String user2 = new UserBuilder()
            .name("Bob")
            .email("bob@email.com")
            .build();
        System.out.println(user2);

        // Observer
        System.out.println("\n--- Observer ---");
        NotificationService notifications = new NotificationService();
        notifications.addObserver(new UserObserver("Angular App"));
        notifications.addObserver(new UserObserver("Admin Panel"));
        notifications.notifyAll("New user registered!");

        System.out.println("\n=== ANGULAR EQUIVALENTS ===");
        System.out.println("Java Pattern     -> Angular");
        System.out.println("----------------------------");
        System.out.println("Singleton        -> providedIn: 'root'");
        System.out.println("Factory          -> Service with switch");
        System.out.println("Builder          -> FormBuilder");
        System.out.println("Observer         -> RxJS Observables");
    }
}
```

30

REST API Backend Angular Integration

30_REST_API_Backend_Angular_Integration\examples\30_REST_API_Backend_Angular_Integration.md

Java Backend and Angular Frontend Integration

Table of Contents

1 Introduction to REST API
2 JSON and Java Objects
3 RESTful Controller Design
4 Angular Service Integration
5 Complete Full-Stack Example
6 Best Practices

---

1. Introduction to REST API

What is REST?

REST (Representational State Transfer) is an architectural style for building web services.

```
■■■
```

```
GET    /api/users          - Get all users
GET    /api/users/1        - Get user by ID
POST   /api/users          - Create new user
PUT    /api/users/1        - Update user
DELETE /api/users/1        - Delete user
```

2. JSON and Java Objects

JSON Basics

```
{
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com",
    "age": 25,
    "isActive": true,
    "roles": ["admin", "user"]
}
```

Java Class to JSON

```
// Java POJO (Plain Old Java Object)
class User {
    private int id;
    private String name;
    private String email;
    private int age;
    private boolean isActive;
    private List<String> roles;

    // Constructors, getters, setters...

    // toString for display
}

// JSON Response (what Angular receives)
{
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com",
    "age": 25,
    "isActive": true,
    "roles": ["admin", "user"]
}
```

3. RESTful Controller Design

Simple REST Controller (without Spring)

```
import java.util.*;

/**
 * User entity - Model for user data
 */
class User {
    private int id;
    private String name;
    private String email;

    public User(int id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    // Getters and setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    // Convert to JSON-like string
    public String toJson() {
        return "{\"id\":" + id + ",\"name\":\"" + name + "\",\"email\":\"" + email + "\"}";
    }
}

/**
 * Simple REST Controller simulation
 */
class UserController {
    private List<User> users = new ArrayList<>();
    private int nextId = 1;

    public UserController() {
        // Initialize with sample data
        users.add(new User(nextId++, "John Doe", "john@example.com"));
        users.add(new User(nextId++, "Jane Smith", "jane@example.com"));
    }

    // GET /api/users - Get all users
    public String getAllUsers() {
        StringBuilder json = new StringBuilder("[");
        for (int i = 0; i < users.size(); i++) {
```

```
            json.append(users.get(i).toJson());
            if (i < users.size() - 1) json.append(",");
        }
        json.append("]");
        return json.toString();
    }

    // GET /api/users/{id} - Get user by ID
    public String getUserById(int id) {
        for (User user : users) {
            if (user.getId() == id) {
                return user.toJson();
            }
        }
        return "{\"error\":\"User not found\"}";
    }

    // POST /api/users - Create user
    public String createUser(String name, String email) {
        User newUser = new User(nextId++, name, email);
        users.add(newUser);
        return newUser.toJson();
    }

    // PUT /api/users/{id} - Update user
    public String updateUser(int id, String name, String email) {
        for (User user : users) {
            if (user.getId() == id) {
                user.setName(name);
                user.setEmail(email);
                return user.toJson();
            }
        }
        return "{\"error\":\"User not found\"}";
    }

    // DELETE /api/users/{id} - Delete user
    public String deleteUser(int id) {
        for (int i = 0; i < users.size(); i++) {
            if (users.get(i).getId() == id) {
                users.remove(i);
                return "{\"message\":\"User deleted\"}";
            }
        }
        return "{\"error\":\"User not found\"}";
    }
}

/**
 * REST Controller Demo
 */
public class RESTControllerDemo {
    public static void main(String[] args) {
```

```
        UserController controller = new UserController();

        System.out.println("=== REST API DEMO ===\n");

        // GET all users
        System.out.println("GET /api/users");
        System.out.println(controller.getAllUsers());

        // GET user by ID
        System.out.println("\nGET /api/users/1");
        System.out.println(controller.getUserById(1));

        // POST - Create user
        System.out.println("\nPOST /api/users (Creating new user)");
        System.out.println(controller.createUser("Alice Brown", "alice@example.com"));

        // GET all users again
        System.out.println("\nGET /api/users (After creation)");
        System.out.println(controller.getAllUsers());

        // PUT - Update user
        System.out.println("\nPUT /api/users/1 (Updating user)");
        System.out.println(controller.updateUser(1, "John Updated", "john.new@example.com"));

        // DELETE - Delete user
        System.out.println("\nDELETE /api/users/2");
        System.out.println(controller.deleteUser(2));

        // GET all users again
        System.out.println("\nGET /api/users (Final)");
        System.out.println(controller.getAllUsers());
    }
}
```

4. Angular Service Integration

How Angular Consumes REST APIs

```
// Angular Service Example (TypeScript)
// This is what you'd create in your Angular frontend

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

export interface User {
  id: number;
  name: string;
```

```
  email: string;
}

@Injectable({
  providedIn: 'root'
})
export class UserService {
  private apiUrl = 'http://localhost:8080/api/users';

  constructor(private http: HttpClient) { }

  // GET - Fetch all users
  getUsers(): Observable<User[]> {
    return this.http.get<User[]>(this.apiUrl);
  }

  // GET - Fetch single user
  getUser(id: number): Observable<User> {
    return this.http.get<User>(`${this.apiUrl}/${id}`);
  }

  // POST - Create user
  createUser(user: User): Observable<User> {
    return this.http.post<User>(this.apiUrl, user);
  }

  // PUT - Update user
  updateUser(id: number, user: User): Observable<User> {
    return this.http.put<User>(`${this.apiUrl}/${id}`, user);
  }

  // DELETE - Delete user
  deleteUser(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${id}`);
  }
}
```

Angular Component Example

```
// Angular Component Example
import { Component, OnInit } from '@angular/core';
import { UserService, User } from './user.service';

@Component({
  selector: 'app-user-list',
  template: `
    <h2>Users</h2>
    <button (click)="loadUsers()">Load Users</button>

    <div *ngFor="let user of users">
```

```
      <p>{{ user.id }} - {{ user.name }} - {{ user.email }}</p>
    </div>
  `
})
export class UserListComponent implements OnInit {
  users: User[] = [];

  constructor(private userService: UserService) { }

  ngOnInit(): void {
    this.loadUsers();
  }

  loadUsers(): void {
    this.userService.getUsers().subscribe(data => {
      this.users = data;
      console.log('Users loaded:', this.users);
    });
  }
}
```

---

5. Complete Full-Stack Example

Java Backend: Product Catalog

```
import java.util.*;

/**
 * Product - Represents a product in e-commerce
 */
class Product {
    private int id;
    private String name;
    private String description;
    private double price;
    private String category;
    private int stock;

    public Product(int id, String name, String description,
                   double price, String category, int stock) {
        this.id = id;
        this.name = name;
        this.description = description;
        this.price = price;
        this.category = category;
        this.stock = stock;
    }
```

```
    // Getters
    public int getId() { return id; }
    public String getName() { return name; }
    public String getDescription() { return description; }
    public double getPrice() { return price; }
    public String getCategory() { return category; }
    public int getStock() { return stock; }

    // Setters
    public void setName(String name) { this.name = name; }
    public void setDescription(String description) { this.description = description; }
    public void setPrice(double price) { this.price = price; }
    public void setCategory(String category) { this.category = category; }
    public void setStock(int stock) { this.stock = stock; }

    // Convert to JSON
    public String toJson() {
        return String.format(
            "{\"id\":%d,\"name\":\"%s\",\"description\":\"%s\",\"price\":%.2f,\"category\":\"%s\"...
            id, name, description, price, category, stock
        );
    }

    // Create from JSON (simplified)
    public static Product fromJson(String json) {
        // Simplified parsing - in real app use Jackson/Gson
        return new Product(0, "Parsed", "From JSON", 0, "Unknown", 0);
    }
}

/**
 * ProductController - REST API endpoints
 */
class ProductController {
    private List<Product> products = new ArrayList<>();
    private int nextId = 1;

    public ProductController() {
        // Initialize sample products
        addSampleData();
    }

    private void addSampleData() {
        products.add(new Product(nextId++, "MacBook Pro", "Apple laptop", 2499.99, "Electronics",...
        products.add(new Product(nextId++, "iPhone 15", "Apple smartphone", 999.99, "Electronics"...
        products.add(new Product(nextId++, "AirPods Pro", "Wireless earbuds", 249.99, "Electronic...
        products.add(new Product(nextId++, "T-Shirt", "Cotton t-shirt", 29.99, "Clothing", 500));
        products.add(new Product(nextId++, "Jeans", "Denim jeans", 79.99, "Clothing", 150));
    }

    // GET /api/products - Get all products
    public String getAllProducts() {
        StringBuilder json = new StringBuilder("[");
```

```
        for (int i = 0; i < products.size(); i++) {
            json.append(products.get(i).toJson());
            if (i < products.size() - 1) json.append(",");
        }
        json.append("]");
        return json.toString();
    }

    // GET /api/products/{id} - Get product by ID
    public String getProductById(int id) {
        for (Product p : products) {
            if (p.getId() == id) {
                return p.toJson();
            }
        }
        return "{\"error\":\"Product not found\"}";
    }

    // GET /api/products/category/{category} - Get products by category
    public String getProductsByCategory(String category) {
        StringBuilder json = new StringBuilder("[");
        boolean first = true;
        for (Product p : products) {
            if (p.getCategory().equalsIgnoreCase(category)) {
                if (!first) json.append(",");
                json.append(p.toJson());
                first = false;
            }
        }
        json.append("]");
        return json.toString();
    }

    // POST /api/products - Create product
    public String createProduct(String name, String description,
                                double price, String category, int stock) {
        Product newProduct = new Product(nextId++, name, description, price, category, stock);
        products.add(newProduct);
        return newProduct.toJson();
    }

    // PUT /api/products/{id} - Update product
    public String updateProduct(int id, String name, String description,
                                double price, String category, int stock) {
        for (Product p : products) {
            if (p.getId() == id) {
                p.setName(name);
                p.setDescription(description);
                p.setPrice(price);
                p.setCategory(category);
                p.setStock(stock);
                return p.toJson();
            }
```

```
        }
        return "{\"error\":\"Product not found\"}";
    }

    // DELETE /api/products/{id} - Delete product
    public String deleteProduct(int id) {
        for (int i = 0; i < products.size(); i++) {
            if (products.get(i).getId() == id) {
                products.remove(i);
                return "{\"message\":\"Product deleted successfully\"}";
            }
        }
        return "{\"error\":\"Product not found\"}";
    }

    // GET /api/products/search?name={name} - Search products
    public String searchProducts(String name) {
        StringBuilder json = new StringBuilder("[");
        boolean first = true;
        for (Product p : products) {
            if (p.getName().toLowerCase().contains(name.toLowerCase())) {
                if (!first) json.append(",");
                json.append(p.toJson());
                first = false;
            }
        }
        json.append("]");
        return json.toString();
    }
}

/**
 * FullStackDemo - Demonstrates complete flow
 */
public class FullStackDemo {
    public static void main(String[] args) {
        ProductController controller = new ProductController();

        System.out.println("=== FULL-STACK DEMO: E-COMMERCE API ===\n");

        // Get all products
        System.out.println("1. GET /api/products");
        System.out.println(controller.getAllProducts());

        // Get products by category
        System.out.println("\n2. GET /api/products/category/Electronics");
        System.out.println(controller.getProductsByCategory("Electronics"));

        // Search products
        System.out.println("\n3. GET /api/products/search?name=Phone");
        System.out.println(controller.searchProducts("Phone"));

        // Create new product
```

```
        System.out.println("\n4. POST /api/products - Create new product");
        System.out.println(controller.createProduct(
            "iPad Pro",
            "Apple tablet",
            1099.99,
            "Electronics",
            75
        ));

        // Update product
        System.out.println("\n5. PUT /api/products/1 - Update product");
        System.out.println(controller.updateProduct(
            1,
            "MacBook Pro M3",
            "Updated Apple laptop",
            2799.99,
            "Electronics",
            30
        ));

        // Delete product
        System.out.println("\n6. DELETE /api/products/3");
        System.out.println(controller.deleteProduct(3));

        // Get all products again
        System.out.println("\n7. GET /api/products - Final state");
        System.out.println(controller.getAllProducts());

        System.out.println("\n=== Integration with Angular ===");
        System.out.println("In Angular, you would call these endpoints:");
        System.out.println("  - this.http.get<Product[]>('/api/products')");
        System.out.println("  - this.http.get<Product[]>('/api/products/category/Electronics')");
        System.out.println("  - this.http.post<Product>('/api/products', productData)");
    }
}
```

6. Best Practices

Java Backend Best Practices

1. **Use proper HTTP methods** - GET for reading, POST for creating, PUT for updating, DELETE for deleting
2. **Return appropriate status codes** - 200 OK, 201 Created, 404 Not Found, 500 Server Error
3. **Use proper JSON serialization** - Use Jackson or Gson library
4. **Validate input** - Check for null values and invalid data
5. **Use DTOs** - Data Transfer Objects for API responses

Angular Frontend Best Practices

1. **Use services** - Centralize HTTP calls in services
2. **Handle errors** - Always handle error responses
3. **Use types** - Define interfaces for type safety
4. **Subscribe properly** - Handle subscriptions and unsubscriptions
5. **Use async pipe** - Prefer async pipe for displaying data

JSON Naming Convention

```
// Java uses camelCase
class User {
    private String firstName;
    private String lastName;
    private Date createdAt;

    // Jackson annotations for JSON
    @JsonProperty("first_name")  // snake_case in JSON
    public String getFirstName() { return firstName; }
}
```

Summary

Key Takeaways

1. **REST API** - Standard way for Angular to communicate with Java backend
2. **JSON** - Data format exchanged between frontend and backend
3. **HTTP Methods** - GET (read), POST (create), PUT (update), DELETE (delete)
4. **Status Codes** - 200 (OK), 201 (Created), 404 (Not Found), 500 (Error)
5. **Integration** - Angular HttpClient maps to Java REST endpoints

Full-Stack Architecture

```
■■■
```

```
■           ■ HTTP (JSON)                                   ■
■           ▼                                               ■
■   ■■■■■■■■■■■■■■■■■■■■■■                                      ■
■   ■  REST API       ■  - Endpoints                        ■
■   ■  Controllers    ■  - Request handling                 ■
■   ■■■■■■■■■■■■■■■■■■■■■■                                      ■
■           ■                                               ■
■   ■■■■■■■■■■▼■■■■■■■■■■                                       ■
■   ■  Java Backend   ■  - Business logic                   ■
■   ■  Services       ■  - Data processing                  ■
■   ■  Repositories   ■  - Database access                  ■
■   ■■■■■■■■■■■■■■■■■■■■■■                                      ■
■                                                            ■
■■■■■■■■■■■■■■■■■■■
```

```
    // Convert to JSON format
    public String toJSON() {
        return String.format("{\"id\":%d,\"email\":\"%s\",\"name\":\"%s\",\"role\":\"%s\"}",
            id, email, name, role);
    }
}
```

UserController.java

UserController.java

```
// UserController.java - REST API GET operations Demo
// Shows how Java backend returns JSON for Angular

import java.util.*;
import java.util.stream.*;

public class UserController {

    // Simulated database
    private static List<User> users = new ArrayList<>();

    static {
        users.add(new User(1, "alice@email.com", "Alice Smith", "USER"));
        users.add(new User(2, "bob@email.com", "Bob Johnson", "ADMIN"));
        users.add(new User(3, "charlie@email.com", "Charlie Brown", "USER"));
    }

    // ===== GET ALL USERS (GET /api/users) =====
    public static List<User> getAllUsers() {
        return users;
    }

    // ===== GET USER BY ID (GET /api/users/{id}) =====
    public static User getUserById(int id) {
        return users.stream()
            .filter(u -> u.getId() == id)
            .findFirst()
            .orElse(null);
    }

    // ===== SEARCH USERS (GET /api/users?role=ADMIN) =====
    public static List<User> searchByRole(String role) {
        return users.stream()
            .filter(u -> u.getRole().equalsIgnoreCase(role))
            .collect(Collectors.toList());
    }

    // Main demo for GET operations
    public static void main(String[] args) {
        System.out.println("=== REST API GET DEMO ===\n");
```

```
        // GET all users
        System.out.println("--- GET /api/users ---");
        List<User> allUsers = getAllUsers();
        for (User u : allUsers) {
            System.out.println(u.toJSON());
        }

        // GET by ID
        System.out.println("\n--- GET /api/users/1 ---");
        User user1 = getUserById(1);
        System.out.println(user1.toJSON());

        // Search by role
        System.out.println("\n--- GET /api/users?role=ADMIN ---");
        List<User> admins = searchByRole("ADMIN");
        for (User u : admins) {
            System.out.println(u.toJSON());
        }
    }
}
```

UserCRUDDemo.java

UserCRUDDemo.java

```
// UserCRUDDemo.java - REST API Create, Update, Delete operations Demo
// Shows POST, PUT, DELETE operations for Angular integration

import java.util.*;

public class UserCRUDDemo {

    // Simulated database
    private static List<User> users = new ArrayList<>();

    static {
        users.add(new User(1, "alice@email.com", "Alice Smith", "USER"));
        users.add(new User(2, "bob@email.com", "Bob Johnson", "ADMIN"));
        users.add(new User(3, "charlie@email.com", "Charlie Brown", "USER"));
    }

    // ===== CREATE USER (POST /api/users) =====
    public static User createUser(String email, String name, String role) {
        int newId = users.size() + 1;
        User newUser = new User(newId, email, name, role);
        users.add(newUser);
        return newUser;
    }

    // ===== UPDATE USER (PUT /api/users/{id}) =====
    public static User updateUser(int id, String name, String role) {
```

```
        for (User user : users) {
            if (user.getId() == id) {
                user.setName(name);
                user.setRole(role);
                return user;
            }
        }
        return null;
    }

    // ===== DELETE USER (DELETE /api/users/{id}) =====
    public static boolean deleteUser(int id) {
        return users.removeIf(u -> u.getId() == id);
    }

    // Main demo for CRUD operations
    public static void main(String[] args) {
        System.out.println("=== REST API CRUD DEMO ===\n");

        // POST create
        System.out.println("--- POST /api/users ---");
        User newUser = createUser("diana@email.com", "Diana Prince", "USER");
        System.out.println(newUser.toJSON());

        // PUT update
        System.out.println("\n--- PUT /api/users/3 ---");
        User updated = updateUser(3, "Charlie Wilson", "ADMIN");
        System.out.println(updated.toJSON());

        // DELETE
        System.out.println("\n--- DELETE /api/users/2 ---");
        boolean deleted = deleteUser(2);
        System.out.println("Deleted: " + deleted);

        System.out.println("\n=== FINAL USER LIST ===");
        for (User u : users) {
            System.out.println(u.toJSON());
        }

        System.out.println("\n=== KEY HTTP METHODS ===");
        System.out.println("POST /api/users - Create new user");
        System.out.println("PUT /api/users/{id} - Update existing user");
        System.out.println("DELETE /api/users/{id} - Delete user");
    }
}
```

31

File IO

Code Examples

UserDataFileDemo.java

UserDataFileDemo.java

```
// UserDataFileDemo - Demonstrates File I/O for User Data Storage
// Use case: Reading/writing user data files for Angular app

import java.io.*;
import java.util.*;

public class UserDataFileDemo {

    // Simple user class
    static class User implements Serializable {
        private int id;
        private String name;
        private String email;

        public User(int id, String name, String email) {
            this.id = id;
            this.name = name;
            this.email = email;
        }

        public int getId() { return id; }
        public String getName() { return name; }
        public String getEmail() { return email; }

        @Override
        public String toString() {
            return id + "," + name + "," + email;
        }

        public static User fromString(String line) {
            String[] parts = line.split(",");
            return new User(
                Integer.parseInt(parts[0]),
                parts[1],
                parts[2]
            );
        }
    }

    // Write users to CSV file
    public static void writeUsersToFile(String filename, List<User> users) {
        try (PrintWriter writer = new PrintWriter(new FileWriter(filename))) {
            for (User user : users) {
                writer.println(user);
            }
```

```
            System.out.println("Written " + users.size() + " users to " + filename);
        } catch (IOException e) {
            System.out.println("Error writing: " + e.getMessage());
        }
    }

    // Read users from CSV file
    public static List<User> readUsersFromFile(String filename) {
        List<User> users = new ArrayList<>();

        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
            String line;
            while ((line = reader.readLine()) != null) {
                users.add(User.fromString(line));
            }
            System.out.println("Read " + users.size() + " users from " + filename);
        } catch (IOException e) {
            System.out.println("Error reading: " + e.getMessage());
        }

        return users;
    }

    // Serialize users to binary file
    public static void serializeUsers(String filename, List<User> users) {
        try (ObjectOutputStream oos = new ObjectOutputStream(
                new FileOutputStream(filename))) {
            oos.writeObject(users);
            System.out.println("Serialized " + users.size() + " users");
        } catch (IOException e) {
            System.out.println("Error serializing: " + e.getMessage());
        }
    }

    // Deserialize users from binary file
    @SuppressWarnings("unchecked")
    public static List<User> deserializeUsers(String filename) {
        try (ObjectInputStream ois = new ObjectInputStream(
                new FileInputStream(filename))) {
            List<User> users = (List<User>) ois.readObject();
            System.out.println("Deserialized " + users.size() + " users");
            return users;
        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Error deserializing: " + e.getMessage());
            return new ArrayList<>();
        }
    }

    // Append single user to file
    public static void appendUser(String filename, User user) {
        try (PrintWriter writer = new PrintWriter(
                new FileWriter(filename, true))) {
            writer.println(user);
```

```
            System.out.println("Appended user: " + user.getName());
        } catch (IOException e) {
            System.out.println("Error appending: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        System.out.println("=== FILE I/O FOR USER DATA ===\n");

        // Create sample users
        List<User> users = Arrays.asList(
            new User(1, "Alice", "alice@email.com"),
            new User(2, "Bob", "bob@email.com"),
            new User(3, "Charlie", "charlie@email.com")
        );

        // Write to CSV
        String csvFile = "users.csv";
        System.out.println("--- Writing CSV ---");
        writeUsersToFile(csvFile, users);

        // Read from CSV
        System.out.println("\n--- Reading CSV ---");
        List<User> readUsers = readUsersFromFile(csvFile);
        for (User u : readUsers) {
            System.out.println(u.getId() + ": " + u.getName() + " (" + u.getEmail() + ")");
        }

        // Serialize
        String binFile = "users.dat";
        System.out.println("\n--- Serializing ---");
        serializeUsers(binFile, users);

        // Deserialize
        System.out.println("\n--- Deserializing ---");
        List<User> deserialized = deserializeUsers(binFile);
        for (User u : deserialized) {
            System.out.println(u.getId() + ": " + u.getName());
        }

        // Append
        System.out.println("\n--- Appending ---");
        appendUser(csvFile, new User(4, "Diana", "diana@email.com"));

        // Read again
        System.out.println("\n--- After Append ---");
        readUsersFromFile(csvFile);

        System.out.println("\n=== ANGULAR USE CASES ===");
        System.out.println("1. Export data to CSV");
        System.out.println("2. Import user lists");
        System.out.println("3. Cache data locally");
        System.out.println("4. Backup/restore functionality");
```

```
        System.out.println("5. Log file management");
    }
}
```

31

Spring Boot Basics

31_Spring_Boot_Basics\examples\31_Spring_Boot_Basics.md

Spring Boot for Angular Developers

Table of Contents

1 Introduction to Spring Boot
2 Spring Boot REST Controllers
3 Spring Data JPA
4 Dependency Injection
5 Spring Boot Application Properties
6 Building a REST API
7 Angular Integration

---

1. Introduction to Spring Boot

What is Spring Boot?

Spring Boot makes it easy to create stand-alone, production-grade Spring-based applications.

```
■■■
```

2. Spring Boot REST Controllers

Creating a REST Controller

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping
    public List<User> getAllUsers() {
        return userService.findAll();
    }

    @GetMapping("/{id}")
    public User getUserById(@PathVariable Long id) {
        return userService.findById(id);
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        return userService.save(user);
    }

    @PutMapping("/{id}")
    public User updateUser(@PathVariable Long id, @RequestBody User user) {
        return userService.update(id, user);
    }

    @DeleteMapping("/{id}")
    public void deleteUser(@PathVariable Long id) {
        userService.delete(id);
    }
}
```

HTTP Method Annotations

Annotation	HTTP Method	Purpose
@GetMapping	GET	Retrieve data
@PostMapping	POST	Create new resource
@PutMapping	PUT	Update existing resource
@PatchMapping	PATCH	Partial update
@DeleteMapping	DELETE	Delete resource

3. Spring Data JPA

Entity Classes

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(unique = true)
    private String email;

    private int age;

    // Constructors, getters, setters
}
```

Repository Interface

```
public interface UserRepository extends JpaRepository<User, Long> {
    // Custom queries
    List<User> findByName(String name);

    @Query("SELECT u FROM User u WHERE u.email = ?1")
    User findByEmail(String email);

    List<User> findByAgeGreaterThan(int age);
}
```

---

4. Dependency Injection

@Service and @Autowired

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;
```

```
    public List<User> findAll() {
        return userRepository.findAll();
    }

    public User save(User user) {
        return userRepository.save(user);
    }
}
```

Constructor Injection (Recommended)

```
@Service
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
```

---

5. Spring Boot Application Properties

application.properties

```
# Server
server.port=8080

# Database
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=password

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# CORS (for Angular)
server.cors.allowed-origins=http://localhost:4200
```

---

6. Building a REST API

Complete Example

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}

// ============ USER ENTITY ============
package com.example.demo.model;

import javax.persistence.*;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(unique = true)
    private String email;

    private int age;

    // Constructors
    public User() {}

    public User(String name, String email, int age) {
        this.name = name;
        this.email = email;
        this.age = age;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
```

```
    public void setEmail(String email) { this.email = email; }

    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
}

// ============ USER REPOSITORY ============
package com.example.demo.repository;

import com.example.demo.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface UserRepository extends JpaRepository<User, Long> {
    List<User> findByName(String name);
    List<User> findByAgeGreaterThan(int age);
}

// ============ USER SERVICE ============
package com.example.demo.service;

import com.example.demo.model.User;
import com.example.demo.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;

@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public List<User> findAll() {
        return userRepository.findAll();
    }

    public Optional<User> findById(Long id) {
        return userRepository.findById(id);
    }

    public User save(User user) {
        return userRepository.save(user);
    }

    public void delete(Long id) {
        userRepository.deleteById(id);
    }
}

// ============ USER CONTROLLER ============
package com.example.demo.controller;
```

```
import com.example.demo.model.User;
import com.example.demo.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/users")
@CrossOrigin(origins = "http://localhost:4200")  // Angular URL
public class UserController {

    @Autowired
    private UserService userService;

    // GET /api/users
    @GetMapping
    public List<User> getAllUsers() {
        return userService.findAll();
    }

    // GET /api/users/1
    @GetMapping("/{id}")
    public ResponseEntity<User> getUserById(@PathVariable Long id) {
        return userService.findById(id)
            .map(user -> ResponseEntity.ok(user))
            .orElse(ResponseEntity.notFound().build());
    }

    // POST /api/users
    @PostMapping
    public ResponseEntity<User> createUser(@RequestBody User user) {
        User saved = userService.save(user);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    // PUT /api/users/1
    @PutMapping("/{id}")
    public ResponseEntity<User> updateUser(@PathVariable Long id, @RequestBody User user) {
        if (!userService.findById(id).isPresent()) {
            return ResponseEntity.notFound().build();
        }
        user.setId(id);
        return ResponseEntity.ok(userService.save(user));
    }

    // DELETE /api/users/1
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
        if (!userService.findById(id).isPresent()) {
            return ResponseEntity.notFound().build();
```

```
        }
        userService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

---

7. Angular Integration

Angular Service

```
// user.service.ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

export interface User {
  id?: number;
  name: string;
  email: string;
  age: number;
}

@Injectable({
  providedIn: 'root'
})
export class UserService {
  private apiUrl = 'http://localhost:8080/api/users';

  constructor(private http: HttpClient) { }

  getUsers(): Observable<User[]> {
    return this.http.get<User[]>(this.apiUrl);
  }

  getUser(id: number): Observable<User> {
    return this.http.get<User>(`${this.apiUrl}/${id}`);
  }

  createUser(user: User): Observable<User> {
    return this.http.post<User>(this.apiUrl, user);
  }

  updateUser(id: number, user: User): Observable<User> {
    return this.http.put<User>(`${this.apiUrl}/${id}`, user);
  }

  deleteUser(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${id}`);
```

```
  }
}
```

Angular Component

```
// user-list.component.ts
import { Component, OnInit } from '@angular/core';
import { UserService, User } from './user.service';

@Component({
  selector: 'app-user-list',
  template: `
    <h2>Users</h2>
    <button (click)="loadUsers()">Refresh</button>
    <button (click)="showCreateForm = true">Add User</button>

    <div *ngFor="let user of users">
      {{ user.name }} - {{ user.email }}
      <button (click)="deleteUser(user.id!)">Delete</button>
    </div>
  `
})
export class UserListComponent implements OnInit {
  users: User[] = [];
  showCreateForm = false;

  constructor(private userService: UserService) { }

  ngOnInit() {
    this.loadUsers();
  }

  loadUsers() {
    this.userService.getUsers().subscribe(data => {
      this.users = data;
    });
  }

  deleteUser(id: number) {
    this.userService.deleteUser(id).subscribe(() => {
      this.loadUsers();
    });
  }
}
```

Summary

Spring Boot Key Concepts

1. **@SpringBootApplication** - Main class annotation
2. **@RestController** - REST API controller
3. **@Service** - Business logic layer
4. **@Repository** - Data access layer
5. **@Entity** - JPA entity class
6. **JpaRepository** - Database operations
7. **@CrossOrigin** - CORS for Angular
8. **@RequestBody/@ResponseBody** - JSON conversion

Spring Boot Complete!

Code Examples

SpringBootConceptsDemo.java

SpringBootConceptsDemo.java

```
// SpringBootConceptsDemo - Demonstrates Spring Boot Concepts
// Conceptual demo of Spring Boot for Angular developers

import java.util.*;

public class SpringBootConceptsDemo {

    public static void main(String[] args) {
        System.out.println("=== SPRING BOOT CONCEPTS DEMO ===\n");

        // Dependency Injection (Conceptual)
        System.out.println("--- Dependency Injection ---");
        // Instead of: UserService service = new UserService();
        // Spring provides: @Autowired UserService service

        // Inversion of Control Container
        System.out.println("\n--- IoC Container ---");
        System.out.println("Spring manages object lifecycle");
        System.out.println("Beans are created and injected automatically");

        // Bean Scopes
        System.out.println("\n--- Bean Scopes ---");
        System.out.println("@Singleton - One instance per container");
        System.out.println("@Prototype - New instance each time");
        System.out.println("@Request - One per HTTP request");
        System.out.println("@Session - One per HTTP session");

        // REST Controller
        System.out.println("\n--- REST Controller Pattern ---");
        System.out.println("@RestController = @Controller + @ResponseBody");
```

```
        // HTTP Methods mapping
        System.out.println("\n--- HTTP Method Annotations ---");
        System.out.println("@GetMapping - Read data");
        System.out.println("@PostMapping - Create data");
        System.out.println("@PutMapping - Update data");
        System.out.println("@DeleteMapping - Delete data");

        // Request/Response
        System.out.println("\n--- Request/Response Annotations ---");
        System.out.println("@RequestBody - JSON -> Java Object");
        System.out.println("@ResponseBody - Java Object -> JSON");
        System.out.println("@PathVariable - URL path parameters");
        System.out.println("@RequestParam - Query parameters");

        // Service Layer
        System.out.println("\n--- Service Layer ---");
        System.out.println("@Service - Business logic annotation");
        System.out.println("Transaction management with @Transactional");

        // Repository Layer
        System.out.println("\n--- Repository Layer ---");
        System.out.println("@Repository - Data access annotation");
        System.out.println("Spring Data JPA automatically generates queries");

        // Configuration
        System.out.println("\n--- Configuration ---");
        System.out.println("@Configuration - Java-based config");
        System.out.println("@Value - Inject properties");
        System.out.println("@ConfigurationProperties - Type-safe properties");

        // Angular Integration
        System.out.println("\n=== ANGULAR INTEGRATION ===");
        System.out.println("Angular HttpClient <-> Spring @RestController");
        System.out.println("JSON serialization/deserialization automatic");
        System.out.println("CORS configuration for cross-origin requests");
    }
}

// Conceptual example of how Spring would handle requests
// In real Spring Boot, these would be annotations

class UserControllerConcept {

    // @GetMapping("/api/users")
    public List<UserConcept> getAllUsers() {
        return Arrays.asList(
            new UserConcept(1, "alice@email.com"),
            new UserConcept(2, "bob@email.com")
        );
    }

    // @GetMapping("/api/users/{id}")
```

```
    public UserConcept getUserById(int id) {
        return new UserConcept(id, "user@email.com");
    }

    // @PostMapping("/api/users")
    public UserConcept createUser(String email) {
        return new UserConcept(99, email);
    }
}

class UserConcept {
    private int id;
    private String email;

    public UserConcept(int id, String email) {
        this.id = id;
        this.email = email;
    }

    public int getId() { return id; }
    public String getEmail() { return email; }
}
```

32

Database JPA Hibernate

32_Database_JPA_Hibernate\examples\32_Database_JPA_Hibernate.md

JWT Authentication for Angular + Java

Table of Contents

1 Introduction to JWT
2 JWT Structure
3 JWT in Java Backend
4 Angular JWT Integration
5 Security Best Practices

---

1. Introduction to JWT

What is JWT?

JSON Web Token (JWT) is a compact, self-contained way to securely transmit information between parties as JSON.

```
■■■
```

2. JWT Structure

JWT Format

```
JWT = Header.Payload.Signature

Example:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0Ijox...
```

Three Parts

1. **Header** - Token type and signing algorithm
2. **Payload** - Claims (data)
3. **Signature** - Verification

Payload Claims

```
{
  "sub": "1234567890",    // Subject (user ID)
  "name": "John Doe",      // Name
  "email": "john@email.com",
  "role": "ADMIN",         // User role
  "iat": 1516239022,      // Issued at
  "exp": 1516242622       // Expiration
}
```

---

3. JWT in Java Backend

JWT Utility Class

```
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
import java.util.Date;

public class JwtUtil {

    // Secret key for signing (in production, use environment variable)
    private static final String SECRET = "mySecretKey12345678901234567890123456789012";
    private static final long EXPIRATION = 86400000; // 24 hours

    // Generate Token
    public static String generateToken(String username, String role) {
```

```
        Date now = new Date();
        Date expiryDate = new Date(now.getTime() + EXPIRATION);

        // In real implementation, use a JWT library like JJWT
        // This is a simplified example
        String payload = String.format(
            "{\"sub\":\"%s\",\"role\":\"%s\",\"iat\":%d,\"exp\":%d}",
            username, role, now.getTime()/1000, expiryDate.getTime()/1000
        );

        // Encode (simplified - use proper JWT library in production)
        String header = "{\"alg\":\"HS256\",\"typ\":\"JWT\"}";
        String encodedHeader = Base64.getUrlEncoder().encodeToString(header.getBytes());
        String encodedPayload = Base64.getUrlEncoder().encodeToString(payload.getBytes());
        String encodedSignature = Base64.getUrlEncoder().encodeToString(SECRET.getBytes());

        return encodedHeader + "." + encodedPayload + "." + encodedSignature;
    }

    // Validate Token
    public static boolean validateToken(String token) {
        try {
            String[] parts = token.split("\\.");
            if (parts.length != 3) return false;

            String payload = new String(Base64.getUrlDecoder().decode(parts[1]));

            // Check expiration
            if (payload.contains("exp")) {
                // Parse and check expiration
                // In production, use JWT library
            }

            return true;
        } catch (Exception e) {
            return false;
        }
    }

    // Get Username from Token
    public static String getUsernameFromToken(String token) {
        try {
            String[] parts = token.split("\\.");
            String payload = new String(Base64.getUrlDecoder().decode(parts[1]));

            // Extract username from JSON payload
            // In production, use proper JSON parsing
            return "user"; // Simplified
        } catch (Exception e) {
            return null;
        }
    }
```

```
    // Get Role from Token
    public static String getRoleFromToken(String token) {
        try {
            String[] parts = token.split("\\.");
            String payload = new String(Base64.getUrlDecoder().decode(parts[1]));

            // Extract role from JSON payload
            // In production, use proper JSON parsing
            return "USER"; // Simplified
        } catch (Exception e) {
            return null;
        }
    }
}
```

Login Controller

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest request) {
        // Validate credentials (in production, check against database)
        if ("admin".equals(request.getUsername()) && "password".equals(request.getPassword())) {

            // Generate JWT token
            String token = JwtUtil.generateToken(request.getUsername(), "ADMIN");

            // Return token
            Map<String, Object> response = new HashMap<>();
            response.put("token", token);
            response.put("username", request.getUsername());
            response.put("role", "ADMIN");

            return ResponseEntity.ok(response);
        }

        return ResponseEntity.status(401).body(Map.of("error", "Invalid credentials"));
    }

    @PostMapping("/register")
    public ResponseEntity<?> register(@RequestBody RegisterRequest request) {
        // In production: save user to database with hashed password
        return ResponseEntity.ok(Map.of("message", "User registered successfully"));
    }
}

// Request classes
class LoginRequest {
```

```
    private String username;
    private String password;

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}

class RegisterRequest {
    private String username;
    private String password;
    private String email;

    // Getters and setters
}
```

Protected Endpoint

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    // Public endpoint
    @GetMapping
    public List<User> getAllUsers() {
        return userService.findAll();
    }

    // Protected endpoint
    @GetMapping("/profile")
    public ResponseEntity<?> getProfile(@RequestHeader("Authorization") String authHeader) {
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            return ResponseEntity.status(401).body(Map.of("error", "No token provided"));
        }

        String token = authHeader.substring(7);

        if (!JwtUtil.validateToken(token)) {
            return ResponseEntity.status(401).body(Map.of("error", "Invalid token"));
        }

        String username = JwtUtil.getUsernameFromToken(token);
        // Return user profile
        return ResponseEntity.ok(Map.of("username", username, "message", "Profile data"));
    }
}
```

4. Angular JWT Integration

Auth Service

```
// auth.service.ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, BehaviorSubject } from 'rxjs';
import { tap } from 'rxjs/operators';

export interface LoginResponse {
  token: string;
  username: string;
  role: string;
}

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private apiUrl = 'http://localhost:8080/api/auth';
  private tokenKey = 'auth_token';

  private loggedIn = new BehaviorSubject<boolean>(this.hasToken());
  isLoggedIn$ = this.loggedIn.asObservable();

  constructor(private http: HttpClient) {}

  login(username: string, password: string): Observable<LoginResponse> {
    return this.http.post<LoginResponse>(`${this.apiUrl}/login`, { username, password })
      .pipe(
        tap(response => {
          this.setToken(response.token);
          this.loggedIn.next(true);
        })
      );
  }

  logout(): void {
    this.removeToken();
    this.loggedIn.next(false);
  }

  getToken(): string | null {
    return localStorage.getItem(this.tokenKey);
  }

  private setToken(token: string): void {
    localStorage.setItem(this.tokenKey, token);
  }
```

```
  private removeToken(): void {
    localStorage.removeItem(this.tokenKey);
  }

  private hasToken(): boolean {
    return !!this.getToken();
  }

  isLoggedIn(): boolean {
    return this.hasToken();
  }
}
```

Auth Interceptor

```
// auth.interceptor.ts
import { Injectable } from '@angular/core';
import { HttpInterceptor, HttpRequest, HttpHandler, HttpEvent } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable()
export class AuthInterceptor implements HttpInterceptor {

  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    const token = localStorage.getItem('auth_token');

    if (token) {
      const cloned = req.clone({
        headers: req.headers.set('Authorization', `Bearer ${token}`)
      });
      return next.handle(cloned);
    }

    return next.handle(req);
  }
}
```

Login Component

```
// login.component.ts
import { Component } from '@angular/core';
import { Router } from '@angular/router';
import { AuthService } from './auth.service';

@Component({
  selector: 'app-login',
  template: `
    <div class="login-form">
```

```
      <h2>Login</h2>
      <input [(ngModel)]="username" placeholder="Username">
      <input type="password" [(ngModel)]="password" placeholder="Password">
      <button (click)="login()">Login</button>
      <p *ngIf="error" class="error">{{ error }}</p>
    </div>
  `
})
export class LoginComponent {
  username = '';
  password = '';
  error = '';

  constructor(private authService: AuthService, private router: Router) {}

  login(): void {
    this.authService.login(this.username, this.password).subscribe({
      next: () => {
        this.router.navigate(['/dashboard']);
      },
      error: (err) => {
        this.error = 'Invalid credentials';
      }
    });
  }
}
```

Protected Route

```
// auth.guard.ts
import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';
import { AuthService } from './auth.service';

@Injectable({
  providedIn: 'root'
})
export class AuthGuard implements CanActivate {

  constructor(private authService: AuthService, private router: Router) {}

  canActivate(): boolean {
    if (this.authService.isLoggedIn()) {
      return true;
    }
    this.router.navigate(['/login']);
    return false;
  }
}
```

App Module Setup

```
// app.module.ts
import { NgModule } from '@angular/core';
import { HTTP_INTERCEPTORS } from '@angular/common/http';
import { AuthInterceptor } from './auth.interceptor';

@NgModule({
  providers: [
    {
      provide: HTTP_INTERCEPTORS,
      useClass: AuthInterceptor,
      multi: true
    }
  ]
})
export class AppModule { }
```

---

5. Security Best Practices

Backend

1. **Use HTTPS** - Always use SSL/TLS
2. **Store secrets securely** - Use environment variables
3. **Set token expiration** - Short-lived tokens (15-60 min)
4. **Use strong signing algorithms** - HS256, RS256
5. **Validate tokens on every request**
6. **Store hashed passwords** - Never store plain text

Frontend

1. **Store tokens securely** - Use httpOnly cookies when possible
2. **Implement token refresh** - Handle expired tokens
3. **Clear tokens on logout**
4. **Use HTTPS**

Summary

JWT Authentication Flow

1. **Login** - User submits credentials
2. **Generate** - Server creates JWT with user info

3 **Store** - Frontend stores JWT (localStorage or cookie)
4 **Send** - Frontend includes JWT in Authorization header
5 **Validate** - Server validates JWT on each request
6 **Authorize** - Server allows/denies access based on claims

---

JWT Authentication Complete!

Code Examples

JPADemo.java

JPADemo.java

```
// JPADemo - Demonstrates JPA/Hibernate Concepts
// Object-Relational Mapping for database operations

import java.util.*;

public class JPADemo {

    public static void main(String[] args) {
        System.out.println("=== JPA/HIBERNATE DEMO ===\n");

        // Entity basics
        System.out.println("--- JPA Entity Annotations ---");
        System.out.println("@Entity - Marks class as database table");
        System.out.println("@Table - Specifies table name");
        System.out.println("@Id - Primary key");
        System.out.println("@GeneratedValue - Auto-generate ID");
        System.out.println("@Column - Column mapping");

        // Entity example
        System.out.println("\n--- Entity Example ---");
        System.out.println("@Entity");
        System.out.println("@Table(name = \"users\")");
        System.out.println("class User {");
        System.out.println("    @Id");
        System.out.println("    @GeneratedValue(strategy = GenerationType.IDENTITY)");
        System.out.println("    private Long id;");
        System.out.println("    ");
        System.out.println("    @Column(nullable = false)");
        System.out.println("    private String name;");
        System.out.println("    ");
        System.out.println("    @Column(unique = true)");
        System.out.println("    private String email;");
        System.out.println("}");

        // Relationships
        System.out.println("\n--- Relationship Annotations ---");
        System.out.println("@OneToOne - One to one relationship");
```

```
        System.out.println("@OneToMany - One to many (list/set)");
        System.out.println("@ManyToOne - Many to one");
        System.out.println("@ManyToMany - Many to many");
        System.out.println("@JoinColumn - Foreign key column");

        // CRUD Operations
        System.out.println("\n--- CRUD with Repository ---");
        System.out.println("save(entity) - Insert or Update");
        System.out.println("findById(id) - Find by primary key");
        System.out.println("findAll() - Find all entities");
        System.out.println("delete(entity) - Delete entity");
        System.out.println("deleteById(id) - Delete by ID");

        // Query Methods
        System.out.println("\n--- Query Methods ---");
        System.out.println("findByEmail(String email)");
        System.out.println("findByAgeGreaterThan(int age)");
        System.out.println("findByNameContaining(String name)");
        System.out.println("@Query(\"SELECT u FROM User u WHERE u.name = ?1\")");

        // Angular Integration
        System.out.println("\n=== ANGULAR INTEGRATION ===");
        System.out.println("Angular <-> REST API <-> JPA Repository <-> Database");
        System.out.println("");
        System.out.println("Angular calls: GET/POST/PUT/DELETE /api/users");
        System.out.println("Spring converts to: userRepository.findAll(), userRepository.save()");
        System.out.println("Database returns: Result set");
        System.out.println("JPA converts to: List<User>");
        System.out.println("REST converts to: JSON array");
        System.out.println("Angular receives: User[]");

        // Best Practices
        System.out.println("\n--- Best Practices ---");
        System.out.println("1. Use DTOs for API responses");
        System.out.println("2. Avoid exposing entities directly");
        System.out.println("3. Use pagination for large lists");
        System.out.println("4. Validate input with Bean Validation");
        System.out.println("5. Use transactions appropriately");
    }
}

// Example of a JPA Entity
// In real JPA, this would have @Entity annotation
class UserEntity {
    private Long id;
    private String name;
    private String email;
    private String password;
    private Date createdAt;
    private Date updatedAt;
    private boolean active;

    // Getters and setters
```

```
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public Date getCreatedAt() { return createdAt; }
    public void setCreatedAt(Date createdAt) { this.createdAt = createdAt; }
}
```

32

Generics

Code Examples

AngularDataServiceDemo.java

AngularDataServiceDemo.java

```
// AngularDataServiceDemo - Demonstrates Generics for Type-Safe Data Services
// Use case: Generic data services for Angular HTTP responses

public class AngularDataServiceDemo {

    // Generic API Response class
    static class ApiResponse<T> {
        private boolean success;
        private T data;
        private String message;

        public ApiResponse(boolean success, T data, String message) {
            this.success = success;
            this.data = data;
            this.message = message;
        }

        public boolean isSuccess() { return success; }
        public T getData() { return data; }
        public String getMessage() { return message; }

        public static <T> ApiResponse<T> ok(T data) {
            return new ApiResponse<>(true, data, "Success");
        }

        public static <T> ApiResponse<T> error(String message) {
            return new ApiResponse<>(false, null, message);
        }
    }

    // Generic User class
    static class User {
        private int id;
        private String name;

        public User(int id, String name) {
            this.id = id;
            this.name = name;
        }

        public int getId() { return id; }
        public String getName() { return name; }

        @Override
        public String toString() {
```

```
            return "User{id=" + id + ", name='" + name + "'}";
        }
    }

    // Generic Repository
    static class Repository<T> {
        private java.util.List<T> items = new java.util.ArrayList<>();

        public void add(T item) {
            items.add(item);
        }

        public T findById(int id) {
            // Simple implementation
            return items.get(id);
        }

        public java.util.List<T> findAll() {
            return new java.util.ArrayList<>(items);
        }

        public void remove(int id) {
            if (id < items.size()) {
                items.remove(id);
            }
        }
    }

    // Generic Service
    static class UserService {
        private Repository<User> userRepo = new Repository<>();

        public void addUser(User user) {
            userRepo.add(user);
        }

        public User getUser(int id) {
            return userRepo.findById(id);
        }

        public ApiResponse<User> getUserResponse(int id) {
            User user = userRepo.findById(id);
            if (user != null) {
                return ApiResponse.ok(user);
            }
            return ApiResponse.error("User not found");
        }

        public ApiResponse<java.util.List<User>> getAllUsersResponse() {
            return ApiResponse.ok(userRepo.findAll());
        }
    }
```

```
    public static void main(String[] args) {
        System.out.println("=== GENERICS FOR ANGULAR DATA SERVICES ===\n");

        // Create user service
        UserService service = new UserService();

        // Add users
        service.addUser(new User(1, "Alice"));
        service.addUser(new User(2, "Bob"));
        service.addUser(new User(3, "Charlie"));

        // Get single user
        System.out.println("--- Single User ---");
        User user = service.getUser(0);
        System.out.println("Found: " + user);

        // Get response with generics
        System.out.println("\n--- API Response with Generics ---");
        ApiResponse<User> response = service.getUserResponse(1);
        System.out.println("Success: " + response.isSuccess());
        System.out.println("Data: " + response.getData());
        System.out.println("Message: " + response.getMessage());

        // Get all users response
        System.out.println("\n--- All Users Response ---");
        ApiResponse<java.util.List<User>> allResponse = service.getAllUsersResponse();
        System.out.println("Success: " + allResponse.isSuccess());
        for (User u : allResponse.getData()) {
            System.out.println("- " + u);
        }

        // Error response
        System.out.println("\n--- Error Response ---");
        ApiResponse<User> errorResponse = service.getUserResponse(999);
        System.out.println("Success: " + errorResponse.isSuccess());
        System.out.println("Message: " + errorResponse.getMessage());

        System.out.println("\n=== ANGULAR USE CASES ===");
        System.out.println("1. Type-safe HTTP responses");
        System.out.println("2. Generic services (UserService, ProductService)");
        System.out.println("3. Reusable data access layers");
        System.out.println("4. Type-safe caching");
        System.out.println("5. Generic form components");
    }
}
```

33

JSON Jackson Integration

33_JSON_Jackson_Integration\examples\33_JSON_Jackson_Integration.md

Java JSON Handling with Jackson

Table of Contents

1 Introduction to JSON
2 Jackson ObjectMapper
3 Jackson Annotations
4 Working with Complex Objects
5 Angular JSON Handling

---

1. Introduction to JSON

JSON in Java/Angular

JSON is the primary data format used for communication between Java backend and Angular frontend.

```
■■■
```

```
import com.fasterxml.jackson.databind.ObjectMapper;

// Create ObjectMapper
ObjectMapper mapper = new ObjectMapper();

// Java Object to JSON String
User user = new User(1, "John", "john@email.com");
String json = mapper.writeValueAsString(user);

// JSON String to Java Object
String jsonString = "{\"id\":1,\"name\":\"John\",\"email\":\"john@email.com\"}";
User user = mapper.readValue(jsonString, User.class);

// JSON to Map
Map<String, Object> map = mapper.readValue(jsonString, Map.class);

// JSON to List
String jsonArray = "[1,2,3,4,5]";
List<Integer> list = mapper.readValue(jsonArray, List.class);
```

ObjectMapper Configuration

```
ObjectMapper mapper = new ObjectMapper();

// Enable pretty printing
mapper.enable(SerializationFeature.INDENT_OUTPUT);

// Ignore unknown properties
mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);

// Handle dates
mapper.registerModule(new JavaTimeModule());
mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
```

---

3. Jackson Annotations

Class-Level Annotations

```
import com.fasterxml.jackson.annotation.*;

@JsonIgnoreProperties({"password", "secret"})
class User {
    private int id;
    private String name;
    private String email;
```

```
    private String password;  // Will be ignored

    @JsonIgnore
    private String secret;    // Will be ignored

    // @JsonProperty - Rename property
    @JsonProperty("user_name")
    private String name;

    // @JsonInclude - Only include non-null
    @JsonInclude(JsonInclude.Include.NON_NULL)
    private String nickname;
}

// @JsonCreator - Custom constructor
class User {
    @JsonCreator
    public User(@JsonProperty("id") int id, @JsonProperty("name") String name) {
        this.id = id;
        this.name = name;
    }
}
```

Method-Level Annotations

```
class User {
    private int id;
    private String name;

    @JsonGetter
    public int getId() { return id; }

    @JsonSetter
    public void setId(int id) { this.id = id; }

    // Ignore in serialization
    @JsonIgnore
    public String getFullName() { return name; }
}
```

---

4. Working with Complex Objects

Nested Objects

```
class User {
    private int id;
    private String name;
    private Address address;  // Nested object

    // Getters and setters
}

class Address {
    private String street;
    private String city;
    private String zipCode;

    // Getters and setters
}

// Serialization
User user = new User();
user.setId(1);
user.setName("John");
user.setAddress(new Address("123 Main St", "NYC", "10001"));

String json = mapper.writeValueAsString(user);
// {"id":1,"name":"John","address":{"street":"123 Main St","city":"NYC","zipCode":"10001"}}
```

Lists and Arrays

```
class User {
    private int id;
    private String name;
    private List<String> roles;
    private PhoneNumber[] phones;

    // Getters and setters
}

class PhoneNumber {
    private String type;
    private String number;

    // Getters and setters
}

// Usage
List<String> roles = Arrays.asList("ADMIN", "USER");
PhoneNumber[] phones = { new PhoneNumber("mobile", "123-456") };
User user = new User(1, "John", roles, phones);

String json = mapper.writeValueAsString(user);
```

Maps

```
class User {
    private int id;
    private String name;
    private Map<String, String> metadata;

    // Getters and setters
}

// Usage
Map<String, String> metadata = new HashMap<>();
metadata.put("department", "IT");
metadata.put("location", "NYC");

User user = new User();
user.setId(1);
user.setName("John");
user.setMetadata(metadata);

String json = mapper.writeValueAsString(user);
// {"id":1,"name":"John","metadata":{"department":"IT","location":"NYC"}}
```

---

5. Angular JSON Handling

Angular HTTP Response

```
// Angular automatically parses JSON
this.http.get<User>('/api/users/1').subscribe(user => {
  console.log(user.name);  // TypeScript knows the type
});

// Interface matches Java class
interface User {
  id: number;
  name: string;
  email: string;
  address?: Address;
}

interface Address {
  street: string;
  city: string;
  zipCode: string;
}
```

JSON Date Handling

```
// Java - Use LocalDateTime
class User {
    private LocalDateTime createdAt;
    private LocalDate birthDate;
}

// Java - Configure Jackson
ObjectMapper mapper = new ObjectMapper();
mapper.registerModule(new JavaTimeModule());

// Results in ISO-8601 format
// {"createdAt":"2024-01-15T10:30:00","birthDate":"2000-05-20"}
```

Code Examples

Complete Demo

```
import com.fasterxml.jackson.databind.*;
import com.fasterxml.jackson.annotation.*;
import java.util.*;
import java.time.*;

class Address {
    private String street;
    private String city;
    private String zipCode;

    public Address() {}
    public Address(String street, String city, String zipCode) {
        this.street = street;
        this.city = city;
        this.zipCode = zipCode;
    }

    public String getStreet() { return street; }
    public void setStreet(String street) { this.street = street; }
    public String getCity() { return city; }
    public void setCity(String city) { this.city = city; }
    public String getZipCode() { return zipCode; }
    public void setZipCode(String zipCode) { this.zipCode = zipCode; }
}

@JsonIgnoreProperties({"password"})
class User {
    private int id;
    private String name;
    private String email;
```

```
    private String password;  // Will be ignored
    private Address address;
    private List<String> roles;
    private Map<String, String> preferences;

    public User() {}

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }

    public Address getAddress() { return address; }
    public void setAddress(Address address) { this.address = address; }

    public List<String> getRoles() { return roles; }
    public void setRoles(List<String> roles) { this.roles = roles; }

    public Map<String, String> getPreferences() { return preferences; }
    public void setPreferences(Map<String, String> preferences) { this.preferences = preferences; }
}

public class JacksonDemo {
    public static void main(String[] args) throws Exception {
        ObjectMapper mapper = new ObjectMapper();
        mapper.enable(SerializationFeature.INDENT_OUTPUT);

        // Create user object
        User user = new User();
        user.setId(1);
        user.setName("John Doe");
        user.setEmail("john@example.com");
        user.setPassword("secret123");  // Will be ignored in JSON

        user.setAddress(new Address("123 Main St", "New York", "10001"));

        user.setRoles(Arrays.asList("ADMIN", "USER"));

        Map<String, String> prefs = new HashMap<>();
        prefs.put("theme", "dark");
        prefs.put("language", "en");
        user.setPreferences(prefs);

        // Serialize to JSON
        System.out.println("=== SERIALIZATION ===");
        String json = mapper.writeValueAsString(user);
```

```
        System.out.println(json);

        // Deserialize from JSON
        System.out.println("\n=== DESERIALIZATION ===");
        String jsonInput = """
            {
              "id": 2,
              "name": "Jane Smith",
              "email": "jane@example.com",
              "address": {
                "street": "456 Oak Ave",
                "city": "Los Angeles",
                "zipCode": "90001"
              },
              "roles": ["USER"],
              "preferences": {
                "theme": "light"
              }
            }
            """;

        User newUser = mapper.readValue(jsonInput, User.class);
        System.out.println("Name: " + newUser.getName());
        System.out.println("City: " + newUser.getAddress().getCity());
        System.out.println("Roles: " + newUser.getRoles());
    }
}
```

Summary

Jackson Key Features

1. **ObjectMapper** - Main class for JSON processing
2. **@JsonIgnore** - Ignore properties
3. **@JsonProperty** - Rename properties
4. **@JsonIgnoreProperties** - Ignore multiple properties
5. **@JsonCreator** - Custom deserialization
6. **@JsonGetter/@JsonSetter** - Custom getters/setters

Angular Integration

1. Angular HttpClient automatically parses JSON
2. Use TypeScript interfaces matching Java classes
3. Handle dates consistently (ISO-8601)
4. Use HttpInterceptor for authentication tokens

JSON Handling Complete!

Code Examples

JSONDemo.java

JSONDemo.java

```
// JSONDemo - Demonstrates JSON handling in Java
// JSON is the data format exchanged between Angular and Java backend

import java.util.*;

public class JSONDemo {

    public static void main(String[] args) {
        System.out.println("=== JSON CONCEPTS DEMO ===\n");

        // JSON Data Types mapping
        System.out.println("--- JSON to Java Mapping ---");
        System.out.println("string -> String");
        System.out.println("number -> Integer, Long, Double");
        System.out.println("boolean -> Boolean");
        System.out.println("null -> null");
        System.out.println("array -> List<T>");
        System.out.println("object -> Map<String, Object> or custom class");

        // Example: JSON Object -> Java Class
        System.out.println("\n--- Example JSON to Java ---");
        System.out.println("JSON: {");
        System.out.println("  \"id\": 1,");
        System.out.println("  \"name\": \"Alice\",");
        System.out.println("  \"email\": \"alice@example.com\",");
        System.out.println("  \"active\": true,");
        System.out.println("  \"roles\": [\"USER\", \"ADMIN\"]");
        System.out.println("}");

        System.out.println("\nJava class:");
        System.out.println("class User {");
        System.out.println("  private int id;");
        System.out.println("  private String name;");
        System.out.println("  private String email;");
        System.out.println("  private boolean active;");
        System.out.println("  private List<String> roles;");
        System.out.println("}");

        // Jackson Annotations (conceptual)
        System.out.println("\n--- Jackson Annotations ---");
        System.out.println("@JsonProperty(\"user_name\") - Property name mapping");
        System.out.println("@JsonIgnore - Ignore property");
        System.out.println("@JsonFormat - Date formatting");
```

```
        System.out.println("@JsonSerialize - Custom serialization");
        System.out.println("@JsonDeserialize - Custom deserialization");

        // REST API Response patterns
        System.out.println("\n--- REST API Response Patterns ---");

        // Single object response
        System.out.println("\nSingle object:");
        System.out.println("{ \"id\": 1, \"name\": \"Alice\" }");

        // List response
        System.out.println("\nList response:");
        System.out.println("[");
        System.out.println("  { \"id\": 1, \"name\": \"Alice\" },");
        System.out.println("  { \"id\": 2, \"name\": \"Bob\" }");
        System.out.println("]");

        // Paginated response
        System.out.println("\nPaginated response:");
        System.out.println("{");
        System.out.println("  \"content\": [... ],");
        System.out.println("  \"page\": 0,");
        System.out.println("  \"size\": 10,");
        System.out.println("  \"totalElements\": 100,");
        System.out.println("  \"totalPages\": 10");
        System.out.println("}");

        // Error response
        System.out.println("\nError response:");
        System.out.println("{");
        System.out.println("  \"status\": 404,");
        System.out.println("  \"error\": \"Not Found\",");
        System.out.println("  \"message\": \"User not found\",");
        System.out.println("  \"timestamp\": \"2024-01-15T10:30:00\"");
        System.out.println("}");

        // Angular HttpClient usage
        System.out.println("\n=== ANGULAR INTEGRATION ===");
        System.out.println("// Angular service call");
        System.out.println("this.http.get<User[]>('/api/users').subscribe(users => ...)");
        System.out.println("");
        System.out.println("// Angular POST");
        System.out.println("this.http.post('/api/users', userData).subscribe(...)");
    }
}

// Example: What a typical Angular interface maps to in Java
class UserEntity {
    private int id;
    private String name;
    private String email;
    private boolean active;
    private List<String> roles;
```

```
    private Date createdAt;

    // Getters and setters (Jackson uses these)
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public boolean isActive() { return active; }
    public void setActive(boolean active) { this.active = active; }

    public List<String> getRoles() { return roles; }
    public void setRoles(List<String> roles) { this.roles = roles; }

    public Date getCreatedAt() { return createdAt; }
    public void setCreatedAt(Date createdAt) { this.createdAt = createdAt; }
}
```

33

Java 17 Features

Code Examples

ModernJavaFeaturesDemo.java

ModernJavaFeaturesDemo.java

```
// ModernJavaFeaturesDemo - Demonstrates Modern Java Features for Angular Backend
// Use case: Modern Java features for Spring Boot backend

public class ModernJavaFeaturesDemo {

    // Simple DTO class (alternative to records for older Java)
    static class UserDTO {
        private final int id;
        private final String name;
        private final String email;

        public UserDTO(int id, String name, String email) {
            this.id = id;
            this.name = name;
            this.email = email;
        }

        public int getId() { return id; }
        public String getName() { return name; }
        public String getEmail() { return email; }

        // Equals, hashCode, toString generated
        @Override
        public String toString() {
            return "UserDTO{id=" + id + ", name='" + name + "', email='" + email + "'}";
        }
    }

    // Functional interface for processing
    @FunctionalInterface
    interface UserProcessor {
        String process(UserDTO user);
    }

    // Switch expression (Java 14+)
    public static String getDayType(String day) {
        return switch (day) {
            case "Saturday", "Sunday" -> "Weekend";
            case "Monday", "Tuesday", "Wednesday", "Thursday", "Friday" -> "Weekday";
            default -> "Invalid";
        };
    }

    // Text block (Java 15+)
    public static String createJSON(String name, int age) {
```

```
        return """
                {
                    "name": "%s",
                    "age": %d,
                    "active": true
                }
                """.formatted(name, age);
    }

    // Var keyword (Java 10+)
    public static void demonstrateVar() {
        // Type inference with var
        var users = java.util.List.of(
            new UserDTO(1, "Alice", "alice@email.com"),
            new UserDTO(2, "Bob", "bob@email.com")
        );

        // Enhanced for loop with var
        for (var user : users) {
            System.out.println("ID: " + user.getId() + ", Name: " + user.getName());
        }
    }

    // Lambda with var
    public static void demonstrateLambda() {
        UserProcessor upper = (var u) -> u.getName().toUpperCase();

        UserDTO user = new UserDTO(1, "John", "john@email.com");
        System.out.println("Processed: " + upper.process(user));
    }

    // Stream improvements
    public static void demonstrateStreams() {
        var users = java.util.List.of(
            new UserDTO(1, "Alice", "alice@email.com"),
            new UserDTO(2, "Bob", "bob@email.com"),
            new UserDTO(3, "Charlie", "charlie@email.com")
        );

        System.out.println("\n--- Stream Operations ---");

        // Filter and map
        users.stream()
            .filter(u -> u.getId() > 1)
            .map(UserDTO::getName)
            .forEach(System.out::println);

        // Method reference
        users.stream()
            .map(Object::toString)
            .forEach(System.out::println);
    }
```

```
    // Optional improvements
    public static void demonstrateOptional() {
        var users = java.util.List.of(
            new UserDTO(1, "Alice", "alice@email.com"),
            new UserDTO(2, "Bob", "bob@email.com")
        );

        // Find first
        var found = users.stream()
            .filter(u -> u.getId() == 1)
            .findFirst();

        found.ifPresentOrElse(
            u -> System.out.println("Found: " + u.getName()),
            () -> System.out.println("Not found")
        );
    }

    public static void main(String[] args) {
        System.out.println("=== MODERN JAVA FEATURES ===\n");

        // User DTO
        System.out.println("--- User DTO ---");
        var user = new UserDTO(1, "John", "john@email.com");
        System.out.println("User: " + user);

        // Switch expressions
        System.out.println("\n--- Switch Expressions ---");
        System.out.println("Monday: " + getDayType("Monday"));
        System.out.println("Saturday: " + getDayType("Saturday"));

        // Text blocks
        System.out.println("\n--- Text Blocks (JSON) ---");
        String json = createJSON("Alice", 25);
        System.out.println(json);

        // Var keyword
        System.out.println("\n--- Var Keyword ---");
        demonstrateVar();

        // Lambda
        System.out.println("\n--- Lambda with Var ---");
        demonstrateLambda();

        // Streams
        demonstrateStreams();

        // Optional
        System.out.println("\n--- Optional ---");
        demonstrateOptional();

        System.out.println("\n=== ANGULAR USE CASES ===");
        System.out.println("1. DTOs for API responses");
```

```
        System.out.println("2. Lambda expressions for streams");
        System.out.println("3. Text blocks for JSON generation");
        System.out.println("4. Switch expressions for status codes");
        System.out.println("5. Optional for null safety");
    }
}
```